

STARUML:

The image shows two screenshots of the StarUML website. The top screenshot displays the main landing page with the heading "A sophisticated software modeler for agile and concise modeling". Below this, it states "Stop using expensive and complex modeling tools. Software models should be simple, and easy to build with straightforward tools." A prominent blue "Download" button is centered on the page. Below the text is a screenshot of the StarUML application interface, which shows a UML class diagram with classes like "Person", "Address", "Phone", "Email", "Date", "Time", "Money", "Weight", "Height", "Age", "Gender", "MaritalStatus", "Religion", "Nationality", "Language", "SkinColor", "HairColor", "EyeColor", "NoseColor", "MouthColor", "LipColor", "CheekColor", "ForeheadColor", "NeckColor", "ChestColor", "BackColor", "ArmColor", "LegColor", "FootColor", "HandColor", "FingerColor", "ThumbColor", "PalmColor", "WristColor", "AnkleColor", "HeelColor", "ToeColor", "NailColor", "HairColor", "SkinColor", "EyeColor", "NoseColor", "MouthColor", "LipColor", "CheekColor", "ForeheadColor", "NeckColor", "ChestColor", "BackColor", "ArmColor", "LegColor", "FootColor", "HandColor", "FingerColor", "ThumbColor", "PalmColor", "WristColor", "AnkleColor", "HeelColor", "ToeColor", "NailColor". The bottom screenshot shows the "Download and start now" section, which provides instructions for downloading StarUML on macOS, Windows, and Linux. It includes system requirements and links to download the software. Below this is a section titled "More about releases" which mentions that StarUML is continuously updated with new features and bug fixes, and provides links to check the changelog, download previous versions, and read the license agreement.

StarUML Download Pricing Extensions Blog Store Help [Download](#)

A sophisticated software modeler for agile and concise modeling

Stop using expensive and complex modeling tools. Software models should be simple, and easy to build with straightforward tools.

[Download](#)

StarUML

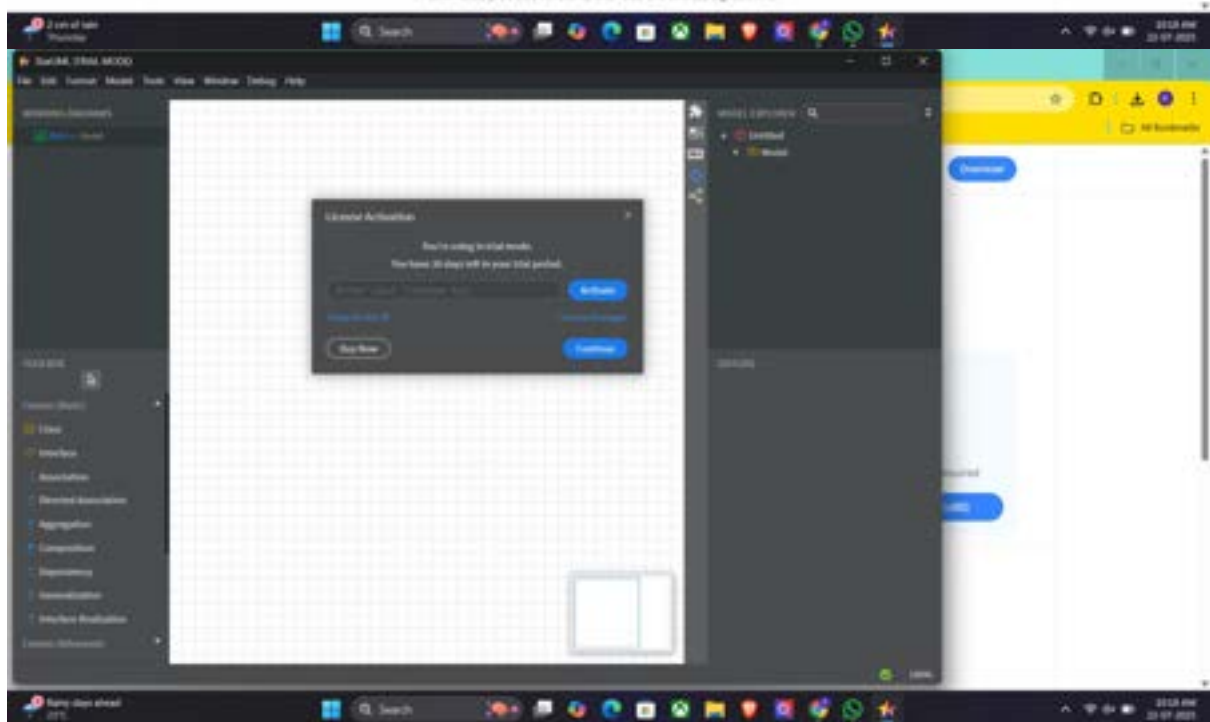
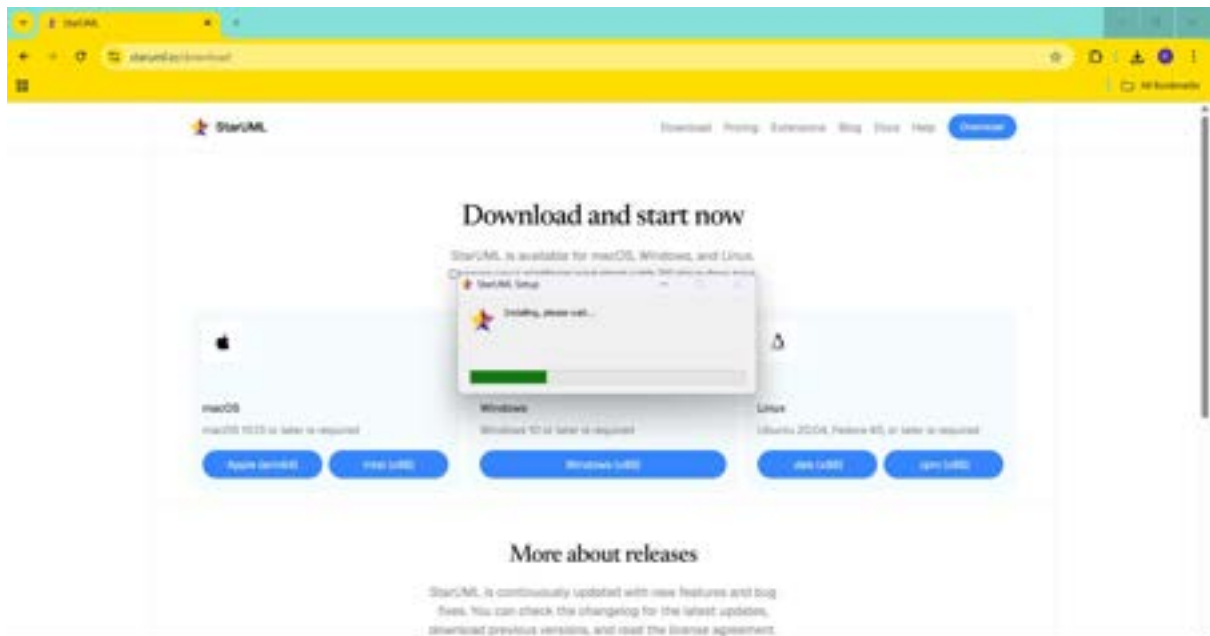
Download and start now

StarUML is available for macOS, Windows, and Linux. Choose your platform and start with 30 days free trial.

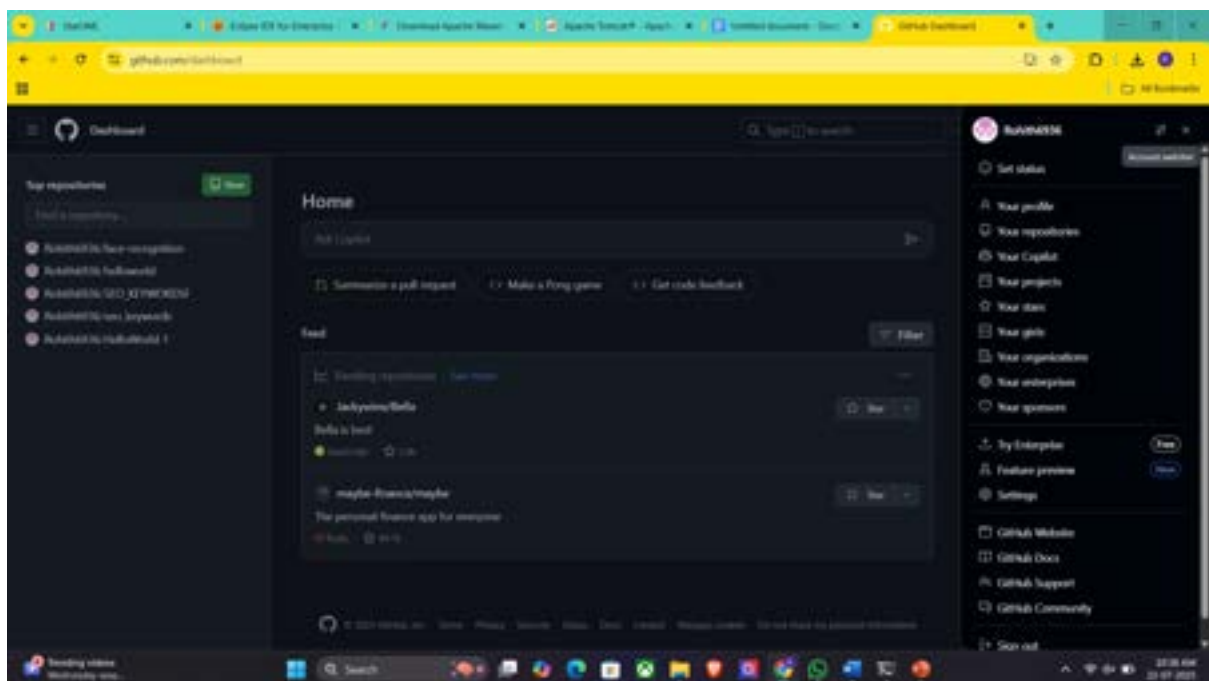
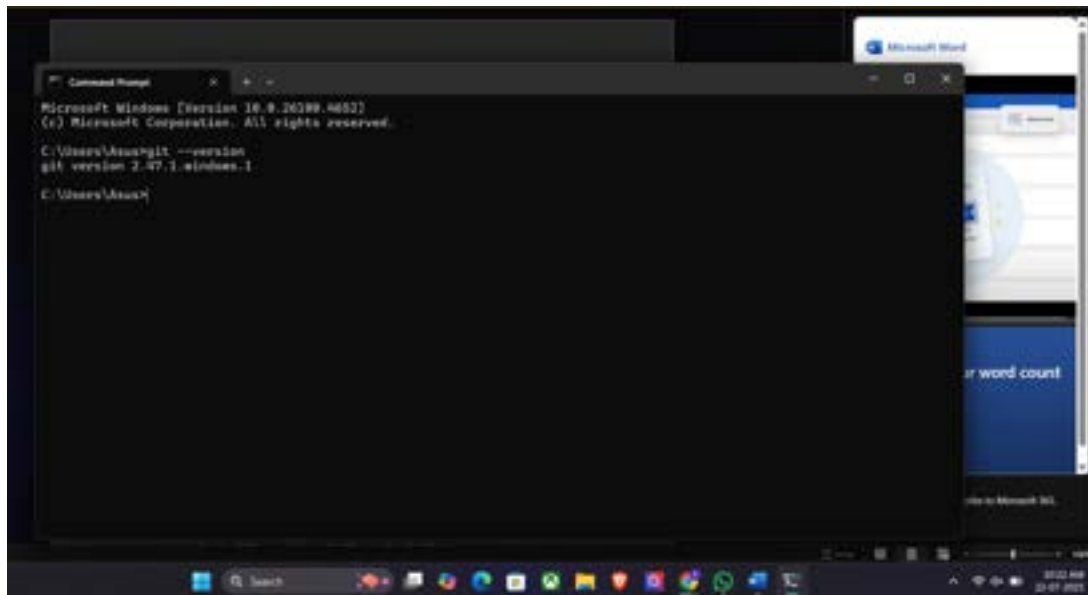
Platform	System Requirements	Download Link
macOS	macOS 10.12 or later is required	Apple download View Screenshot
Windows	Windows 10 or later is required	Windows download View Screenshot
Linux	Ubuntu 20.04, Fedora 35, or later is required	Linux download View Screenshot

More about releases

StarUML is continuously updated with new features and bug fixes. You can check the changelog for the latest updates, download previous versions, and read the license agreement.



GIT:



MAVEN:

```
Command Prompt
Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Asus>git --version
git version 2.47.1.windows.1

C:\Users\Asus>mvn --version
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: C:\apache-maven-3.9.11
Java version: 23.0.1, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-23
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

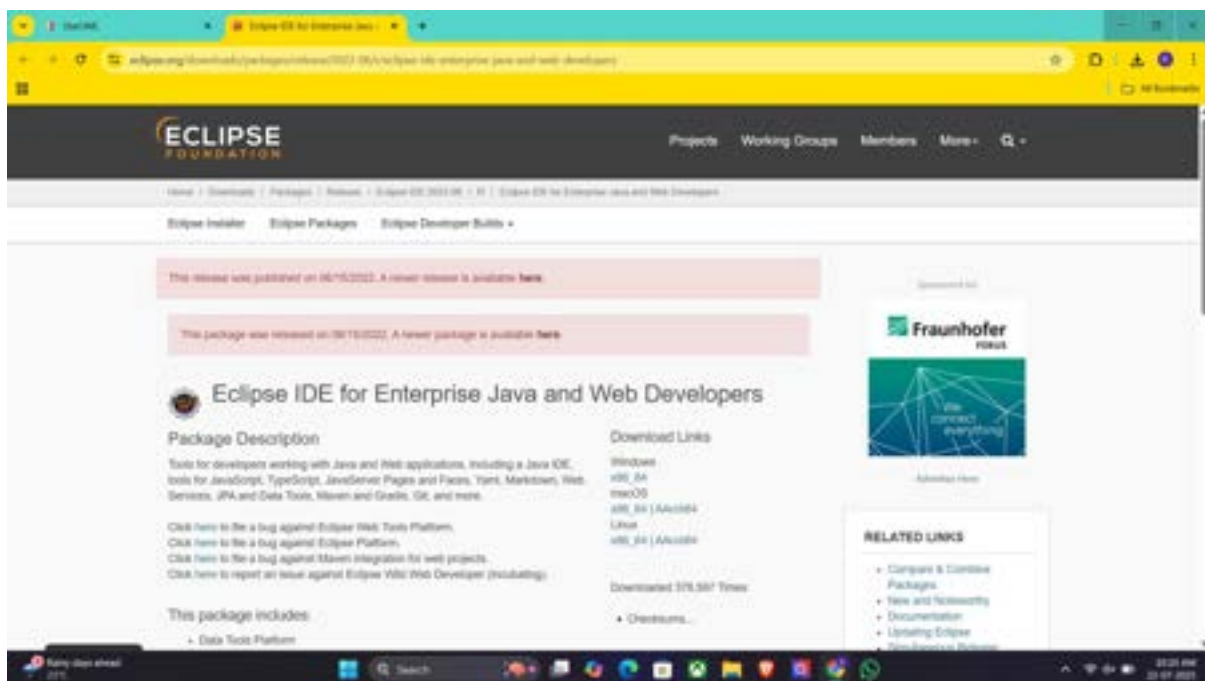
C:\Users\Asus>
```

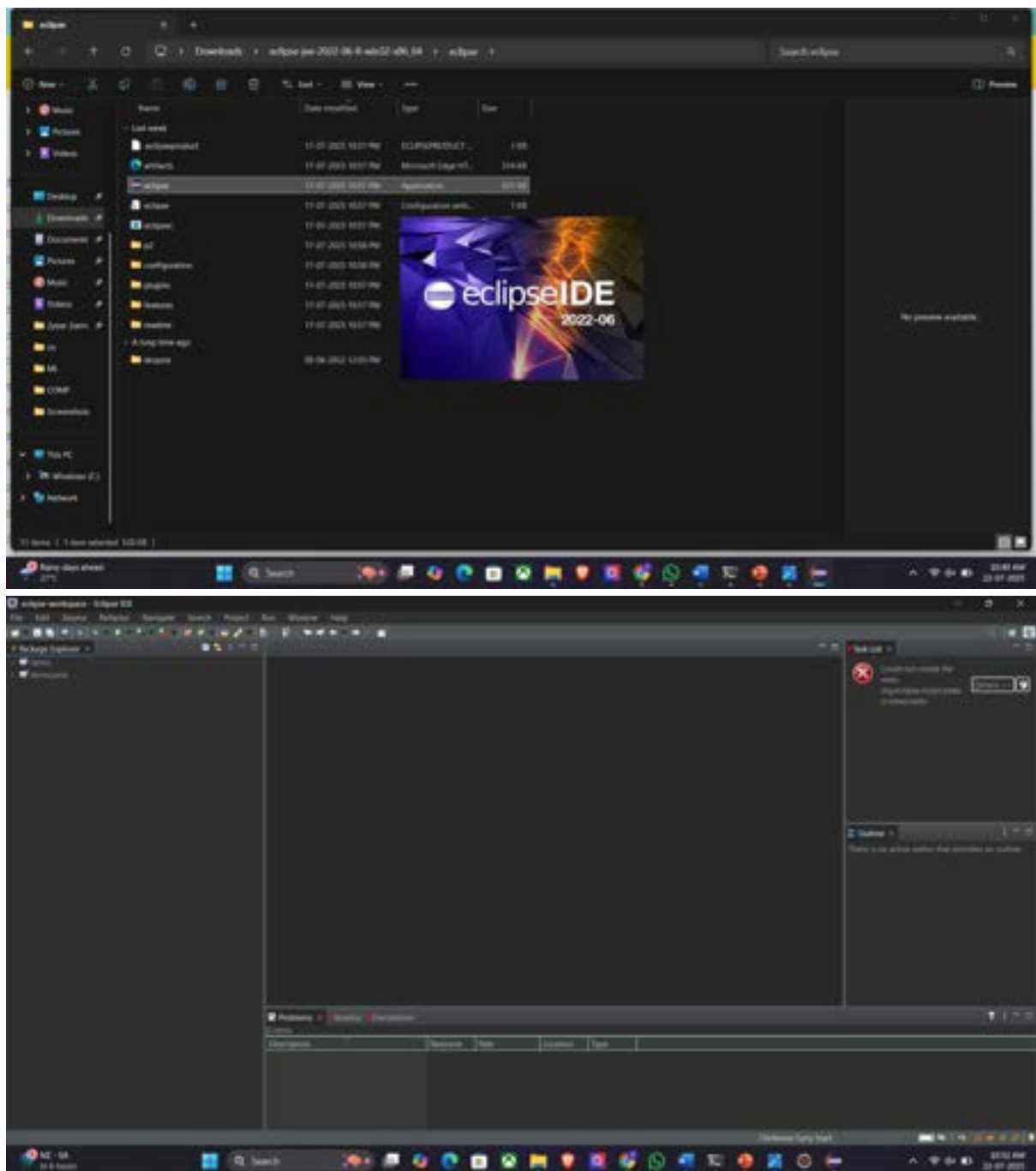
JAVA DEVELOPMENT KIT:

```
C:\Users\Asus>java --version
java 23.0.1 2024-10-15
Java(TM) SE Runtime Environment (build 23.0.1+11-39)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)

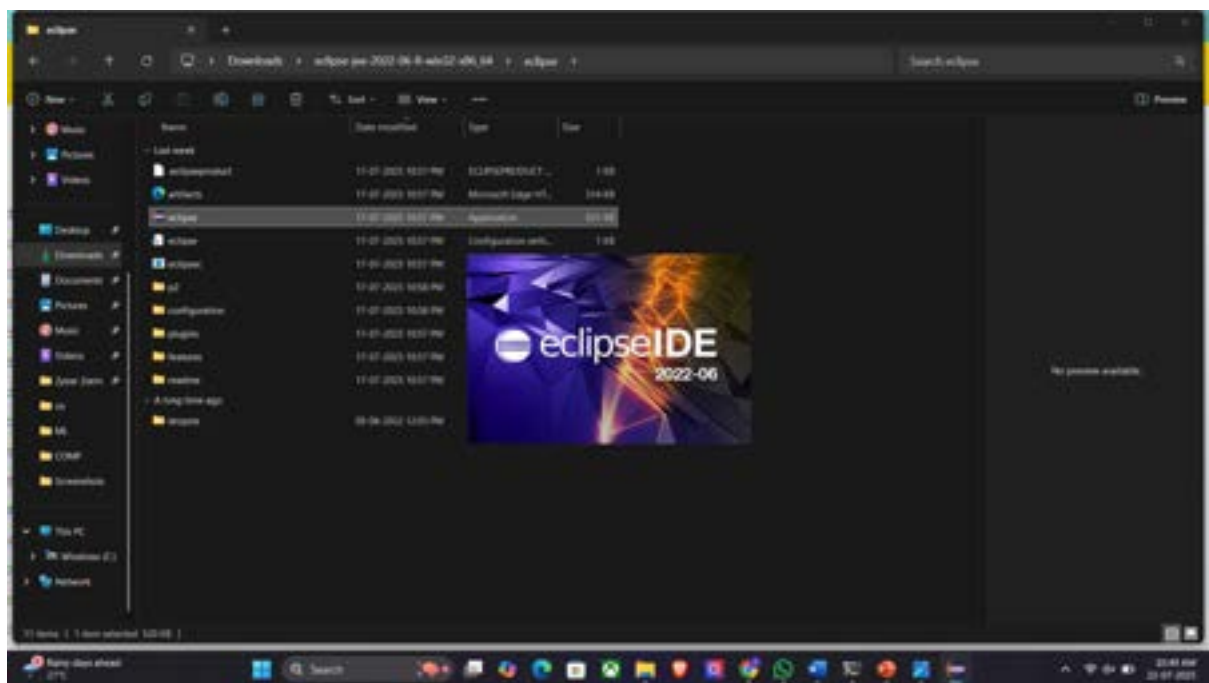
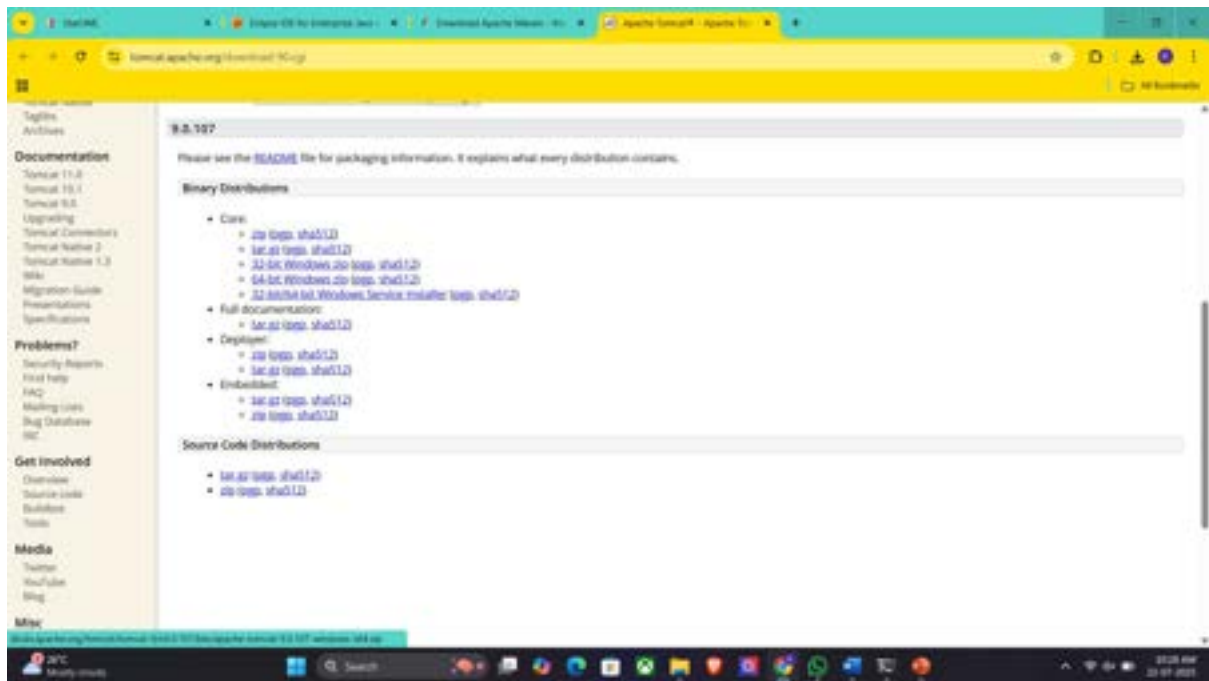
C:\Users\Asus>
```

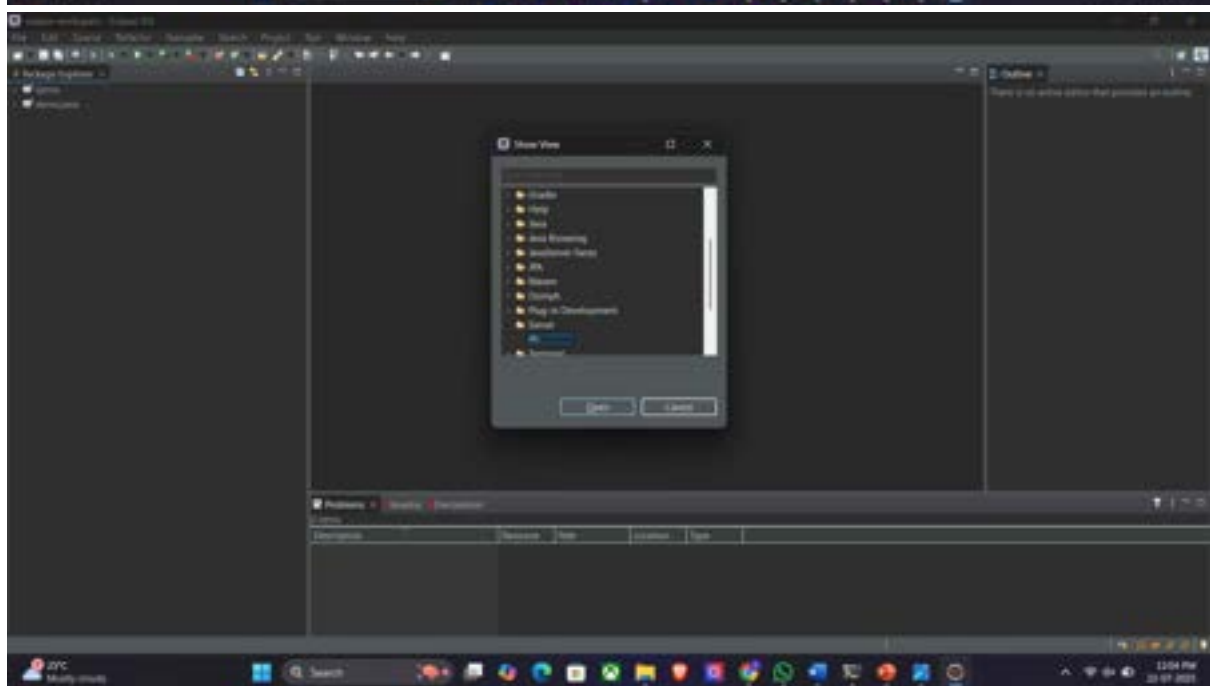
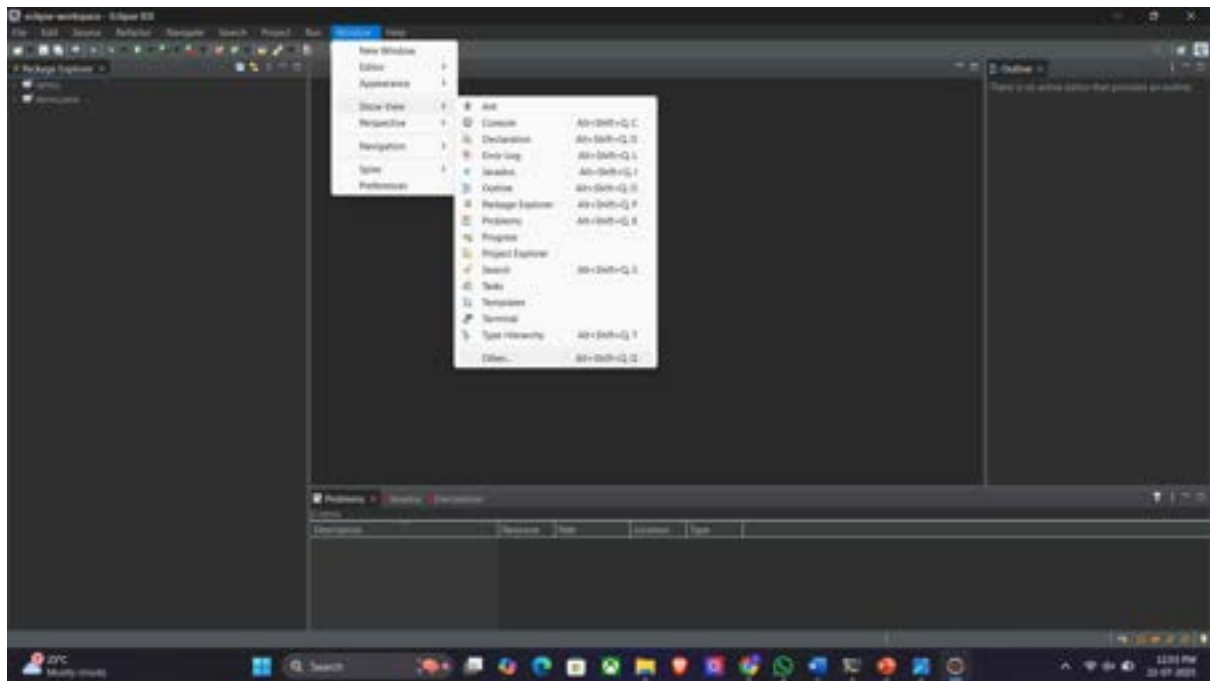
Eclipse IDE:

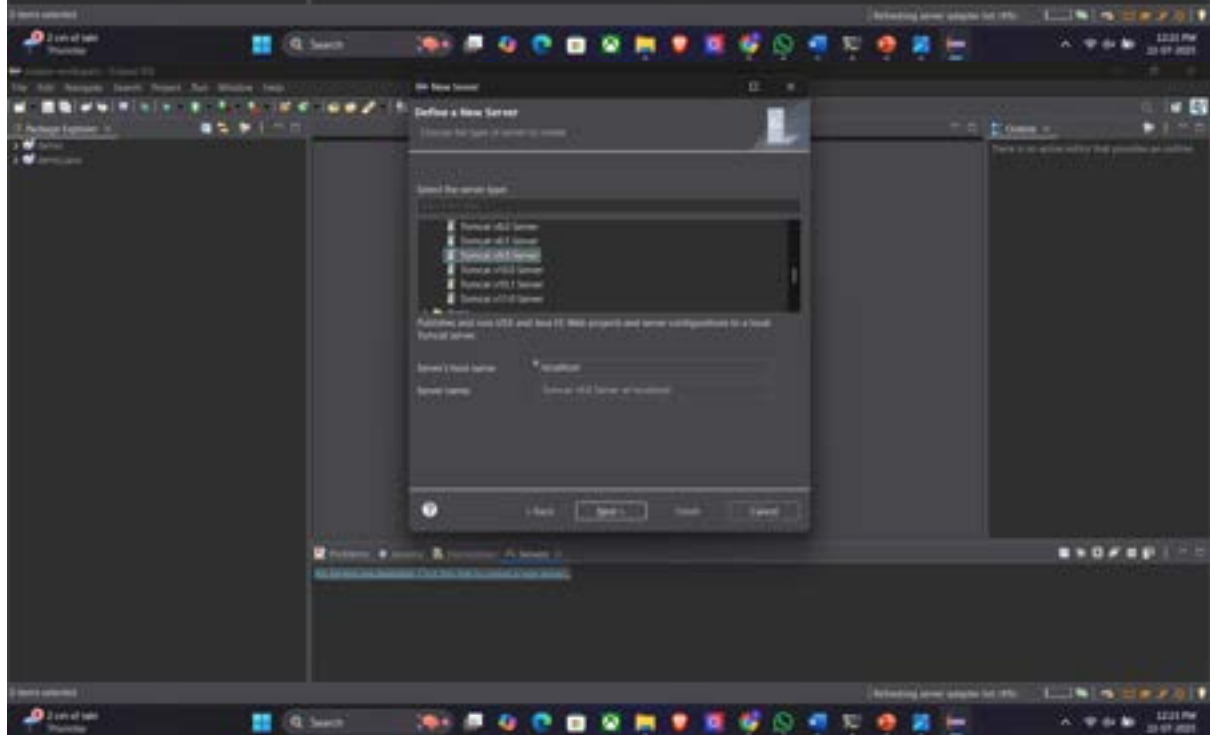
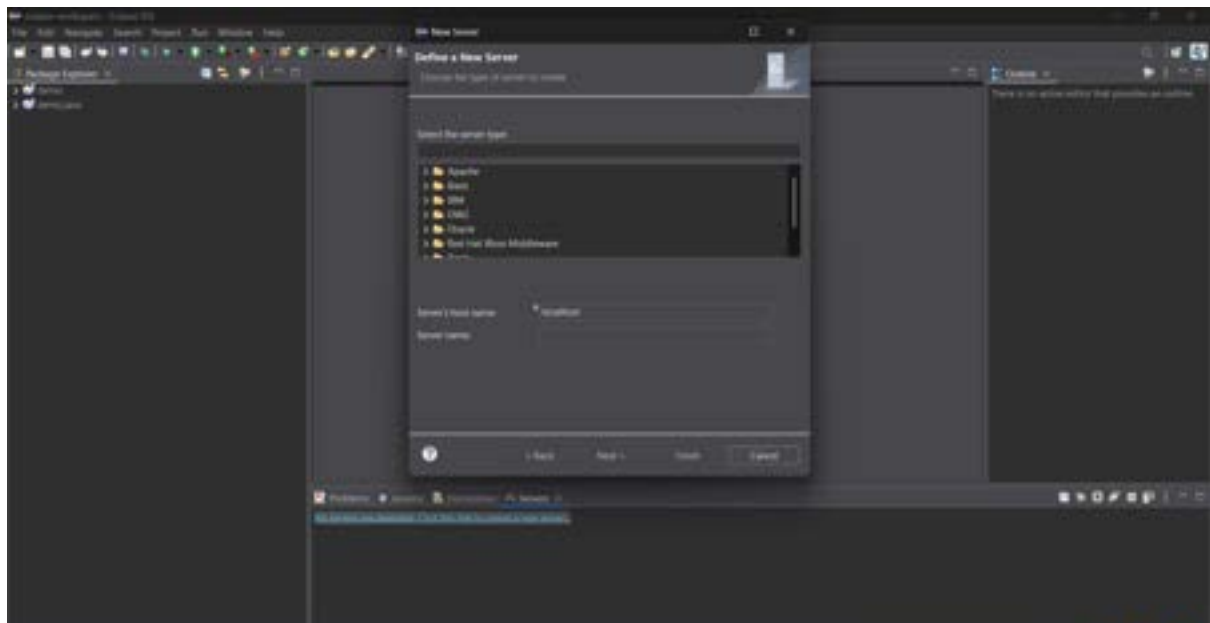


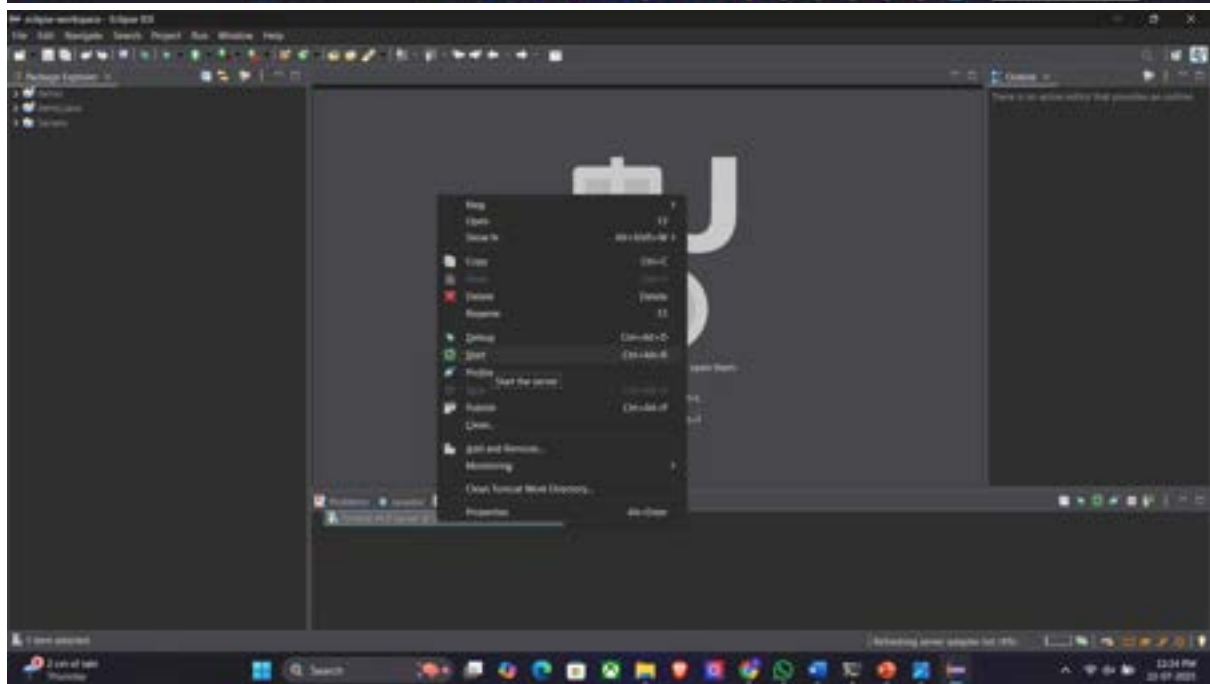
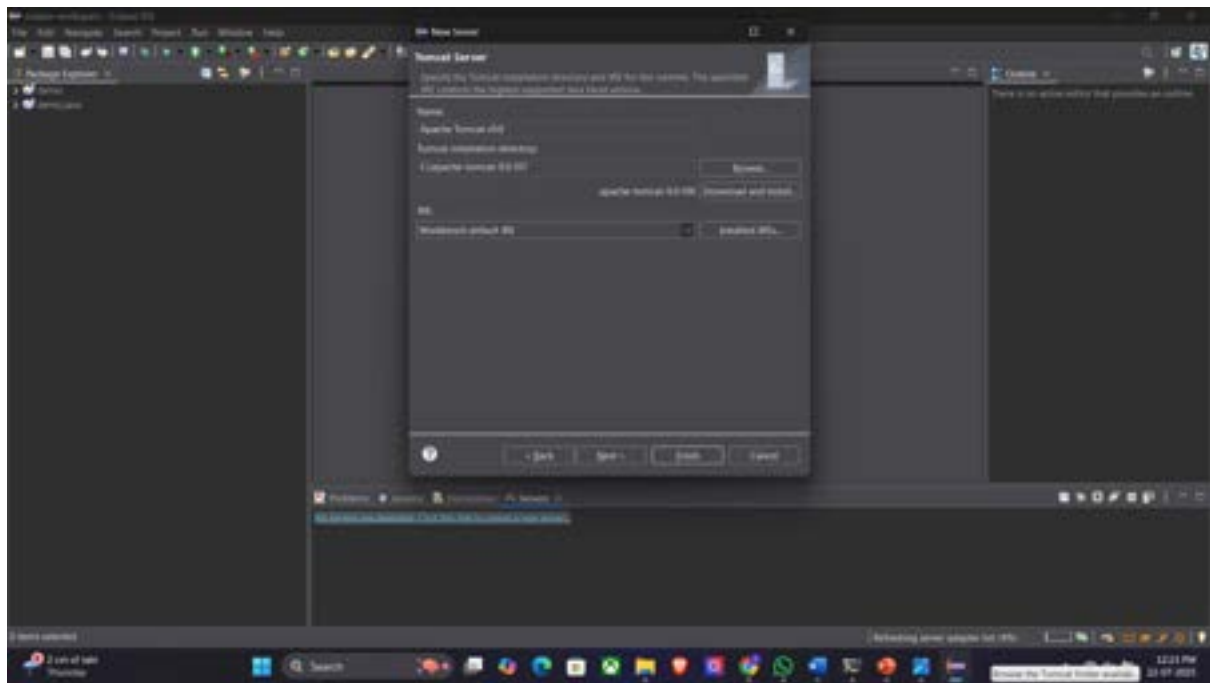


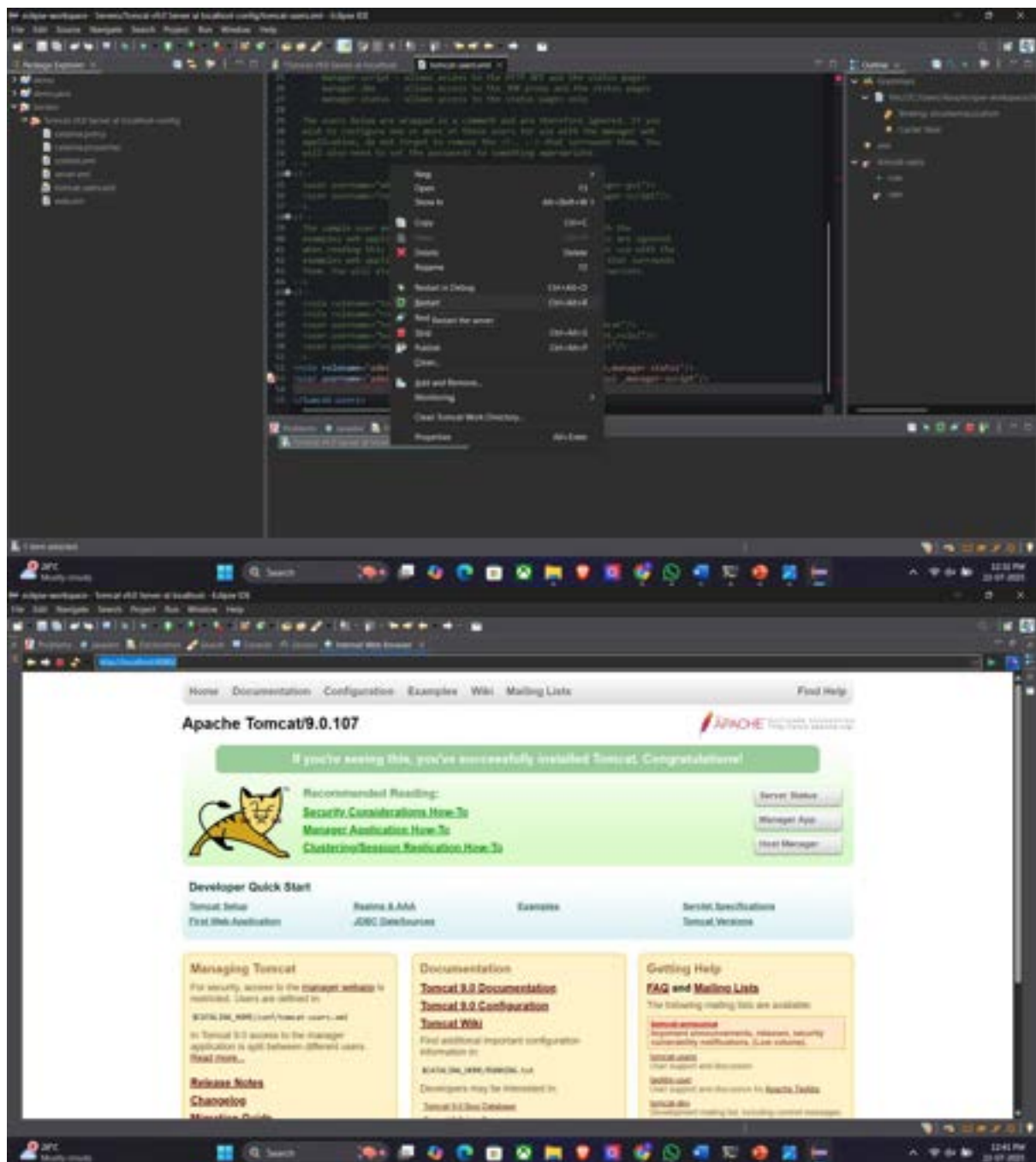
Apache Tomcat:











PROJECT: WHATSAPP

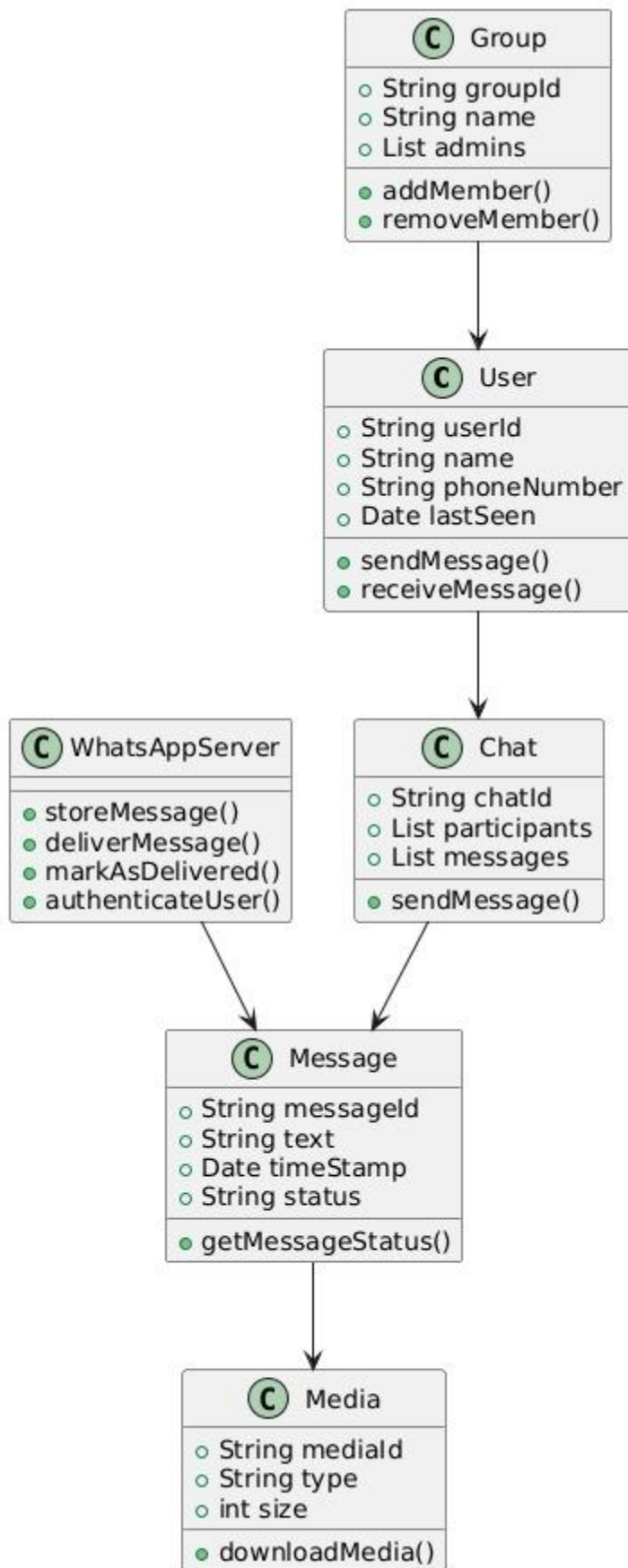
Abstract:

WhatsApp is a widely used instant messaging application that enables users to communicate in real time through text messages, images, videos, voice notes, documents, and voice/video calls. It operates on a client-server architecture where the mobile application interacts with a central WhatsApp server to authenticate users, synchronize contacts, and manage message delivery.

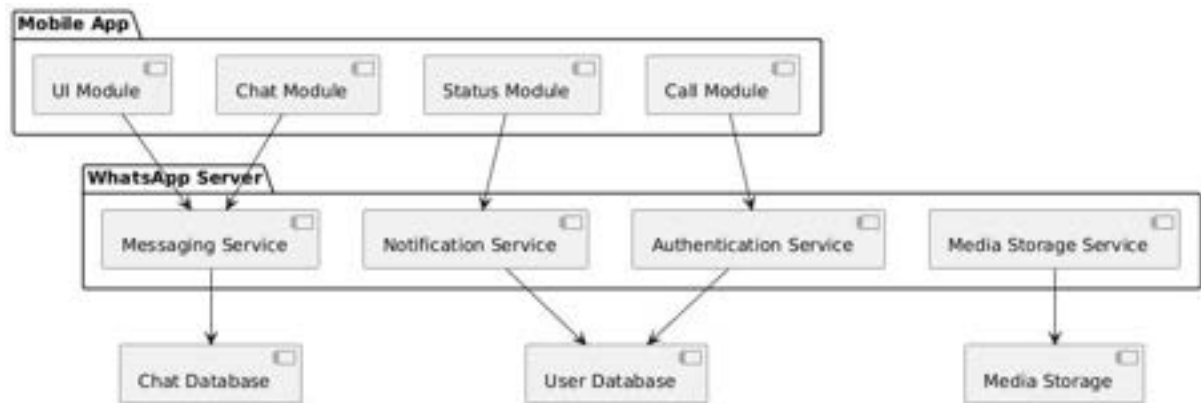
The system allows users to engage in one-to-one chats as well as participate in group conversations. Messages are transmitted using end-to-end encryption, ensuring secure communication between users. WhatsApp also supports features such as media sharing, status updates, group management, message synchronization, and push notifications, enabling seamless communication across devices.

In this project, core functionalities such as user management, chat handling, message flow, media transmission, and server interactions are analyzed. UML diagrams—including use case, class, sequence, and component diagrams—are used to model the internal structure and behavior of WhatsApp. This analysis provides a clear understanding of how a modern real-time messaging system works, focusing on reliability, scalability, and secure communication

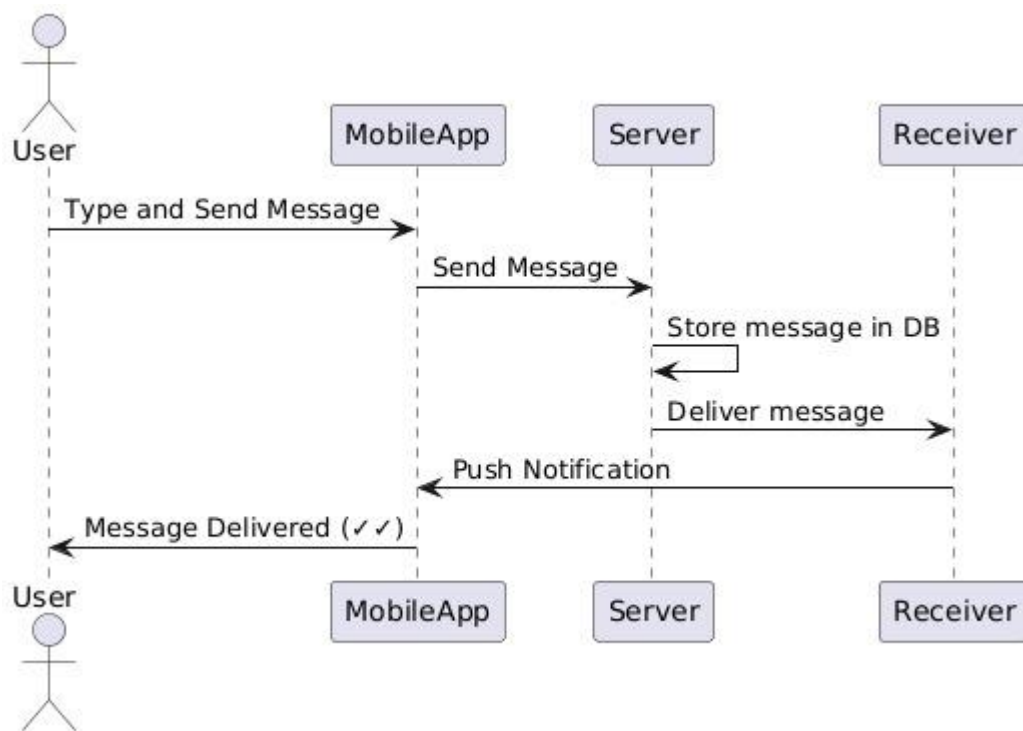
Class Diagram:



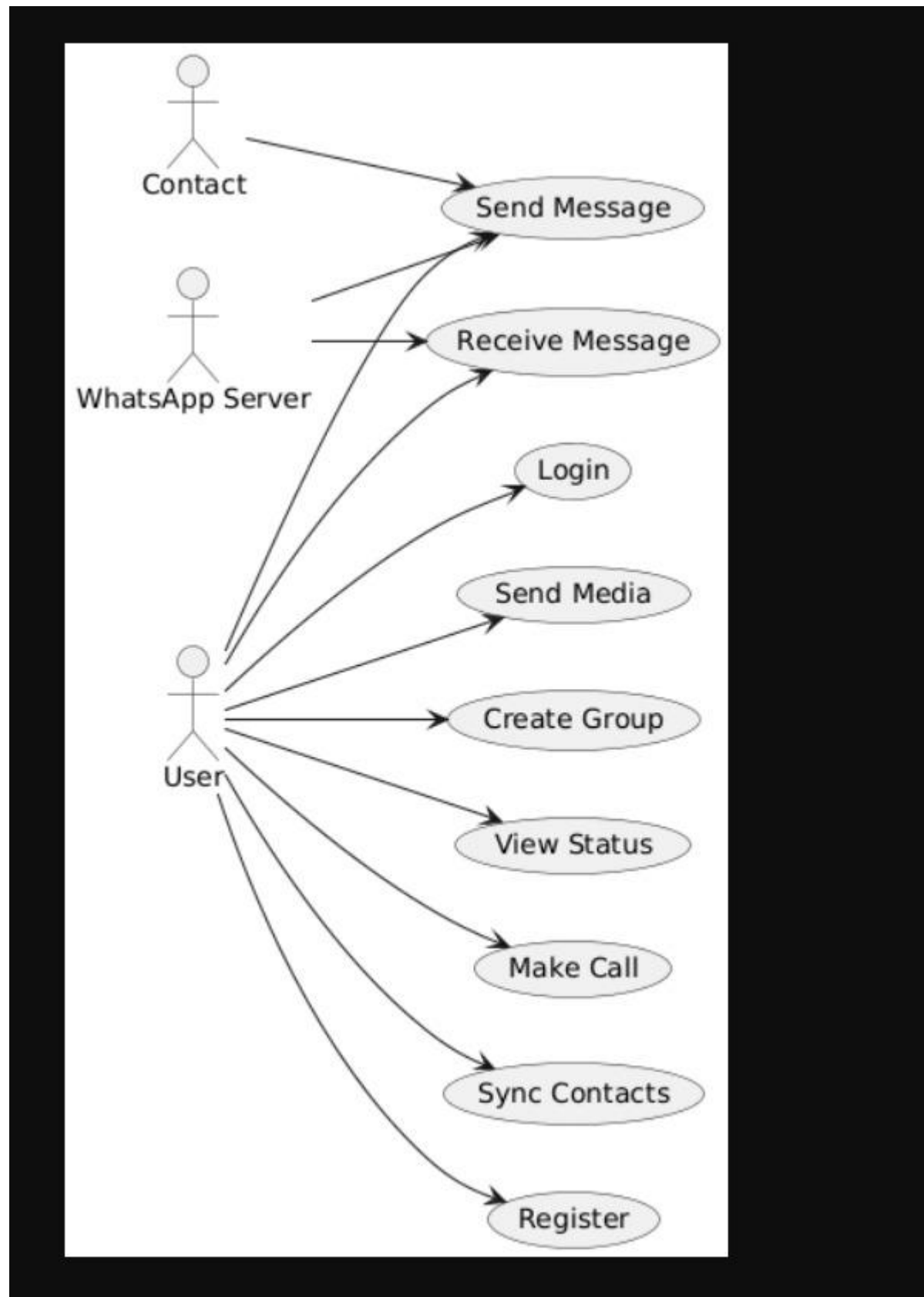
COMPONENT DIAGRAM:



SEQUENCE DIAGRAM:



USE CASE DIAGRAM:



LAB ACTION PLAN FOR WEEK 2

Objective:

1. Pushing multi-folder project into private repository (by student).
2. Students must explore all listed git commands on the multi-folder project in local and remote repository.
3. Students must explore all git commands on given scenario-based question.
4. Students need to make note in observation book and also upload the executed exercise on scenario-based question.

Task 1: Pushing multi-folder project into private repository(Note: student take screenshots for upload).

1. Create a Private Repository on GitHub

- Go to <https://github.com>
- Click **New repository**
- Give it a name
- Check **Private**
- Click **Create repository**

2. Initialize Git in Your Local Project (if not already done)

Open a terminal in the **root** of your multi-folder project:

```
cd /path/to/your/project  
git init
```

3. Add the Remote Repository (in bash)

You'll see a URL on GitHub after creating the repo. It will look as below , run this command in git bash:

```
git remote add origin https://github.com/yourusername/your-private-repo.git
```

4. Add and Commit Your Files(in bash)

```
git add .  
git commit -m "Initial commit"
```

5. Push to the Private Repository

If the repo is new and empty, push like this from git bash:

```
git push -u origin master
```

If you're using the main branch (GitHub default):

```
git push -u origin main
```

You may be prompted to log in (via username/password or token).

Task 2: Students must explore all listed git commands on the multi-folder project in local and remote repository.

1. Git Commands:

- **git version** : The command git version is used to check the version of git.
git --version
- **git config**: Configures Git settings. Commonly used to set up user information
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
- **git config --list**: Displays all the Git configurations for the current user.

2. Repository Management

- **git init**: Initializes a new Git repository in the current directory
git init
- **git clone**: Creates a copy of an existing Git repository from a remote source (e.g., GitHub) to your local machine
git clone <https://github.com/username/repository.git>
- **git remote -v** : To see the remote repository that is connected to your local repository
git remote -v
- **git remote add** : To add a new remote repository to your local repository
git remote add origin <https://github.com/username/repository.git>
- **Pushing Changes to Remote:** To push your local commits to a remote repository:
git push <remote_name> <branch_name>
git push origin main
- **Pulling Updates from Remote:** To pull the latest changes from the remote repository and merge them into your local branch
git pull <remote_name> <branch_name>
git pull origin main
- **git remote remove**: To remove a remote repository from your local configuration:
git remote remove <remote_name>
git remote remove origin
- **Renaming a Remote:** to rename an existing remote:

```
git remote rename <old_name> <new_name>
```

git remote rename origin upstream

- **Fetching Updates from Remote:** To fetch updates from the remote repository but not merge them into your local branch

```
git fetch <remote_name>
```

- **Changing Remote URL :** To change the URL of a remote (e.g., after changing the remote repository address):

```
git remote set-url <remote_name> <new_url>
```

```
git remote set-url origin https://github.com/username/new-repository.git
```

- **Viewing the Remote Repository's Information:** to see detailed information about a remote repository

```
git remote show <remote_name>
```

```
git remote show origin
```

3. Staging and Committing

- **git status:** Shows the status of changes in your working directory and staging area. It tells you which files are untracked, modified, or ready to be committed.

```
git status
```

- **git add:** Adds changes in the working directory to the staging area.

```
git add filename.txt # Adds a specific file
```

```
git add . # Adds all changes in the directory
```

- **git commit:** Commits the staged changes to the repository with a descriptive message. The -m option allows you to include a commit message.

```
git commit -m "Commit message describing changes"
```

4. Branching and Merging

- **git branch:** Lists all branches or creates a new branch

```
git branch # Lists all branches or
```

```
git branch -a # Lists all branches
```

```
git branch branch-name # Creates a new branch
```

```
git branch -d <branch_name> # Deletes a branch
```

Use -D to force-delete if it hasn't been merged.

- **Listing All Remote Branches** : To you want to list all branches on the remote repository

git branch -r

- **git checkout**: Switches to a different branch

git checkout branch-name # Switches to an existing branch

git checkout -b new-branch # Creates and switches to a new branch

- **Pruning Deleted Remotes** : If a branch was deleted on the remote but still shows up locally.

git remote prune origin

When someone **deletes a branch on the remote**, your local Git doesn't automatically remove the corresponding remote-tracking branch (like origin/old-feature).
git remote prune origin removes these outdated references.

- **Fetching a Specific Remote Branch**: To fetch a specific branch from a remote.
git fetch <remote_name> <branch_name>

git fetch origin feature-branch

- **Setting the Upstream Branch for Pushing** : When pushing for the first time and want to set the remote branch you are pushing to:

git push --set-upstream <remote_name> <branch_name>

git push --set-upstream origin feature-branch

- **git merge**: Merges the specified branch into the current branch. This command integrates the changes from the feature branch into the main branch.

git checkout main # Switch to the main branch

git merge branch-name # Merge branch-name into main

- **Rebasing a Local Branch onto a Remote Branch**: If you want to rebase your local branch onto a remote branch (this can be useful to keep your history linear)

git fetch <remote_name>

git rebase <remote_name>/<branch_name>

Example:

git fetch origin

git rebase origin/main

5. Undoing Changes

- **git reset** : Removes the specified file from the staging area but leaves the working directory unchanged. **git reset --hard** can also reset the working directory and staging area to the last commit.

git reset <file>:

- **git revert**: Creates a new commit that undoes the changes from a specified commit, leaving the history intact.

git revert <commit>:

6. Viewing History

- **git log**: Shows a history of commits in the repository, including commit hashes, messages, and timestamps. Use **git log --oneline** for a more concise view.

git log:

- **git diff**: Displays differences between various commits, the working directory, and the staging area. **git diff** without arguments shows changes not yet staged.

git diff:

- **git show**: Shows the details of a specific commit, including the changes made and the commit message.

git show <commit>

7. To undo the changes made to file before staging it.

- **git restore filename**

8. To correct committed with the wrong message

- **git commit --amend -m "Corrected commit message"**

9. Recovering deleted branches

- **git reflog**
- **git checkout -b feature-ui <commit_hash>**

10. To download the latest changes from the remote without merging

- **git fetch origin**

11. To remove accidentally committed sensitive file from Git history.

- **git filter-branch --force --index-filter **
- **"git rm --cached --ignore-unmatch secrets.txt" **
- **--prune-empty --tag-name-filter cat -- --all**

12. To merge changes from another branch

First switch to branch where changes are to applied and then merge another-branch

- **git checkout previous-branch**
- **git merge another-branch**

13. To resolve a merge conflict manually

You tried to merge two branches and Git reported a conflict in x.js. Then for conflict resolution do:

1. **Open x.js and resolve/remove the conflict markers** (<<<<<<<, =====, >>>>>>>)
2. After resolving:
 - ✓ **git add app.js**
 - ✓ **git commit # If Git didn't auto-create a merge commit**

14. The .gitignore

The .gitignore file is used to tell Git which files or directories to ignore in your project. It's a useful way to avoid committing unnecessary files like log files, build outputs, and IDE configurations.

a. Create a .gitignore File

In the root directory of your Git project, create a file called .gitignore

```
touch .gitignore
```

b. Add Rules to .gitignore

Inside the .gitignore file, you add patterns for the files and directories you want Git to ignore. Each pattern should be written on a new line.

Here are some common examples:

- Ignore all .log files: ***.log**
- Ignore a specific file: **secret_file.txt**
- Ignore all files in a temp/ directory: **temp/**
- Ignore all files except important_file.txt inside a folder:

```
folder/*
!folder/important_file.txt
```

- Ignore files with a specific extension: ***.bak**

15. To see who changed a particular line in a file

- **git blame script.py**

16. Git stash

You modified files A and B, but not committed them.

git stash # saves changes and reverts to clean state

git switch another-branch

do something else...

git switch main

git stash apply # brings back your changes

17. To check branch is merge

- **git branch --merged**

If branch not merged then displays its name

18. To Delete multiple local branches at once

- **git branch -d branch1 branch2 branch3**

LAB ACTION PLAN FOR WEEK 3

Objective:

1. To work on collaborative coding by:
 - a. Creating Organization.
 - b. Coordinating with others through a shared repository
2. To resolve conflicts when collaborating on same part of code.
3. To create and apply patch
4. Students need to make note in observation book and also upload the screenshots of above executed exercise and scenario-based question.

1. Collaborative coding.

Code collaboration refers to the process of multiple people working together on the same codebase.

There are **two ways** of collaborative coding:

- a. Creating Organization
- b. Coordinating with others through a shared repository

a. Facilitating Collaborative Work by creating Organization

Organization - Organizations are shared accounts where businesses and open-source projects can collaborate across many projects at once. Owners and administrators can manage member access to the organization's data and projects with sophisticated security and administrative features. Steps to facilitate collaborative work by creating organization are:

Step 1: Click on New organization in Git hub

Click on Create a free organization.

Step 2: Set up your organization by entering the details like name, associated email, account verification, etc.

Click on Next

Step 3: Complete the setup by

- ✓ adding members to the group.
- ✓ Now, the invitation goes to other collaborator and they must accept.
- ✓ Next click on complete setup
- ✓ Goto Github and click on

Settings → Member privileges → Base permissions → Select write → change base permission to “Write”

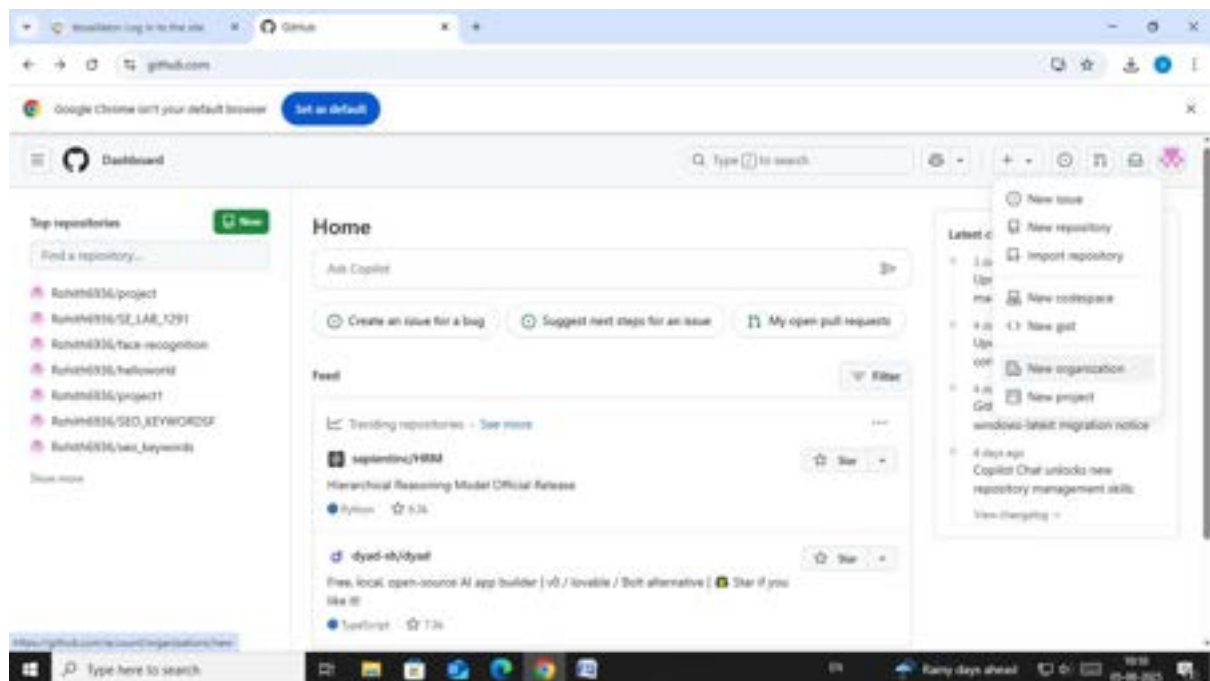
Next under,

Projects base permissions → Select Write → change base permission to “Write”

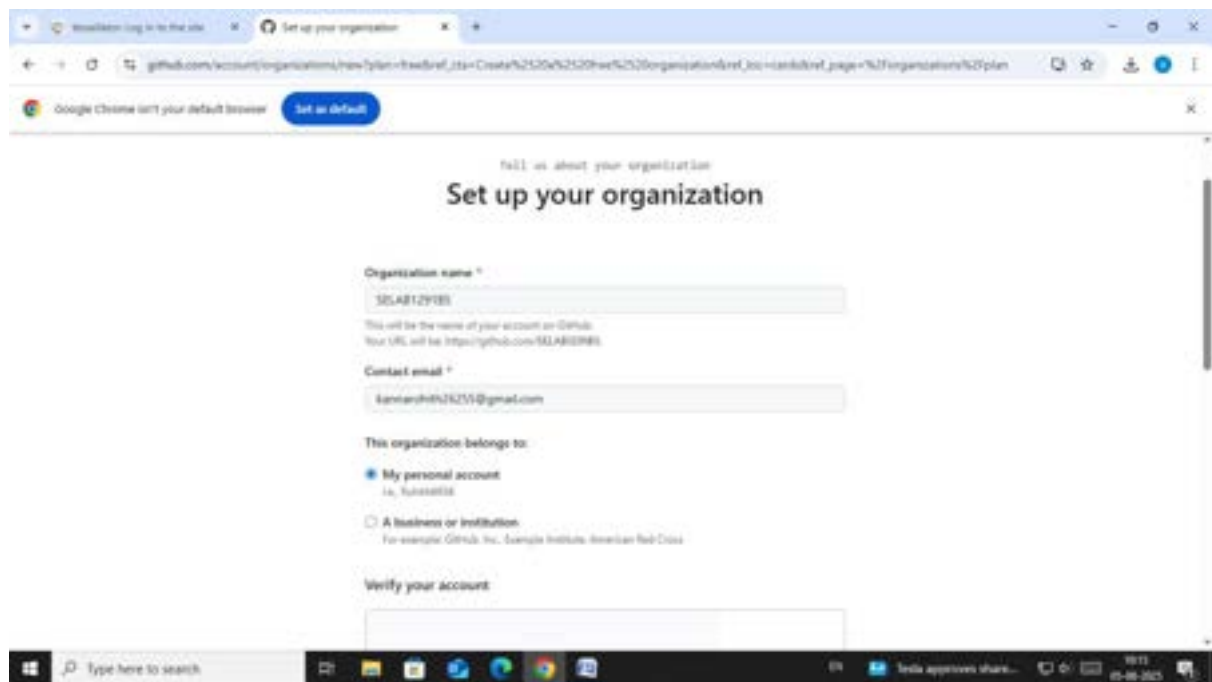
Step 4: Create the remote repository for storing the project related files in Organization. This repository is accessible to every member of the team as per the permissions given.

Note: The repository can be made private so that it is accessible only to the group members rather than being in a public domain.

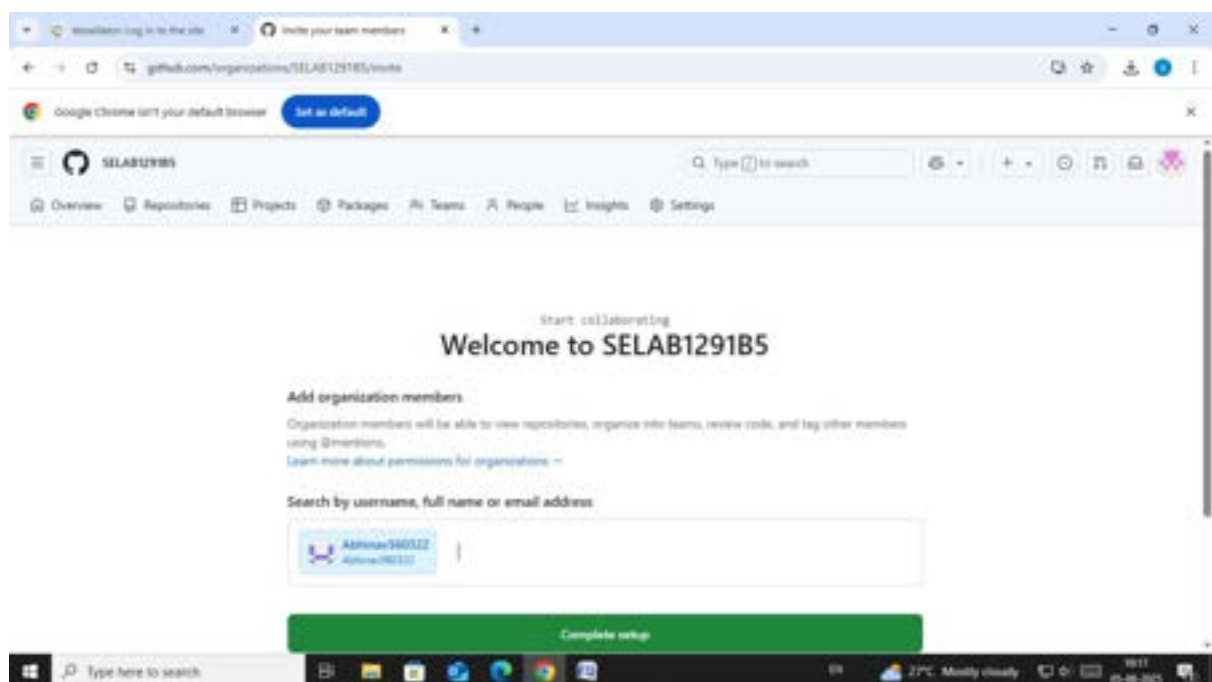
Now all the members of the team can contribute to the development of the project and the different files with all the versions and modification notices will be available in the respective repositories and is accessible to all the members



Organization details:



Collaborator adding:



Settings:

Browser tabs: [Workflow log in to the site](#) | [Member privileges](#)

Address bar: github.com/organizations/SELAB129185/settings/member_privileges?enable_for=Basic-permissions

Google Chrome isn't your default browser [Set as default](#)

Organization: SELAB129185 [Switch settings context](#) [Go to your organization profile](#)

Navigation: Overview | Repositories | Projects | Packages | Teams | People | Insights | **Settings**

Member privileges

Base permissions

Base permissions to the organization's repositories apply to all members and excludes outside collaborators. Since organization members can have permissions from multiple sources, members and collaborators who have been granted a higher level of access than the base permissions will retain that higher permission privileges.

Write

Repository creation

Members will be able to create only selected repository types. Outside collaborators can never create repositories.

☐ Public
Members will be able to create public repositories, visible to anyone. [Why is this option disabled?](#)

☒ Private

Browser tabs: [Workflow log in to the site](#) | [Member privileges](#)

Address bar: github.com/organizations/SELAB129185/settings/member_privileges?enable_for=Basic-permissions

Google Chrome isn't your default browser [Set as default](#)

Navigation: Models | **Webhooks** | Discussions | Packages | Pages

Security

☒ Authentication security

☒ Advanced Security

☐ Deploy keys

☐ Compliance

☐ Verified and approved domains

☐ Secrets and variables

Third-party Access

☐ GitHub Apps

☐ OAuth app policy

☐ Personal access tokens

Allow forking of private repositories

If enabled, forking is allowed on private and public repositories. If disabled, forking is only allowed on public repositories. This setting is also configurable per repository.

Save

Projects base permissions

Projects created by members will default to the selected role below.

Write

Pages creation

Members will be able to publish sites with only the selected access controls.

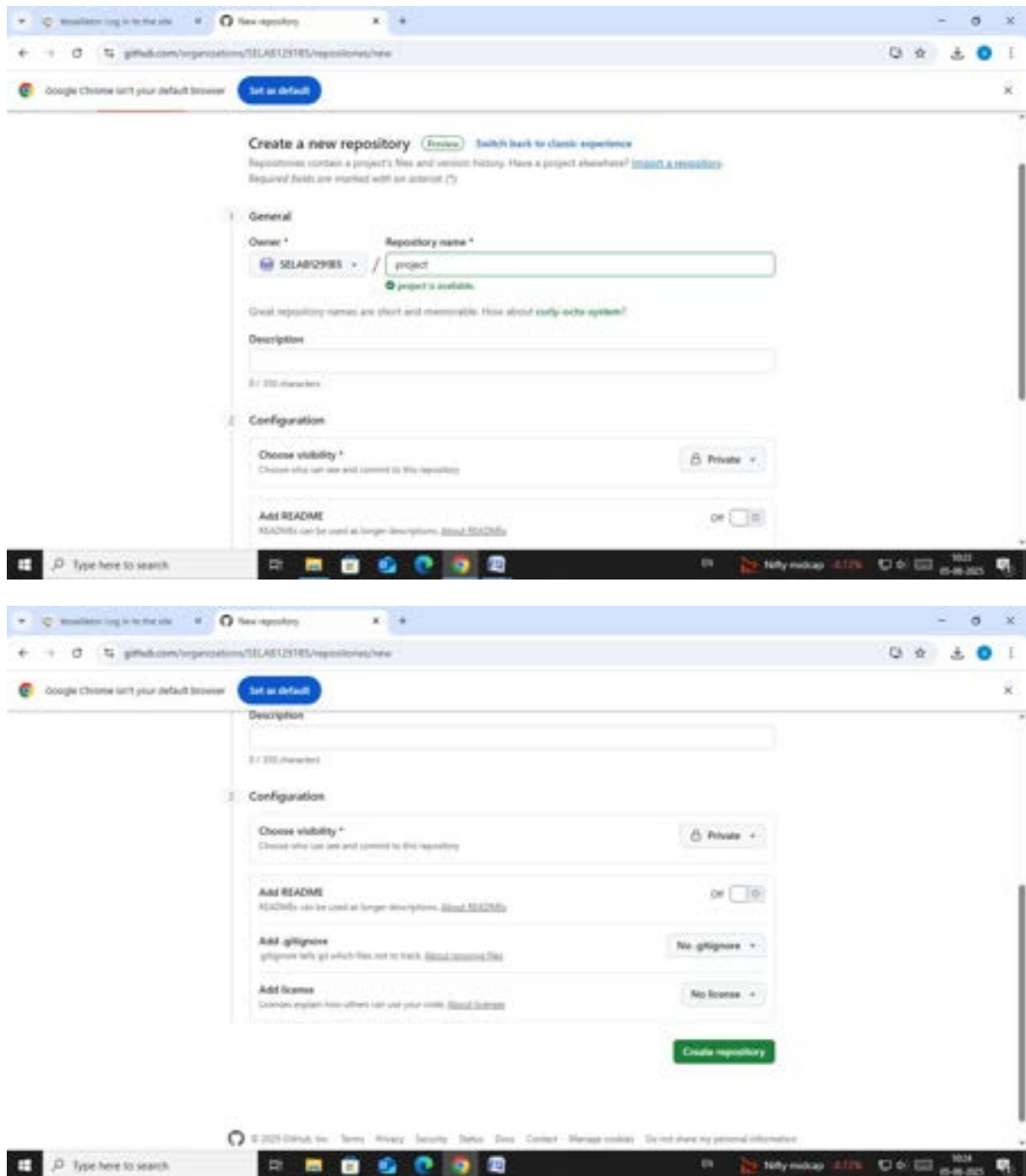
☒ Public
Members will be able to create public sites, visible to anyone.

☐ Private
Members will be able to create private sites, visible to anyone with permission. [Why is this option disabled?](#)

Save

Integration access requests

☒ Allow integration requests from outside collaborators
Outside collaborators will be able to request access for GitHub or OAuth apps to access this organization and its resources.



b. To collaborate effectively on GitHub shared repository by sharing code and coordinating with others through a shared repository.

→ **Steps for Collaborator 1 are:** (collaborator-1 has repository that he wants to share with collaborator-2)

- ✓ Go to Settings > collaborators
- ✓ Now in Manage Access click on Add people
- ✓ Now you will be able to see ONE collaborator added
- ✓ After this invitation goes to collaborator 2 and they need to accept it.

→ **Steps for Collaborator 2 are:**

✓ 1. Set Up Git and GitHub

In git bash

```
git config --global user.name "Your Name"
```

```
git config --global user.email you@example.com
```

Note : run `$ git remote -v`

```
If origin  git@github.com:OrgYOG/commonREPO.git (fetch)
   origin  git@github.com:OrgYOG/commonREPO.git (push)
then
```

```
$ git remote -v // Note now, not connected to any remote
```

✓ 2. Goto Git hub and copy url of shared repository by collaborator 1

- Copy url (option 1)

✓ 3. Clone the Repository

Get a local copy of the repo:

In git bash

```
git clone https://github.com/username/repository-name.git
```

```
cd repository-name
```

✓ 4. Create a Branch for Your Work

Always work on a separate branch:

In git bash

```
git checkout -b feature/your-feature-name
```

✓ 5. Make Changes and Commit

Edit files, then stage and commit your changes:

In git bash to existing file / new file

```
git add .
```

```
git commit -m "Add a clear and descriptive commit message"
```

✓ 6. Push Your Branch to GitHub

➔ In git bash

➔ `git push origin feature/your-feature-name`

✓ 7. Keep Your Branch Updated (has to be done by owner collaborator 1)

Before making new changes:

In git bash

```
git checkout main
```

```
git pull origin main
```

```
git checkout feature/your-new-feature
```

```
git merge main
```

method-1:

```
MINGW64~/c/Users/ranga/OneDrive/Desktop/collaborator

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git config --global user.name "Abhinav560322"

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git config --global user.email "rangaabhinav56@gmail.com"

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git clone https://github.com/SELAB129185/project.git
Cloning into 'project'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ |
```

MINGW64:/c/Users/ranga/OneDrive/Desktop/collaborator/project

GNU nano 8.5

a.txt

```
hi this is kmit  
hi this is collaborator2  
hi from git bash
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

4 AIRTELPP
+1.38%



Search




```

MINGW64~/c/Users/ranga/OneDrive/Desktop/collaborator/project
Cloning into 'project'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ nano a.txt

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ cd project

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ nano a.txt

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git add .

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git branch collaborator2

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git checkout collaborator2
M      a.txt
Switched to branch 'collaborator2'

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ nano b.txt

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git add .
warning: in the working copy of 'b.txt', LF will be replaced by CRLF the next time Git touches it

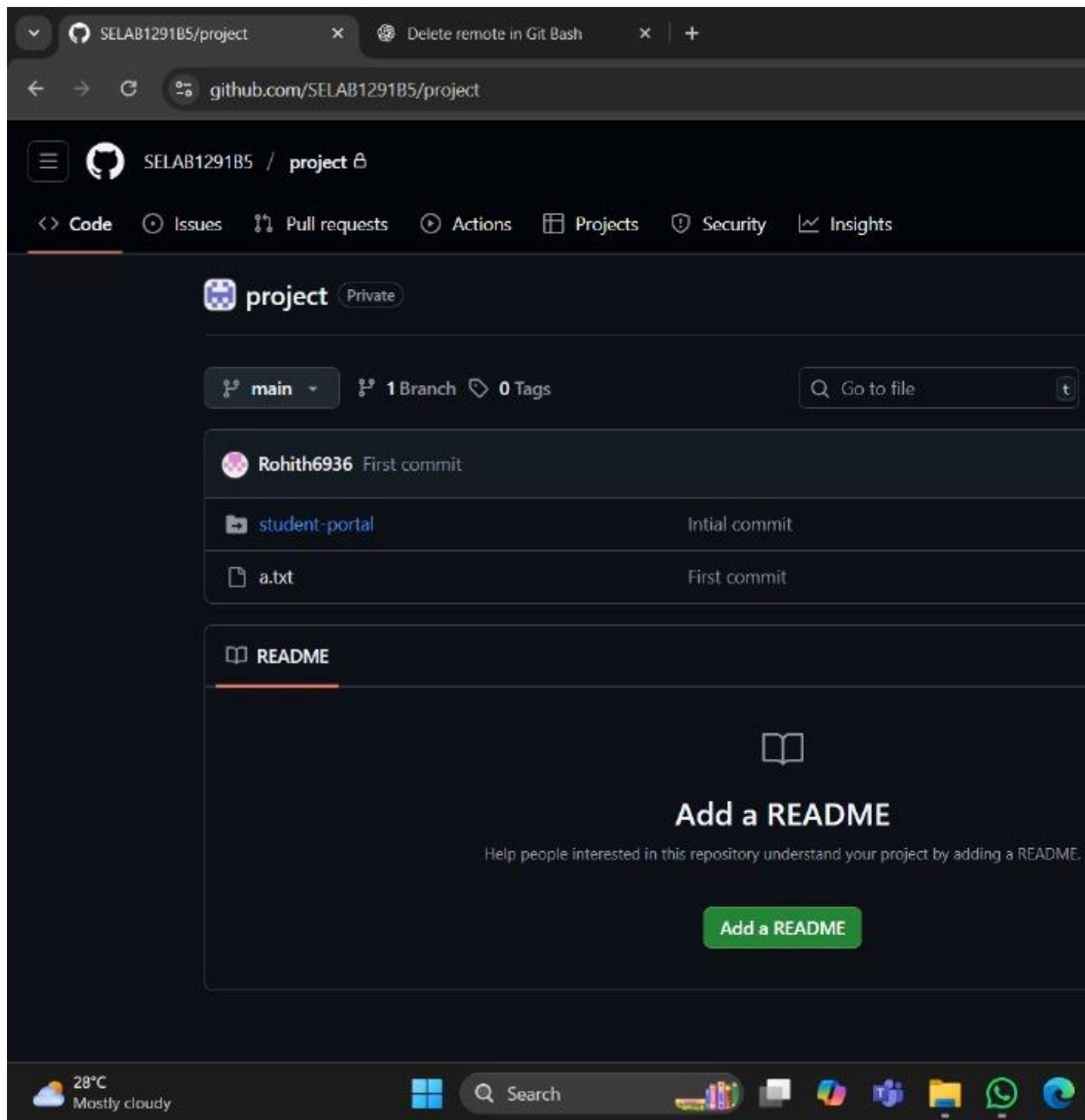
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git commit -m "changes by collaborator2"
[collaborator2 8072299] changes by collaborator2
 2 files changed, 3 insertions(+)
 create mode 100644 b.txt

ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git push -u origin collaborator2
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 397 bytes | 397.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'collaborator2' on GitHub by visiting:
remote:   https://github.com/SELAB129185/project/pull/new/collaborator2
remote:
To https://github.com/SELAB129185/project.git
 * [new branch]      collaborator2 -> collaborator2
branch 'collaborator2' set up to track 'origin/collaborator2'.

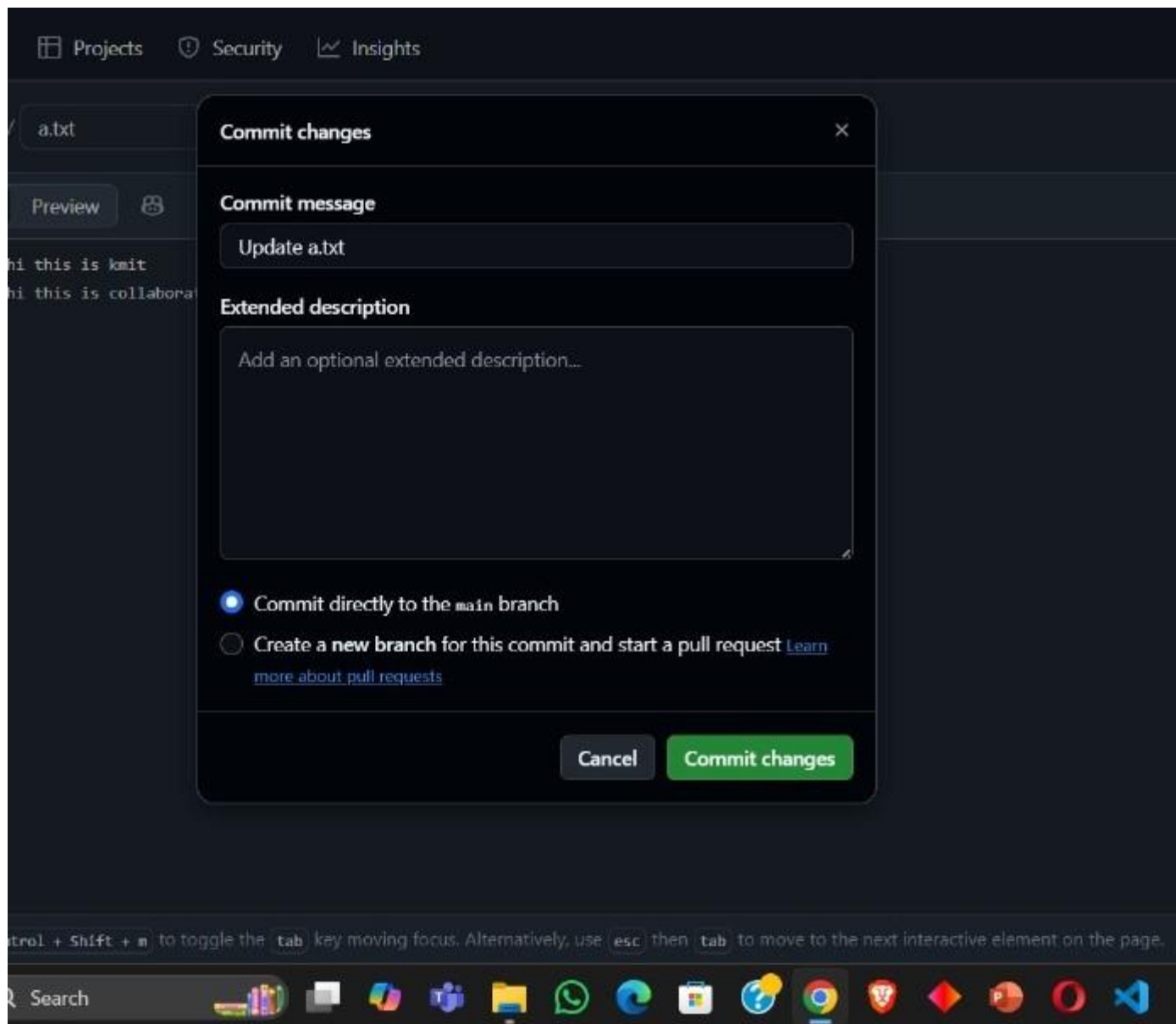
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ |

```

Method -2:



Update file:



main 3 Branches 0 Tags			Go to file	Add file	Code
Rohith5936	Remove sensitive file	fradSeT - 38 minutes ago	6 Commits		
student-portal	Initial commit	1 hour ago			
a.txt	changes by collaborator2	1 hour ago			
b.txt	changes by collaborator2	1 hour ago			

Exercise on Fork

- If you're contributing to an existing project: (in public repository)
Visit the repository page and click Fork

Note: Following steps needed if you forked from any public repository to contribute

1. Go to: `https://github.com/othername/awesome-project`
2. Click the **"Fork"** button at the top-right of the page.
3. GitHub will create a copy of that repository in your account:

`https://github.com/your-username/awesome-project`

4. Clone your forked repo: in bash

- ✓ `git clone https://github.com/your-username/awesome-project`
- ✓ Make changes.
- ✓ Commit

- ✓ `git push origin main`

5. Create a **"pull request" back to the original repo (othername/awesome-project) to suggest your changes.**

- ✓ Click at top Pull request
- ✓ Add title
- ✓ Create pull request

Now you see on page message **"NO conflicts with base branch"**

- ✓ Add comment any thing
- ✓ Click on comment
- ✓ Submit the pull request for review

✓ 6. Review and Discuss Changes

- Collaborators can comment, request changes, or approve the PR(Pull Request)
- Make edits if needed and push again; they will update the PR automatically

✓ 7. Merge the Pull Request (has to be done by both collaborators)

Once approved:

- The repo owner can click Merge pull request
- Or you can squash and merge, depending on the team's workflow

✓ 8. Keep Your Branch Updated (has to be done by owner collaborator 1)

Before making new changes:

```
In git bash
git checkout main
git pull origin main
git checkout feature/your-new-feature
git merge main
cd repository-name
```

gumidellisoujanya / student-portal

Q Type to search

+ -

+

+

+

+

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

student-portalPublic

Watch0

Fork0

Star0

master1 Branch0 Tags

Go to file

Add file

Existing forks

You don't have any forks of this repository.

Create a new fork

gumidellisoujanyainitial commitc6e0e60 · 2 weeks ago1 Commit

backend	initial commit	2 weeks ago
config	initial commit	2 weeks ago
frontend	initial commit	2 weeks ago
.gitignore	initial commit	2 weeks ago
README.md	initial commit	2 weeks ago
pom.xml	initial commit	2 weeks ago

README

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Packages

No packages published

https://github.com/gumidellisoujanya/student-portal/fork

Type here to search

EN

31°C Cloudy

12:45

05-08-2025

affecting the original project.

Required fields are marked with an asterisk (*).

Owner *



Repository name *

student-portal

✔ student-portal is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)

☒ Copy the `master` branch only

Contribute back to gumidellisojanyu/student-portal by adding your own branch. [Learn more.](#)

 You are creating a fork in your personal account.

Create fork

Browser tabs: (2) WhatsApp, Rohith6936/student-portal

Address bar: github.com/Rohith6936/student-portal

Navigation bar: Rohith6936 / student-portal

Search bar: Type to search

Navigation links: Code, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings

Repository: student-portal (Public)

forked from gumidellisoujanya/student-portal

Buttons: Pin, Watch 0, Fork 0

Branches: master (1 Branch), 0 Tags

Buttons: Go to file, Add file, Code

Status: This branch is up to date with gumidellisoujanya/student-portal:master

Buttons: Contribute, Sync fork

Commit history:

Commit	Initial commit	c6e0e60 · 2 weeks ago	1 Commit
backend	initial commit	2 weeks ago	
config	initial commit	2 weeks ago	
frontend	initial commit	2 weeks ago	
.gitignore	initial commit	2 weeks ago	
README.md	initial commit	2 weeks ago	
pom.xml	initial commit	2 weeks ago	

Right sidebar:

About: No description, website, or topics

Readme

Activity

0 stars

0 watching

0 forks

Releases: No releases published, Create a new release

Packages: No packages published

Windows taskbar: Type here to search, Taskbar icons, EN, 31°C Cloudy

2. To resolve conflicts when collaborating on same part of code.

Resolving conflicts when collaborating with GitHub (or Git in general) is a normal part of team development. Conflicts happen when two people make changes to the same part of the code, and Git doesn't know which version to keep.

✔ Steps to Resolve a Conflict

1. See Which Files Are in Conflict (Say fl.txt)

```
In git bash
git status
Conflicted files will be marked like:
In git bash
both modified: (Say fl.txt)
```

3. Open the File and Look for Conflict Markers by clicking on

✔ Click on **Resolve Conflict**

Git will insert conflict markers in the file like this:

```
def greet():
```

```
<<<<<<< HEAD
```

```
    print("Hello from First collaborator ")
```

```
=====
```

```
    print("Hello from second collaborator ")
```

```
>>>>>>> feature/second collaborator branch
```

This shows the two conflicting changes:

- HEAD is your version
- The section below ===== is the other branch's version or second collaborator branch version

3. Edit the File to Fix the Conflict

✔ Choose one version or combine them:

```
def greet():
    print("Hello from First collaborator ")
    print("Hello from second collaborator ")
```

✔ Then **delete** all the **conflict markers** (<<<<<<<, =====, >>>>>>>).

4. Mark the Conflict as Resolved

Once you've edited and saved the file:

✔ git add fl.txt

5. Complete the Merge or Pull

✔ git commit # Only needed if you're merging

Or if you're rebasing:

✔ git rebase --continue

⚠ If You Want to Abort the Merge

If it gets messy and you want to cancel:

```
git merge --abort
```

Here below screen shot shows conflict raised when collaborator 1 wanted to merge changes to same line that already collaborator 2 added the line of code at that line in same file but in their own branch.

Example :

Merge Conflict – Changes made to a file by two different users leads to merge conflict as below.

Say I am collaborator 2, now

- ➔ **Accept invitation from collaborator 1**
- ➔ **Next goes to Shared repository by collaborator 1 (say abcrepo)**
- ➔ **Now open Git bash and clone the shared repo as below steps**

Step 1: first connect to this “abcrepo” to local repository by clone

- ✓ Now change directory to “abcrepo”
- ✓ Switch to branch feature-abc
- ✓ Now add line in README.md file
- ✓ Now git add, commit
- ✓ Now see status and push to remote branch feature-abc
- ✓ Now go to Github and refresh or click on “abcrepo”, you can see README.md

Step 2: Now collaborator 2 adding line in README.md, save, git add, commit, push. No conflicts

Step 3: Also collaborator 1 adding line in README.md at same line collaborator 2 added, then git add, commit, push gives **merge conflict**.

NOTE IMPORTANT :

Now collaborator 2 has to give git pull command then open conflicting file to see conflict markers.

Now Edit the File to Fix the Conflict by keeping either or both collaborator changes and remove conflict markers. Add file, commit, and push.

3. To create and apply patch

A patch in Git is a text file that represents changes between two sets of files, or commits. It's essentially the output of the git diff command, packaged in a format that can be applied to another set of files.

Steps:

1. Goto github and take public project url
2. Clone into git bash
3. Now open file and edit it, add, commit

4. Run git log
5. Create patch with commit hash code as:

```
git format-patch -1 commit-hash-code
```

This creates 0001-commit-from-cloned-person.patch file.

Note: git format-patch -1 <commit-hash>

- -1 means "create a patch from 1 commit"
- This creates a .patch file for a single commit.
- Example:

```
git format-patch -1 abc1234
```

- This creates a file like 0001-Your-commit-message.patch
- To create patches for the **last 3 commits**: git format-patch -3

or

Create patches from multiple commits

```
git format-patch <base_commit>..HEAD
```

6. Get the path of above patch file and email or what's up to the remote owner of file.
7. Once the remote owner gets the patch file 0001-commit-from-cloned-person.patch next the owner need to apply patch file in his git bash using git command below:

Syntax:

```
git apply my-changes.patch
```

Or, if it's a patch made by git format-patch, use:

```
git am 0001-Your-commit-message.patch
```

✓ **git apply 0001-Your-commit-message.patch**

Conclusion:

The effective use of Git in student collaborative projects promotes **teamwork, organization, and clear communication**, allowing students to manage code efficiently and track contributions accurately. By following structured Git workflows and best practices—such as regular commits, feature branching, and resolving conflicts early—students gain valuable experience in real-world development processes. This not only enhances their technical skills but also prepares them for **professional teamwork in software engineering**, where version control and collaboration are essential. Ultimately, mastering Git empowers students to deliver cleaner, more reliable code and succeed in collaborative environments.

LAB ACTION PLAN FOR WEEK 4

Objective:

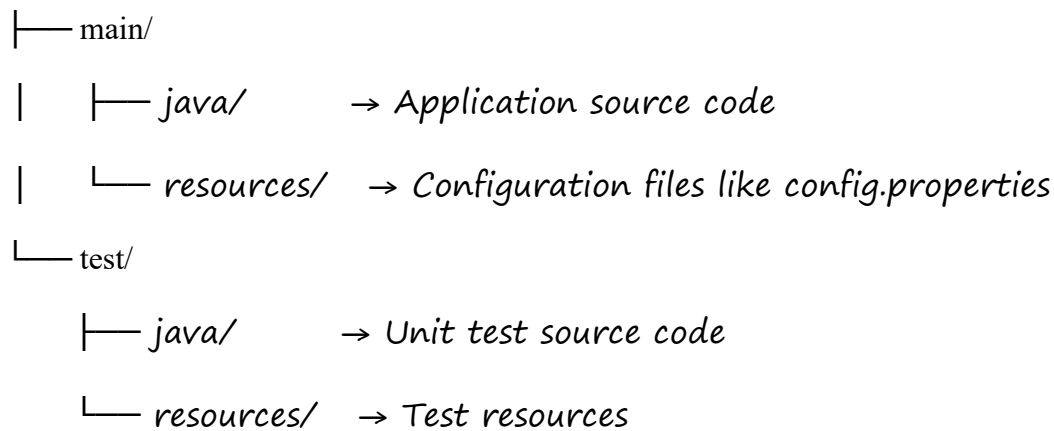
To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications. `src/`



The compiled files and reports are generated in the `target/` directory after a successful build.

2. Steps to Perform Maven Build and Testing

---mvn clean install

- **clean:** Deletes the previous build (`target/` folder)
- **install:** Builds, tests, and installs the package to local `.m2` repository

After execution:

- `target/` folder contains compiled `.class` files and the final `.jar` or `.war`

- Artifact is stored in:
`~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In `pom.xml`:

```
<dependency>  
  <groupId>com.google.code.gson</groupId>  
  <artifactId>gson</artifactId>  
  <version>2.10</version>  
</dependency>
```

After running:

```
mvn clean install
```

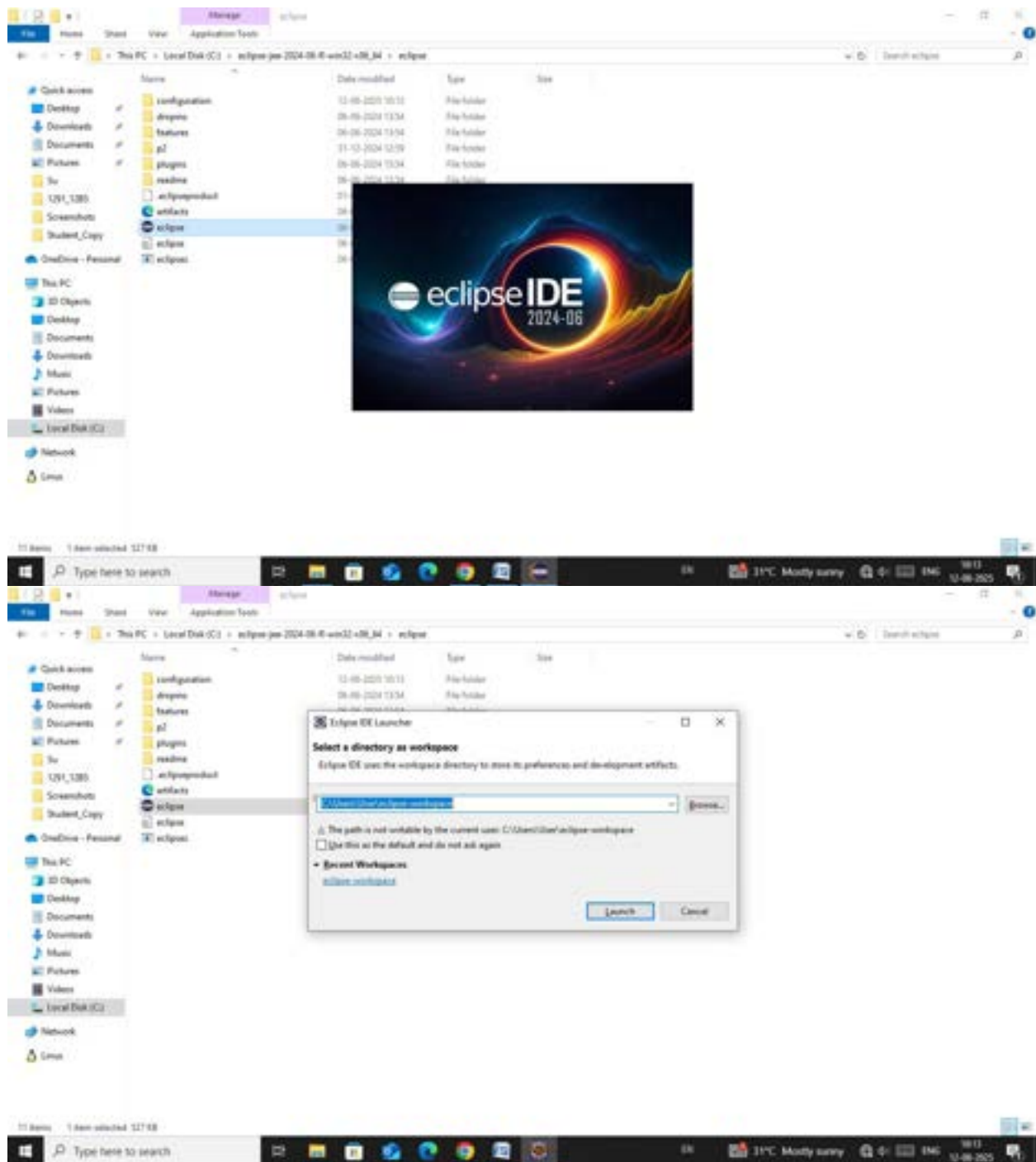
Check:

- Gson JAR is downloaded to `.m2/repository/com/google/code/gson/gson/`
- If version is wrong or not found → **BUILD FAILURE** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└─ 1.1. Launch Eclipse workspace



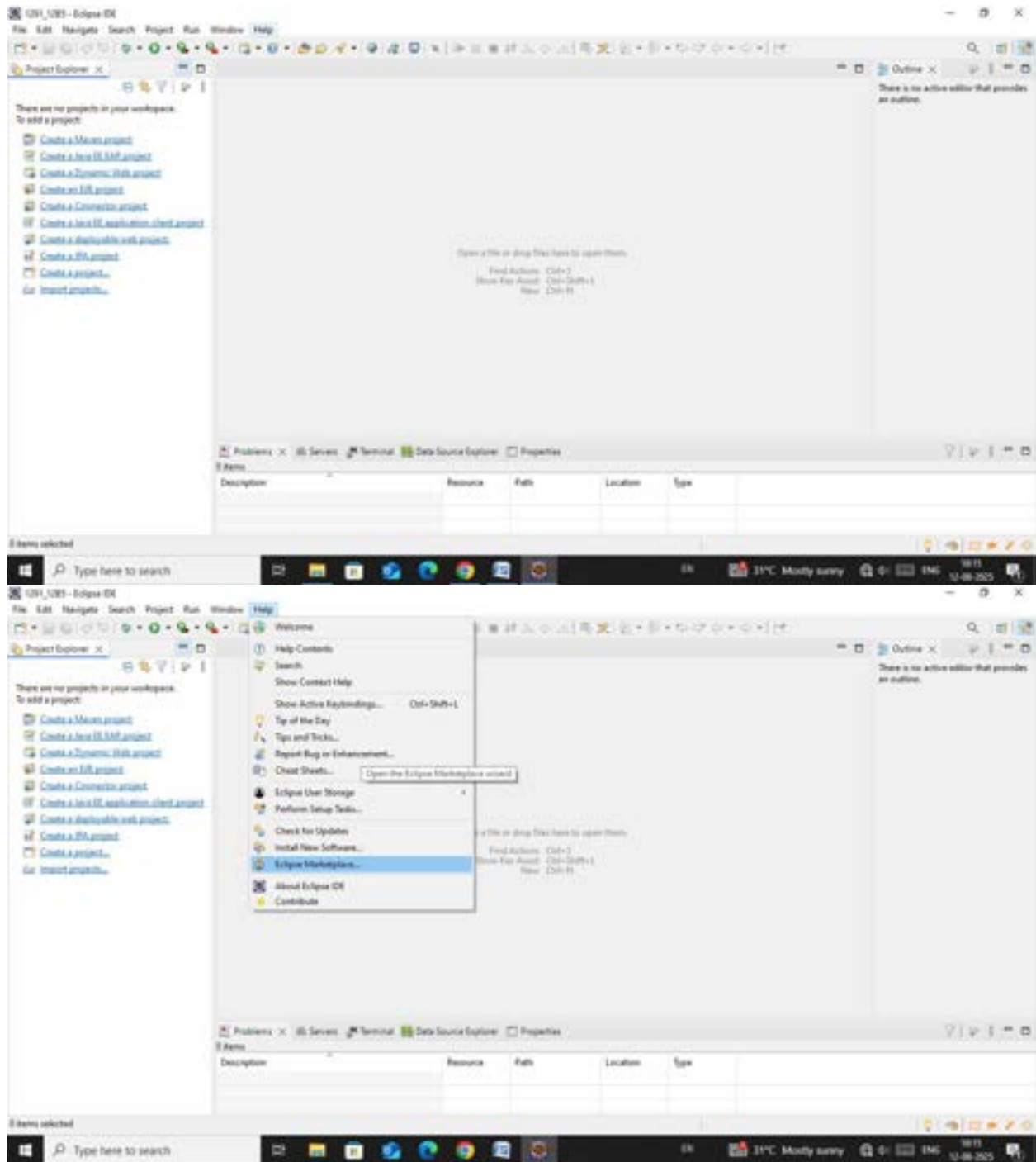
Step 2. Install Maven Plugin (if not installed)

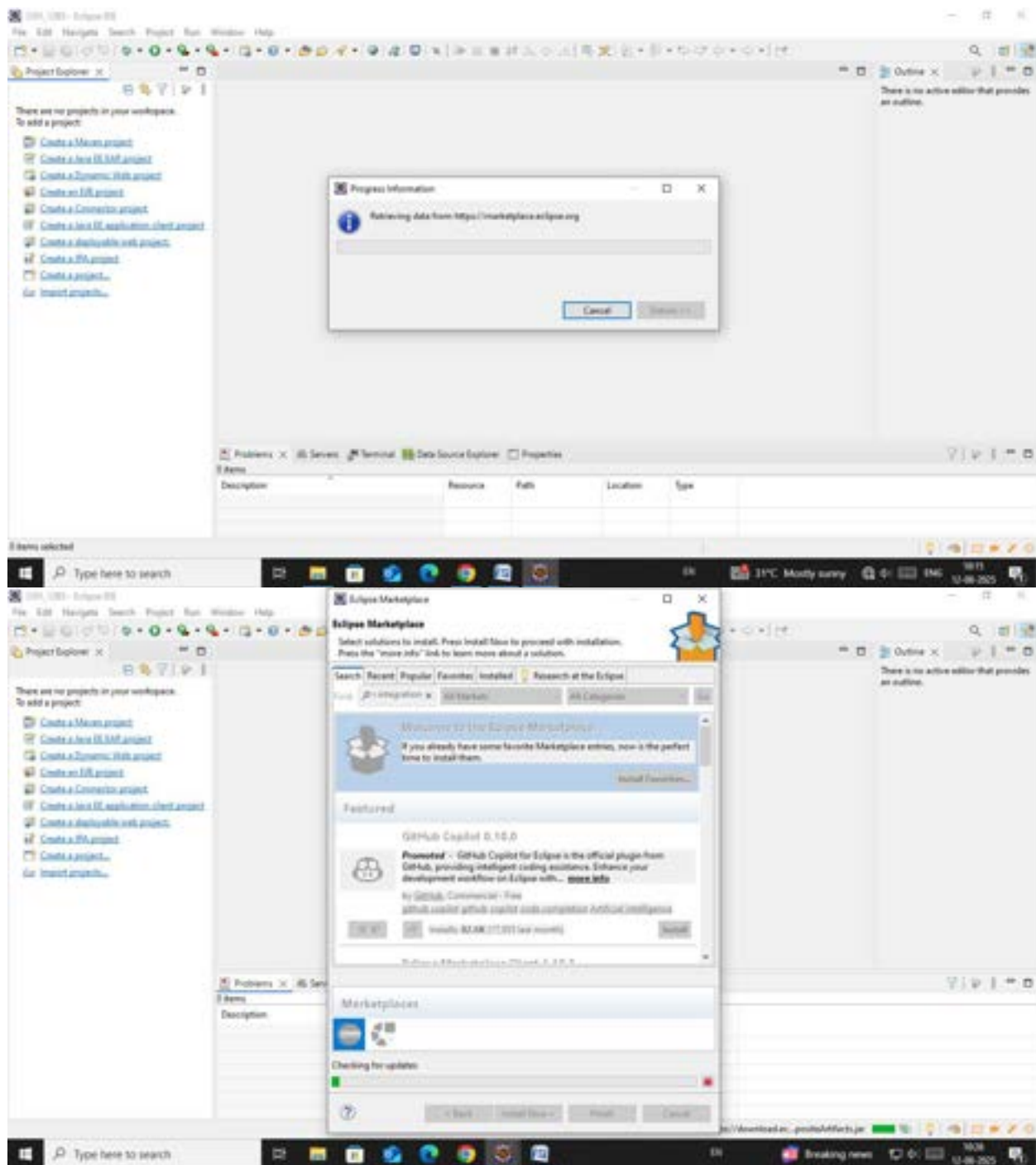
— 2.1. Go to "Help" in the top menu

— 2.1.1. Click "Eclipse Marketplace"

— 2.1.2. Search for "Maven Integration for Eclipse"

2.1.3. Install the plugin if not already installed





Step 3. Create a New Maven Project

└─ 3.1. File -> New -> Project...

└─ 3.1.1. Expand "Maven"

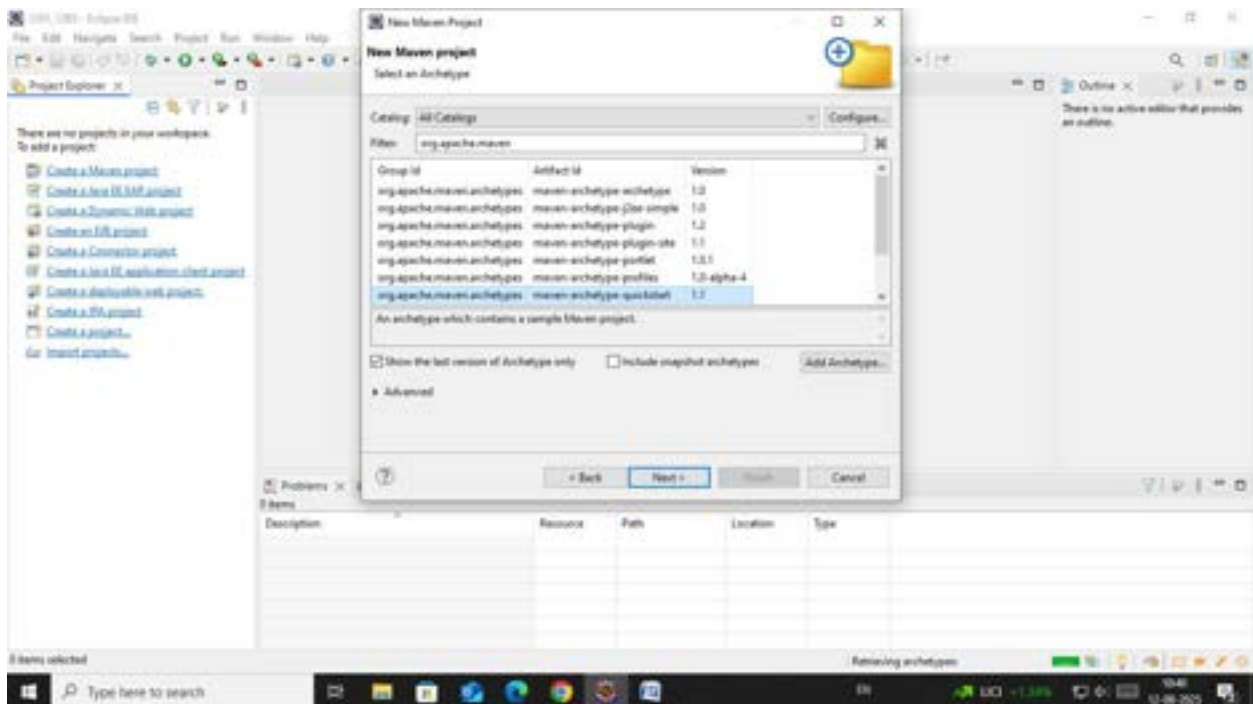
└─ 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

- └─ 4.1. Select workspace location (default or custom)
- └─ 4.2. Click "Next"

Step 5. Choose Maven Archetype

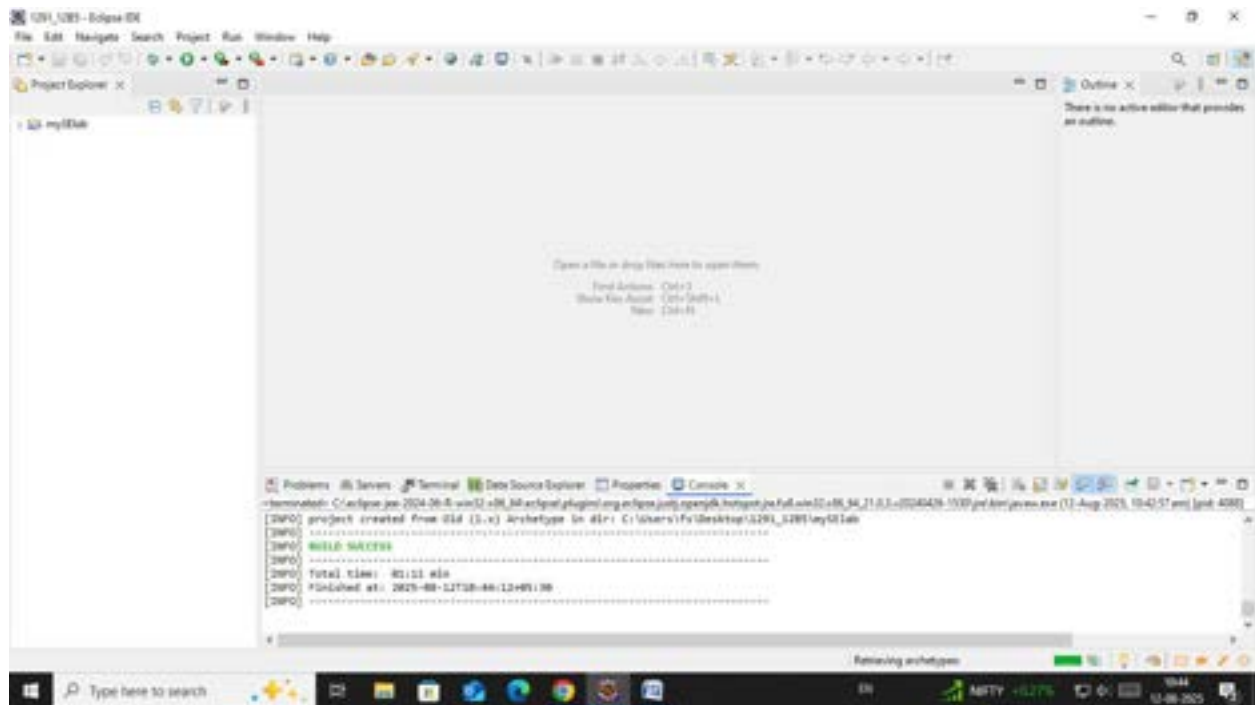
- └─ 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")
- └─ 5.2. Click "Next"



Step 6. Define Project Metadata

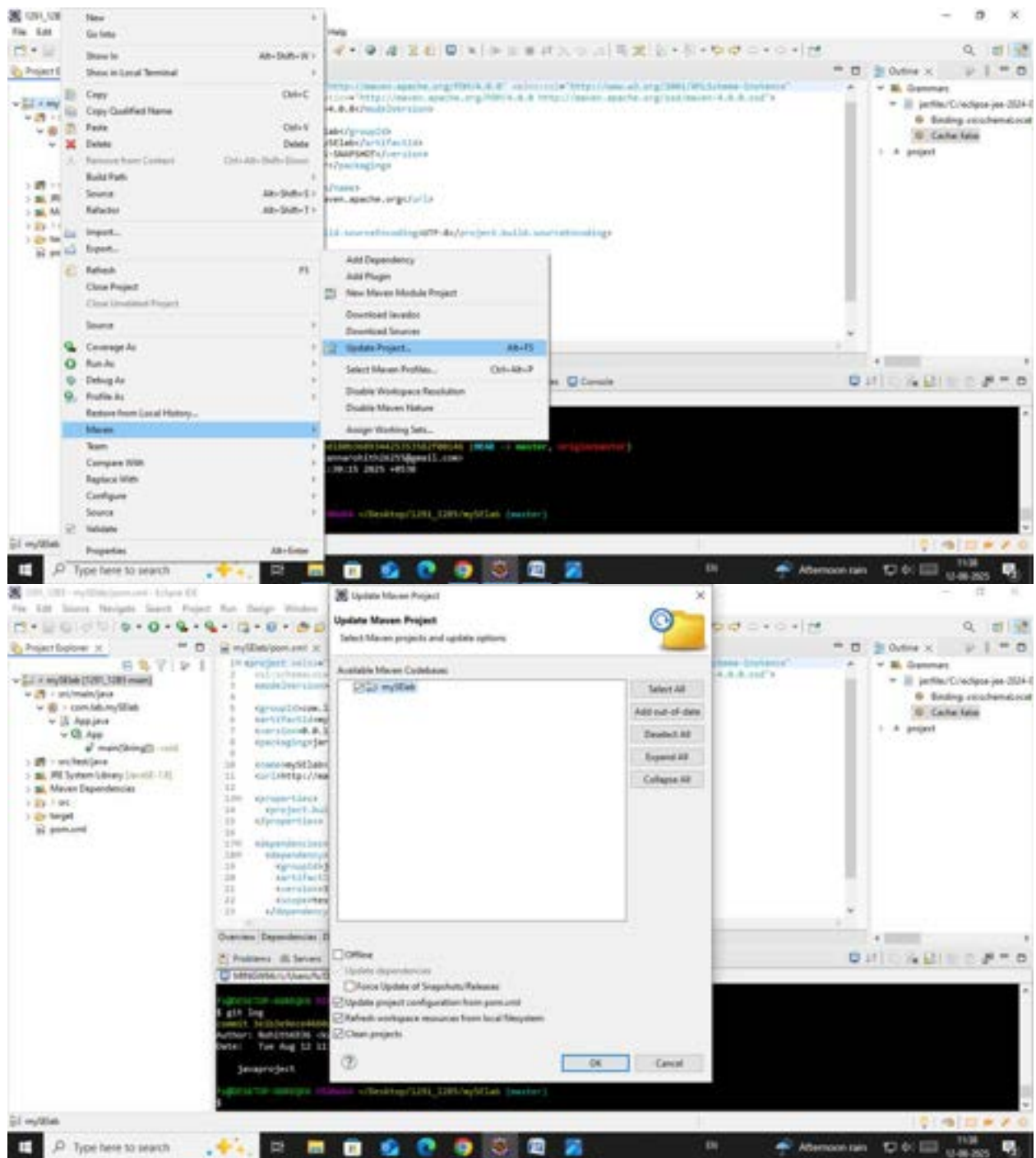
- └─ 6.1. Group ID: (e.g., com.example)
- └─ 6.2. Artifact ID: (e.g., my-maven-project)
- └─ 6.3. Version: (default is usually fine)
- └─ 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.



Step 7. Maven Project Created

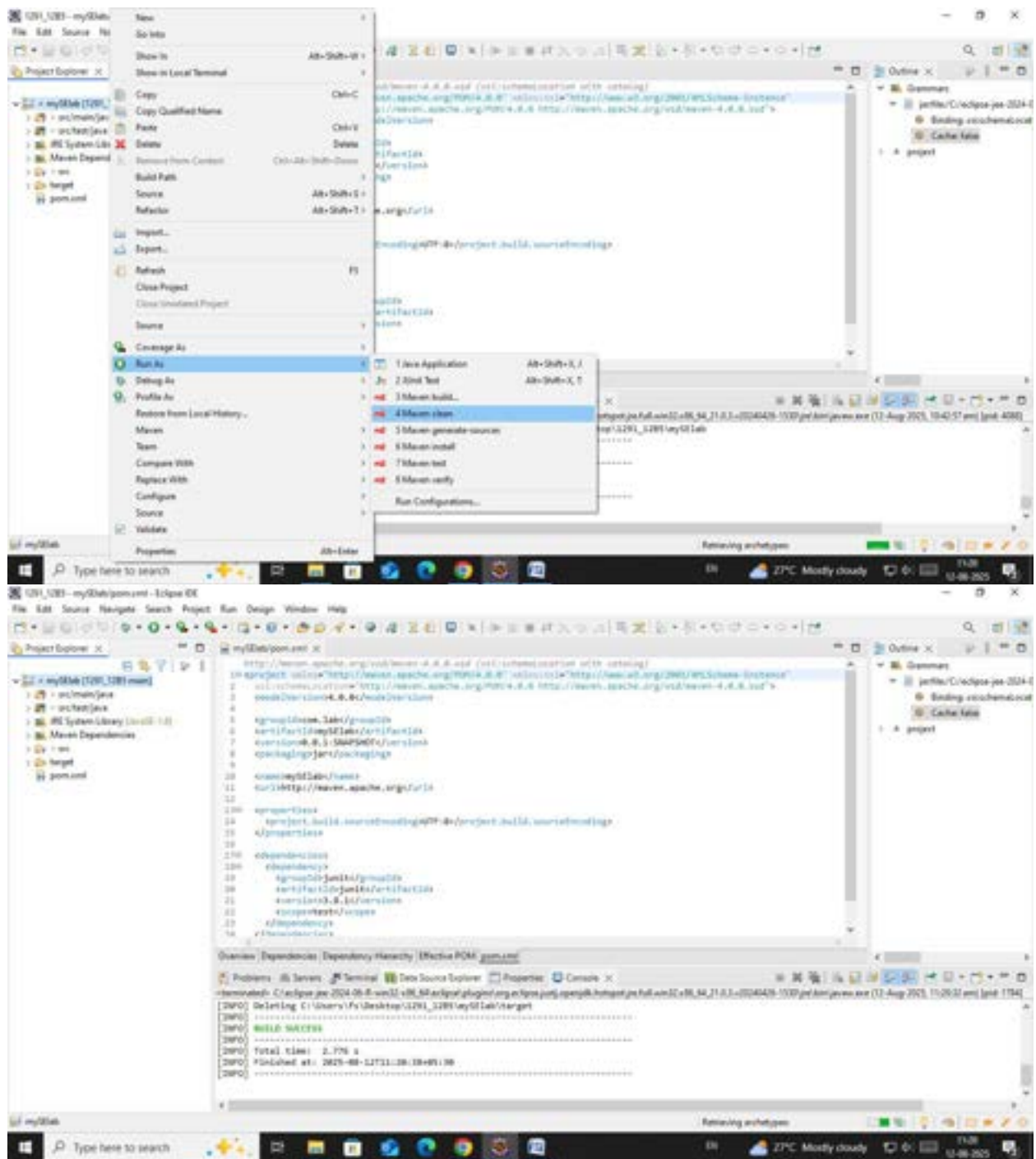
- └─ 7.1. Project structure is generated with a standard Maven layout
- └─ 7.2. Includes:
 - └─ src/main/java (for Java source code)
 - └─ src/test/java (for test code)
 - └─ pom.xml (Maven configuration file)

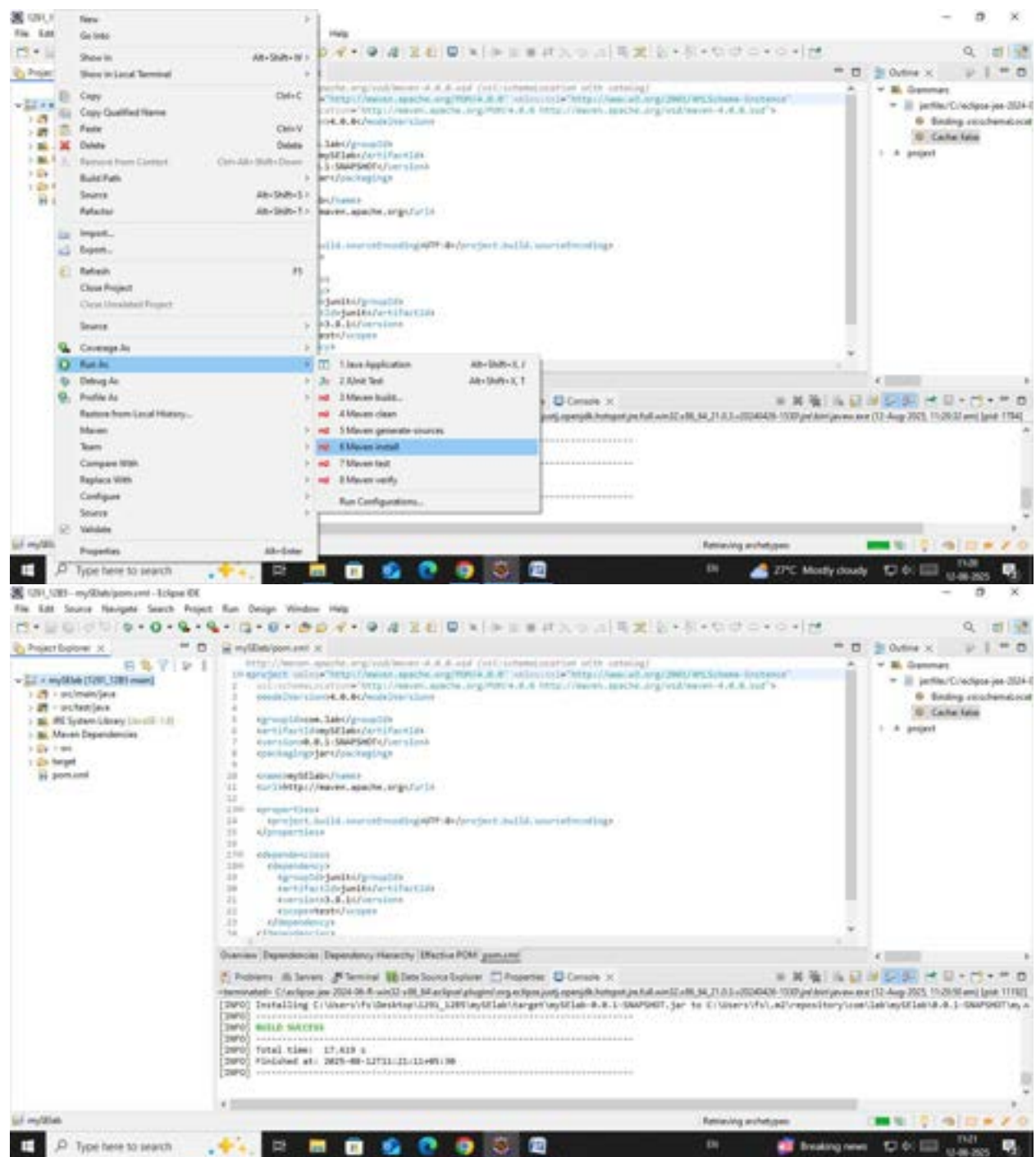


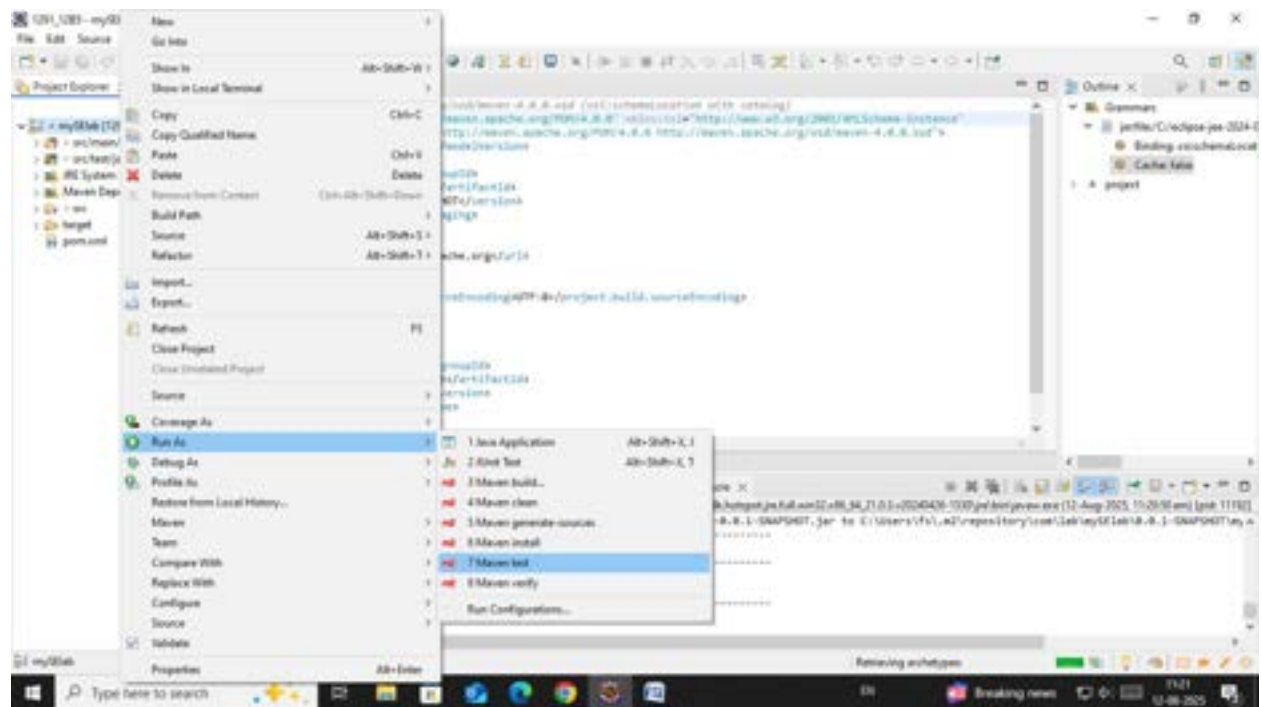
Step 9. Build and Run Maven Project

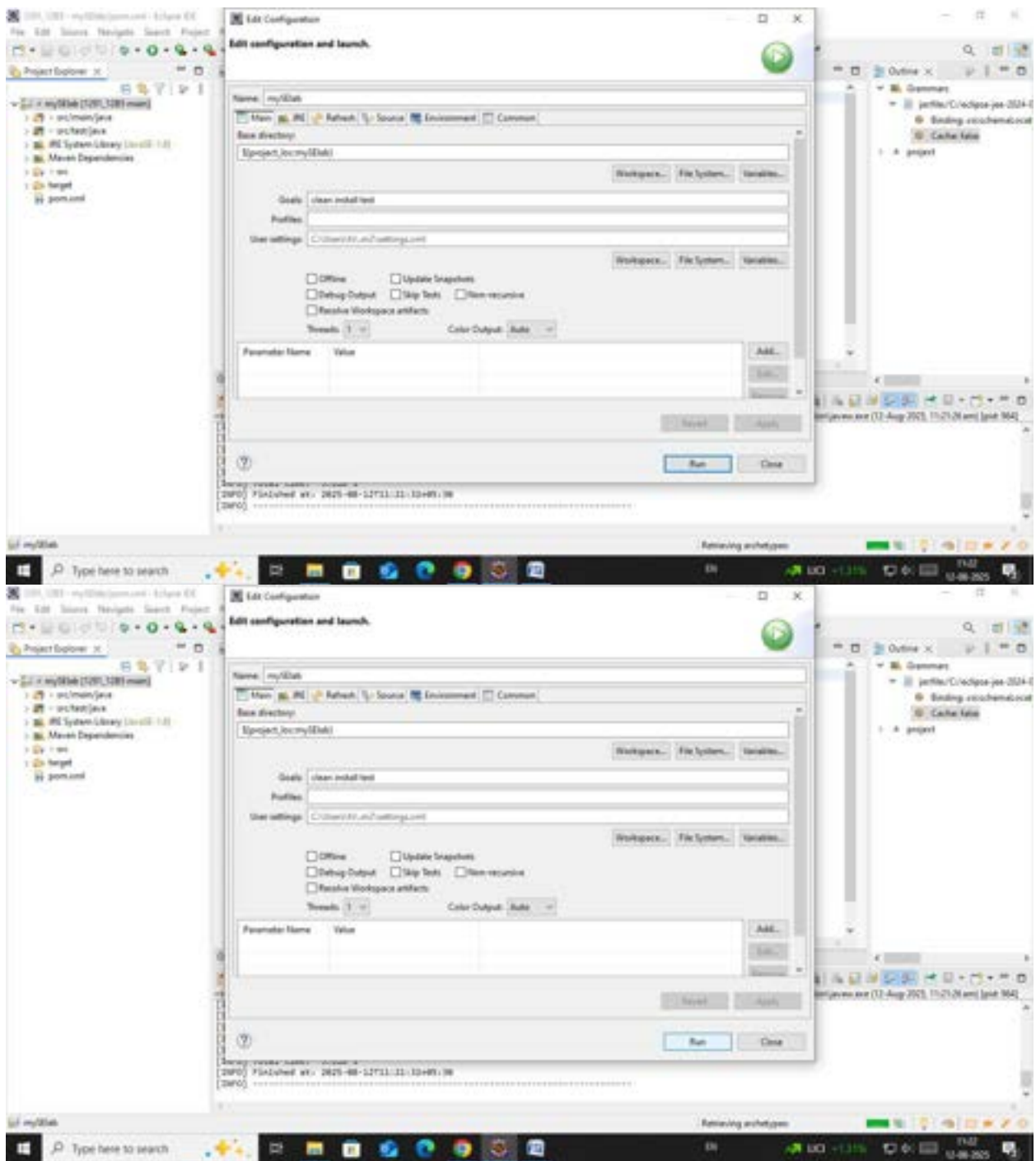
- └─ 9.1. Right-click on App.java -> Run As -> Maven Clean
 - └─ 9.1.1. Right-click on App.java -> Run As -> Maven Install
 - └─ 9.1.2. Right-click on App.java -> Run As -> Maven Test

9.1.3. Right-click on App.java -> Run As -> Maven Build

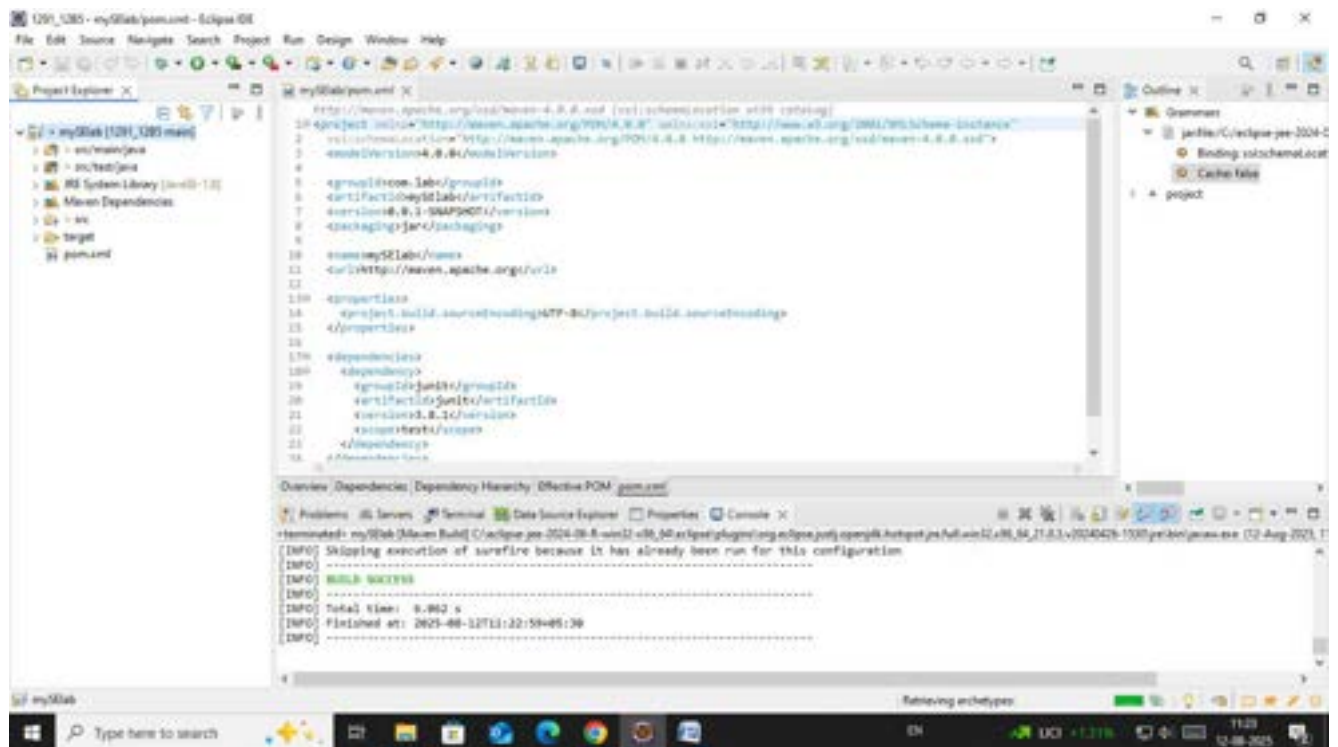






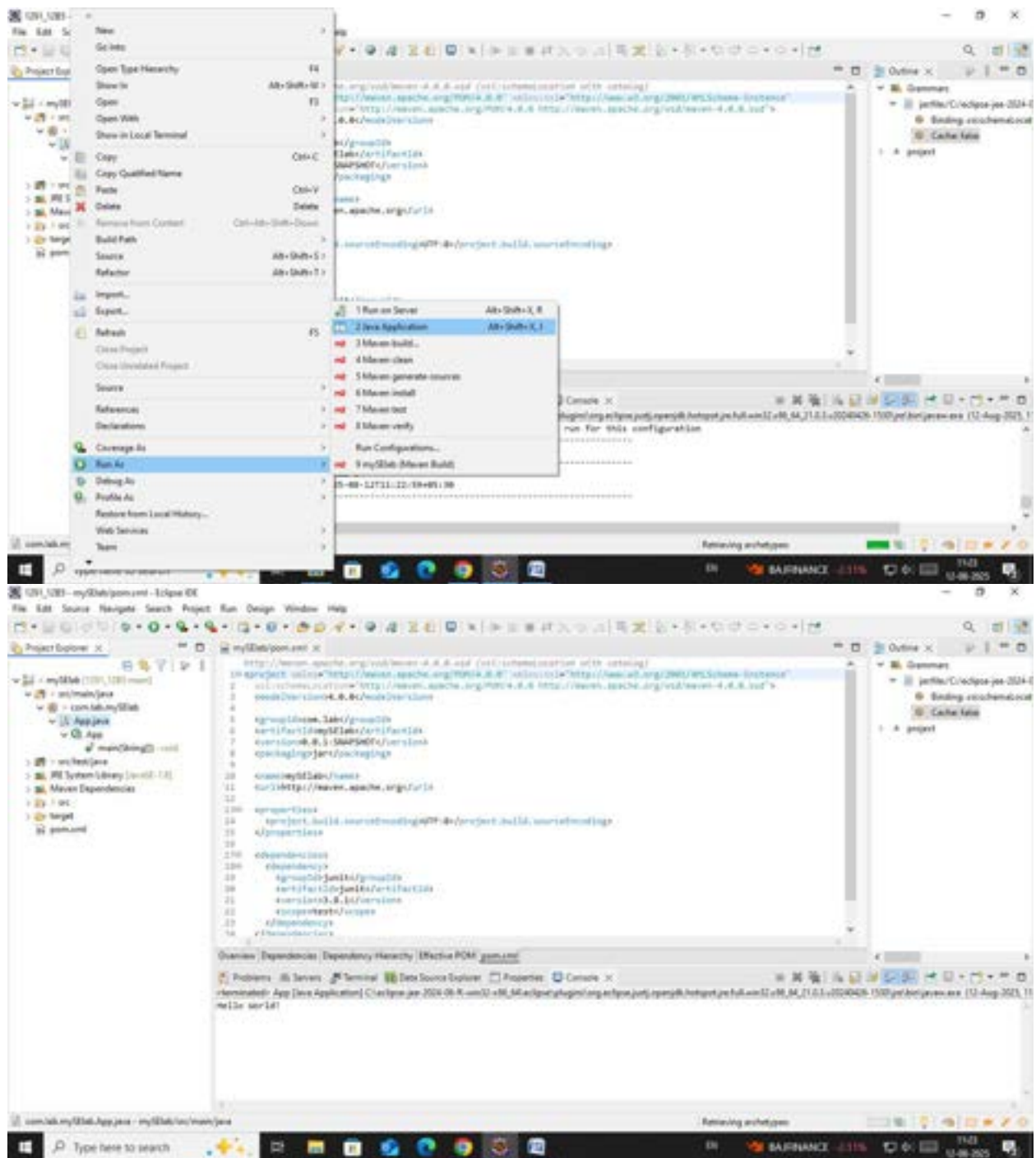


Step 11. Check console for BUILD SUCCESS message.



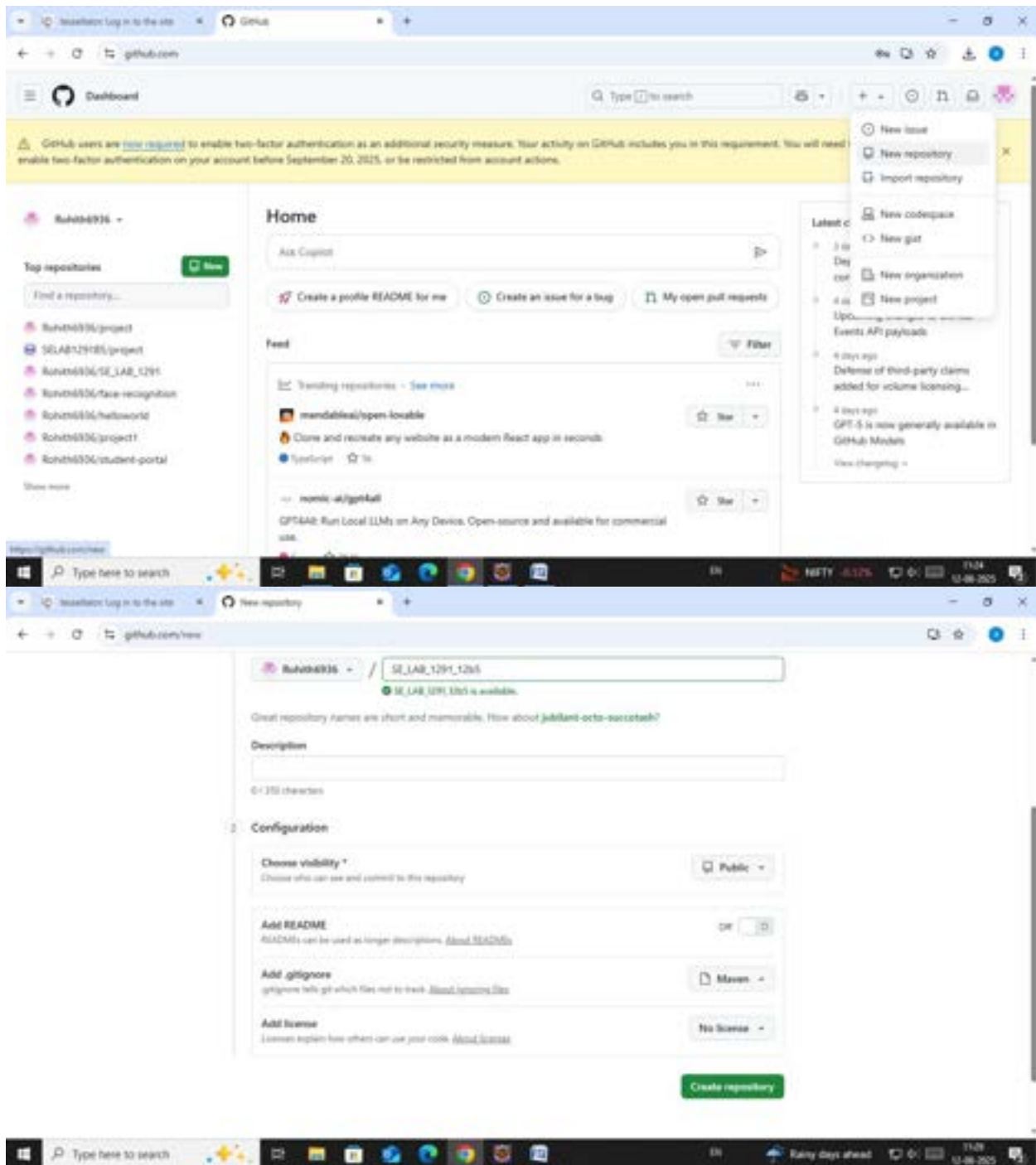
Step 12. Run the application:

- Right-click on App.java -> Run As -> Java Application
- Output: "Hello World" displayed.

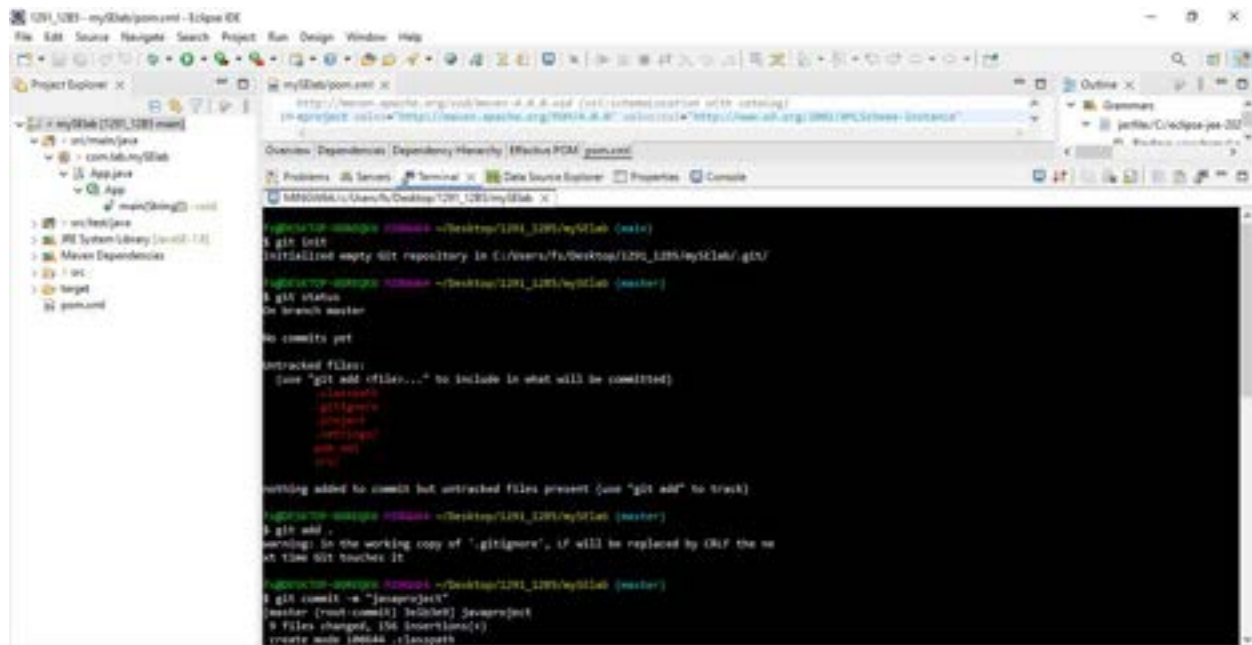


GITHUB:

Step: create a repository







The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays a project named 'myStab' with sub-projects 'src/main/java' and 'com.lab.myStab'. The 'App.java' file is selected. The main editor window shows the 'myStab/pom.xml' file. The terminal window at the bottom displays the following commands and output:

```
fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git init
Initialized empty Git repository in C:/Users/Fu/Desktop/1201_1205/myStab/.git/

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git status
On branch master

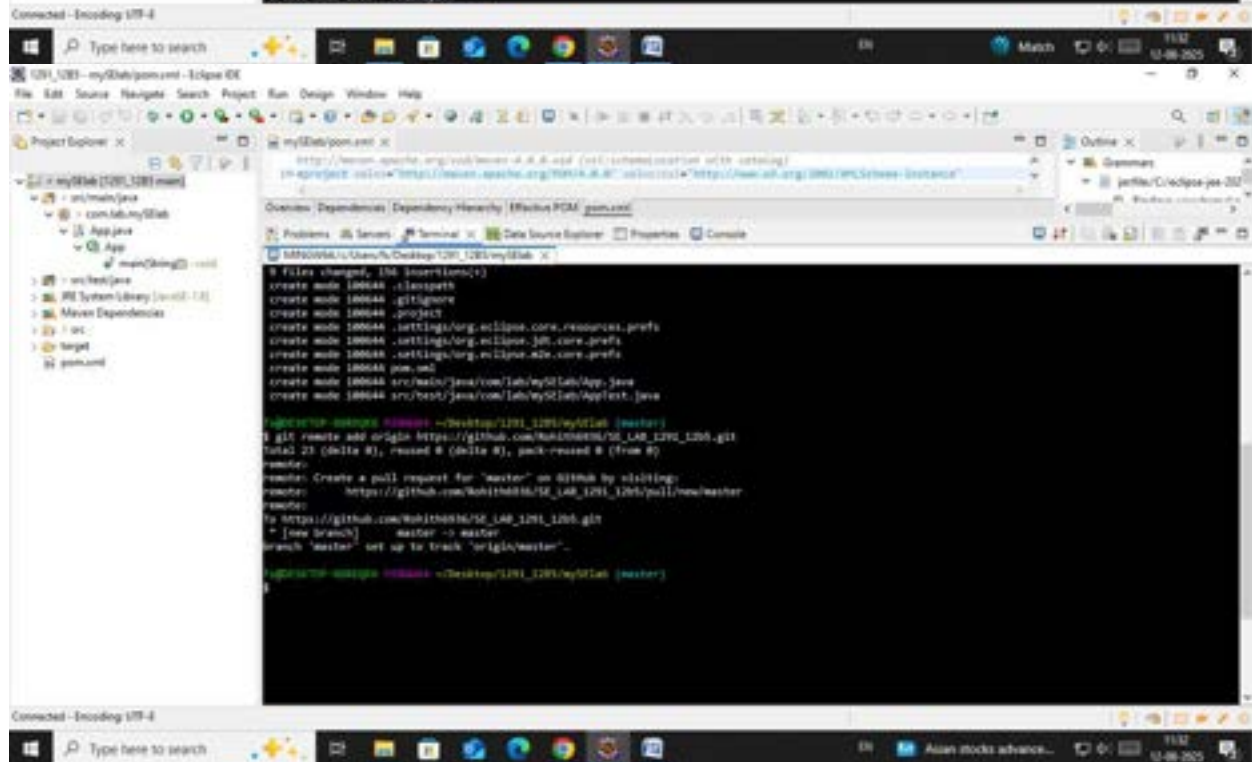
no commits yet

untracked files:
  (use "git add <file>..." to include in what will be committed)
  .classpath
  .gitignore
  .project
  .settings/org.eclipse.core.resources.prefs
  pom.xml

nothing added to commit but untracked files present (use "git add" to track)

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git add .
warning: in the working copy of ".gitignore", it will be replaced by OLF the next time Git touches it

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git commit -m "jassraj"
[master (root commit) 7e1c8e9] jassraj
5 files changed, 156 insertions(+)
create mode 100644 .classpath
```



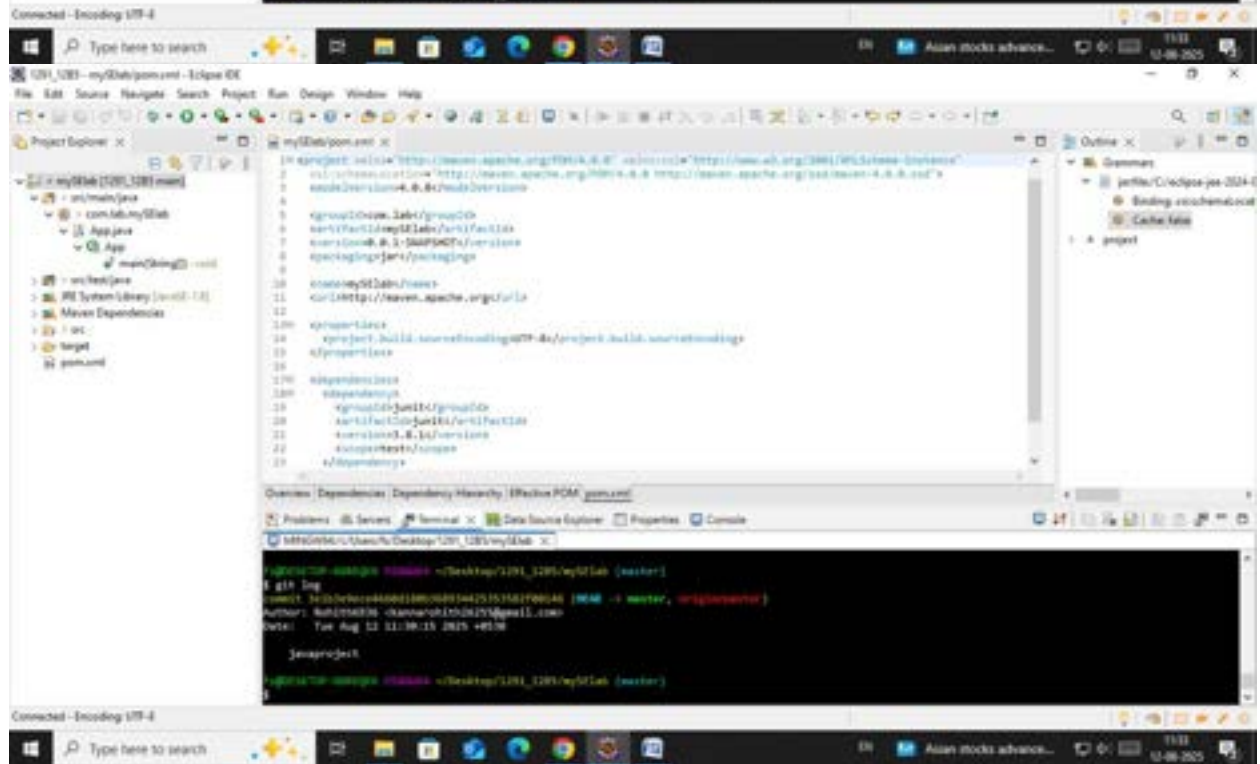
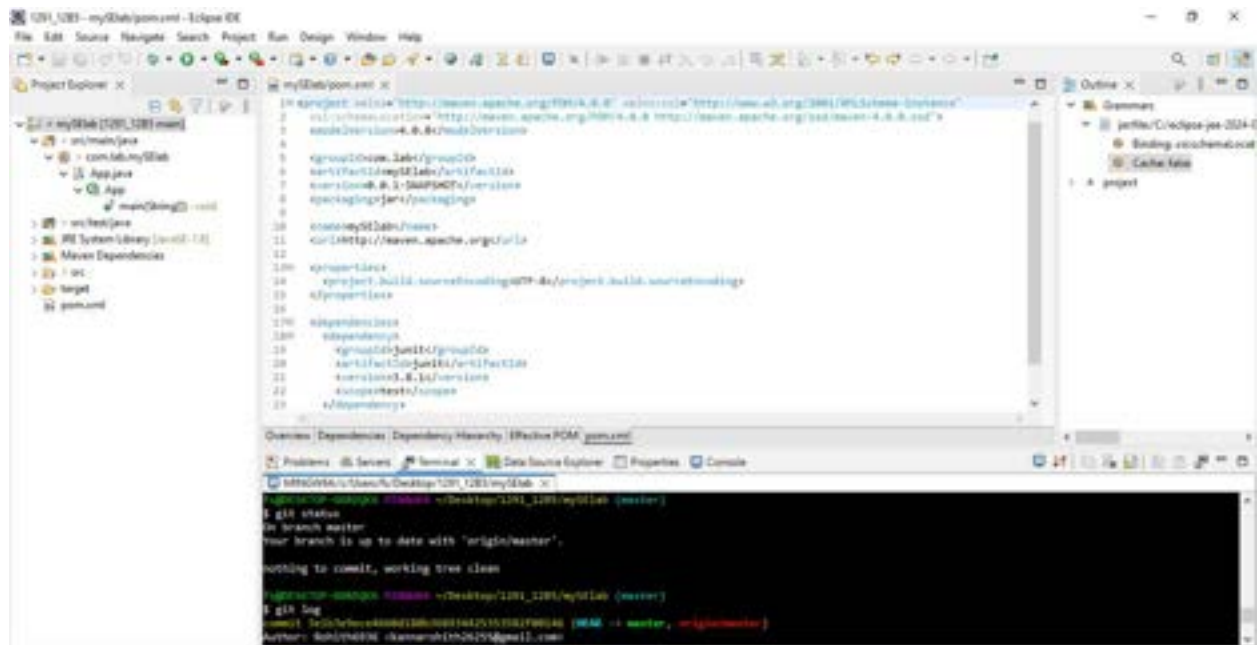
The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays a project named 'myStab' with sub-projects 'src/main/java' and 'com.lab.myStab'. The 'App.java' file is selected. The main editor window shows the 'myStab/pom.xml' file. The terminal window at the bottom displays the following commands and output:

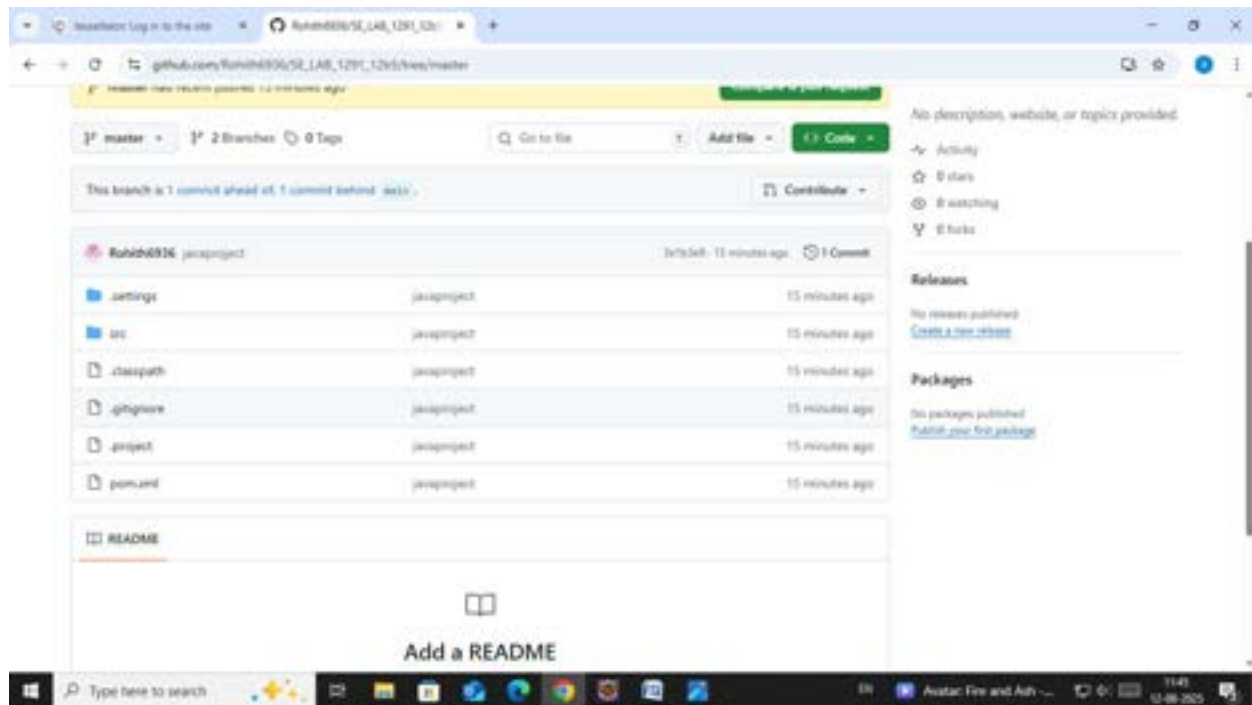
```
5 files changed, 156 insertions(+)
create mode 100644 .classpath
create mode 100644 .gitignore
create mode 100644 .project
create mode 100644 .settings/org.eclipse.core.resources.prefs
create mode 100644 .settings/org.eclipse.jdt.core.prefs
create mode 100644 .settings/org.eclipse.wst.core.prefs
create mode 100644 pom.xml
create mode 100644 src/main/java/com/lab/myStab/App.java
create mode 100644 src/test/java/com/lab/myStab/AppTest.java

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git remote add origin https://github.com/hukith086/SE_140_1201_1205.git
total 23 (delta 8), reused 8 (delta 8), pack-reused 8 (from 8)

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$ git push
To https://github.com/hukith086/SE_140_1201_1205.git
 * [new branch] master -> master
branch 'master' set up to track 'origin/master'.

fu@ubuntu-vm:~/Desktop/1201_1205/myStab$
```

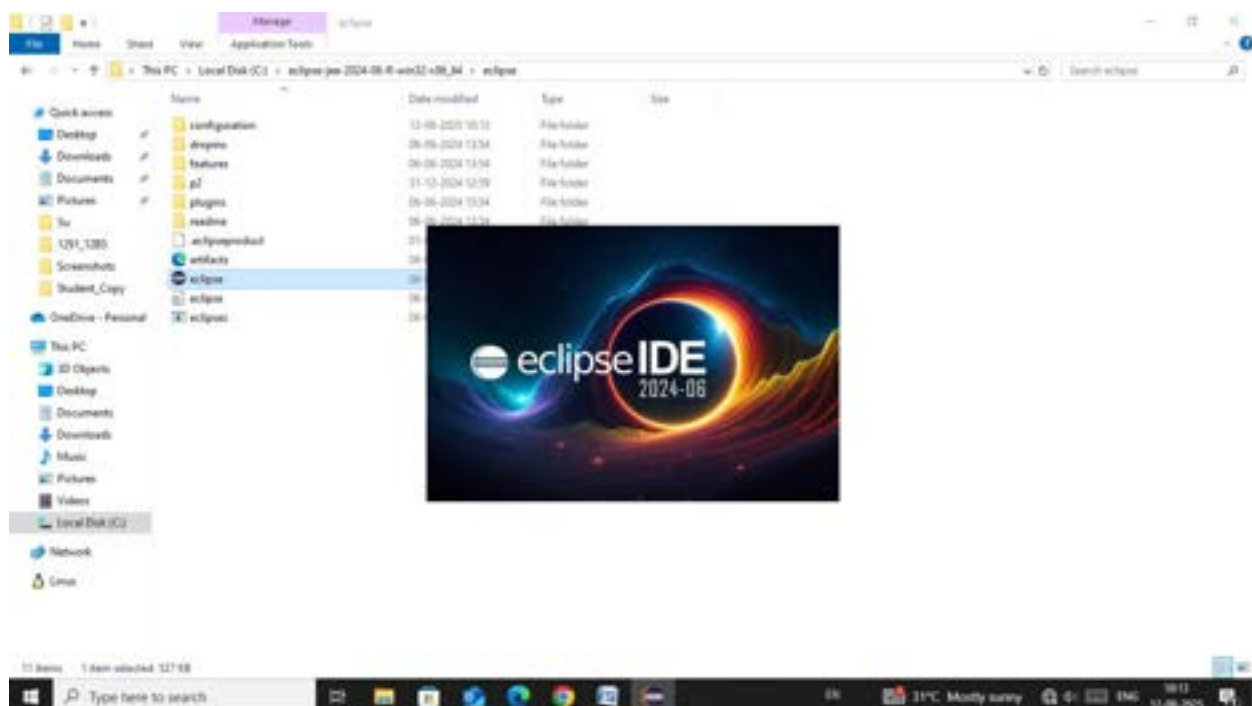




Creation of Maven web Java Project

Step 1: Open Eclipse

- └─ 1.1 Launch Eclipse IDE.
- └─ 1.2 Select or create a workspace.

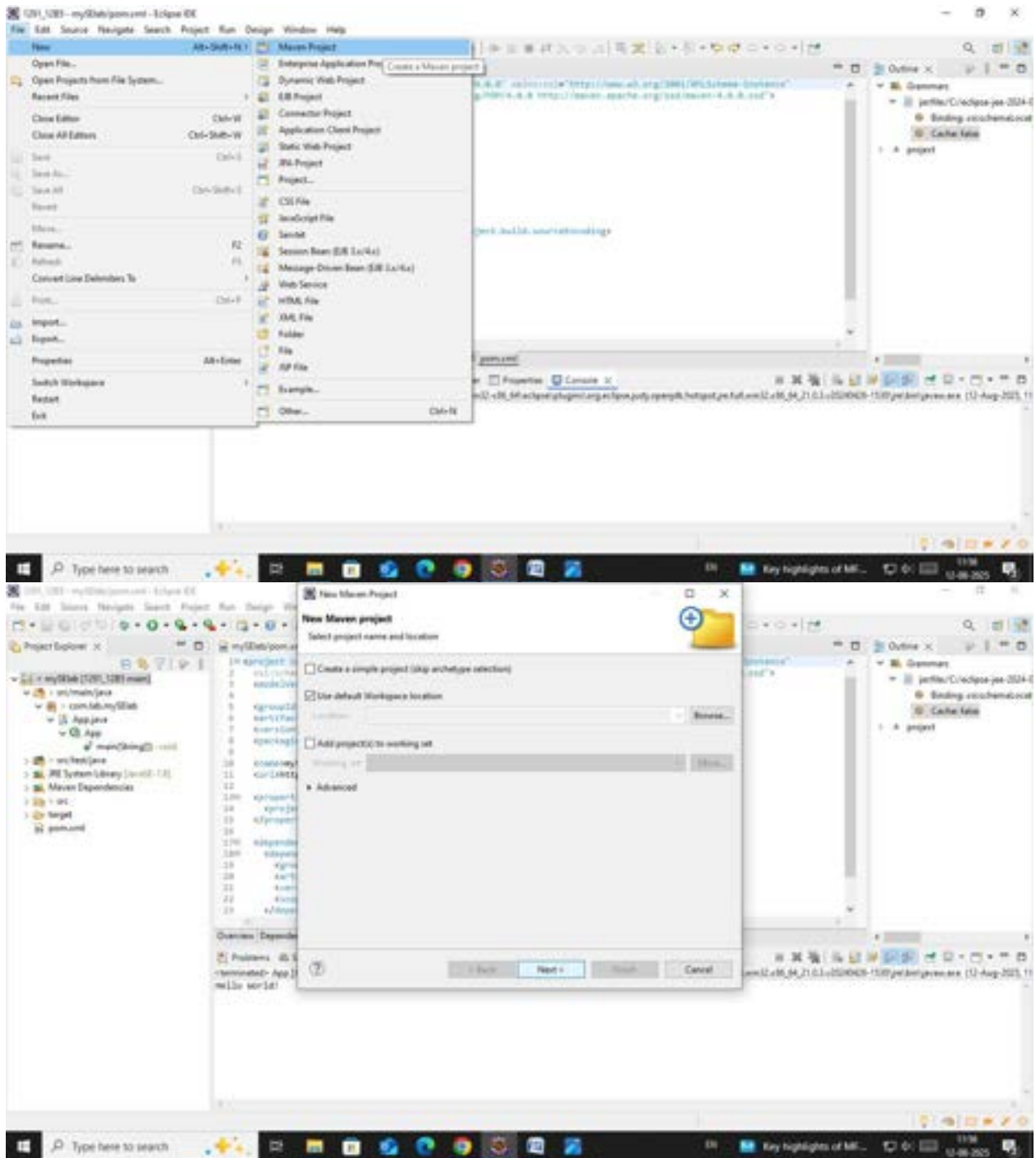


Step 2: Create a New Maven Project

└─ 2.1. File -> New -> Project...

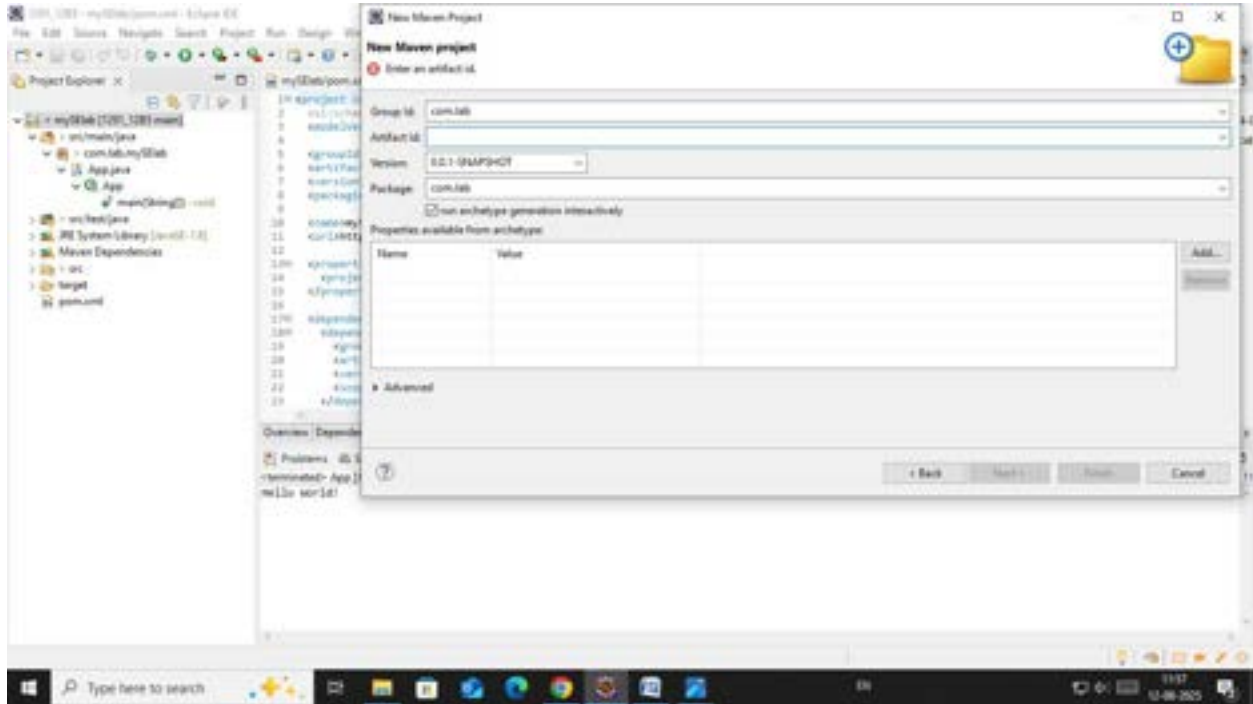
└─ 2.1.1. Expand "Maven"

└─ 2.1.2. Select "Maven Project" and click "Next"



Step 3: Choose Maven Archetype

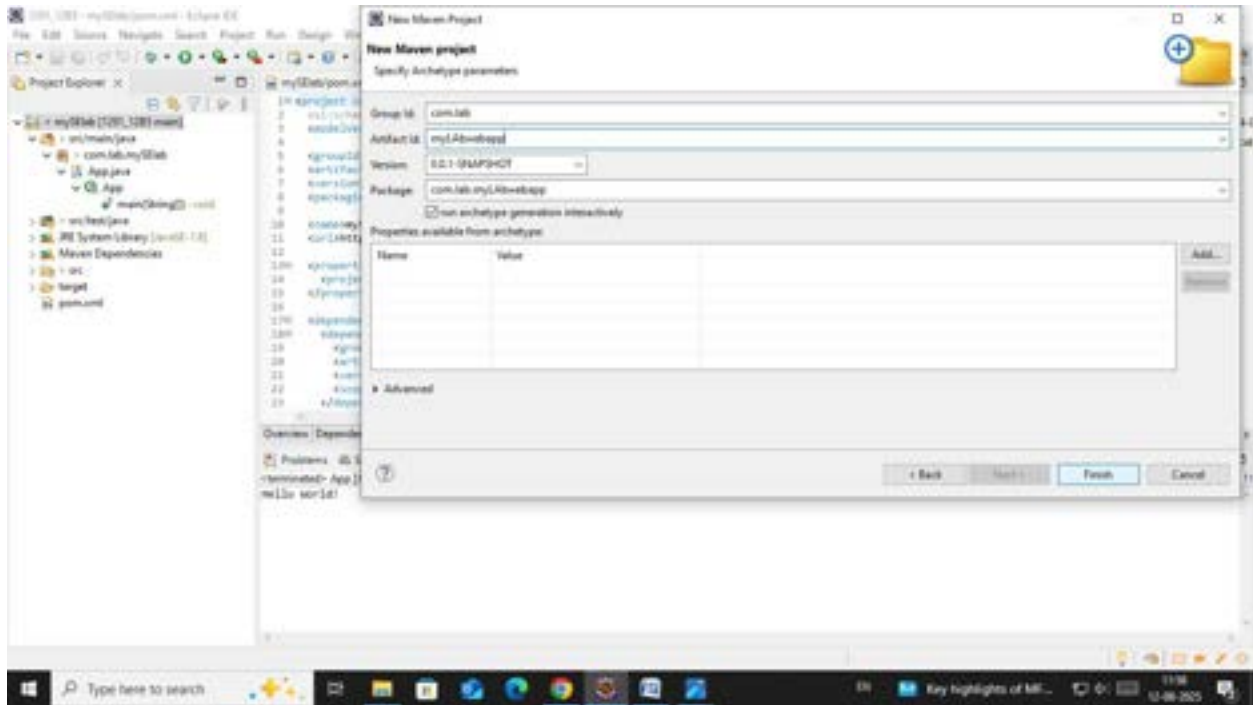
- └─ 3.1. Select an archetype(e.g. "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")
- └─ 3.2. Click "Next"



Step 4: Configure the Maven Project

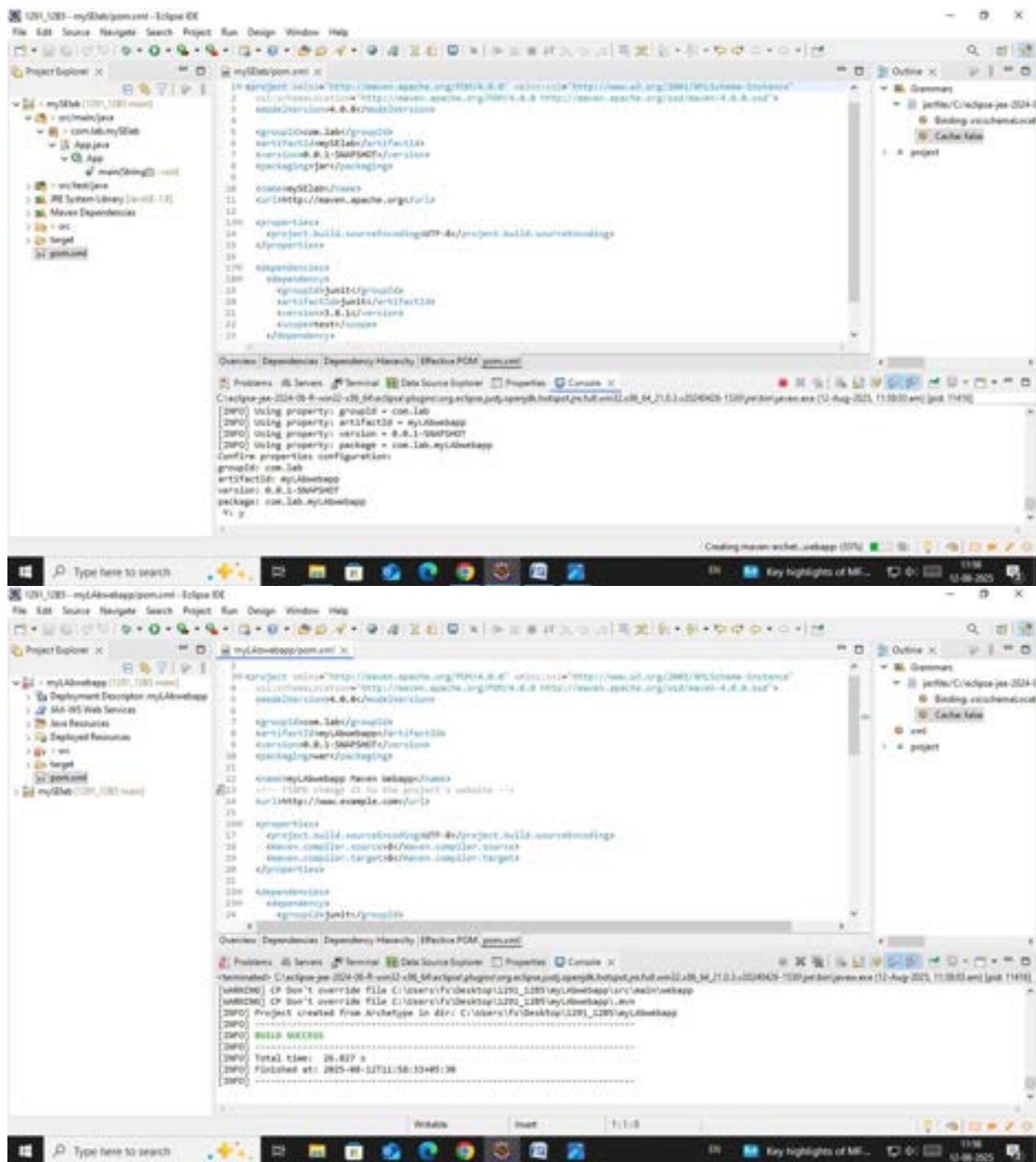
- └─ 4.1 Group Id: Enter a group ID (e.g., com.example).
- └─ 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

└─ 4.3 Click ****Finish**** to create the project.



Step 5: Add Maven Dependencies

└─ 5.1 Open the ****pom.xml**** file in the Maven project.



— 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

The screenshot shows the Maven Repository search results for the artifact 'java:javax.servlet-api'. The search bar at the top contains 'java:javax.servlet-api'. The results are sorted by Relevance. The first result is '1. Java Servlet API' with 20,379 usages. The second result is '2. JavaServlet(TM) Specification' with 13,614 usages. The third result is '3. SLF4J API Module' with 77,618 usages. The left sidebar shows a list of repositories and groups. The right sidebar shows a list of popular tags.

The screenshot shows the Maven Repository artifact page for 'jakarta.servlet:jakarta.servlet-api'. The page displays the artifact's details, including its license (GPL 2.0), categories (Java Specifications), tags (jakarta, standard, servlet, api, specs), homepage (https://projects.eclipse.org/projects/ee4j/servlet), ranking (#87 in Maven Repository), and usage (4,927 artifacts). The page also shows a table of versions and their release dates.

Version	Vulnerabilities	Repository	Usage	Date
6.1.0		Central	5,423	Jun 21, 2024
6.1.x		Central	82	Feb 27, 2024
6.1.0-RT		Central	55	Nov 23, 2023
6.0.0		Central	3,944	Jun 21, 2022
5.0.0		Central	4,723	Jun 21, 2022

Example:

```
<<xml
```

```
<dependency>
```

```
<groupId>javax.servlet</groupId>
```

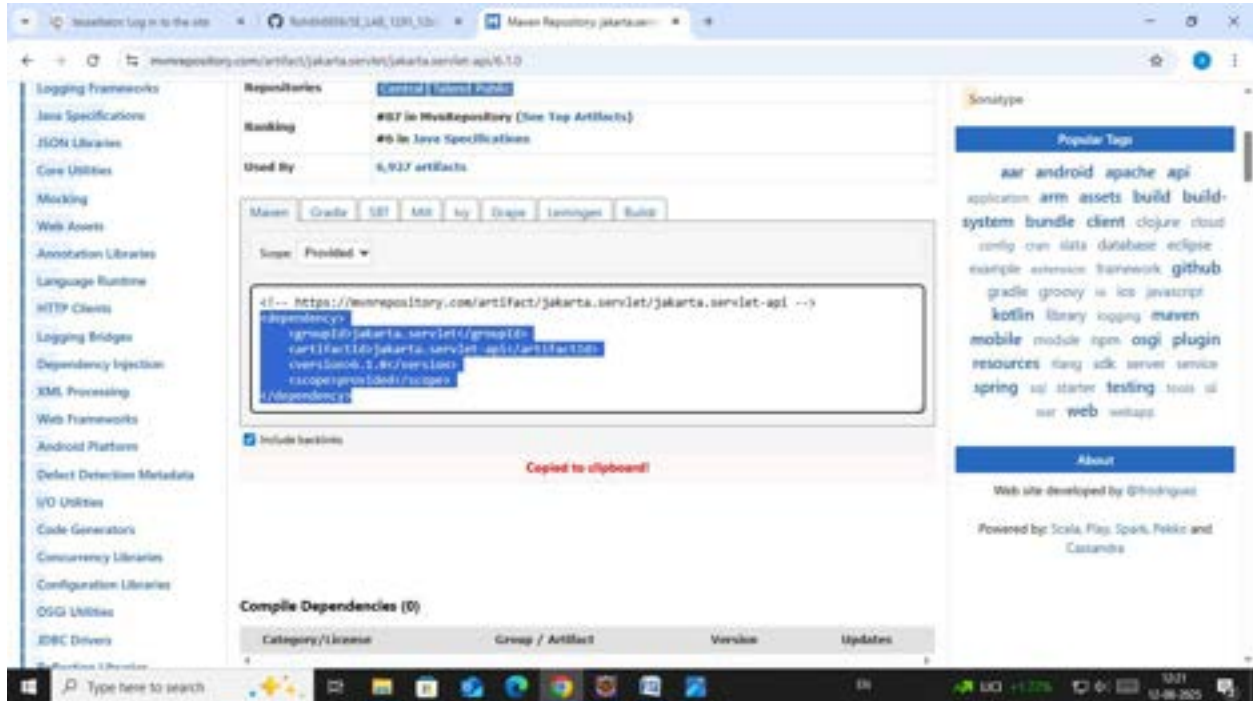
<artifactId>javax.servlet-api</artifactId>

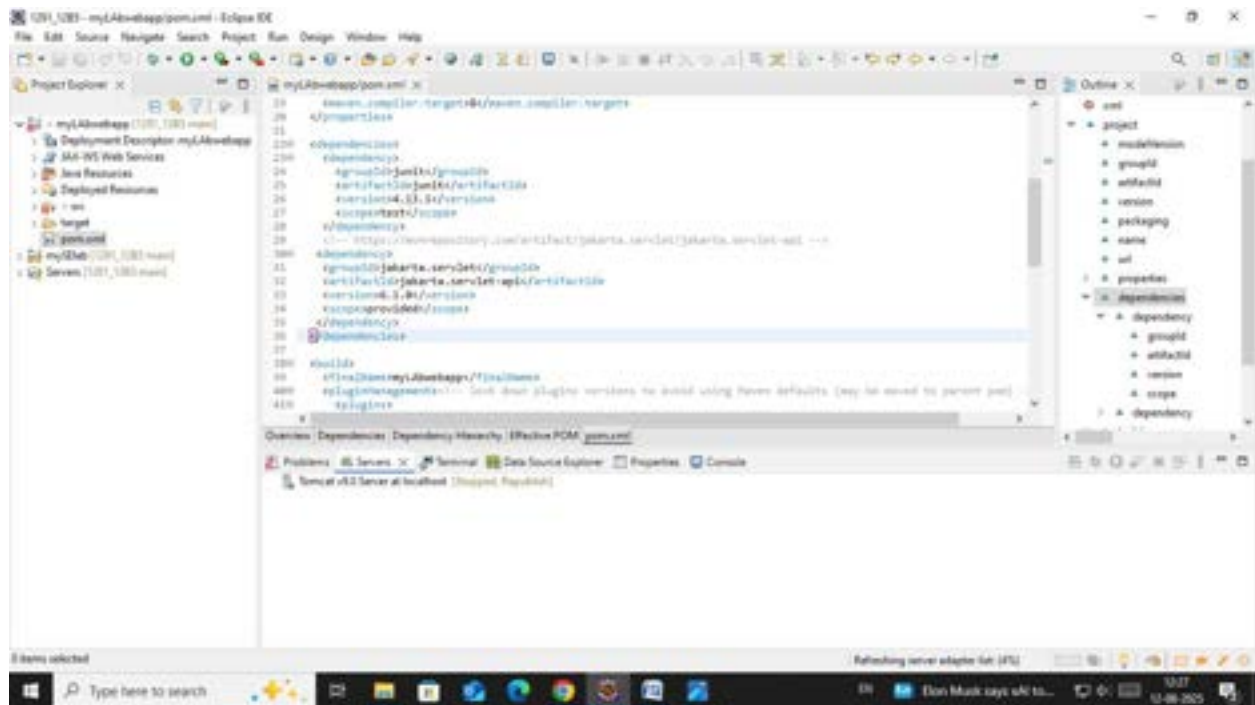
<version>4.0.1</version>

<scope>provided</scope>

</dependency>

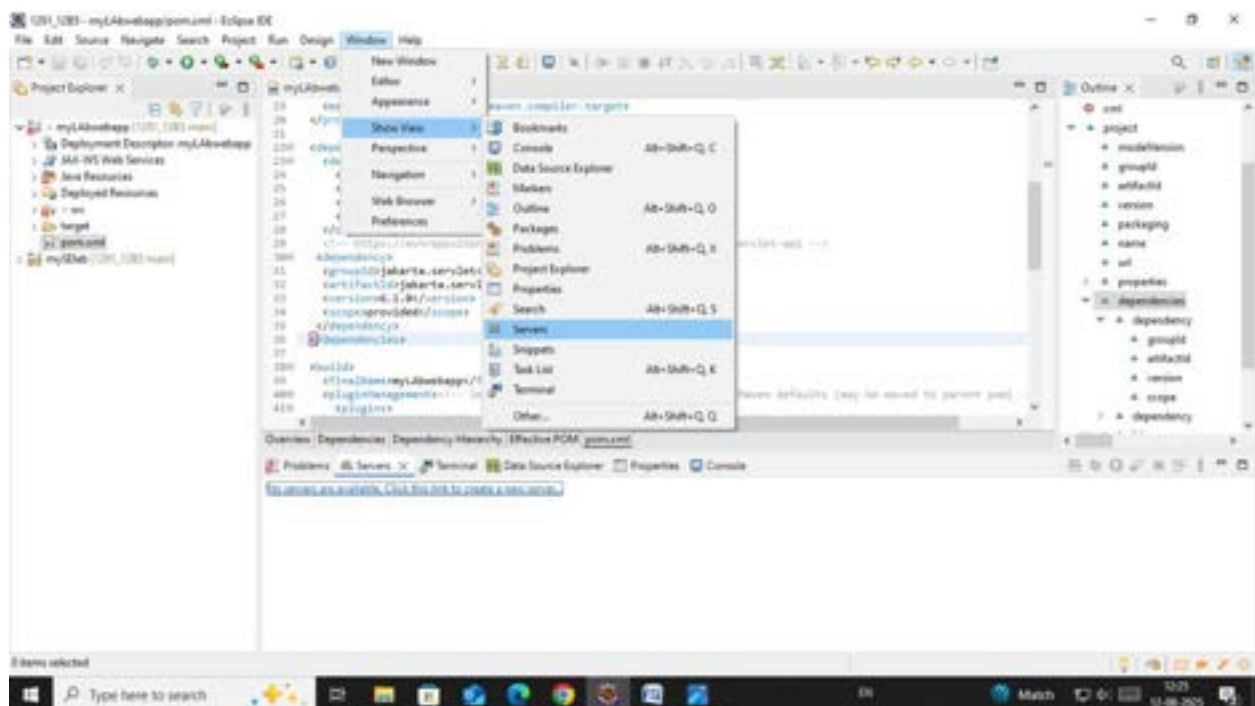
...

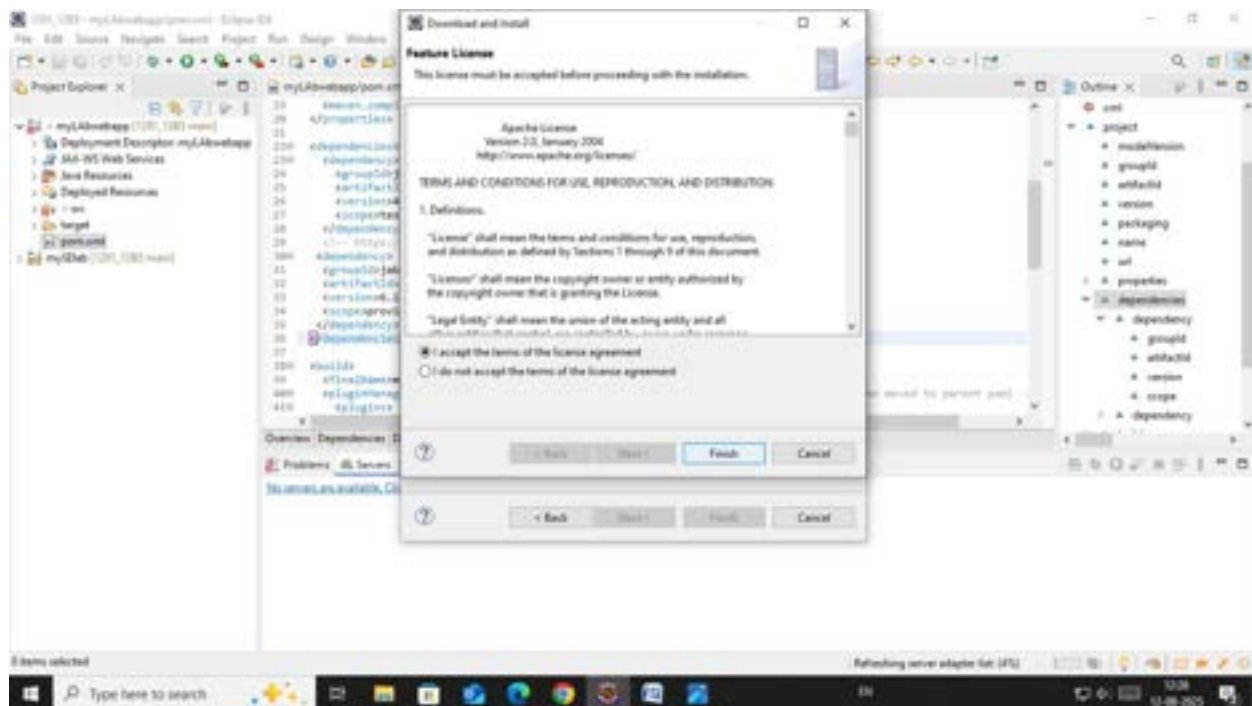
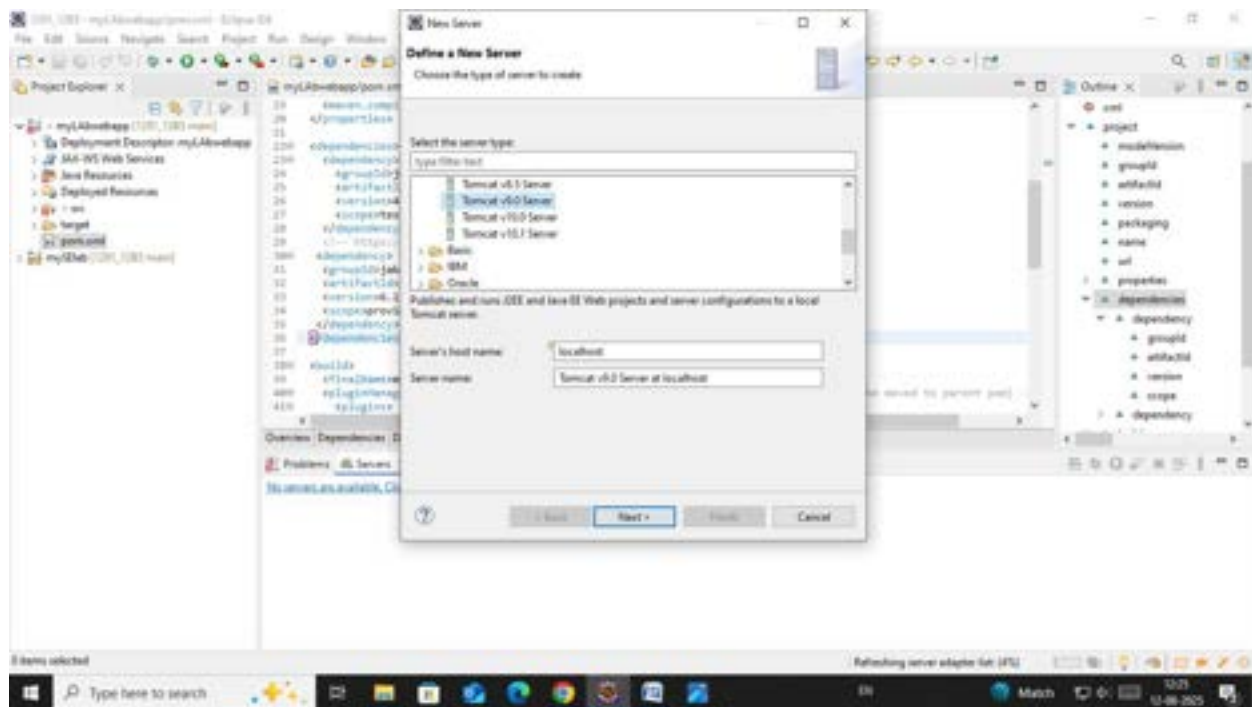


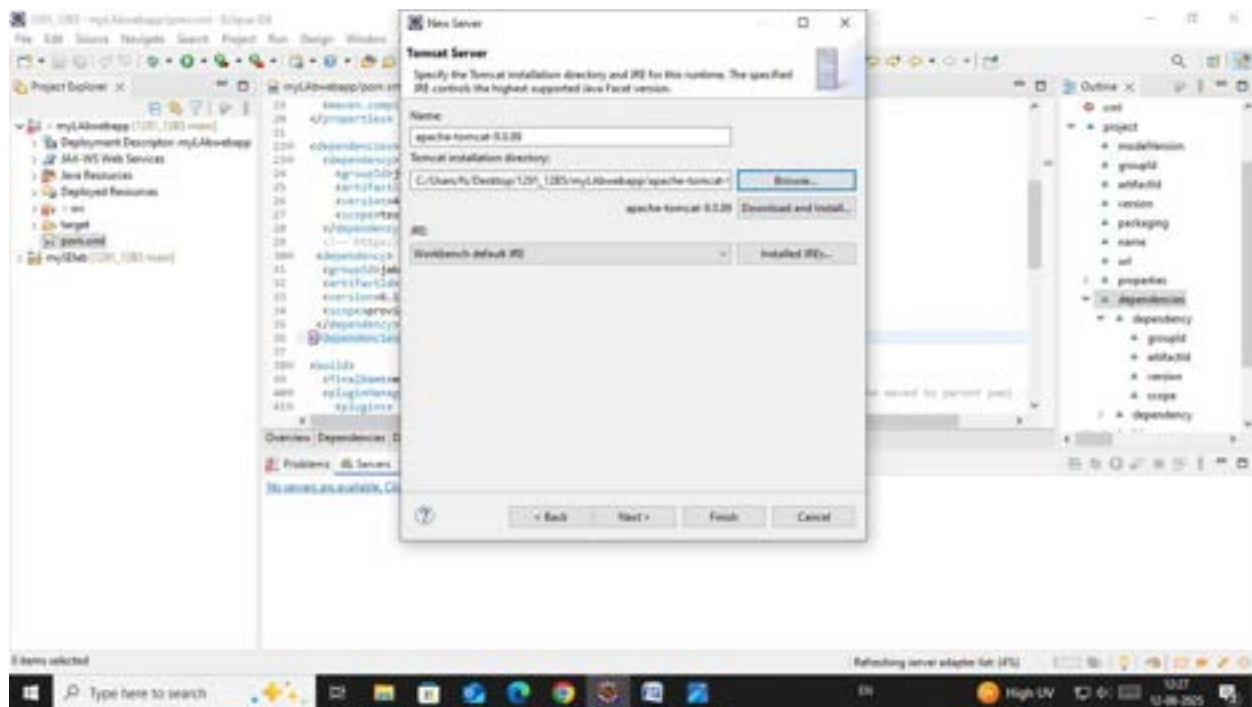


Step 6:- Configure server:

- └ Window -> Show View -> Servers
- └ Add server -> Select Tomcat v9.0 server -> Click Next
- └ Configure server options (e.g., ports, server location).

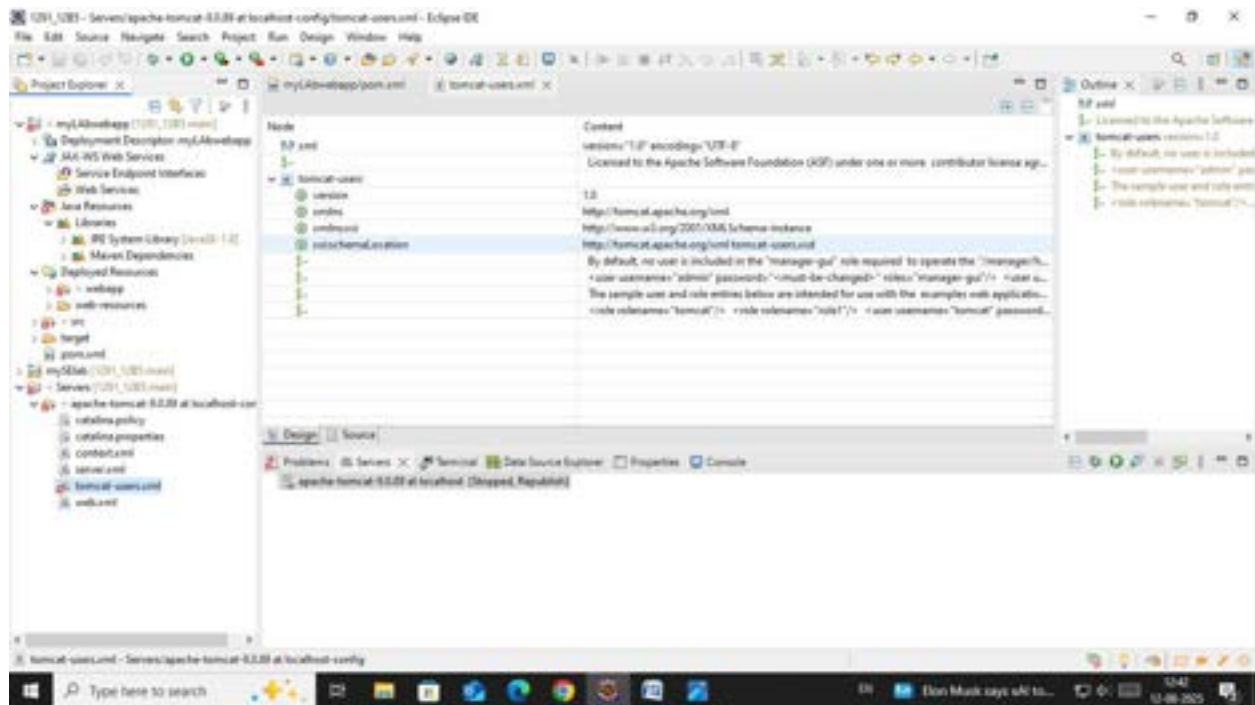






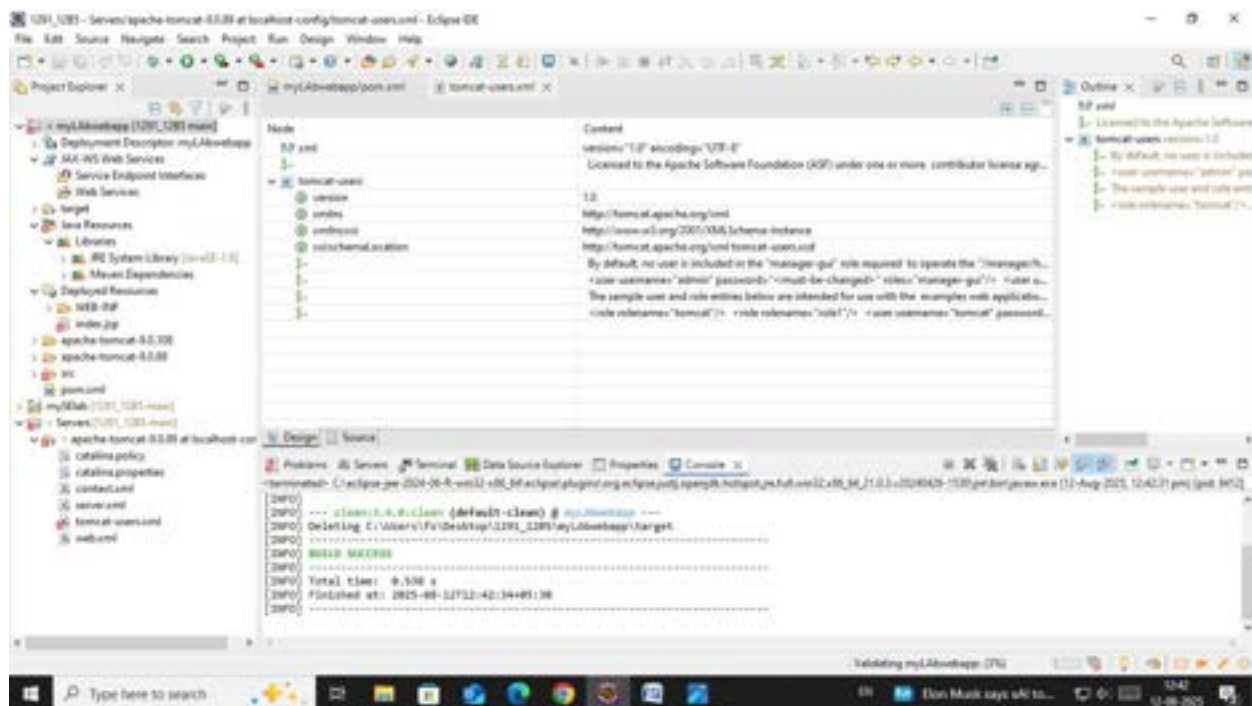
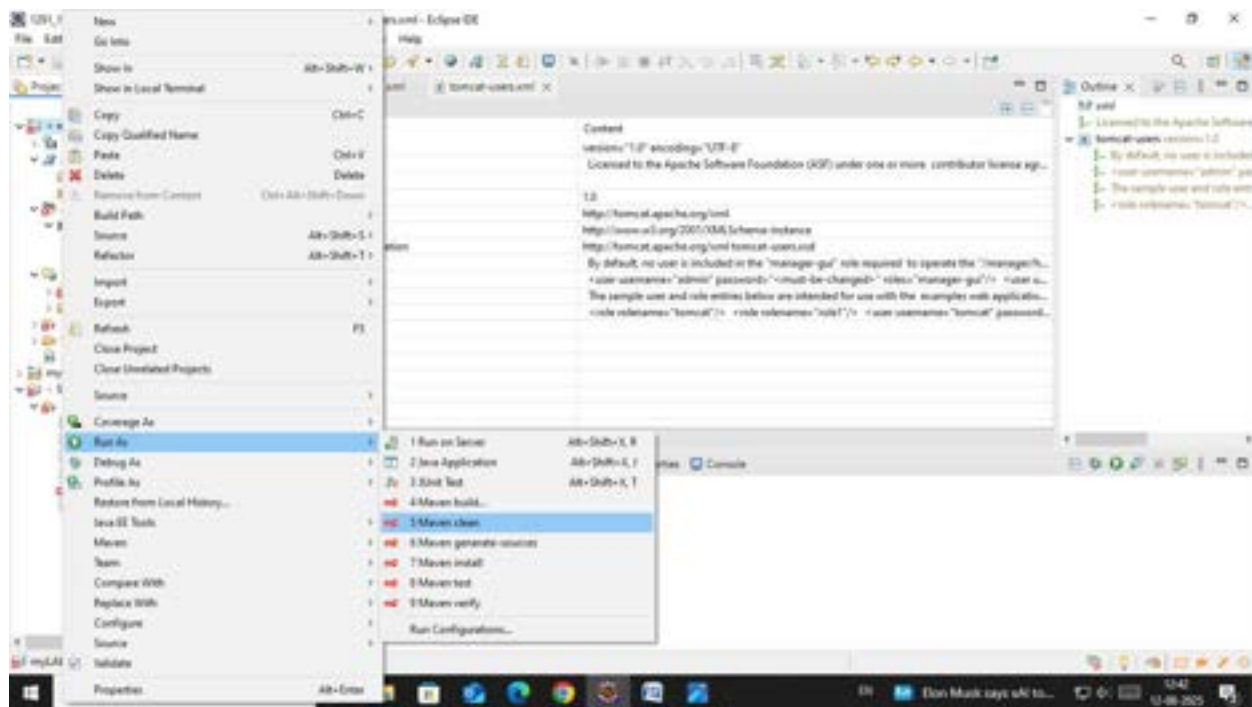
Step 7:- Modify 'tomcat-users.xml':

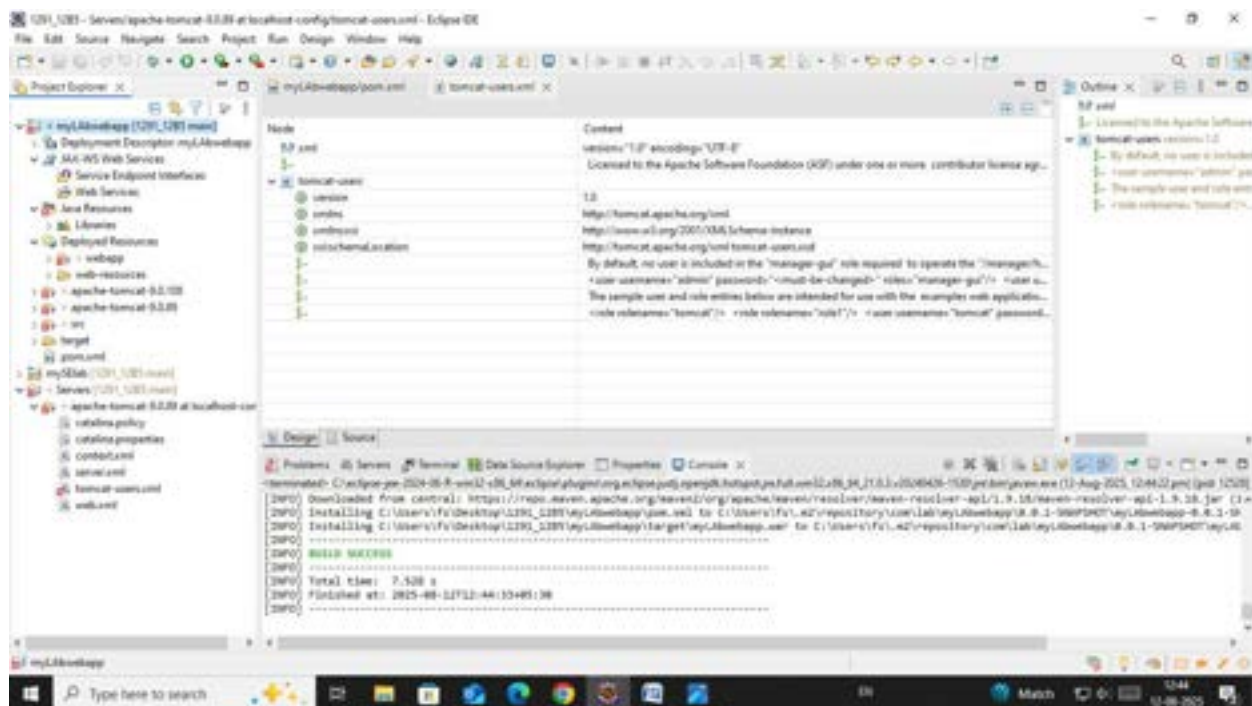
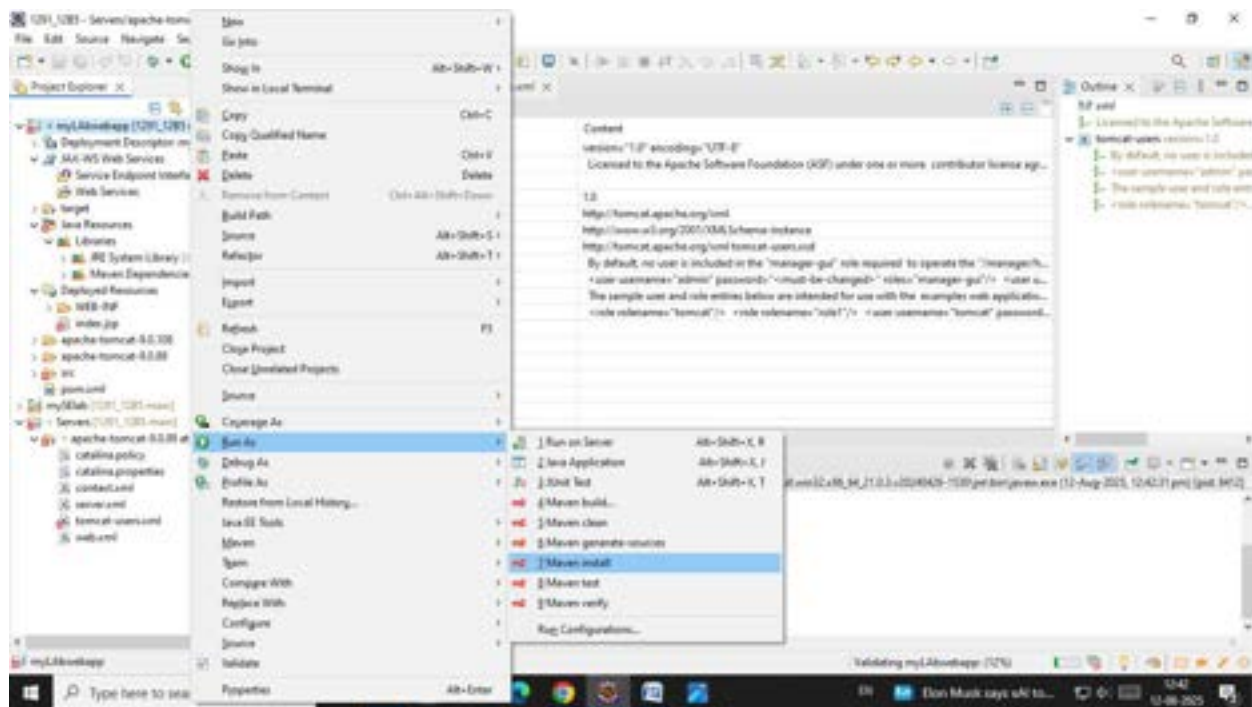
└─ Add role and user details under <tomcat-users> tag.

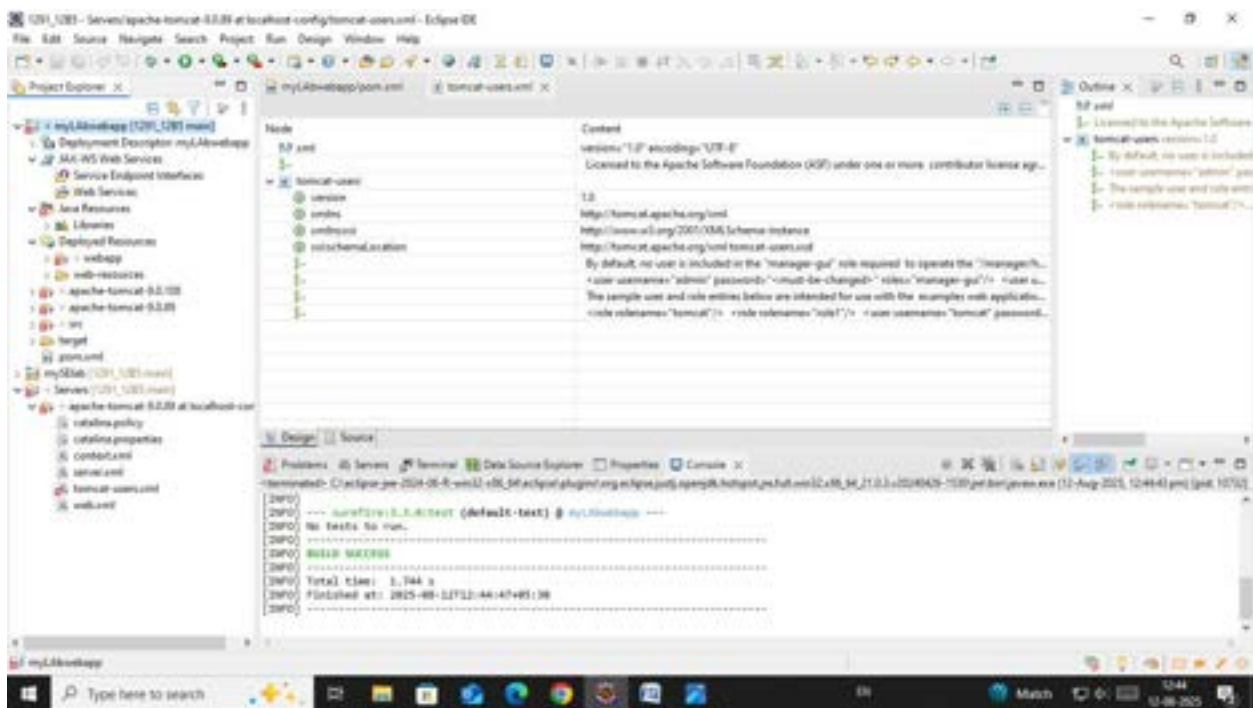
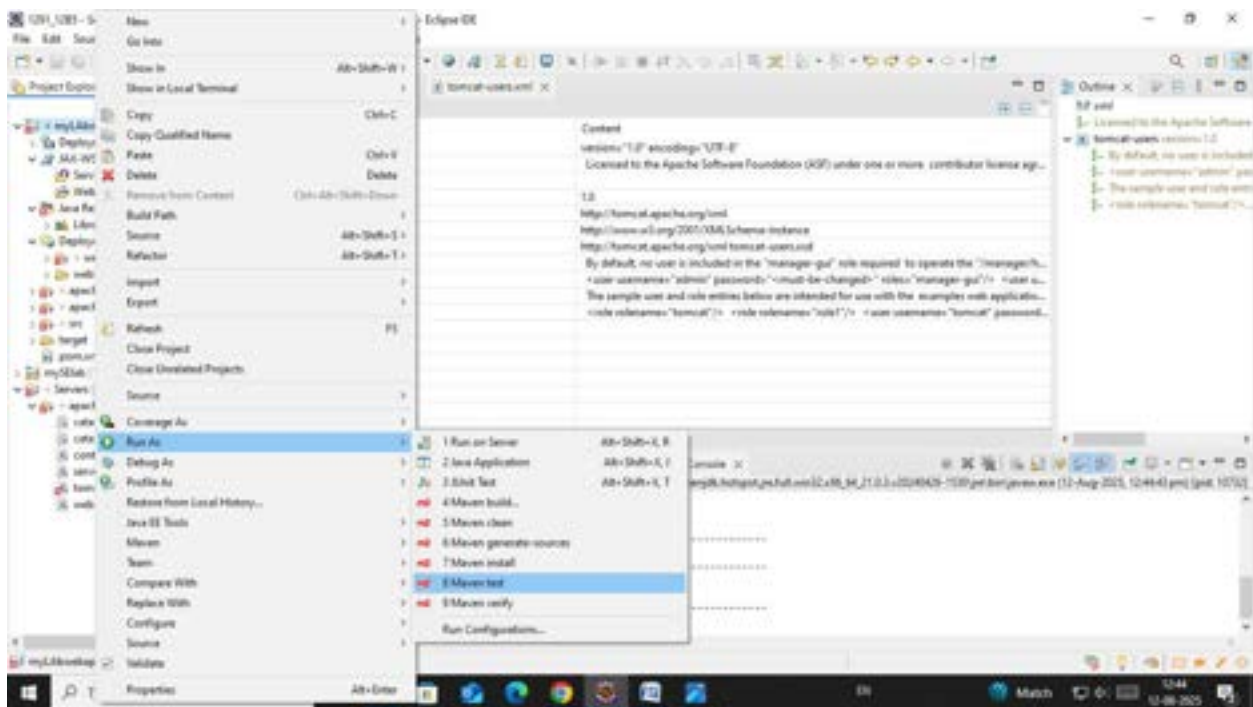


Step 8:. Build the project:

- └ Right-click on index.jsp -> Run As -> Maven Clean
- └ Right-click on index.jsp -> Run As -> Maven Install
- └ Right-click on index.jsp -> Run As -> Maven Test
- └ Right-click on index.jsp -> Run As -> Maven Build



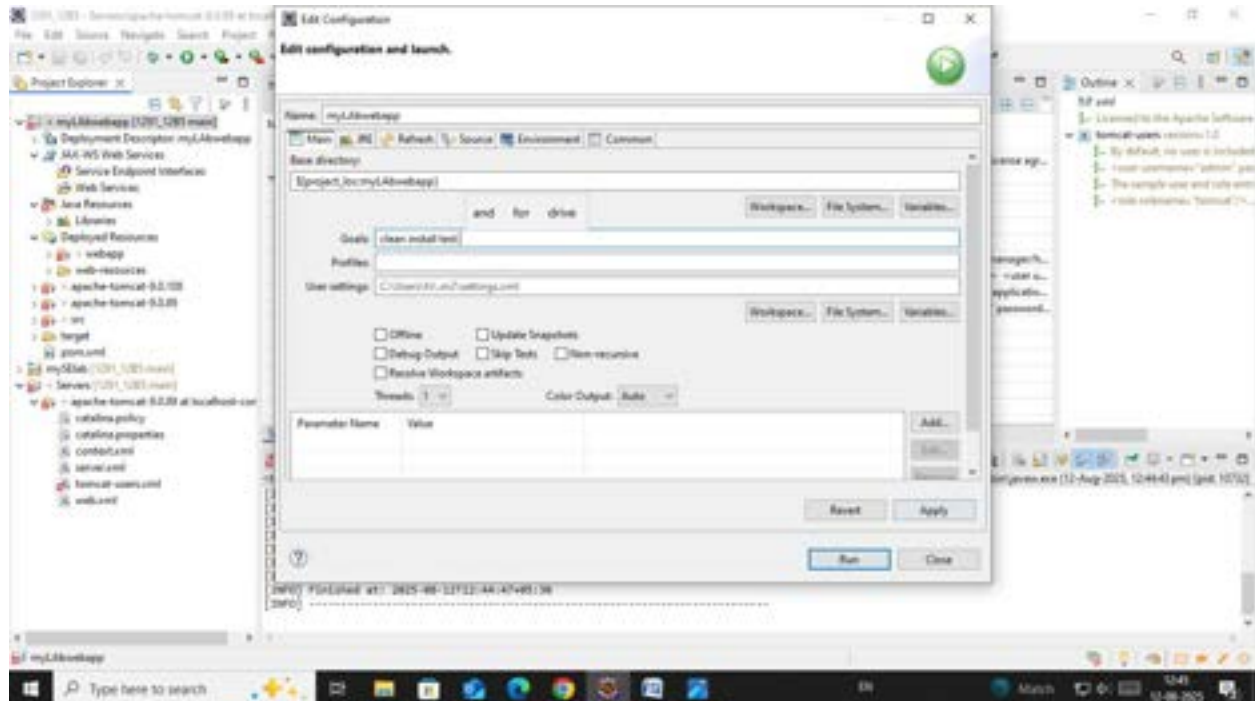
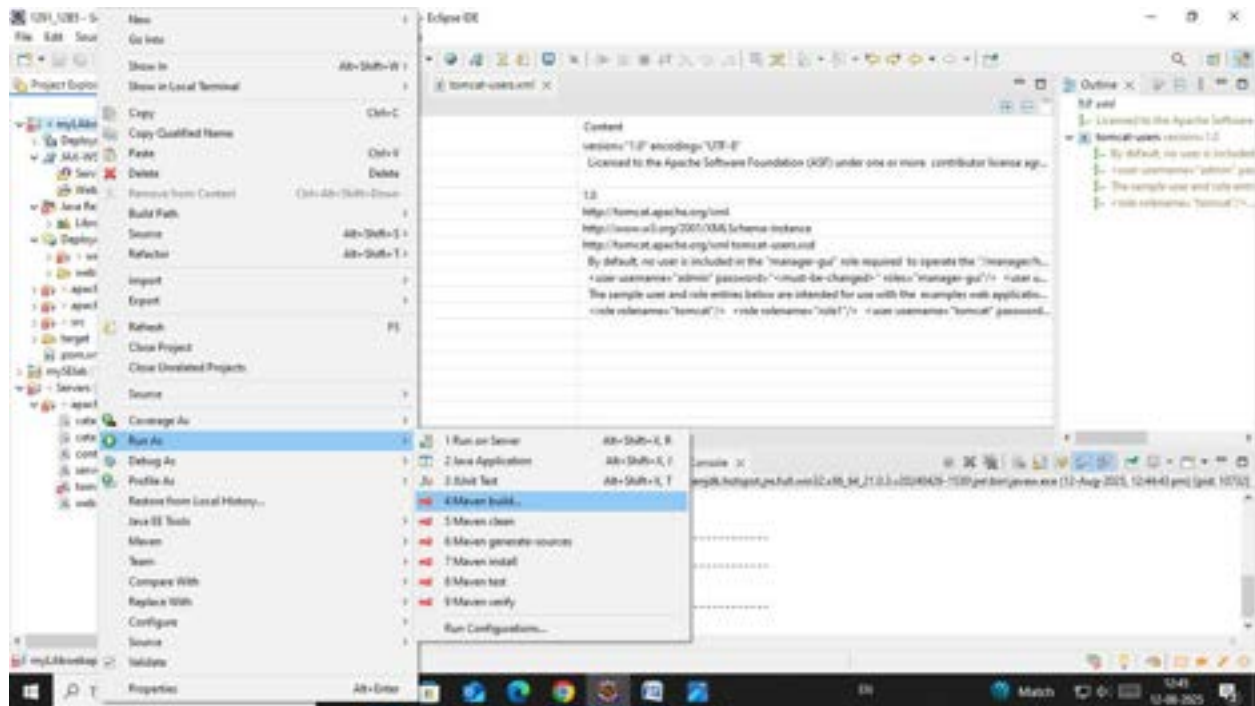


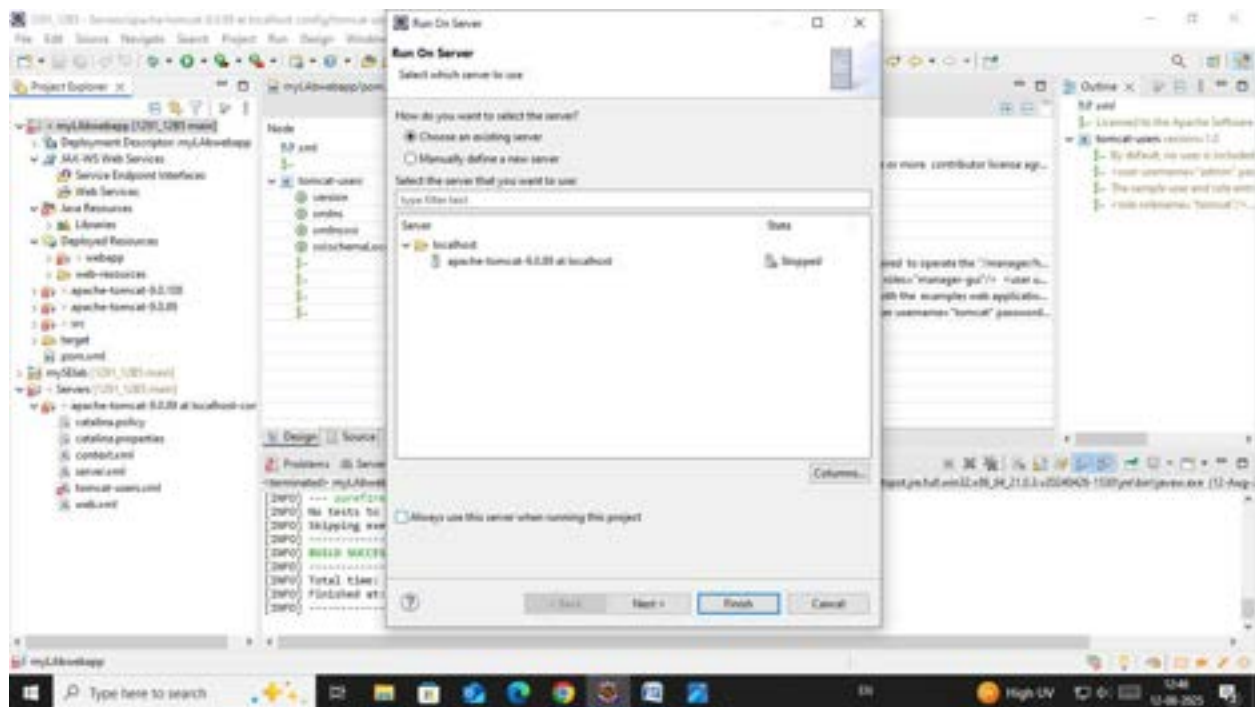


Step 9. In the Maven Build dialog:

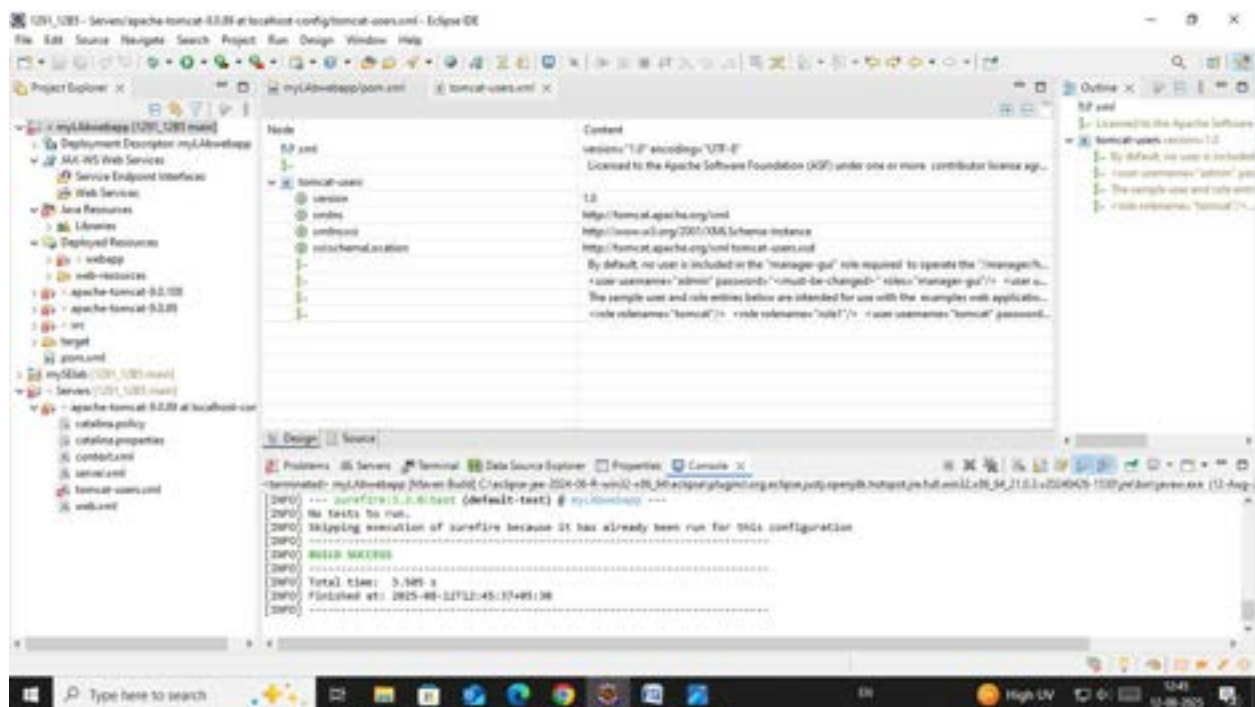
└ Enter Goals: clean install test

└ Click on Apply -> Click on Run





Step 10. Check console for BUILD SUCCESS message.



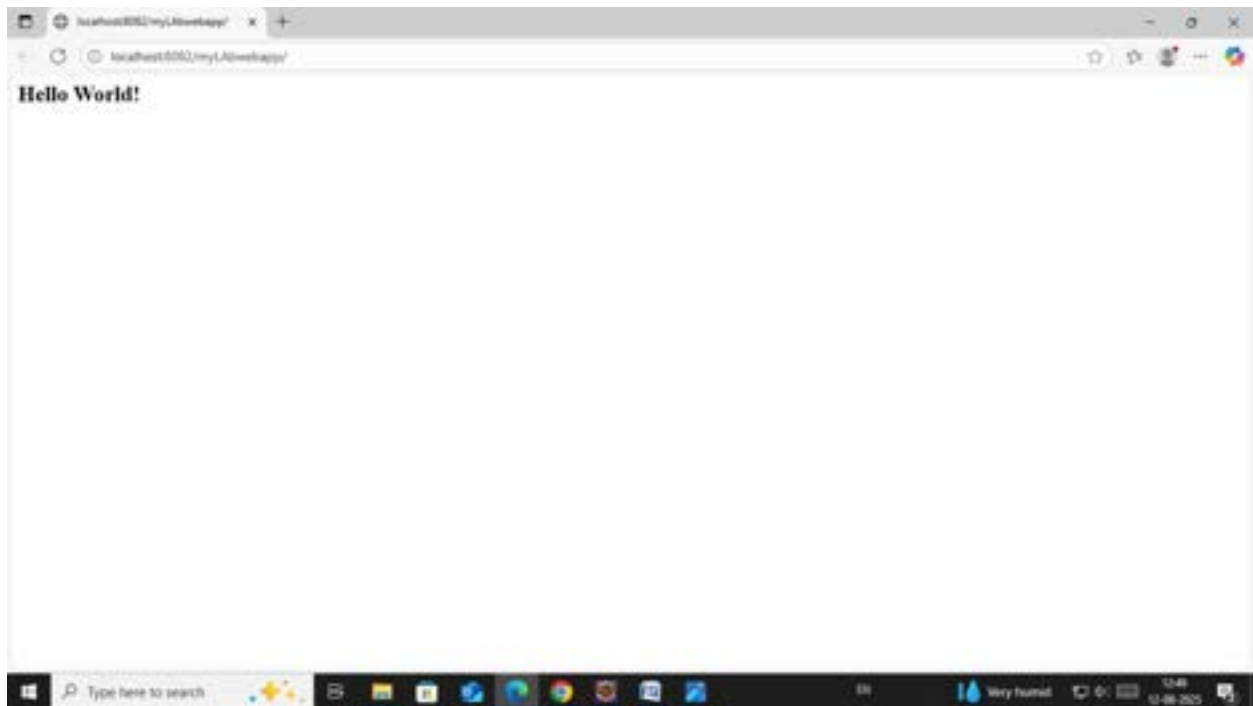
Step 11. Run the application:

└─ Right-click on index.jsp -> Run As -> Run on Server

└─ Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github



3. Configure Java Version via Compiler Plugin

In `pom.xml`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <source>17</source>
    <target>17</target>
  </configuration> </plugin>
```

Check:

- Run `mvn package`
- Inspect generated JAR: `jar tf target/myapp.jar`

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```
public class AppTest {  
    @Test  
    public void testSum() {  
        assertEquals(5, 2 + 3);  
    }  
}
```

Run:

`mvn test`

Check:

- `target/test-classes/` → *compiled test .class files*
- `target/surefire-reports/` → *.txt or .xml test results*
If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause BUILD FAILURE
- Maven shows precise error in console
- *Fix the dependency tag → re-run `mvn clean install`*

6. Adding Resource Files

Put `config.properties` inside:

`src/main/resources/config.properties`

To read:

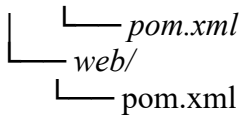
```
InputStream input =  
getClass().getClassLoader().getResourceAsStream("config.properties");
```

After build, check `target/classes/` → file should exist there.

7. Multi-Module and Parent Projects

Structure:

```
parent/  
├── pom.xml (packaging: pom)  
└── core/
```



- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

8. Executable JARs

Add to `pom.xml`:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.Main</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Run:

```
mvn package
java -jar target/myapp.jar
```

9. Building a WAR File

Create a Maven web project with structure:

```
src/main/webapp/
└─ WEB-INF/web.xml
```

Add:

```
<packaging>war</packaging>
```

Command:

mvn package

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.

LAB ACTION PLAN FOR WEEK 5

Objectives:

Students will be able to:

- Learn how to pull, run, stop, start, remove, and inspect containers and images.
- Gain the ability to create, monitor, and troubleshoot running containers.
- Configure and manage networks for container communication.
- Create and manage persistent storage for containers.
- Learn how to list, remove, and manage images efficiently.

Docker:

Docker is an open-source platform that automates the process of building, packaging, and running applications inside lightweight, portable containers, ensuring they run consistently across different environments.

Docker Images:

A read-only template that contains the application code, runtime, libraries, and dependencies. It is like a blueprint for creating containers.

Docker Containers:

A running instance of a Docker image.

Working with Docker CLI Commands:

Docker CLI Commands with redis

Step 1: Pull the redis Image

Command:

```
docker pull redis
```

```
Administrator: Windows PowerShell
-H, --host list          Daemon socket to connect to
-l, --log-level string   Set the logging level ("debug", "info",
                        "warn", "error", "fatal") (default "info")
--tls                   Use TLS; implied by --tlsverify
--tlscacert string       Trust certs signed only by this CA (default
                        "C:\\Users\\Asus\\.docker\\ca.pem")
--tlscert string         Path to TLS certificate file (default
                        "C:\\Users\\Asus\\.docker\\cert.pem")
--tlskey string          Path to TLS key file (default
                        "C:\\Users\\Asus\\.docker\\key.pem")
--tlsverify             Use TLS and verify the remote
-v, --version            Print version information and quit

Run 'docker COMMAND --help' for more information on a command.

For more help on how to use Docker, head to https://docs.docker.com/go/guides/

PS C:\\WINDOWS\\system32> docker --version
Docker version 27.4.0, build bde2b89
PS C:\\WINDOWS\\system32> docker pull redis
Using default tag: latest
latest: Pulling from library/redis
81bad6e5066: Pull complete
e51f68c71d8e: Pull complete
311d9cf20af5: Pull complete
f22261c94fe4: Pull complete
6f34ca96de3f: Pull complete
4f4fb708ef54: Pull complete
63edde0222ea: Pull complete
Digest: sha256:cc2dfb8f5151da2684b4a09bd04b567f92d07591d91980eb3eca21df07e12760
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\\WINDOWS\\system32>
```

Step 2: Run a Redis Container

Command:

```
docker run --name my-redis -d redis
```

What It Does:

- 🔧 Creates and starts a container named my-redis from the redis image.
- 🔧 The -d flag runs the container in the background.

```
Administrator: Windows PowerShell
311d9cf20af5: Pull complete
f22261c94fe4: Pull complete
6f34ca96de3f: Pull complete
4f4fb708ef54: Pull complete
63edde0222ea: Pull complete
Digest: sha256:cc2dfb8f5151da2684b4a09bd04b567f92d07591d91980eb3eca21df07e12760
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\\WINDOWS\\system32> docker run --name my-redis -d redis
ae502c13ea51eb33f620dc2cb6d7c6976edb431d99760c54fa832f560eca7faf
PS C:\\WINDOWS\\system32>
```

Step 3: Check Running Containers

Command:

docker ps

What It Does:

🖥️ Lists all running containers.

```
Administrator: Windows PowerShell
311d9cf28af5: Pull complete
f22261c94fe4: Pull complete
6f34ca96de3f: Pull complete
4f4fb700ef54: Pull complete
63edde0222ea: Pull complete
Digest: sha256:cc2dfb8f5151da2684b4a09bd004b567f92d07591d91980eb3eca21df07e12760
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\WINDOWS\system32> docker run --name my-redis -d redis
ae502c13ea51eb33f620dc2cb6d7c6976edb431d99760c54fa832f560eca7faf
PS C:\WINDOWS\system32> docker exec -it my-redis redis-cli
127.0.0.1:6379>
```

Step 4: Access Redis

Command:

docker exec -it my-redis redis-cli

Opens the Redis command-line tool (redis-cli) inside the container.

Example Redis Commands:

127.0.0.1:6379> SET name "Alice"

OK

127.0.0.1:6379> GET name

"Alice"

```
Administrator: Windows PowerShell
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> docker stop my-redis
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'my-redis'
127.0.0.1:6379> exit
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> _
```

```
Administrator: Windows PowerShell
(error) ERR wrong number of arguments for 'set' command
127.0.0.1:6379> SET name "Alice"
OK
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> _
```

Step 5: Stop the Redis Container

Command:

`docker stop my-redis`

What It Does:

🔴 Stops the Redis container but doesn't delete it.

```
Administrator: Windows PowerShell
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> docker stop my-redis
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'my-redis'
127.0.0.1:6379> exit
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> _
```

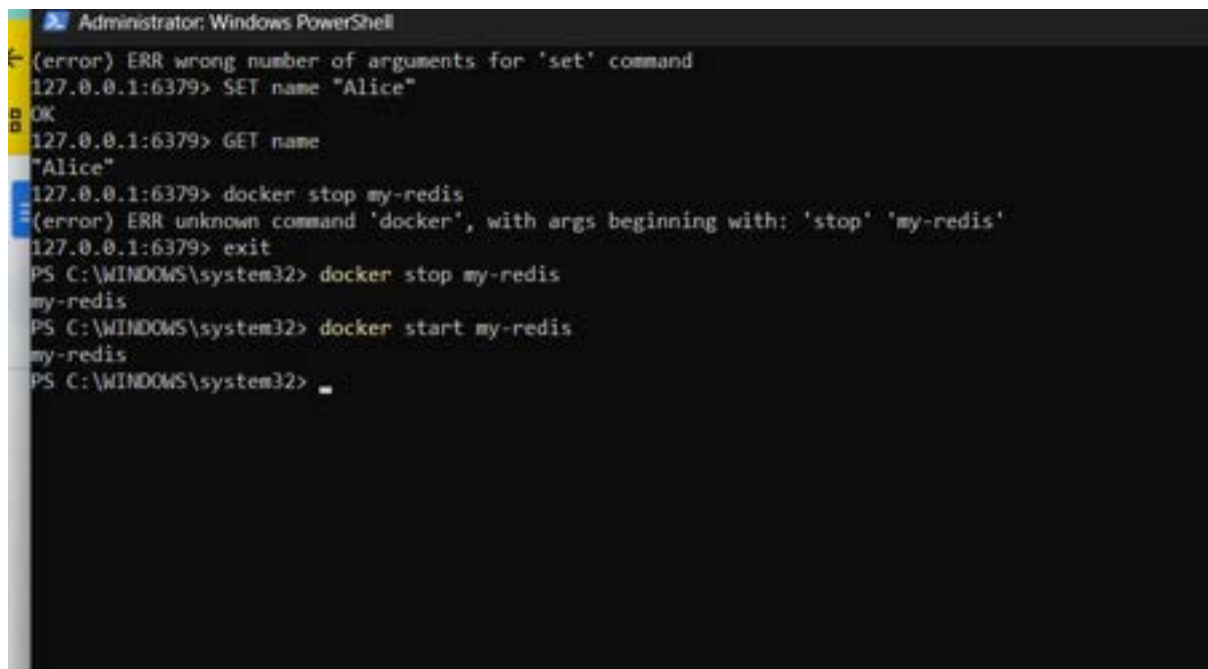
Step 6: Restart the Redis Container

Command:

`docker start my-redis`

What It Does:

🔧 Restarts the stopped container.



```
Administrator: Windows PowerShell
(error) ERR wrong number of arguments for 'set' command
127.0.0.1:6379> SET name "Alice"
OK
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> docker stop my-redis
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'my-redis'
127.0.0.1:6379> exit
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> docker start my-redis
my-redis
PS C:\WINDOWS\system32>
```

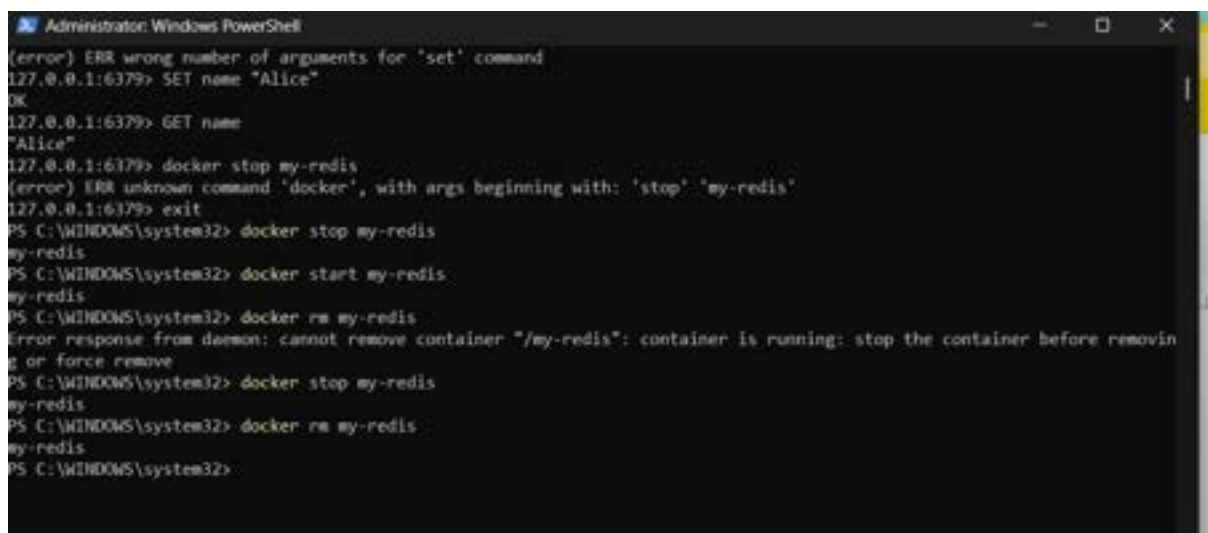
Step 7: Remove the Redis Container

Command:

`docker rm my-redis`

What It Does:

🔧 Deletes the container permanently.



```
Administrator: Windows PowerShell
(error) ERR wrong number of arguments for 'set' command
127.0.0.1:6379> SET name "Alice"
OK
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> docker stop my-redis
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'my-redis'
127.0.0.1:6379> exit
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> docker start my-redis
my-redis
PS C:\WINDOWS\system32> docker rm my-redis
Error response from daemon: cannot remove container "/my-redis": container is running: stop the container before removing or force remove
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> docker rm my-redis
my-redis
PS C:\WINDOWS\system32>
```

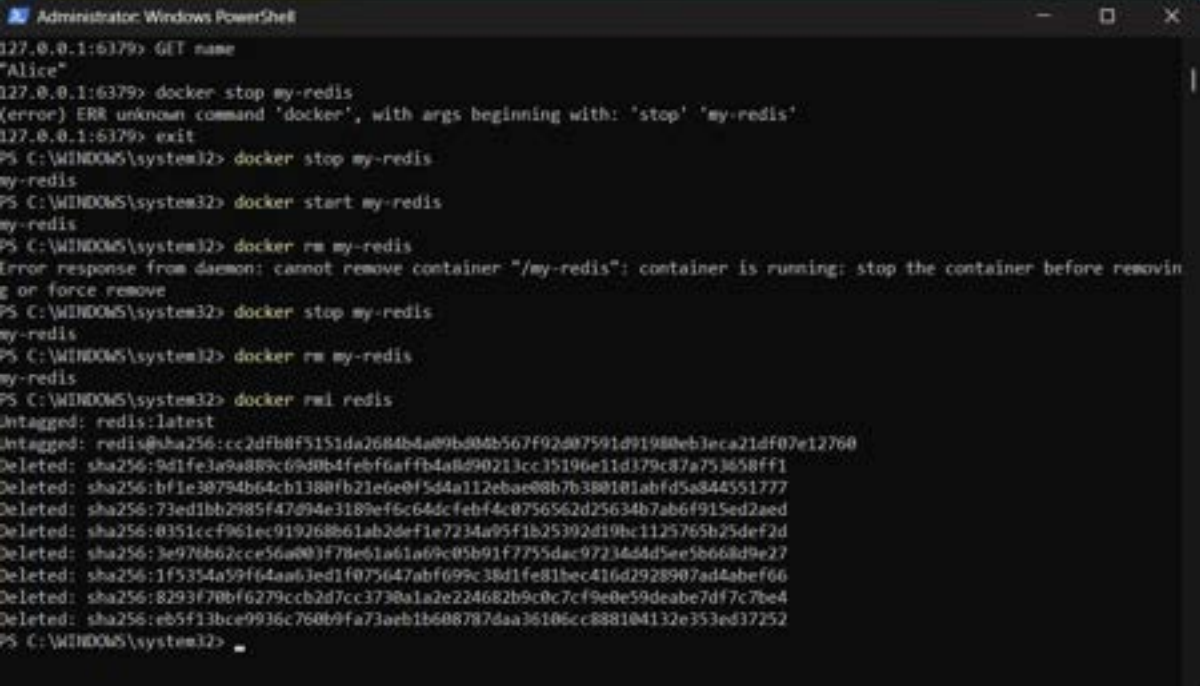
Step 8: Remove the Redis Image

Command:

```
docker rmi redis
```

What It Does:

🗑️ Deletes the Redis image from your local system.



```
Administrator: Windows PowerShell
127.0.0.1:6379> GET name
"Alice"
127.0.0.1:6379> docker stop my-redis
(error) ERR unknown command 'docker', with args beginning with: 'stop' 'my-redis'
127.0.0.1:6379> exit
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> docker start my-redis
my-redis
PS C:\WINDOWS\system32> docker rm my-redis
Error response from daemon: cannot remove container "/my-redis": container is running: stop the container before removing or force remove
PS C:\WINDOWS\system32> docker stop my-redis
my-redis
PS C:\WINDOWS\system32> docker rm my-redis
my-redis
PS C:\WINDOWS\system32> docker rmi redis
Untagged: redis:latest
Deleted: sha256:cc2dfb0f5151da2684b4a09b804b567f92d07591d91988eb3eca21df07e12760
Deleted: sha256:9d1fe3a9a889c69d0b4feb6affb4a8d90213cc35196e11d379c87a753658ff1
Deleted: sha256:bfe30794b64cb1380fb21e6e0f54a112ebae88b7b380101abfd5a844551777
Deleted: sha256:73ed1bb2985f47094e3189ef6c64dcfeb4c0756562d25634b7ab6f915ed2aed
Deleted: sha256:0351ccf961ec919268b61ab2def1e7234a95f1b25392d19bc1125765b25def2d
Deleted: sha256:3e978b62cce56a003f78e61a61a69c05b91f7755dac9723454d5ee5b668d9e27
Deleted: sha256:1f5354a59f64a63ed1f075647abf699c38d1fe81bec416d2928907ad4abef66
Deleted: sha256:8293f70bf6279ccb2d7cc3738a1a2e224682b9c0c7cf9e0e59deabe7df7c7be4
Deleted: sha256:eb5f13bce9936c760b9fa73aeb1b608787daa36106cc888104132e353ed37252
PS C:\WINDOWS\system32> _
```

2.Working with Docker file

A Docker file is a text file with instructions to create a custom Docker image.

Step 1: Set Up Your Folder

1. Windows:

- o Create a folder like C:\DockerProjects\Redis.

- o Open Git Bash and navigate to the folder:

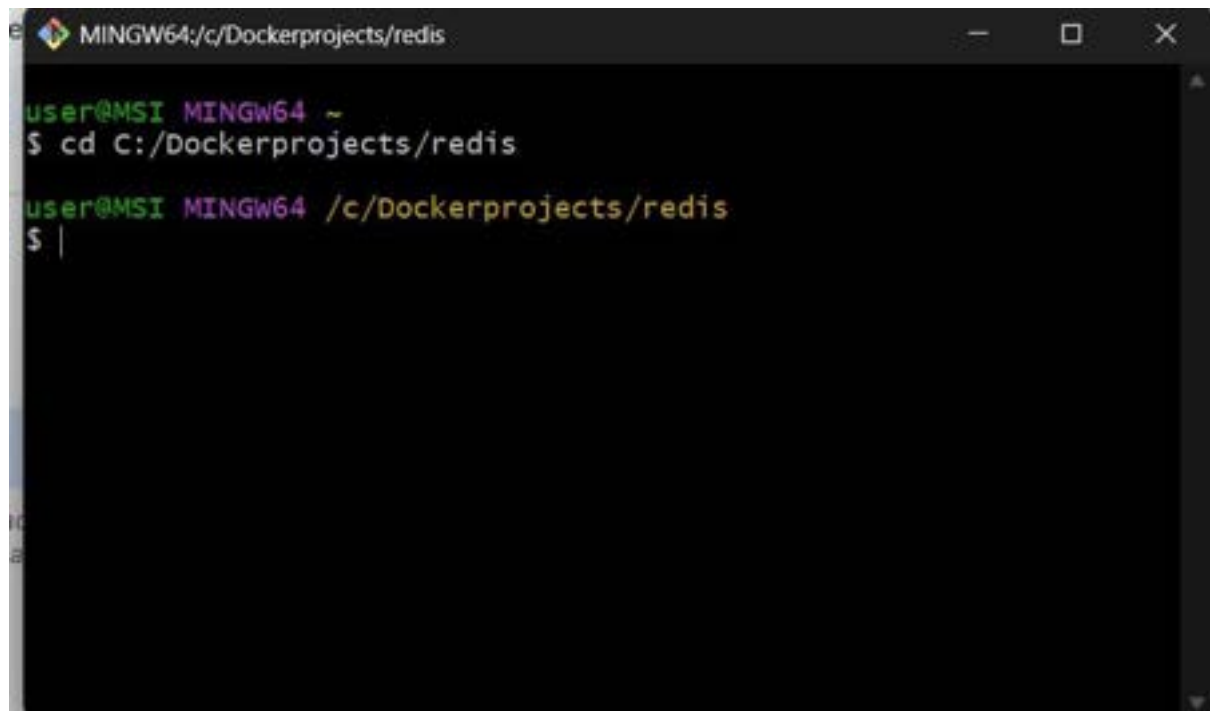
```
cd /c/DockerProjects/Redis
```

2. Mac/Linux:

- o Create a folder:

```
mkdir ~/DockerProjects/Redis
```

```
cd ~/DockerProjects/Redis
```



A terminal window titled 'MINGW64:/c/Dockerprojects/redis'. The prompt is 'user@MSI MINGW64 ~'. The first command entered is '\$ cd C:/Dockerprojects/redis'. The second command entered is '\$ |'.

```
user@MSI MINGW64 ~
$ cd C:/Dockerprojects/redis

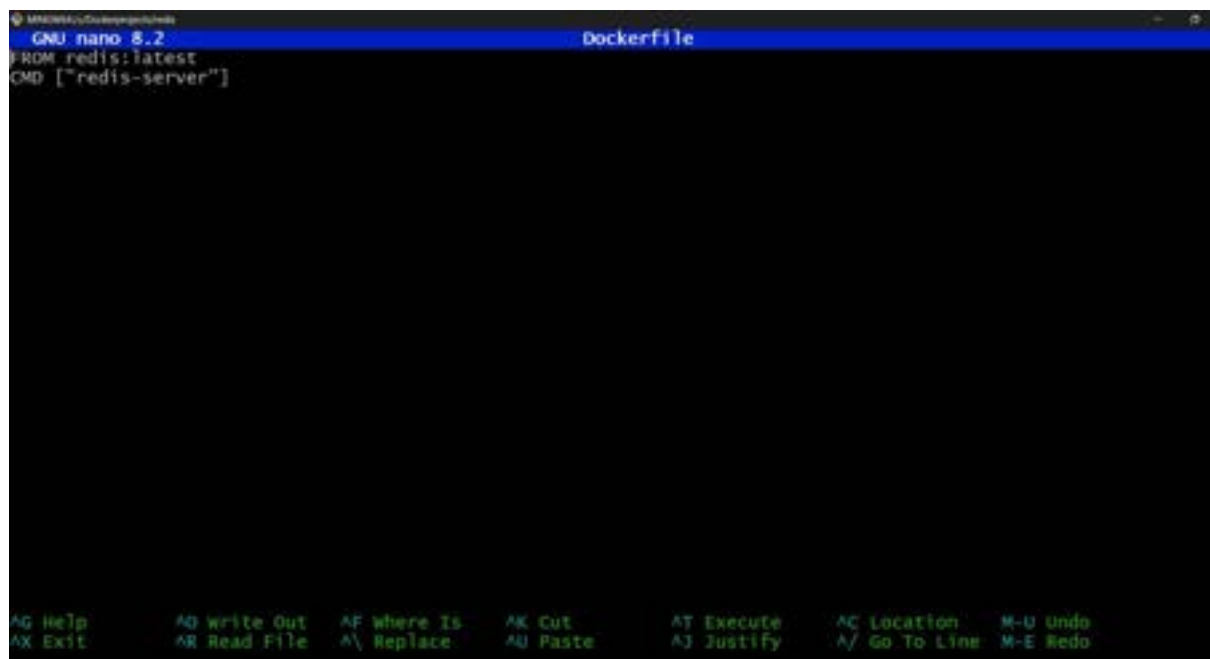
user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

Step 2: Write the Dockerfile

1. Inside the folder, create a file named Dockerfile (no extension).
2. Add the following content:

FROM redis:latest

CMD ["redis-server"]



A screenshot of the GNU nano 8.2 text editor editing a file named 'Dockerfile'. The content of the file is 'FROM redis:latest' followed by 'CMD ["redis-server"]' on the next line. The bottom status bar shows various keyboard shortcuts.

```
GNU nano 8.2 Dockerfile
FROM redis:latest
CMD ["redis-server"]

AG Help      AQ Write Out  AF Where Is   AK Cut        AT Execute    AC Location   M-U Undo
AX Exit      AR Read File  AL Replace    AU Paste      AJ Justify    AL Go To Line M-E Redo
```

Docker Commands (Step-by-step):

1. **docker build -t redisnew .**

What it does:

📦 This creates (builds) a Docker image using the recipe (Dockerfile) in the current folder (.).

📦 -t redisnew: Gives the image a name/tag ("redisnew"), so you can find it easily.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker build -t redisnew .
[+] Building 18.6s (5/5) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 76B                      0.0s
=> [internal] load metadata for docker.io/library/redis:1 3.9s
=> [internal] load .dockerignore                       0.0s
=> => transferring context: 2B                            0.0s
=> [1/1] FROM docker.io/library/redis:latest@sha256:cc2d 14.6s
=> => resolve docker.io/library/redis:latest@sha256:cc2df 0.0s
=> => sha256:311d9cf20af57b45fca01f6e264cac4a 876B / 876B 1.1s
=> => sha256:cc2dfb8f5151da2684b4a09bd0 10.22kB / 10.22kB 0.0s
=> => sha256:9d1fe3a9a889c69d0b4feb6aaffb 6.33kB / 6.33kB 0.0s
=> => sha256:e51f60c71d8e10e0555ce03c32fd 1.10kB / 1.10kB 0.7s
=> => sha256:a240ca6b723fb655f76edd7fbba6 2.29kB / 2.29kB 0.0s
=> => sha256:b1badc6e50664185acfaa0ca2 28.23MB / 28.23MB 12.9s
=> => sha256:f22261c94fe4aaa2fc0818ade2 24.19MB / 24.19MB 6.9s
=> => sha256:6f34ca96de3ff70793fb163e4f3b823438 97B / 97B 1.7s
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1 32B / 32B 2.7s
=> => sha256:63edde0222ea5d1f451341c58985 2.12kB / 2.12kB 3.6s
=> => extracting sha256:b1badc6e50664185acfaa0ca255d80760 1.1s
=> => extracting sha256:e51f60c71d8e10e0555ce03c32fd4407a 0.0s
=> => extracting sha256:311d9cf20af57b45fca01f6e264cac4a6 0.0s
=> => extracting sha256:f22261c94fe4aaa2fc0818ade274d1b98 0.3s
=> => extracting sha256:6f34ca96de3ff70793fb163e4f3b82343 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc 0.0s
=> => extracting sha256:63edde0222ea5d1f451341c589856bd7a 0.0s
=> exporting to image                                0.0s
=> => exporting layers                                  0.0s
=> => writing image sha256:f5b47042e2a5a960bc6bac4a2ef05f 0.0s
=> => naming to docker.io/library/redisnew              0.0s

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

2. docker run --name myredisnew -d redisnew

What it does:

📦 Starts a new container (mini computer) from the redisnew image.

📦 --name myredisnew: Names the container "myredisnew" so it's easy to identify.

📦 -d: Runs the container in the background.


```
MINGW64:/c/Dockerprojects/redis
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1 32B / 32B 2.7s
=> => sha256:63edde0222ea5d1f451341c58985 2.12kB / 2.12kB 3.6s
=> => extracting sha256:b1badc6e50664185acfaa0ca255d80760 1.1s
=> => extracting sha256:e51f60c71d8e10e0555ce03c32fd4407a 0.0s
=> => extracting sha256:311d9cf20af57b45fca01f6e264cac4a6 0.0s
=> => extracting sha256:f22261c94fe4aaa2fc0818ade274d1b98 0.3s
=> => extracting sha256:6f34ca96de3ff70793fb163e4f3b82343 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc 0.0s
=> => extracting sha256:63edde0222ea5d1f451341c589856bd7a 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:f5b47042e2a5a960bc6bac4a2ef05f 0.0s
=> => naming to docker.io/library/redisnew 0.0s

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker run --name myredisnew -d redisnew
e2e2d69decec51e8fffc808e0b2b333e96f8881b77b5c41671b83e6e19343318

user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

3. docker ps

What it does:

📖 Shows a list of containers that are running right now.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED
STATUS        PORTS     NAMES
e2e2d69decec   redisnew  "docker-entrypoint.s..." 26 seconds ago
Up 26 seconds  6379/tcp  myredisnew

user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

4. docker stop myredisnew

What it does:

📖 Stops the container named "myredisnew" (like turning off a computer).

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker stop myredisnew
myredisnew

user@MSI MINGW64 /c/Dockerprojects/redis
```

5. docker login

What it does:

📖 Logs you into your Docker Hub account, so you can upload images.


```
user@MSI MINGW64 /c/Dockerprojects/redis  
$ docker login  
Authenticating with existing credentials...  
Login Succeeded
```

6. docker ps -a

What it does:

- 🖨️ Shows a list of all containers, including stopped ones.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS         NAMES
e2e2d69decec   redisnew      "docker-entrypoint.s..." 10 minutes
ago          Exited (0)    myredisnew
8588a6892d40   ubuntu        "/bin/bash"              3 months a
go          Exited (255)  happy_nobel
b8d1ae283578   ubuntu        "/bin/bash"              3 months a
go          Exited (255)  confident_carson
f0e68f11eaa8   ubuntu        "/bin/bash"              3 months a
go          Exited (255)  mystifying_rubin
f2fc9ad71114   8a5693abca66 "/bin/bash"              3 months a
go          Exited (0)    intelligent_mayer
17985489d97a   8a5693abca66 "/bin/bash"              3 months a
go          Exited (0)    lucid_mahavira
2ab2e7ac2d6a   d11c13fac209 "/bin/bash"              3 months a
go          Exited (255)  elastic_aryabhata
f685ef32f244   58d643b91c96 "/bin/bash"              3 months a
go          Exited (255)  great_tharp
4141b686aecf   58d643b91c96 "/bin/bash"              3 months a
go          Exited (1)    quizzical_jang
29e8140a9f26   58d643b91c96 "/bin/bash"              4 months a
go          Exited (1)    optimistic_tharp
d2e76707a21d   58d643b91c96 "/bin/bash"              4 months a
go          Exited (255)  flamboyant_pare
1d5ee7758aac   58d643b91c96 "/bin/bash"              4 months a
go          Exited (0)    boring_tesla
f9a8b63be0d9   ubuntu        "/bin/bash"              4 months a
go          Exited (127)  quizzical_khorana
0d08d43b7fbb   58d643b91c96 "/bin/bash"              4 months a
go          Exited (0)    trusting_moser
240a163e91c8   58d643b91c96 "/bin/bash"              4 months a
go          Exited (0)    bold_chebyshev
6d2872e4e3aa   58d643b91c96 "/bin/bash"              4 months a
go          Exited (0)    serene_chandrasekh
ar
7e46e1969cd2   9b24e6f54c27 "/bin/bash"              4 months a
go          Exited (0)    sad_golick
a99350d09dea   9b24e6f54c27 "/bin/bash"              4 months a
go          Exited (0)    charming_agnesi
55a367ad63bc   0de6c07bff58 "/bin/bash"              4 months a
go          Exited (0)    relaxed_yalow
fecf5cd1f362   a0c3eaf8c629 "/bin/bash"              4 months a
go          Exited (0)    brave_lederberg
d1d062568223   a0c3eaf8c629 "/bin/bash"              4 months a
go          Exited (0)    quizzical_swanson
ab8b59225fb4   7d9df11d39f0 "/bin/bash"              4 months a
go          Exited (0)    affectionate_satos
hi
```

7. docker commit 0e993d2009a1 budarajumadhurika/redis1

What it does:

🔧 Takes a snapshot (saves changes) of the container with ID 0e993d2009a1 and creates a new image called budarajumadhurika/redis1.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker commit e2e2d69decec kannarohith369/redis
sha256:dee473af92a5449ec8dad7a308abf96e2076fc25adf0e996195e976ea7a1d607

user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

8. docker images

What it does:

🔧 Lists all images saved on your system.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
kannarohith369/redis latest      dee473af92a5  35 seconds ago 137MB
redisnew             latest      f5b47042e2a5  13 hours ago   137MB
ubuntuos             latest      ba0725824e67  3 months ago   347MB
<none>               <none>      0dea92c08728  5 months ago   347MB
<none>               <none>      6a51c5ed6fdf  5 months ago   347MB
<none>               <none>      86d36e5d4bac  5 months ago   347MB
<none>               <none>      75be11f731f5  5 months ago   347MB
linux1               latest      0d3955e89158  5 months ago   331MB
<none>               <none>      57ff21045edb  5 months ago   331MB
<none>               <none>      85107a5f3881  5 months ago   341MB
ubuntu               latest      a04dc4851cbc  6 months ago   78.1MB
```

9. docker push budarajumadhurika/redis1

What it does:

🔧 Uploads the image budarajumadhurika/redis1 to Docker Hub, so others can download it.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker push kannarohith369/redis
Using default tag: latest
The push refers to repository [docker.io/kannarohith369/redis]
803d653c7ad7: Mounted from library/redis
5f70bf18a086: Mounted from library/redis
cfb4bd9f69a7: Mounted from library/redis
7f32c228fb8a: Mounted from library/redis
0256a7df0b75: Mounted from library/redis
5b34e7b7ec41: Mounted from library/redis
eb5f13bce993: Mounted from library/redis
latest: digest: sha256:ace3d716af5c57aa3441219bcf1ed67b72337a80b1f82cd13
d3875b1059b8ea7 size: 1776

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

10. docker rm 0e993d2009a1

What it does:

📺 Deletes the container with ID 0e993d2009a1.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm kannarohith369/redis
Error response from daemon: No such container: kannarohith369/redis

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm kannarohith369/redis
Error response from daemon: No such container: kannarohith369/redis

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm e2e2d69dece
e2e2d69dece

user@MSI MINGW64 /c/Dockerprojects/redis
```

11. docker rmi budarajumadhurika/redis1

What it does:

📺 Deletes the image budarajumadhurika/redis1 from your system.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker pull kannarohith369/redis
Using default tag: latest
latest: Pulling from kannarohith369/redis
Digest: sha256:ace3d716af5c57aa3441219bcf1ed67b72337a80b1f82cd13d3875b10
59b8ea7
Status: Image is up to date for kannarohith369/redis:latest
docker.io/kannarohith369/redis:latest

user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

12. docker ps -a

What it does:

Shows all containers again to confirm changes.


```

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS         NAMES
e2e2d69decec   redisnew      "docker-entrypoint.s..." 10 minutes ago
Exited (0) 4 minutes ago    myredisnew
8588a6892d40   ubuntuos     "/bin/bash"              3 months ago
Exited (255) 38 minutes ago    happy_nobel
b8d1ae283578   ubuntuos     "/bin/bash"              3 months ago
Exited (255) 38 minutes ago    confident_carson
f0e68f11eaa8   ubuntu       "/bin/bash"              3 months ago
Exited (255) 38 minutes ago    mystifying_rubin
f2fc9ad71114   8a5693abca66 "/bin/bash"              3 months ago
Exited (0) 3 months ago      intelligent_mayer
17985489d97a   8a5693abca66 "/bin/bash"              3 months ago
Exited (0) 3 months ago      lucid_mahavira
2ab2e7ac2d6a   d11c13fac209 "/bin/bash"              3 months ago
Exited (255) 3 months ago    elastic_aryabhata
f685ef32f244   58d643b91c96 "/bin/bash"              3 months ago
Exited (255) 3 months ago    great_tharp
4141b686aecf   58d643b91c96 "/bin/bash"              3 months ago
Exited (1) 3 months ago      quizzical_jang
29e8140a9f26   58d643b91c96 "/bin/bash"              4 months ago
Exited (1) 4 months ago      optimistic_tharp
d2e76707a21d   58d643b91c96 "/bin/bash"              4 months ago
Exited (255) 4 months ago    flamboyant_pare
1d5ee7758aac   58d643b91c96 "/bin/bash"              4 months ago
Exited (0) 4 months ago      boring_tesla
f9a8b63be0d9   ubuntu       "/bin/bash"              4 months ago
Exited (127) 4 months ago    quizzical_khorana
0d08d43b7fbb   58d643b91c96 "/bin/bash"              4 months ago
Exited (0) 4 months ago      trusting_moser
240a163e91c8   58d643b91c96 "/bin/bash"              4 months ago
Exited (0) 4 months ago      bold_chebyshev
6d2872e4e3aa   58d643b91c96 "/bin/bash"              4 months ago
Exited (0) 4 months ago      serene_chandrasekh
ar
7e46e1969cd2   9b24e6f54c27 "/bin/bash"              4 months ago
Exited (0) 4 months ago      sad_golick
a99350d09dea   9b24e6f54c27 "/bin/bash"              4 months ago
Exited (0) 4 months ago      charming_agnesi
55a367ad63bc   0de6c07bff58 "/bin/bash"              4 months ago
Exited (0) 4 months ago      relaxed_yalow
fecf5cd1f362   a0c3eaf8c629 "/bin/bash"              4 months ago
Exited (0) 4 months ago      brave_lederberg
d1d062568223   a0c3eaf8c629 "/bin/bash"              4 months ago
Exited (0) 4 months ago      quizzical_swanson
ab8b59225fb4   7d9df11d39f0 "/bin/bash"              4 months ago
Exited (0) 4 months ago      affectionate_satos
hi

```

13. docker logout

What it does:

- 📌 Logs you out of Docker Hub.

14. docker pull budarajumadhurika/redis1

What it does:

- 📦 Downloads the image budarajumadhurika/redis1 from Docker Hub.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker pull kannarohith369/redis
Using default tag: latest
latest: Pulling from kannarohith369/redis
Digest: sha256:ace3d716af5c57aa3441219bcf1ed67b72337a80b1f82cd13d3875b10
59b8ea7
Status: Image is up to date for kannarohith369/redis:latest
docker.io/kannarohith369/redis:latest

user@MSI MINGW64 /c/Dockerprojects/redis
$ |
```

15. docker run --name myredis -d budarajumadhurika/redis1

What it does:

- 📦 Starts a new container using the image budarajumadhurika/redis1.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker run --name myredis -d kannarohith369/redis
bfff37deb78a0d7a749b4a68c0dbfa979939cc967a4d096ad60391fb04ea61558

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

16. docker exec -it myredis redis-cli

What it does:

- 📦 Opens the Redis command-line interface (like a terminal) inside the running container myredis.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> |
```

17. SET name "Abcdef"

What it does:

- 📦 Saves a key-value pair in Redis (key = name, value = Abcdef).

18. GET name

What it does:

- 📦 Retrieves the value of the key name from Redis (it will return "Abcdef").

19. exit

What it does:

🖥️ Exits the Redis CLI.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "rohith"
OK
127.0.0.1:6379> GET name
"rohith"
127.0.0.1:6379> exit

user@MSI MINGW64 /c/Dockerprojects/redis
$
```


20. docker ps -a

What it does:

- 🖨️ Shows all containers again to check their status.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND PORTS NAMES	CREATED	STATUS
8588a6892d40	ubuntuos	"/bin/bash"	3 months ago	Exited
(255) About an hour ago		happy_nobel		
b8d1ae283578	ubuntuos	"/bin/bash"	3 months ago	Exited
(255) About an hour ago		confident_carson		
f0e68f11eaa8	ubuntu	"/bin/bash"	3 months ago	Exited
(255) About an hour ago		mystifying_rubin		
f2fc9ad71114	8a5693abca66	"/bin/bash"	3 months ago	Exited
(0) 3 months ago		intelligent_mayer		
17985489d97a	8a5693abca66	"/bin/bash"	3 months ago	Exited
(0) 3 months ago		lucid_mahavira		
2ab2e7ac2d6a	d11c13fac209	"/bin/bash"	3 months ago	Exited
(255) 3 months ago		elastic_aryabhata		
f685ef32f244	58d643b91c96	"/bin/bash"	3 months ago	Exited
(255) 3 months ago		great_tharp		
4141b686aecf	58d643b91c96	"/bin/bash"	3 months ago	Exited
(1) 3 months ago		quizzical_jang		
29e8140a9f26	58d643b91c96	"/bin/bash"	4 months ago	Exited
(1) 4 months ago		optimistic_tharp		
d2e76707a21d	58d643b91c96	"/bin/bash"	4 months ago	Exited
(255) 4 months ago		flamboyant_pare		
1d5ee7758aac	58d643b91c96	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		boring_tesla		
f9a8b63be0d9	ubuntu	"/bin/bash"	4 months ago	Exited
(127) 4 months ago		quizzical_khorana		
0d08d43b7fbb	58d643b91c96	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		trusting_moser		
240a163e91c8	58d643b91c96	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		bold_chebyshev		
6d2872e4e3aa	58d643b91c96	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		serene_chandrasekhar		
7e46e1969cd2	9b24e6f54c27	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		sad_golick		
a99350d09dea	9b24e6f54c27	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		charming_agnesi		
55a367ad63bc	0de6c07bff58	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		relaxed_yalow		
fecf5cd1f362	a0c3eaf8c629	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		brave_lederberg		
d1d062568223	a0c3eaf8c629	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		quizzical_swanson		
ab8b59225fb4	7d9df11d39f0	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		affectionate_satoshi		
eef07383f792	7d9df11d39f0	"/bin/bash"	4 months ago	Exited
(0) 4 months ago		fervent_williams		

21. docker stop myredis

What it does:

📄 Stops the container myredis.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker stop myredis
myredis
```

22. docker rm 50a6e4a9c326

What it does:

📄 Deletes the container with ID 50a6e4a9c326.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm bff37debf8a0
bff37debf8a0
```

23. docker images

What it does:

📄 Lists all images again to confirm which ones remain.

```
$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
kannarohith369/redis latest       dee473af92a5      9 minutes ago   137MB
redisnew             latest       f5b47042e2a5      14 hours ago    137MB
ubuntuos             latest       ba0725824e67      3 months ago    47MB
<none>               <none>       0dea92c08728      5 months ago    47MB
<none>               <none>       6a51c5ed6fdf      5 months ago    47MB
<none>               <none>       86d36e5d4bac      5 months ago    47MB
<none>               <none>       75be11f731f5      5 months ago    47MB
linux1               latest       0d3955e89158      5 months ago    31MB
<none>               <none>       57ff21045edb      5 months ago    31MB
<none>               <none>       85107a5f3881      5 months ago    41MB
ubuntu               latest       a04dc4851cbc      6 months ago    8.1MB

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

24. docker rmi budarajumadhurika/redis1

What it does:

🗑️ Deletes the image budarajumadhurika/redis1 again.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rmi kannarohith369/redis
Untagged: kannarohith369/redis:latest
Untagged: kannarohith369/redis@sha256:ace3d716af5c57aa3441219bcf1
ed67b72337a80b1f82cd13d3875b1059b8ea7
Deleted: sha256:dee473af92a5449ec8dad7a308abf96e2076fc25adf0e9961
95e976ea7a1d607

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

Step 3: Remove Login Credentials (Optional)

If you no longer need to be logged in, you can log out:

docker logout

What It Does:

🗑️ Logs you out from Docker Hub and removes your stored credentials.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker logout
Removing login credentials for https://index.docker.io/v1/

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

Conclusion:

Through this hands-on exploration of Docker commands and Dockerfile usage, students will get a clear picture about the end-to-end container workflow—from pulling images, creating and running containers, to customizing them with Dockerfiles. By practicing key commands such as `docker build`, `docker run`, `docker ps`, and `docker exec`, students will gain practical skills in managing containerized environments. The Dockerfile exercise reinforced the importance of layered builds, efficient image creation, and environment reproducibility. Overall, these activities strengthened the understanding of containerization principles, enabling building, testing and deploying applications in isolated, consistent environments.

LAB ACTION PLAN FOR WEEK 6

Objectives:

Students will be able to:

- Learn how to define and run multiple interdependent services (e.g., web server, database) in a single configuration file.
- Gain skills in writing and interpreting docker-compose.yml files for service setup.
- Deploy the same setup across different machines without manual configuration.
- Configure container networking and persistent storage within Compose.
- Reduce setup time and enable faster iteration during application development.

Students will perform Hands-on running multi container applications using docker compose and make a document that includes the execution of each step along with scenario-based question solutions after completing the tasks.

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications. It allows you to define services, networks, and volumes that your application needs, all in a single file. This makes it easier to manage complex applications that require multiple containers (e.g., a web server and a database).

Structure of a Docker Compose File

A Docker Compose file is written in YAML (YAML Ain't Markup Language), a human-readable data format used for configuration. In the Docker context, it helps define the services, networks, and volumes needed for an application.

Basic Components of a Docker Compose YAML File

1. **Version:** Specifies which version of the Docker Compose file format you're using. Each version comes with its own set of features.

```
version:'3.8'
```

2. **Services:** This section defines all the containers you want to run for your application. Each service represents a container (like a web server or database).

Each service includes:

- **Image:** The Docker image to use.
- **Ports:** Ports to map between the container and your host machine.
- **Environment:** Environment variables required by the service.
- **Dependencies:** Which other services a container depends on.

Example:

```
services:
web:
image:nginx
ports:
-"8060:80"
db:
image:tomee
ports:
-"8050:8080"
```

3. **Networks:** Define networks to allow different services to communicate with each other. Docker Compose automatically creates a default network if not specified.
4. **Volumes:** Used for persistent storage, allowing data to persist even after the container stops or is removed.

Example Docker Compose File (Simple)

Here's a basic example of a Docker Compose file that runs WordPress and MySQL together:

```
version: '3.8' # Docker Compose file format version

services:
wordpress: # WordPress service
image: wordpress:latest
ports:
- "8080:80" # Map port 80 of the container to port 8080 of the host
environment:
WORDPRESS_DB_HOST: db:3306 # Database host
WORDPRESS_DB_USER: wordpress
WORDPRESS_DB_PASSWORD: wordpress
WORDPRESS_DB_NAME: wordpress
depends_on:
```

- db # Ensures the db service starts first

```
db: # MySQL service
  image: mysql:5.7
  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: wordpress
    MYSQL_USER: wordpress
    MYSQL_PASSWORD: wordpress
```

How to Run It

To run a multi-container setup like the one above:

- 1. Save the file as docker-compose.yml.**

Or

docker-compose.yaml

- 2. To Start the compose**

docker-compose up -d

- 3. To stop the containers**

docker-compose down

- 4. To scale the container**

docker-compose up --scale<service name>=2 -d

1. Define and run multiple interdependent services

Task:

- I. Create a new folder compose-lab**

Inside it, create a file docker-compose.yml with the following content:

```
version: "3.9"
```

```
services:
```

```
  web:
```


image: nginx:latest

ports:

- "8080:80"

db:

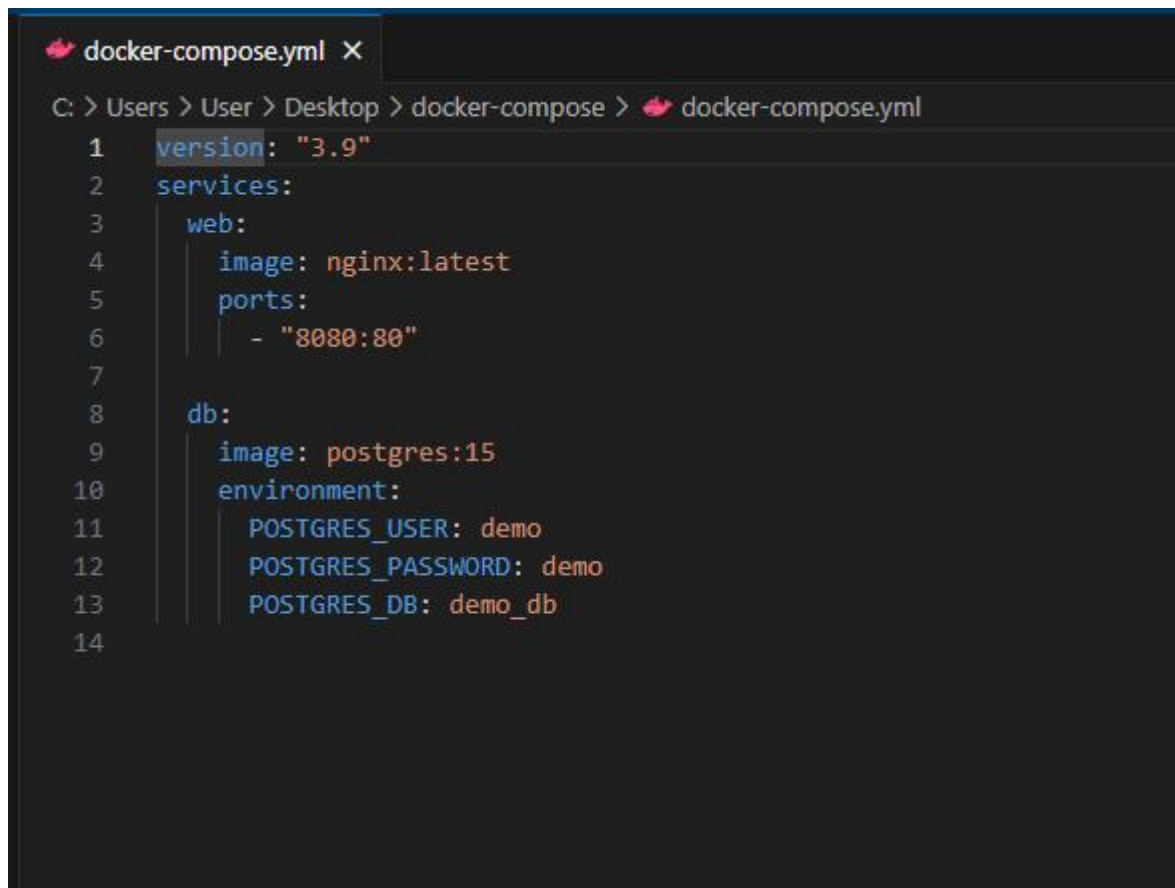
image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

POSTGRES_DB: demo_db



```
docker-compose.yml X
C: > Users > User > Desktop > docker-compose > docker-compose.yml
1  version: "3.9"
2  services:
3    web:
4      image: nginx:latest
5      ports:
6        - "8080:80"
7
8    db:
9      image: postgres:15
10     environment:
11       POSTGRES_USER: demo
12       POSTGRES_PASSWORD: demo
13       POSTGRES_DB: demo_db
14
```

II.Run the setup:

docker compose up -d

III.Open your browser and visit: <http://localhost:8080>.

IV.Expected Output:

Nginx welcome page is displayed.

db container runs in the background.



2.Write and interpret docker-compose.yml files

Task:

I. Modify docker-compose.yml to add a Redis cache:

redis:

image: redis:alpine

II.Add a depends_on so web waits for Redis:

web:

image: nginx:latest

ports:

- "8080:80"

depends_on:

- redis

```
docker-compose.yml X
C: > Users > User > Desktop > docker-compose > docker-compose.yml
1  version: "3.9"
2  services:
3    web:
4      image: nginx:latest
5      ports:
6      - "8081:80"
7      depends_on:
8      - redis
9
10   redis:
11     image: redis:alpine
12
13   db:
14     image: postgres:15
15     environment:
16       POSTGRES_USER: demo
17       POSTGRES_PASSWORD: demo
18       POSTGRES_DB: demo_db
19
```

III.Restart the setup:

docker compose up -d

```
C:\Users\User\Desktop\docker-compose>docker compose up -d
time="2025-08-26T11:23:34+05:30" level=warning msg="C:\Users\User\Desktop\docker-compose\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
   ✔ Network docker-compose_default Created
   ✔ Container docker-compose-db-1 Started
   ✔ Container docker-compose-redis-1 Started
   ✔ Container docker-compose-web-1 Started
```

docker compose ps

```
C:\Users\User\Desktop\docker-compose>docker compose ps
time="2025-08-26T11:23:34+05:30" level=warning msg="C:\Users\User\Desktop\docker-compose\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
   ✔ redis Pulled
     61cf9b0ee7f3 Download complete
     18a08e15d3 Download complete
     1f79d4c62d4 Download complete
     444f979e9f5d Already exists
     98a081f937a Download complete
     f1ea0d4450a Download complete
     9824c27679d3 Download complete
[+] Running 3/3
   ✔ Container docker-compose-db-1 Running
   ✔ Container docker-compose-redis-1 Started
   ✔ Container docker-compose-web-1 Running

NAME                IMAGE              COMMAND                  SERVICE     CREATED       status       PORTS
docker-compose-db-1 postgres:15         "docker-entrypoint.sh"   db          7 minutes ago Up 7 minutes 5432/tcp
docker-compose-redis-1 redis:alpine        "docker-entrypoint.sh"   redis       28 seconds ago Up 28 seconds 6379/tcp
docker-compose-web-1 nginx:latest        "docker-entrypoint..." web         2 minutes ago Up 2 minutes 0.0.0.0:8081->80/tcp
```

IV.Expected Output:

Three services (web, db, redis) are listed as running.

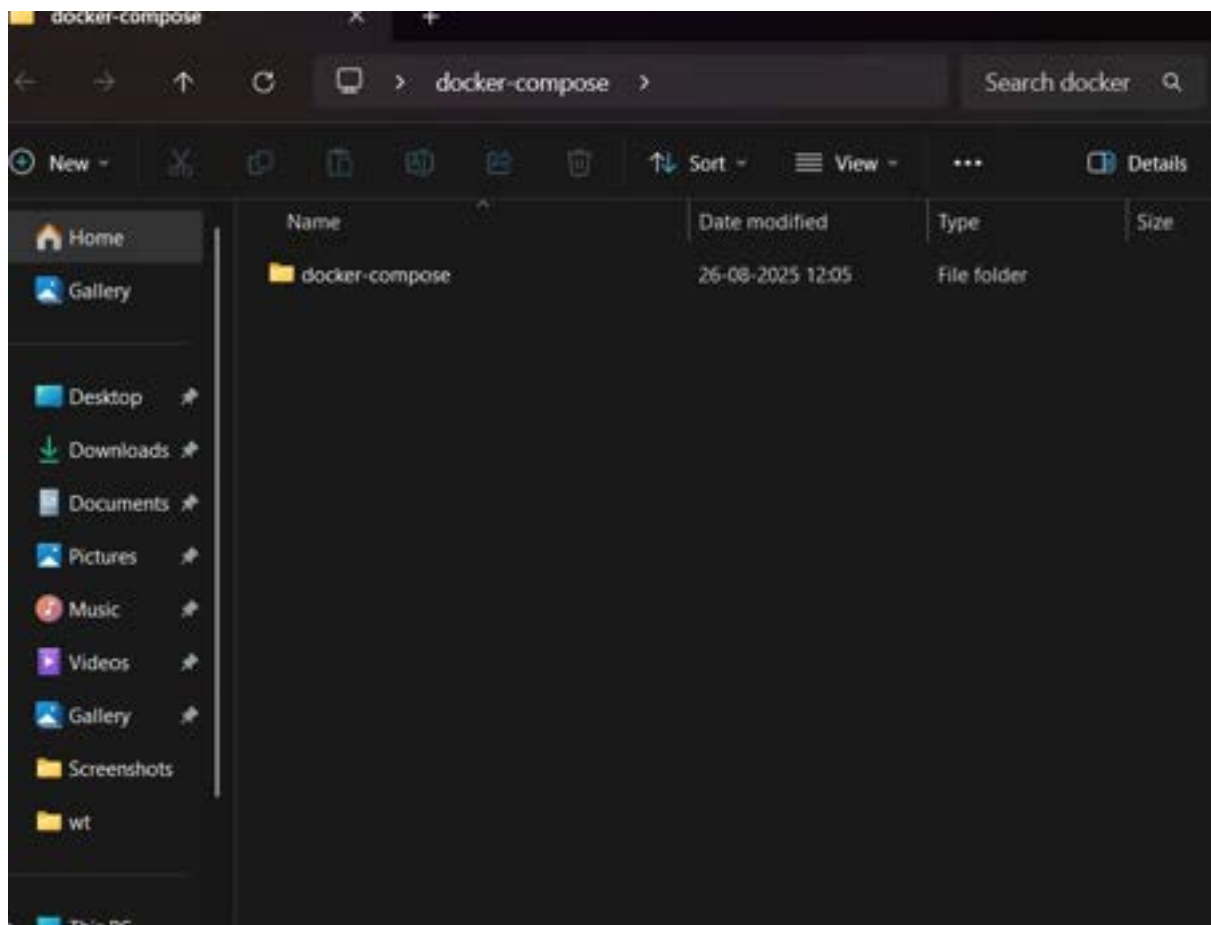
```
C:\Users\User\Desktop\docker-compose>docker compose ps
time="2025-08-26T11:28:53+05:30" level=warning msg="C:\Users\User\Desktop\docker-compose\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE              COMMAND                  SERVICE   CREATED      STATUS      PORTS
docker-compose-db-1 postgres:15        "docker-entrypoint.sh"  db        38 seconds ago  Up 11 seconds  5432/tcp
docker-compose-redis-1 redis:alpine        "docker-entrypoint.sh"  redis     38 seconds ago  Up 11 seconds  6379/tcp
docker-compose-web-1 nginx:latest        "/docker-entrypoint..." web        23 seconds ago  Up 9 seconds   0.0.0.0:8080->80/tcp
```

3.Deploy across different machines

Task:

I. Zip your compose-lab folder.

Transfer it to another machine with Docker Compose installed.



II.Run:

docker compose up -d

Check that Nginx and Postgres work there as well.

```

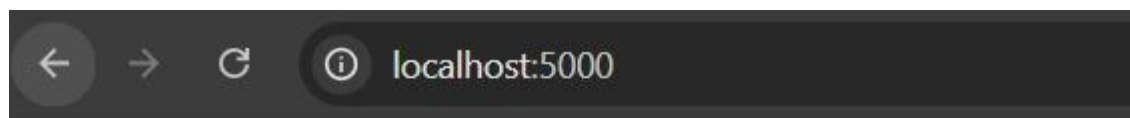
✓5d4d240bc6e Pull complete
✓ba7836bb8a68 Pull complete
✓799f859abbbd Pull complete
✓d981e777080c Pull complete
✓94adcb462801 Pull complete
✓5c10925b29ff Pull complete
[*] Running 8/1.8s (4/8)
[*] Running 8/1 Building
[*] Building 27.4s (7/8)
[*] Building 28.2s (10/10) FINISHED
[web internal] load build definition from Dockerfile
=> => transferring dockerfile: 142B
[web internal] load metadata for docker.io/library/python:3.10-slim
[web internal] load .dockerignore
=> => transferring context: 2B
[web 1/4] FROM docker.io/library/python:3.10-slim@sha256:420fbb0e468d3eaf0f7e93eaf7a48792cbcad39d43ac95b96bee2a
=> resolve docker.io/library/python:3.10-slim@sha256:420fbb0e468d3eaf0f7e93eaf7a48792cbcad39d43ac95b96bee2a
=> sha256:7732878445d9e71f91ce50493915297ccalbd6392045d9dccc0f378a200967bf 1.29MB / 1.29MB
=> sha256:72e8e193aa9ad19c7f1bcb09737a83d1906bcc1e51965c2873f081eb07bd3a8 14.18MB / 14.18MB
=> sha256:3a195ff1e16155a2ca71ee2cc2c4e467119c644d8360b7c2f6e6d963349358b 250B / 250B
=> sha256:420fbb0e468d3eaf0f7e93eaf7a48792cbcad39d43ac95b96bee2a 10.37kB / 10.37kB
=> sha256:c30c37eb1623e9086442038c1273a7eff89cfff44d7c2ece721fa61560af7e1f 1.73kB / 1.73kB
=> sha256:c2c9cfb78b6af7fa4408488d4bd0482017fa3bbc3571025c19b53d66676fc8 5.38kB / 5.38kB
=> extracting sha256:7732878445d9e71f91ce50493915297ccalbd6392045d9dccc0f378a200967bf
=> extracting sha256:72e8e193aa9ad19c7f1bcb09737a83d1906bcc1e51965c2873f081eb07bd3a8
=> extracting sha256:3a195ff1e16155a2ca71ee2cc2c4e467119c644d8360b7c2f6e6d963349358b
[web internal] load build context
=> => transferring context: 221B
[web 2/4] WORKDIR /app
[web 3/4] COPY app.py /app/
[web 4/4] RUN pip install flask
[web] exporting to image
=> => exporting layers
=> => writing image sha256:4fda6735cc10585e46a42c8b8a2c9086942cbc4d1dbc287e5e3381022f92ebd
[*] Running 8/0 docker.io/library/docker-compose-web
✓Service web Built
✓Network docker-compose_app-net Created
✓Network docker-compose_default Created
✓Volume "docker-compose_db-data" Created
✓Container docker-compose-db-1 Started
✓Container docker-compose-web-1 Started

```

III.Expected Output:

The same services run on the new machine without changes.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
docker-compose-db-1	postgres:15	"docker-entrypoint.s..."	db	About a minute ago	Up About a minute	5432/tcp
docker-compose-web-1	docker-compose-web	"python app.py"	web	About a minute ago	Up About a minute	0.0.0.0:5000->5000/tcp



Hello Docker Compose!

4.Networking and persistent storage

Task:

- I. Update your docker-compose.yml to add a custom network and volume:

networks:

app-net:

volumes:

db-data:

services:

web:

image: nginx:latest

ports:

- "8080:80"

networks:

- app-net

depends_on:

- db

db:

image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

POSTGRES_DB: demo_db

volumes:

- db-data:/var/lib/postgresql/data

networks:

- app-net

```
docker-compose.yml
C: > Users > User > Desktop > docker-compose > docker-compose.yml

1  services:
2    web:
3      image: nginx:latest
4      ports:
5        - "8081:80"
6      networks:
7        - app-net
8      depends_on:
9        - db
10
11   db:
12     image: postgres:15
13     environment:
14       POSTGRES_USER: demo
15       POSTGRES_PASSWORD: demo
16       POSTGRES_DB: demo_db
17     volumes:
18       - db-data:/var/lib/postgresql/data
19     networks:
20       - app-net
21
22   networks:
23     app-net:
24
25   volumes:
26     db-data:
27
```

II.Run:

docker compose up -d

```
[*] Running @/desktop/docker-compose/docker compose up -d
[*] Creating network docker-compose_app-net
[*] Starting 4/18733136137495/30" level=warning msg="Found orphan containers ([docker-compose-radius-1]) for this project. If you removed or renamed this service in your
[*] Network docker-compose_app-net Created 0.11s
[*] Volume "docker-compose_db-data" Created 0.04s
[*] Container docker-compose-web-1 Started 0.15s
[*] Container docker-compose-db-1 Started 0.02s
```

III.Insert some data into Postgres (optional with psql).

```
C:\Users\User\Desktop\docker-compose>docker compose exec db psql -U demo -d demo_db
psql (15.14 (Debian 15.14-1.pgdg13+1))
Type "help" for help.

demo_db=# CREATE TABLE test (id SERIAL PRIMARY KEY, name VARCHAR(50));
CREATE TABLE
demo_db=# INSERT INTO test (name) VALUES ('Sample data');
INSERT 0 1
demo_db=# SELECT * FROM test;
 id |      name
-----+-----
  1 | Sample data
(1 row)

demo_db=# \q
```

IV.Remove containers:

docker compose down

```
[+] Running 3/3esktop\docker-compose>docker compose down
[+] Container docker-compose-web-1 Removed
[+] Container docker-compose-db-1 Removed
[+] Network docker-compose_app-net Removed
```

V.Start again:

docker compose up -d

```
C:\Users\User\Desktop\docker-compose>docker compose up -d
[+] Running 6/6
  Network docker-compose_app-net Creating
  [+] Running 5/5
    Network 0.0.0.0:4134->4134/tcp redis=redis Found orphan containers ([docker-compose-redis-1]) for this project. If you removed or renamed this service in your
    Network docker-compose_app-net Created
    Container docker-compose-db-1 Started
    Container docker-compose-web-1 Started
```

VI.Expected Output:

Database data persists across restarts.

Services communicate via the app-net network using service names.

```
C:\Users\User\Desktop\docker-compose>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
docker-compose-db-1 postgres:15          "docker-entrypoint.s..." db         About a minute ago Up About a minute 5432/tcp
docker-compose-redis-1 redis:alpine         "docker-entrypoint.s..." redis      14 minutes ago Up 14 minutes 6379/tcp
docker-compose-web-1 nginx:latest         "/docker-entrypoint..." web        About a minute ago Up About a minute 0.0.0.0:8081->80/tcp
```

5.Faster iteration during development

Task:

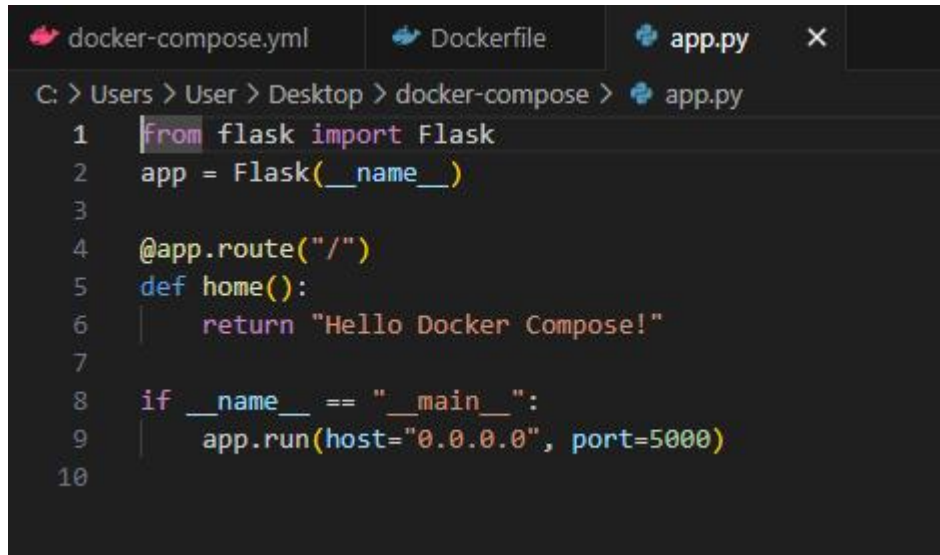
I. Create a simple Flask app in app.py:

from flask import Flask

app = Flask(__name__)

@app.route("/")


```
def home():  
    return "Hello from Flask + Docker!"  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

A screenshot of a code editor with three tabs: 'docker-compose.yml', 'Dockerfile', and 'app.py'. The 'app.py' tab is active, showing a Python script. The path in the editor is 'C: > Users > User > Desktop > docker-compose > app.py'. The code in 'app.py' is:

```
1 from flask import Flask  
2 app = Flask(__name__)  
3  
4 @app.route("/")  
5 def home():  
6     return "Hello Docker Compose!"  
7  
8 if __name__ == "__main__":  
9     app.run(host="0.0.0.0", port=5000)  
10
```

II.Add a Dockerfile in the same folder:

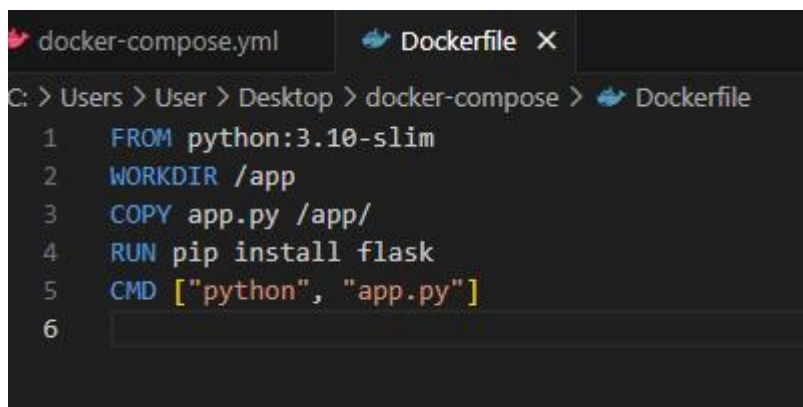
FROM python:3.10-slim

WORKDIR /app

COPY app.py /app/

RUN pip install flask

CMD ["python", "app.py"]

A screenshot of a code editor with two tabs: 'docker-compose.yml' and 'Dockerfile'. The 'Dockerfile' tab is active, showing a Dockerfile. The path in the editor is 'C: > Users > User > Desktop > docker-compose > Dockerfile'. The code in 'Dockerfile' is:

```
1 FROM python:3.10-slim  
2 WORKDIR /app  
3 COPY app.py /app/  
4 RUN pip install flask  
5 CMD ["python", "app.py"]  
6
```

III.Update docker-compose.yml:

web:

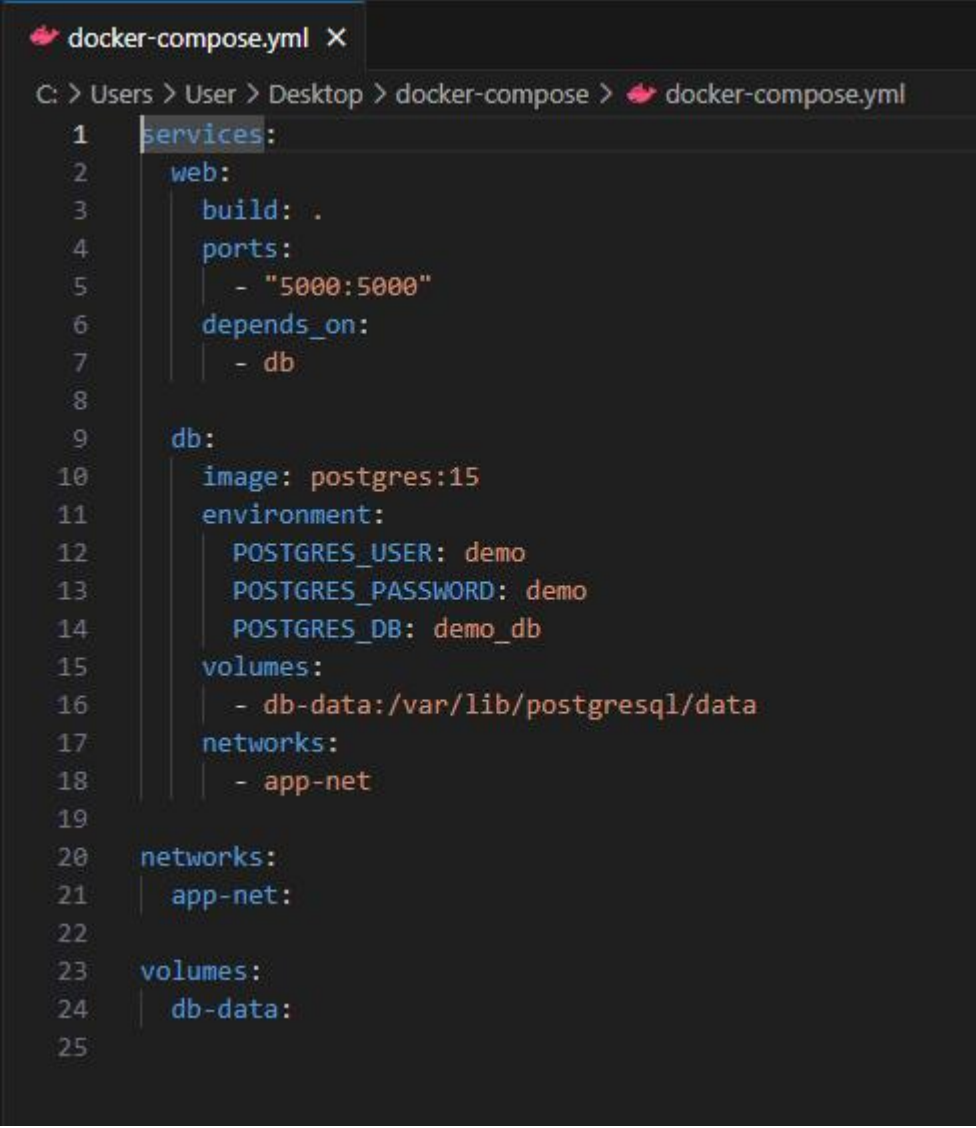
build: .

ports:

- "5000:5000"

depends_on:

- db

A screenshot of a code editor window titled 'docker-compose.yml'. The editor shows the following YAML configuration:

```
1 services:
2   web:
3     build: .
4     ports:
5       - "5000:5000"
6     depends_on:
7       - db
8
9   db:
10    image: postgres:15
11    environment:
12      POSTGRES_USER: demo
13      POSTGRES_PASSWORD: demo
14      POSTGRES_DB: demo_db
15    volumes:
16      - db-data:/var/lib/postgresql/data
17    networks:
18      - app-net
19
20 networks:
21   app-net:
22
23 volumes:
24   db-data:
25
```

The editor has a dark theme and shows line numbers from 1 to 25 on the left margin.

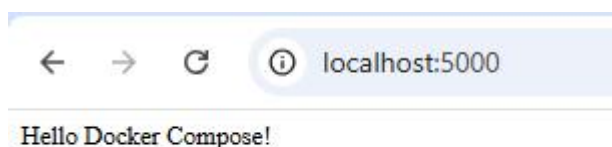
IV.Run:

docker compose up --build


```
root@web:~# docker-compose up --build
[+] Building 0.0s [web] exporting to image
[+] Exporting layers
[+] Exporting manifest sha256:1d9e9791a5b8d108711b88ad8e8f7d8e0/3126d617b6b8a409976ead
[+] Exporting config sha256:6d1796663747b4b026c17a6d31d61b2421f7e0b17e00a1b41f0e0
[+] Exporting stratification manifest sha256:1d9b8c2b831a72611b1d4e01f8d3c30e710a00f0e0
[+] Exporting manifest list sha256:7b1c1e0b7f320d1121c7b0c04871a011b0001f0b0e791b001e12d0b
[+] Sending to docker://library/docker-compose-web:latest
[+] Sending to docker://library/docker-compose-web:latest
[+] Removing previous manifests for manifest file
time="2025-08-26T11:51:07.951387" level=warning msg="Found orphan containers ([docker-compose-radius-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up."
[+] Starting app
[+] Container docker-compose-web-1 Created
[+] Container docker-compose-web-1 Recreated
Attaching to web-1, web-2
web-1 | PostgreSQL database directory appears to contain a database; skipping initialization
web-1 | 2025-08-26 06:21:09.581 UTC [1] LOG: starting PostgreSQL 15.14 (Debian 15.14-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
web-1 | 2025-08-26 06:21:09.581 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
web-1 | 2025-08-26 06:21:09.581 UTC [1] LOG: listening on IPv6 address "::", port 5432
web-1 | 2025-08-26 06:21:09.579 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
web-1 | 2025-08-26 06:21:09.600 UTC [29] LOG: database system was interrupted; last known up at 2025-08-26 06:18:17 UTC
web-1 | 2025-08-26 06:21:09.601 UTC [29] LOG: database system was not properly shut down; automatic recovery in progress
web-1 | 2025-08-26 06:21:09.611 UTC [29] LOG: redo starts at 0/180F170
web-1 | 2025-08-26 06:21:09.671 UTC [29] LOG: invalid record length at 0/180F710: wanted 24, got 0
web-1 | 2025-08-26 06:21:09.671 UTC [29] LOG: redo done at 0/180F710 system usage: CPU: user=0.00 s, system=0.00 s, elapsed=0.00 s
web-1 | 2025-08-26 06:21:09.681 UTC [27] LOG: checkpoint starting: end-of-recovery; immediate wait
web-1 | 2025-08-26 06:21:09.713 UTC [27] LOG: checkpoint complete: write 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.000 s, sync=0.000 s, total=0.013 s; sync files=2, logseq=0.000 s, average=0.001 s; distance=0 kB, estimate=0 kB
web-1 | 2025-08-26 06:21:09.710 UTC [1] LOG: database system is ready to accept connections
web-1 | * Serving flask app 'app'
web-1 | * Debug mode: off
web-2 | WARNING: This is a development server. Do not use it in a production deployment; use a production WSGI server instead.
web-2 | * Running on all addresses (0.0.0.0)
web-2 | * Running on http://172.17.0.1:5000
web-2 | * Running on http://172.17.0.1:5000
web-2 | Press CTRL+C to quit
web-2 | 172.17.0.1 - - [26/Aug/2025:06:21:11] "GET / HTTP/1.1" 200 -
```

VI.Expected Output:

New message appears instantly after rebuild.



Conclusion:

Docker Compose simplifies multi-container application management by allowing developers to define, configure, and run services in a single YAML file. It streamlines deployment, ensures consistent environments, and reduces manual setup, making it an essential tool for orchestrating complex, container-based applications efficiently.

LAB ACTION PLAN FOR WEEK 7

Objective:

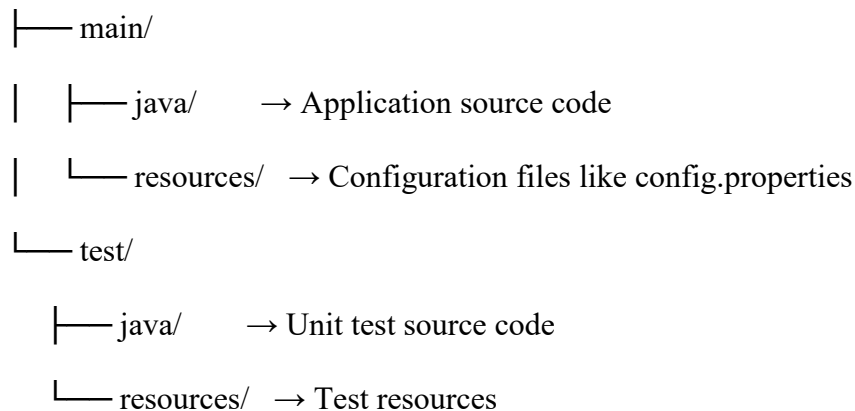
To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications.
src/



The compiled files and reports are generated in the **target/** directory after a successful build.

2. Steps to Perform Maven Build and Testing

----mvn clean install

- **clean:** Deletes the previous build (**target/** folder)
- **install:** Builds, tests, and installs the package to local **.m2** repository

After execution:

- **target/** folder contains compiled **.class** files and the final **.jar** or **.war**

- Artifact is stored in:
`~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In `pom.xml`:

```
<dependency>  
  
<groupId>com.google.code.gson</groupId>  
  
<artifactId>gson</artifactId>  
  
<version>2.10</version>  
  
</dependency>
```

After running:

```
mvn clean install
```

Check:

- Gson JAR is downloaded to `.m2/repository/com/google/code/gson/gson/`
- If version is wrong or not found → **BUILD FAILURE** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└─ 1.1. Launch Eclipse workspace

Step 2. Install Maven Plugin (if not installed)

└─ 2.1. Go to "Help" in the top menu

└─ 2.1.1. Click "Eclipse Marketplace"

└─ 2.1.2. Search for "Maven Integration for Eclipse"

└─ 2.1.3. Install the plugin if not already installed

Step 3. Create a New Maven Project

└─ 3.1. File -> New -> Project...

└─ 3.1.1. Expand "Maven"

- └─ 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

- └─ 4.1. Select workspace location (default or custom)
- └─ 4.2. Click "Next"

Step 5. Choose Maven Archetype

- └─ 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")
- └─ 5.2. Click "Next"

Step 6. Define Project Metadata

- └─ 6.1. Group ID: (e.g., com.example)
- └─ 6.2. Artifact ID: (e.g., my-maven-project)
- └─ 6.3. Version: (default is usually fine)
- └─ 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.

Step 7. Maven Project Created

- └─ 7.1. Project structure is generated with a standard Maven layout
- └─ 7.2. Includes:
 - └─ src/main/java (for Java source code)
 - └─ src/test/java (for test code)
 - └─ pom.xml (Maven configuration file)

Step 8. Update Project Settings (if needed)

- └─ 8.1. Right-click on the project -> Maven -> Update Project...
- └─ 8.2. Ensure dependencies are up to date

Step 9. Build and Run Maven Project

- └─ 9.1. Right-click on App.java -> Run As -> Maven Clean
 - └─ 9.1.1. Right-click on App.java -> Run As -> Maven Install

- └─ 9.1.2. Right-click on App.java -> Run As -> Maven Test

- └─ 9.1.3. Right-click on App.java -> Run As -> Maven Build

Step 10. In the Maven Build dialog:

- └─ Enter Goals: clean install test

- └─ Click on Apply -> Click on Run

Step 11. Check console for BUILD SUCCESS message.

Step 12. Run the application:

- └─ Right-click on App.java -> Run As -> Java Application

- └─ Output: "Hello World" displayed.

Creation of Maven web Java Project

Step 1: Open Eclipse

- └─ 1.1 Launch Eclipse IDE.

- └─ 1.2 Select or create a workspace.

Step 2: Create a New Maven Project

- └─ 2.1. File -> New -> Project...

- └─ 2.1.1. Expand "Maven"

- └─ 2.1.2. Select "Maven Project" and click "Next"

Step 3: Choose Maven Archetype

- └─ 3.1. Select an archetype(e.g "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

- └─ 3.2. Click "Next"

Step 4: Configure the Maven Project

- └─ 4.1 Group Id: Enter a group ID (e.g., com.example).

- └─ 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

- └─ 4.3 Click ****Finish**** to create the project.

Step 5: Add Maven Dependencies

└─ 5.1 Open the ****pom.xml**** file in the Maven project.

└─ 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

└─ Example:

```
``xml

<dependency>

<groupId>javax.servlet</groupId>

<artifactId>javax.servlet-api</artifactId>

<version>4.0.1</version>

<scope>provided</scope>

</dependency>

...
```

Step 6:- Configure server:

- └─ Window -> Show View -> Servers
- └─ Add server -> Select Tomcat v9.0 server -> Click Next
- └─ Configure server options (e.g., ports, server location).

Step 7:- Modify 'tomcat-users.xml':

- └─ Add role and user details under <tomcat-users> tag.

Step 8:- Build the project:

- └─ Right-click on index.jsp -> Run As -> Maven Clean
- └─ Right-click on index.jsp -> Run As -> Maven Install
- └─ Right-click on index.jsp -> Run As -> Maven Test
- └─ Right-click on index.jsp -> Run As -> Maven Build

Step 9. In the Maven Build dialog:

- └ Enter Goals: clean install test
- └ Click on Apply -> Click on Run

Step 10. Check console for BUILD SUCCESS message.

Step 11. Run the application:

- └ Right-click on index.jsp -> Run As -> Run on Server
- └ Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github

3. Configure Java Version via Compiler Plugin

In `pom.xml`:

```
<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>17</source>

<target>17</target>

</configuration></plugin>
```

Check:

- Run `mvn package`
- Inspect generated JAR: `jar tf target/myapp.jar`

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```

public class AppTest {

    @Test

    public void testSum() {

        assertEquals(5, 2 + 3);

    }

}

```

Run:

```
mvn test
```

Check:

- target/test-classes/ → compiled test .class files
 - target/surefire-reports/ → .txt or .xml test results
- If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause BUILD FAILURE
- Maven shows precise error in console
- Fix the dependency tag → re-run `mvn clean install`

6. Adding Resource Files

Put config.properties inside:

```
src/main/resources/config.properties
```

To read:

```

InputStream input =
getClass().getClassLoader().getResourceAsStream("config.properties");

```

After build, check target/classes/ → file should exist there.

7. Multi-Module and Parent Projects

Structure:

```

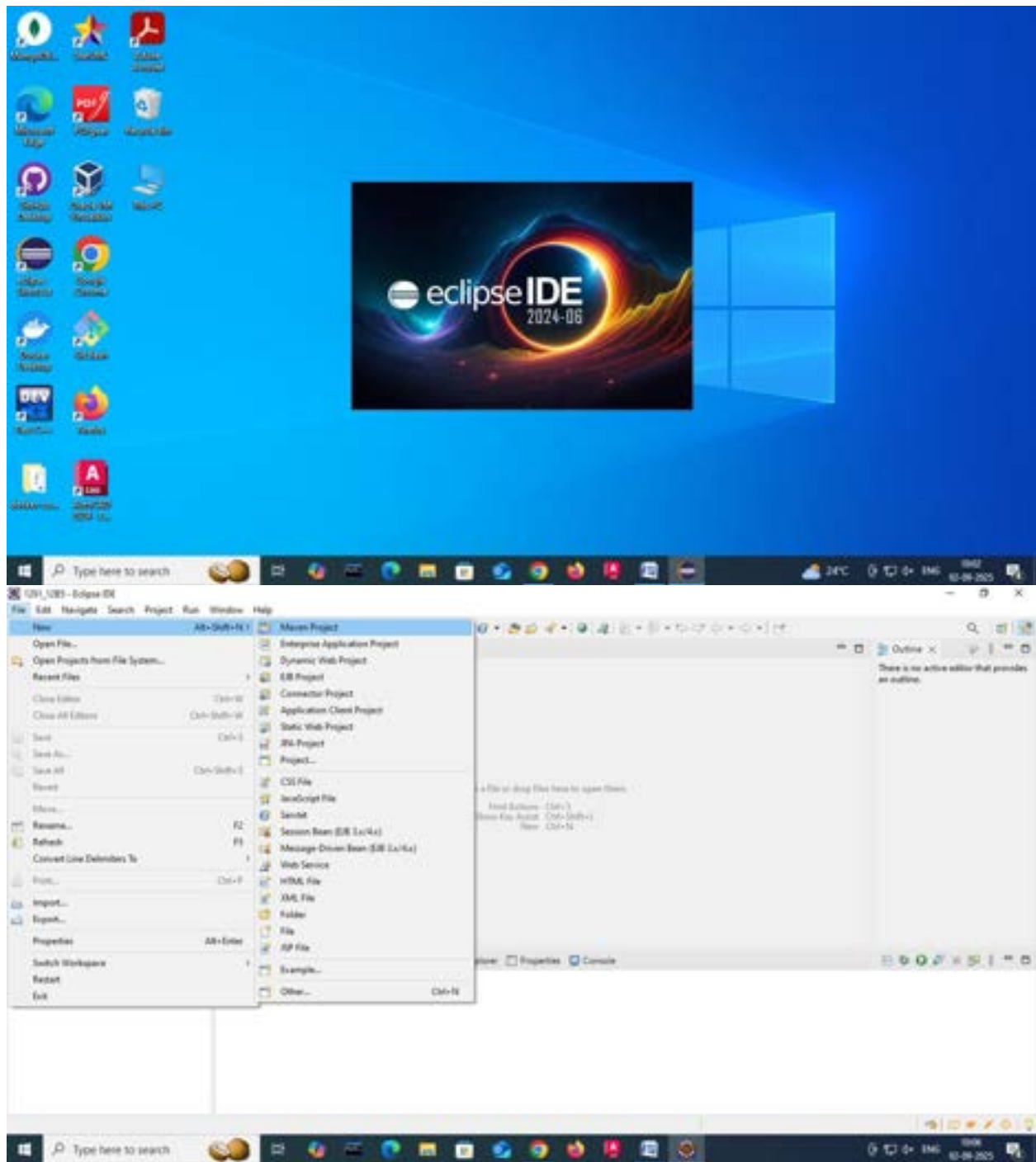
parent/
├── pom.xml (packaging: pom)
├── core/
│   └── pom.xml
└── web/
    └── pom.xml

```

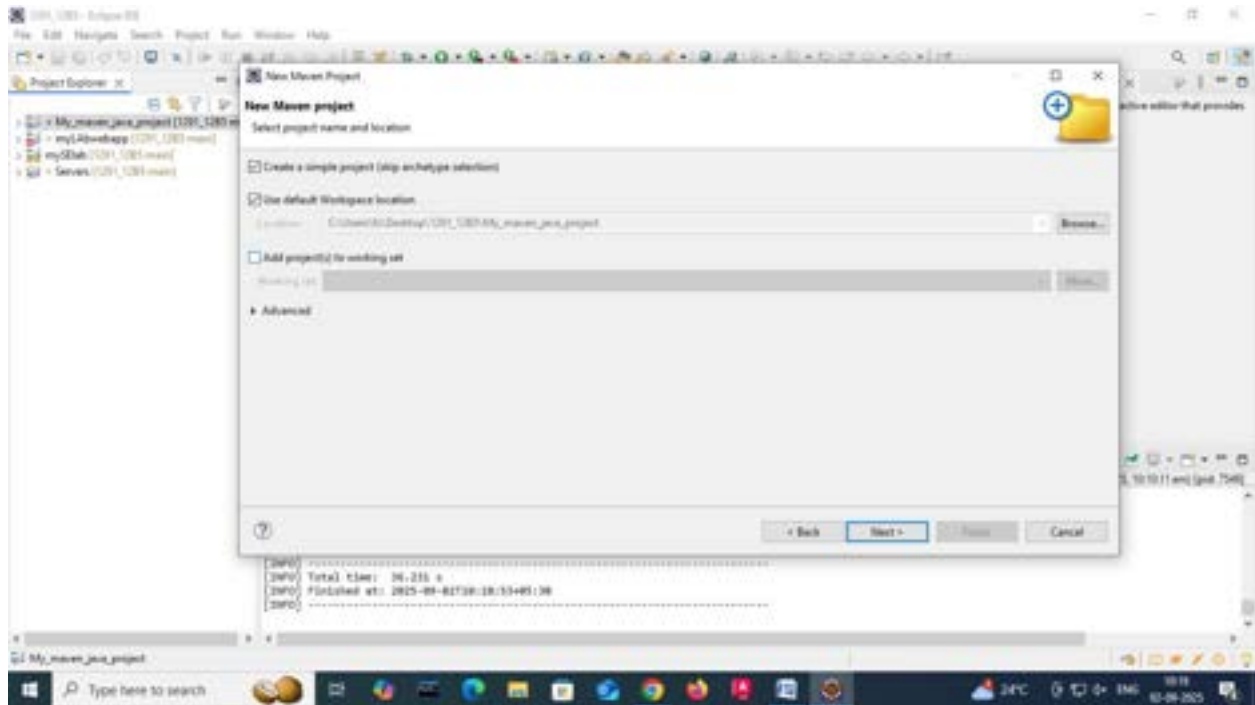
- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

Creating a Multi-Module Maven Project:

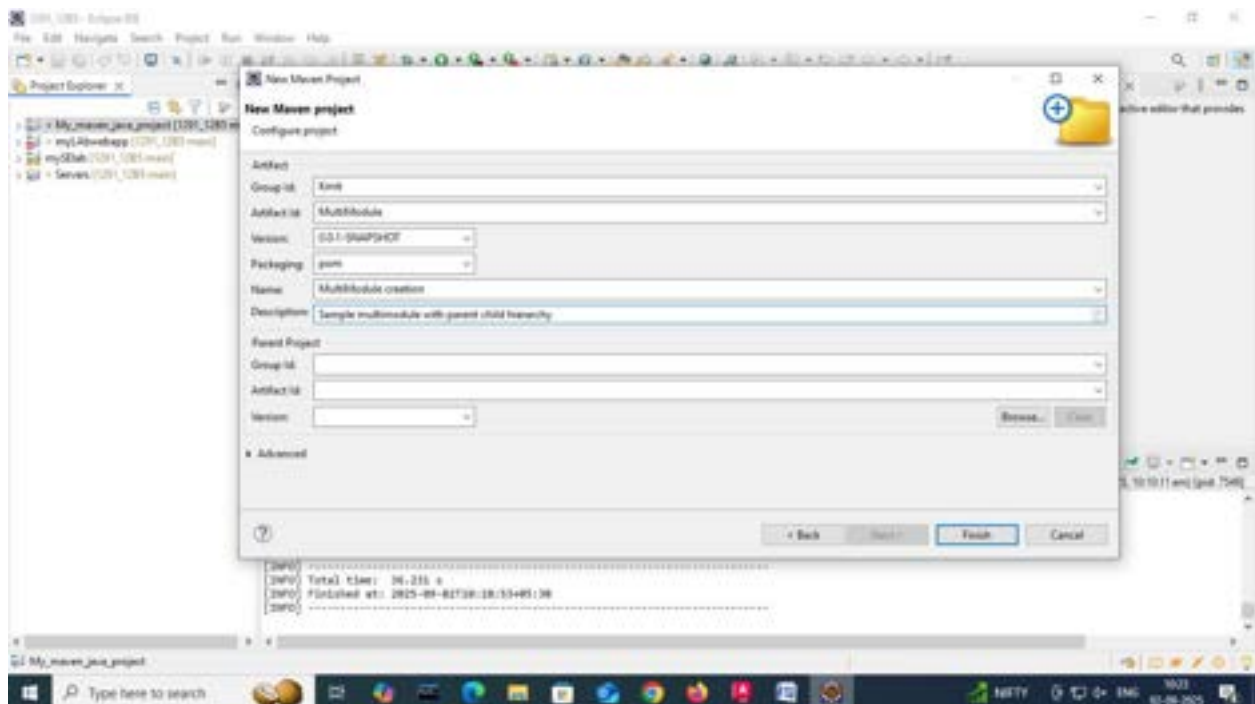
1. Open eclipse-> file-> new -> other -> Maven -> MavenProject ->click on next.



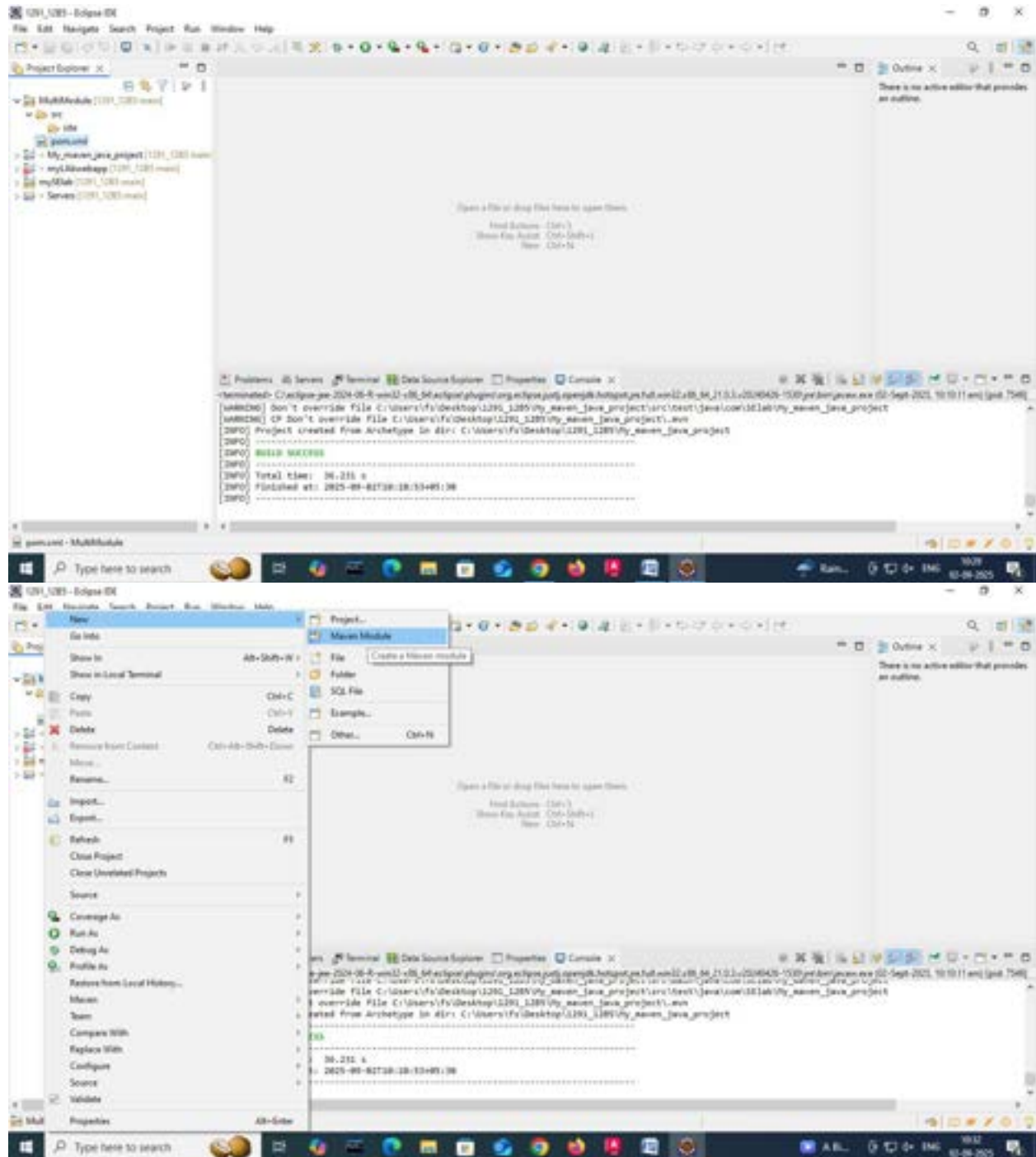
2. Check the first option: Create a Simple project(skip archetype selection) and click on next.



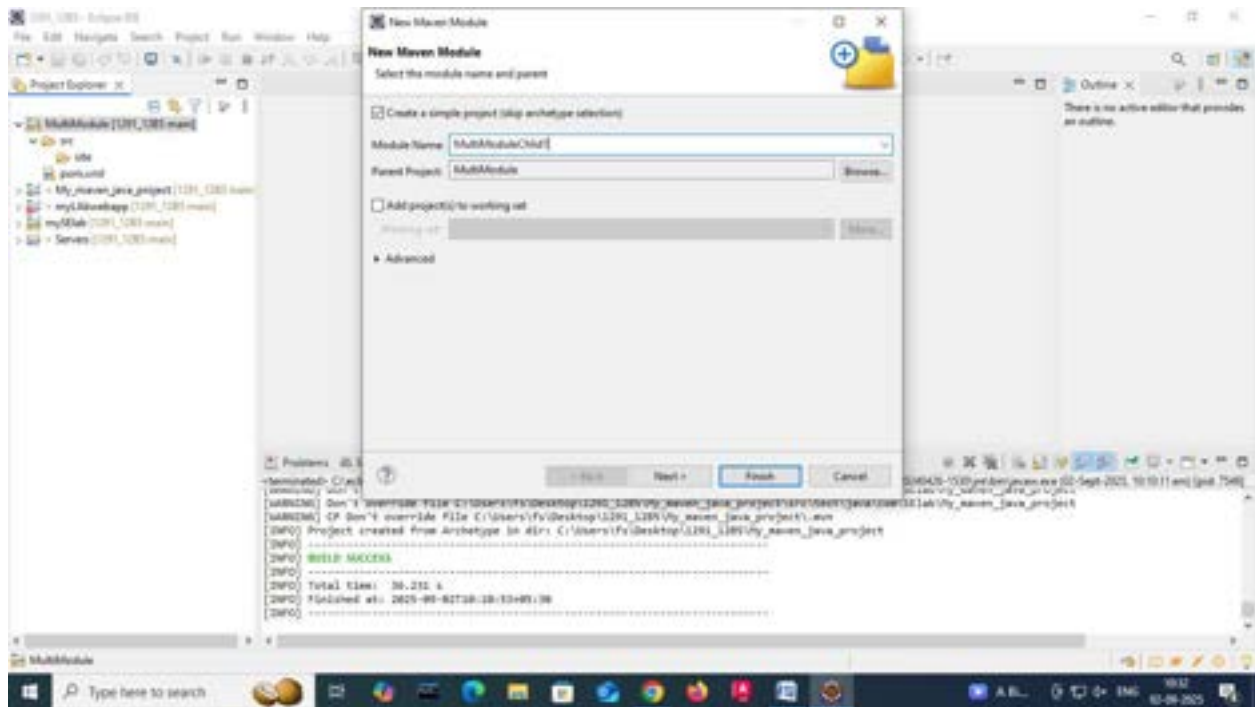
3. Give groupId: KMIT, artifact ID: MultiModule, in the packaging tab select the drop down and opt for pom.
4. Give Name: Multimodule creation and Description: Sample multimodule with parent child hierarchy and click on next.



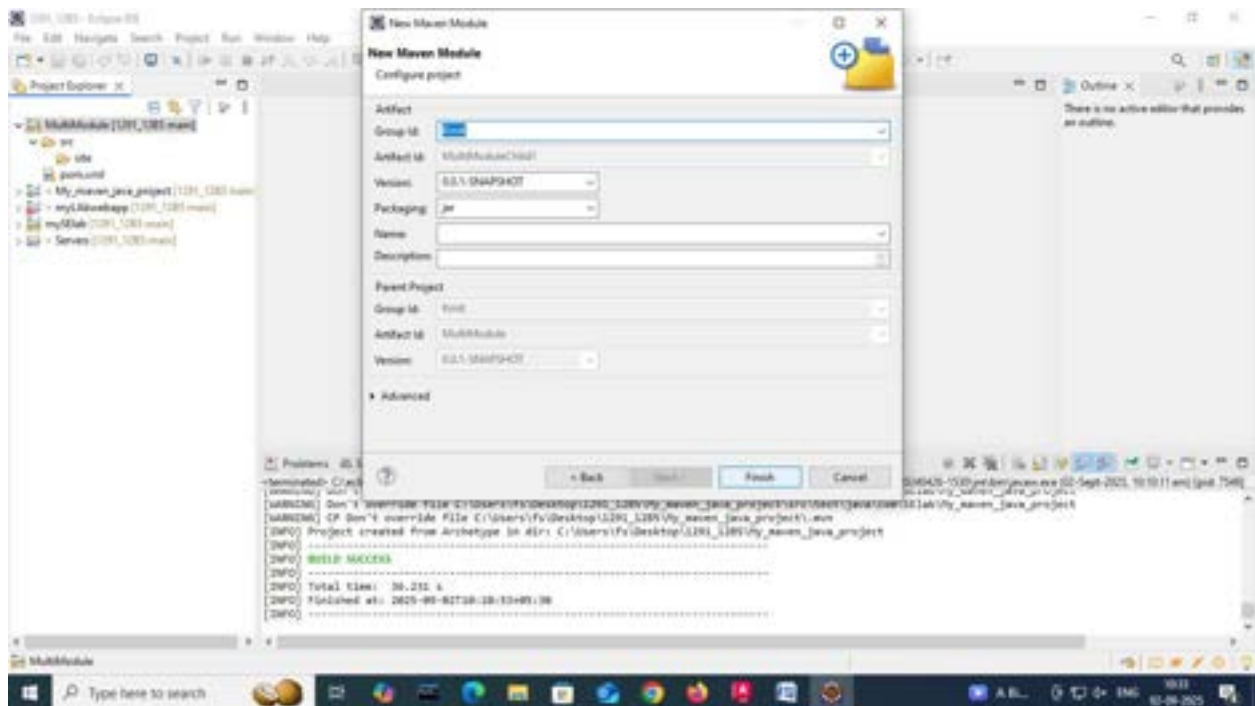
- Now the Multimodule folder is created and displayed in the project explorer on the left. Right click on the folder and opt new -> mavenModule

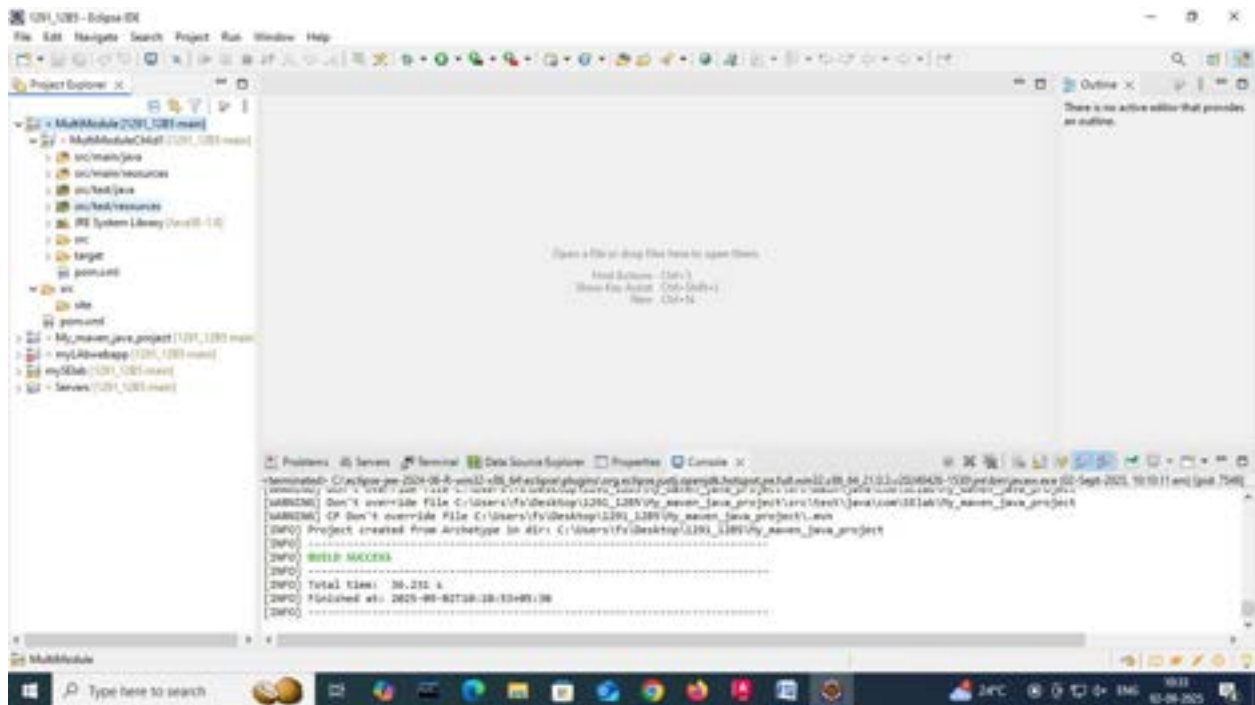


6. Check the option Create a Simple project(skip archetype selection, give the artifactID: MultiModuleChild1 and click on next and finish.

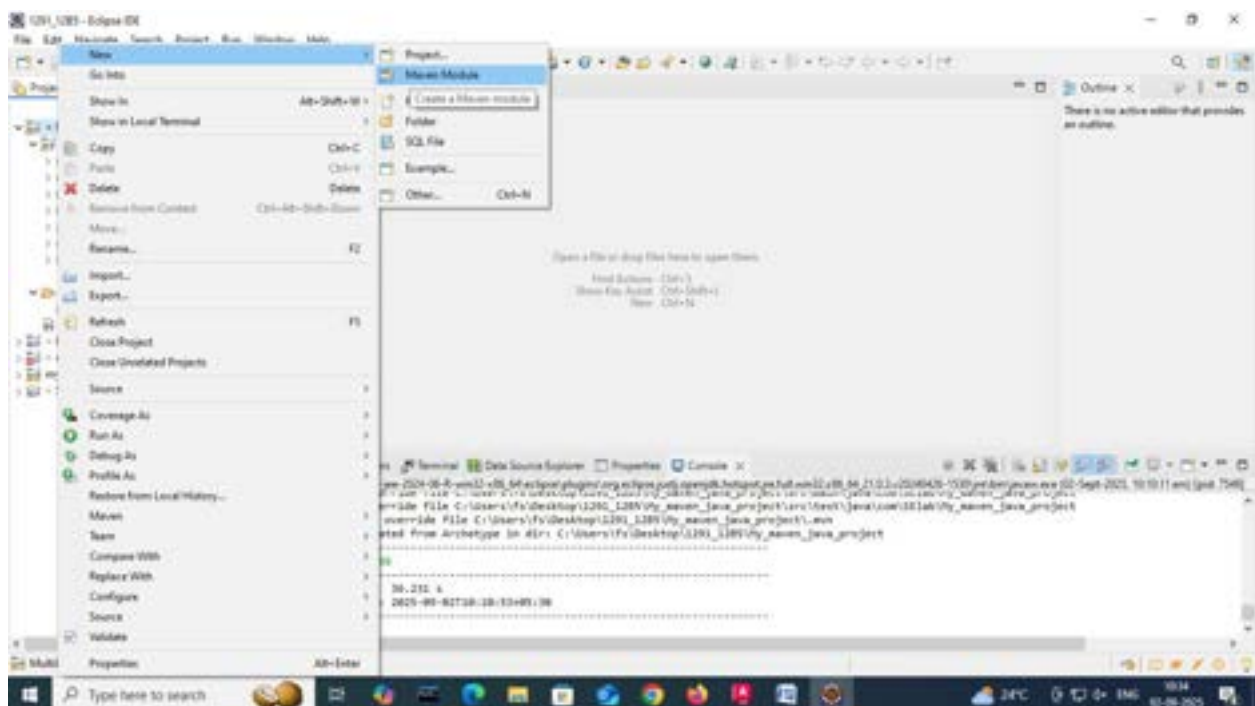


7. Now you can see, the module1 we created is displayed in the project explorer with a separate pom.xml file.

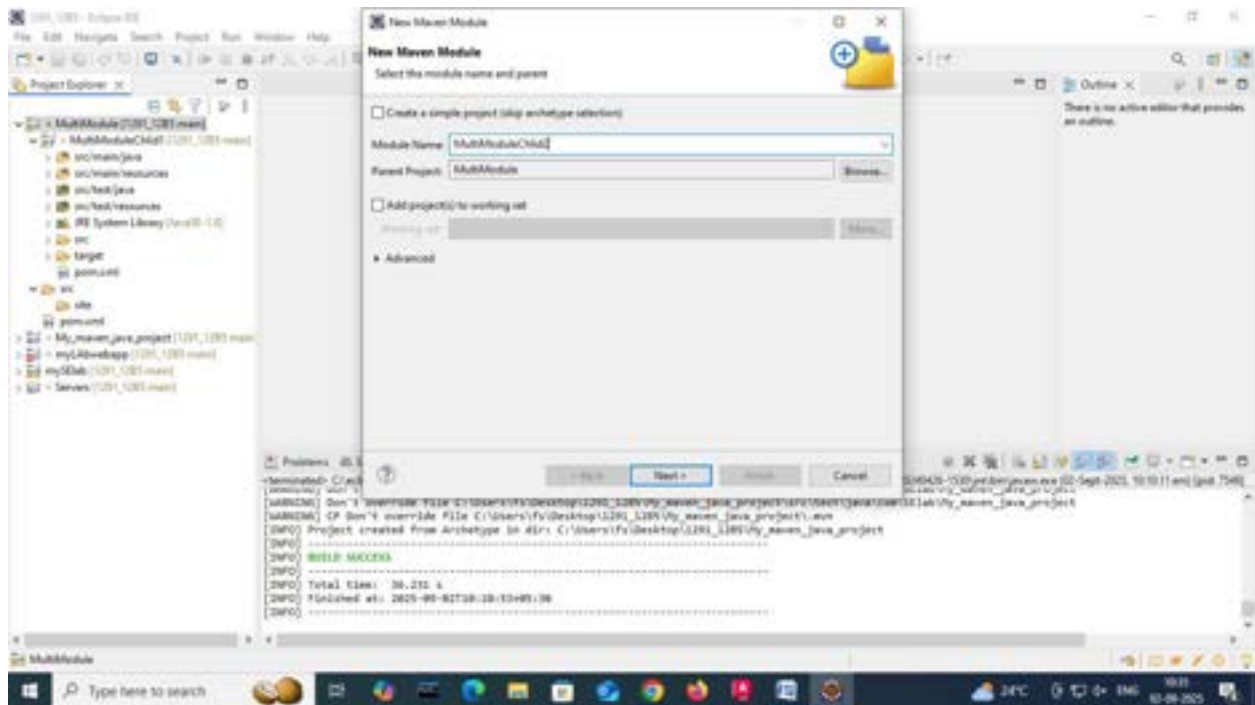




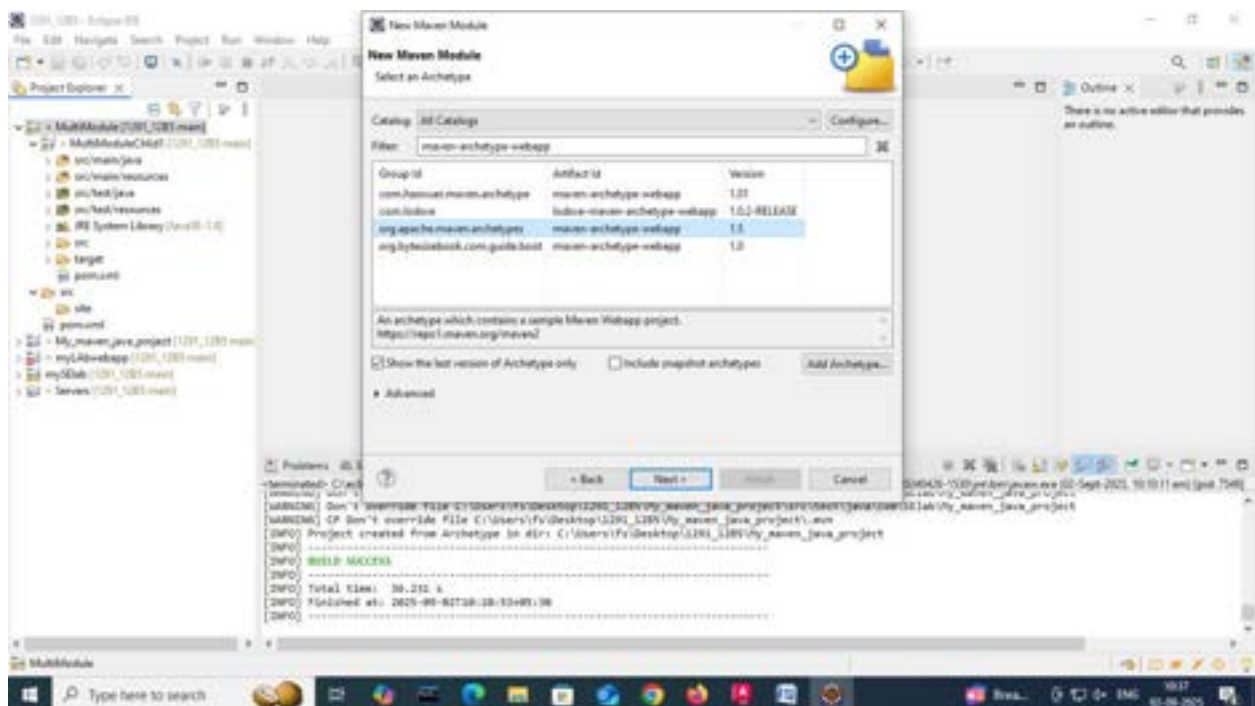
8. . Right click on the MultiModule folder in the project explorer and opt new -> mavenModule



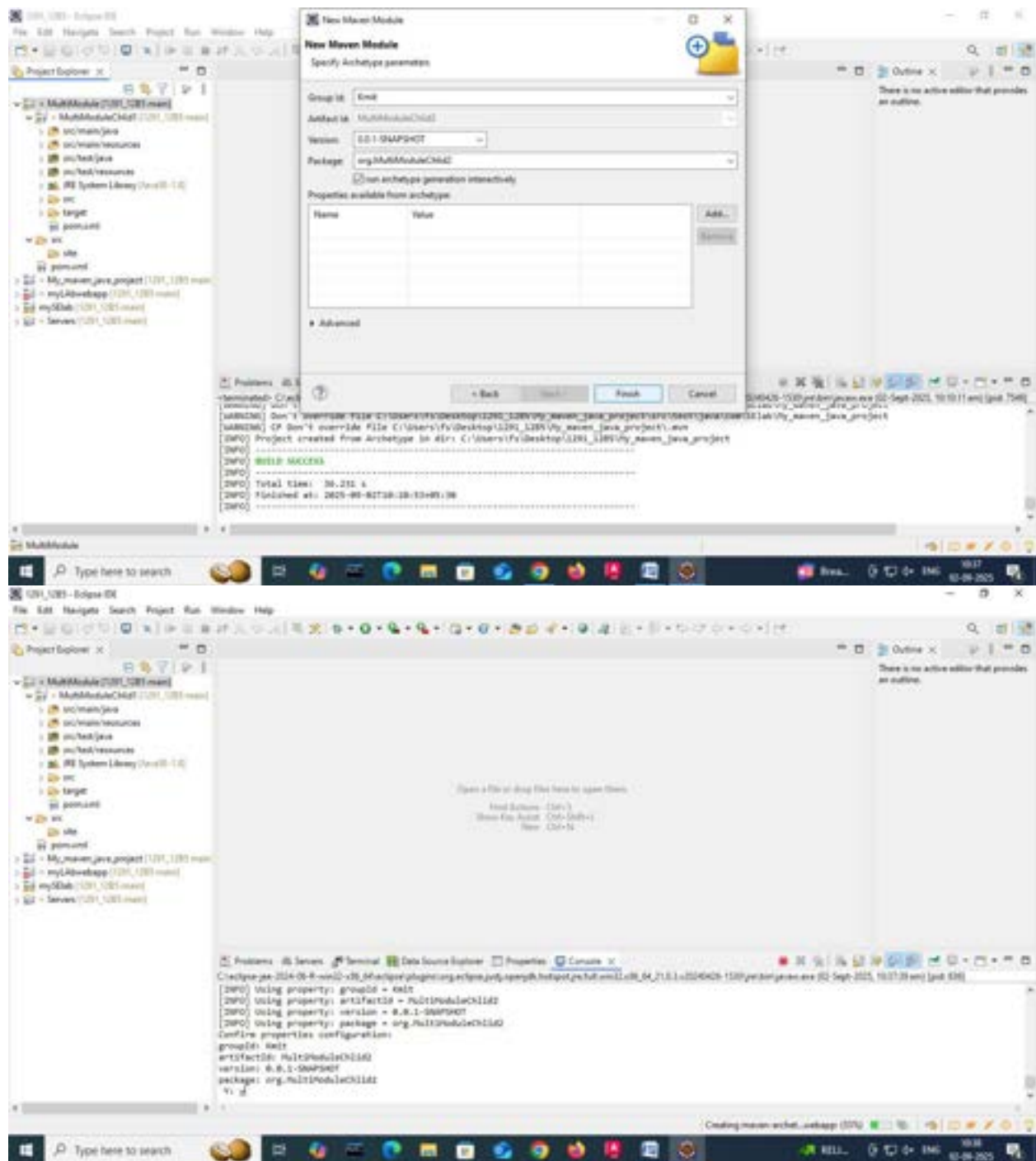
9. Give the artifactID: MultiModuleChild2 and click on next.



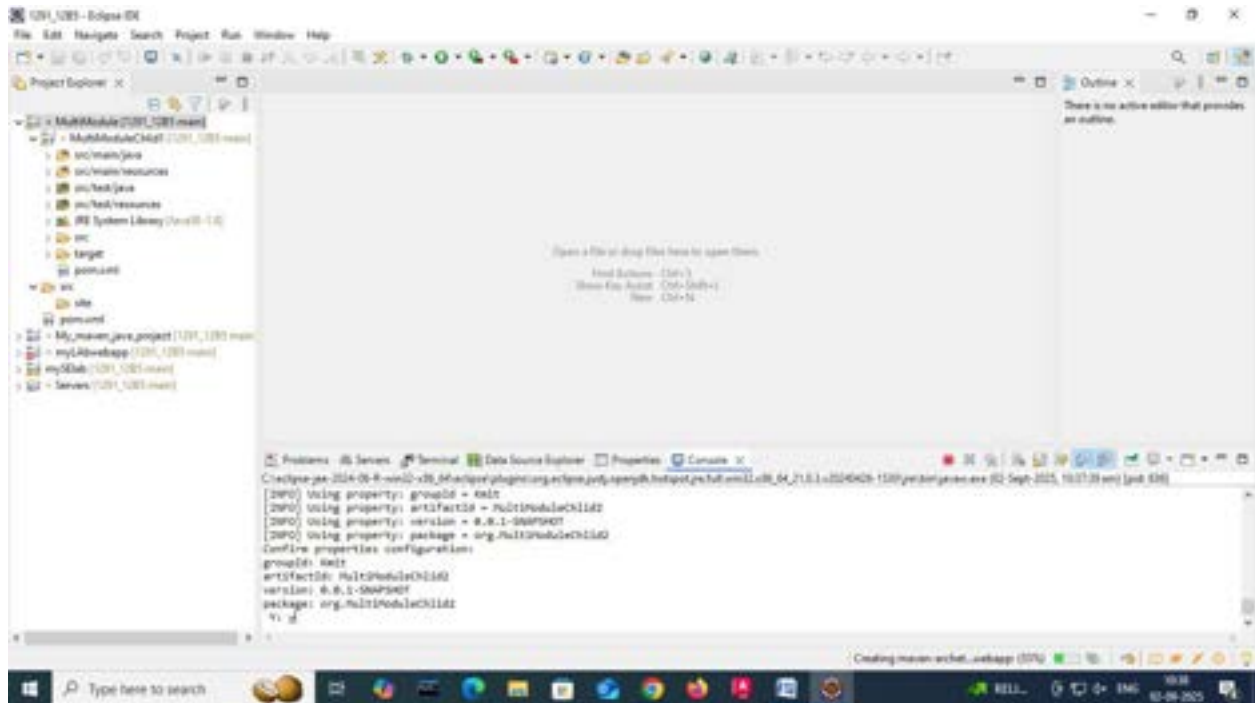
10. In filters search for maven-archetype-webapp, select the version 1.1/1.5 and click on next.



11. Now click on finish, and confirm with a Y in the console.



12. Now you can watch the new child is been added to the parent project.



Note: Childs or Modules can be added to the projects who have the packaging type as pom. If we try to create a module within child1 it is not possible since its packaging is opted to JAR.

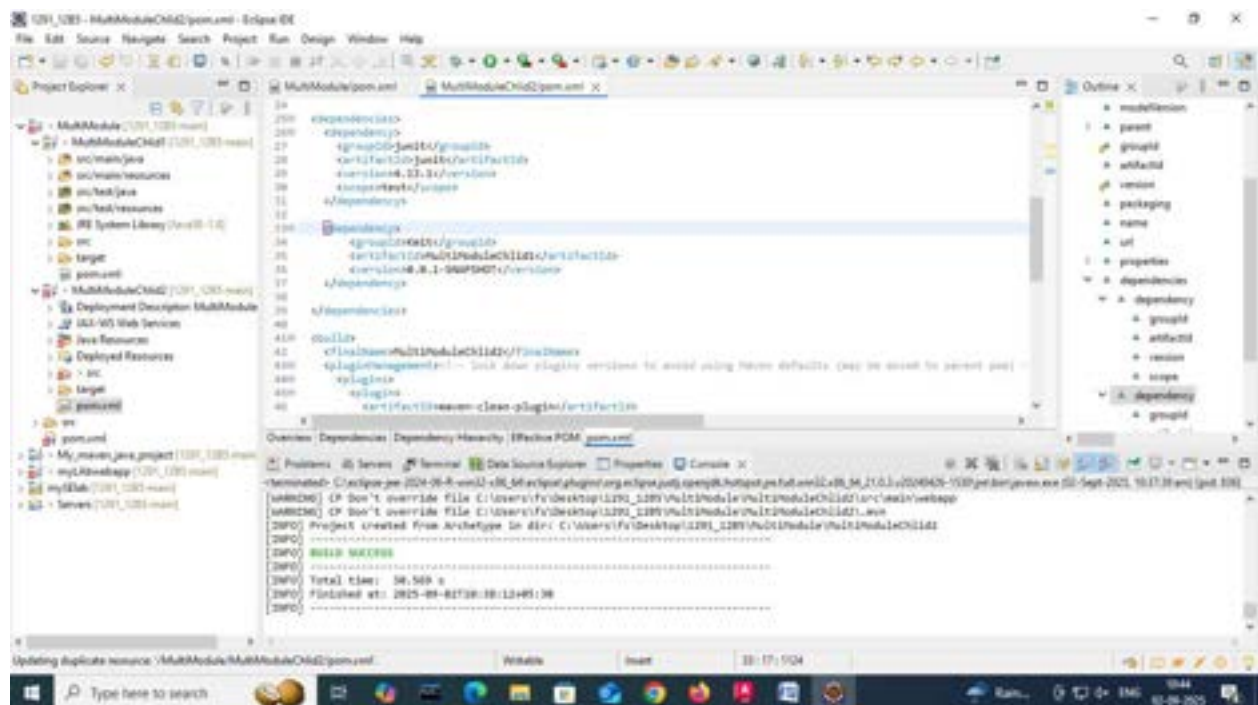
Linking two modules in a Parent Project:

Go to the module u want to connect. And add a dependency to other module on which the current module depends. For example if my child2 is dependent on child1. I have to change the dependencies in the pom.xml by adding a new dependency of child1.

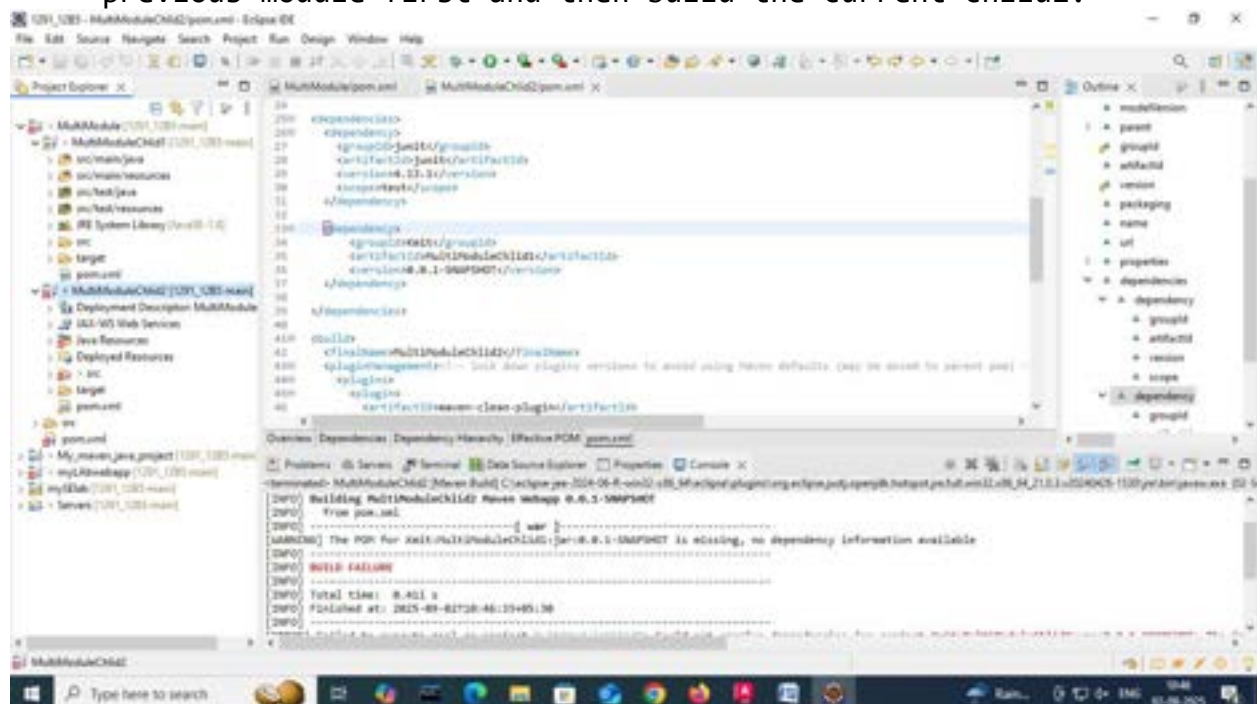
Steps to add dependency:

1. Goto pom.xml of child2.
2. Navigate to dependencies tag. Update the dependencies with child1 dependency as given below, and save pom.xml after update.

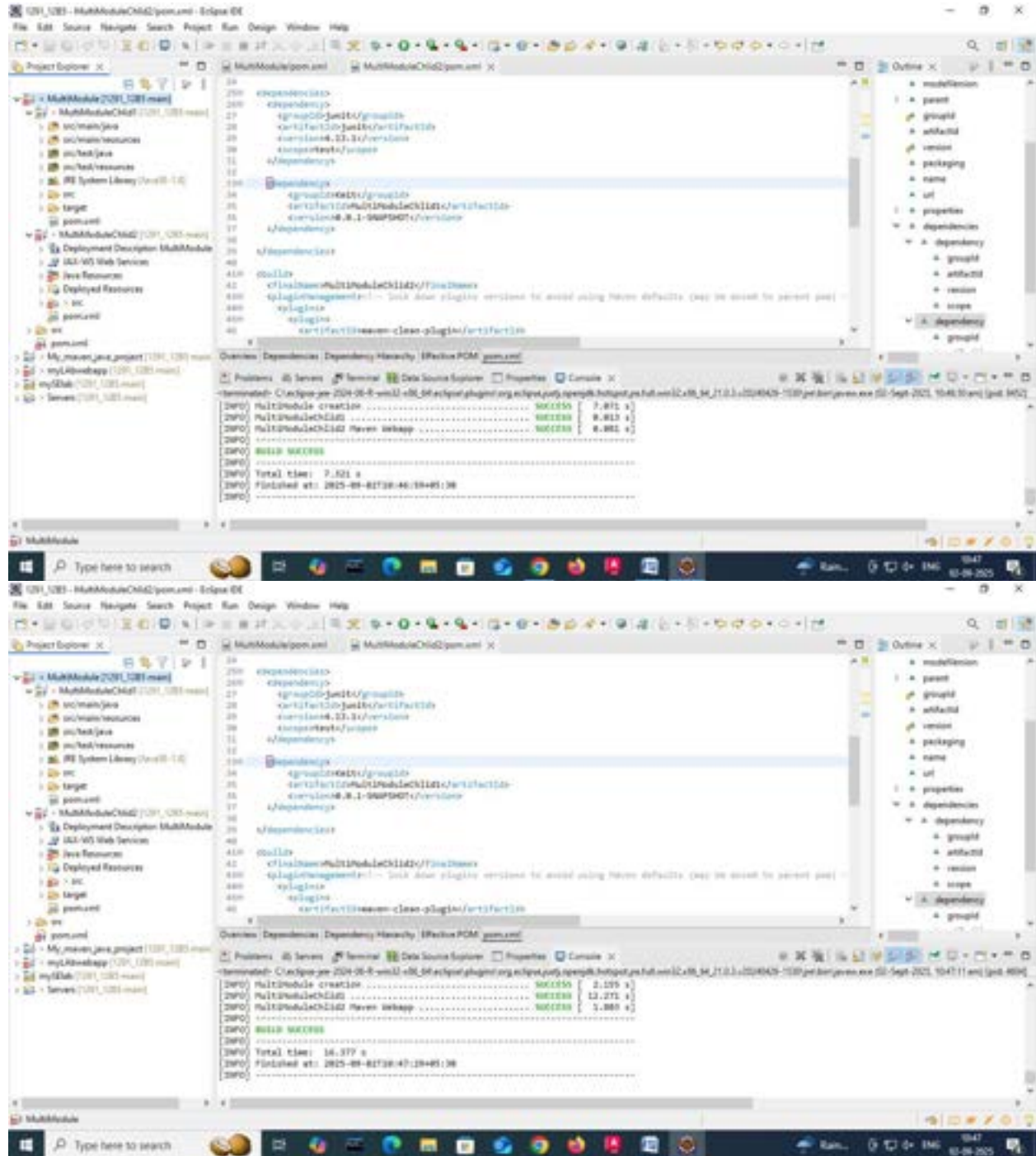
```
<dependency>
    <groupId>KMIT</groupId>
    <artifactId>MultimoduleChild1</artifactId>
    <version>0.0.1-SNAPSHOT</version>
</dependency>
```

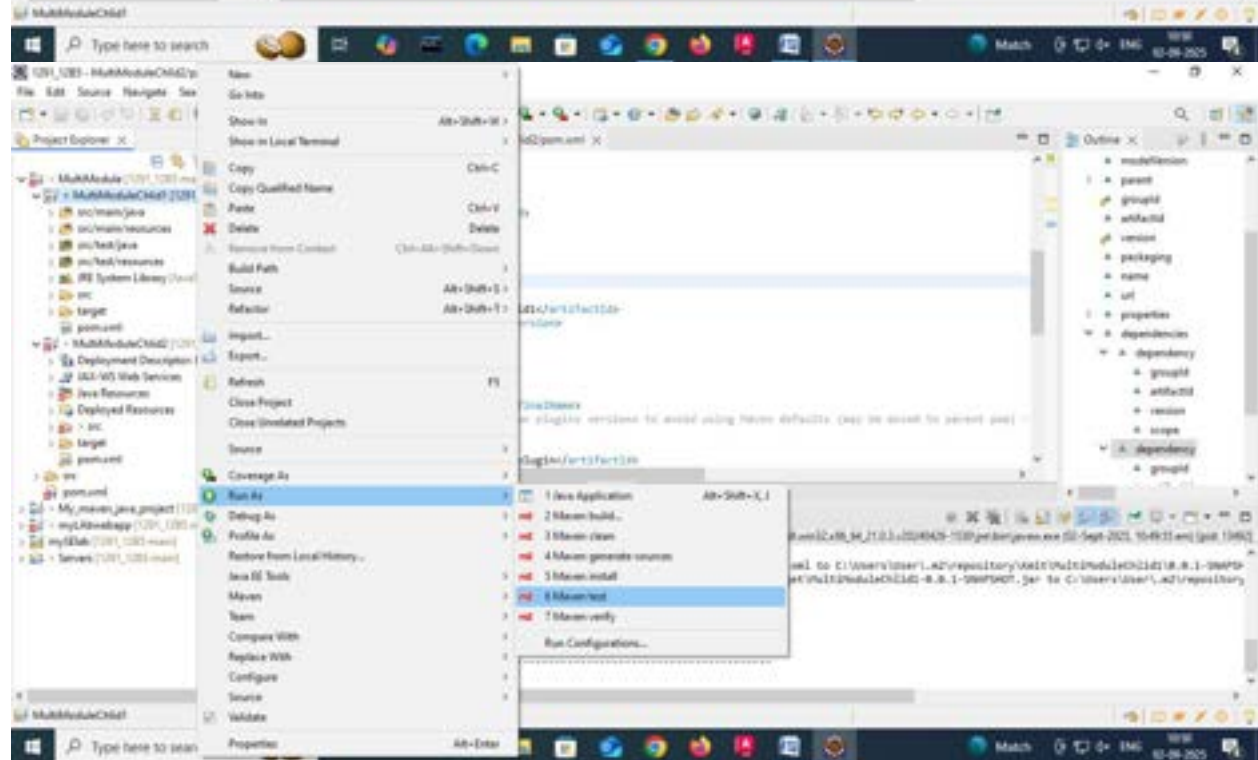
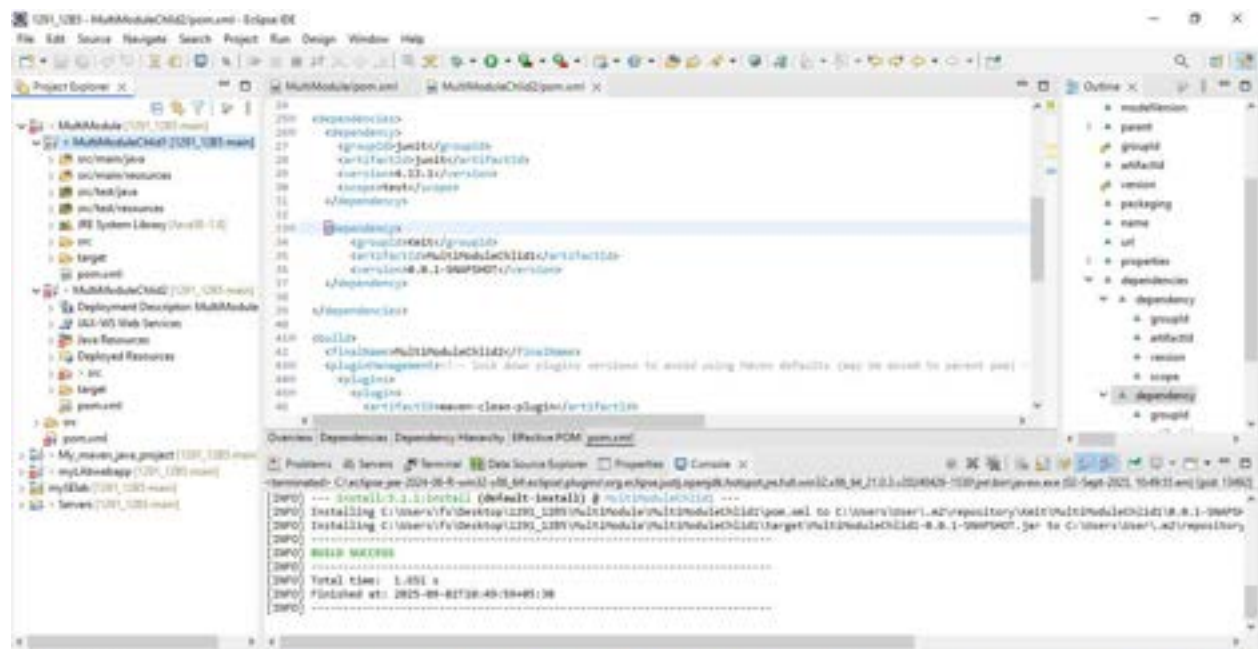



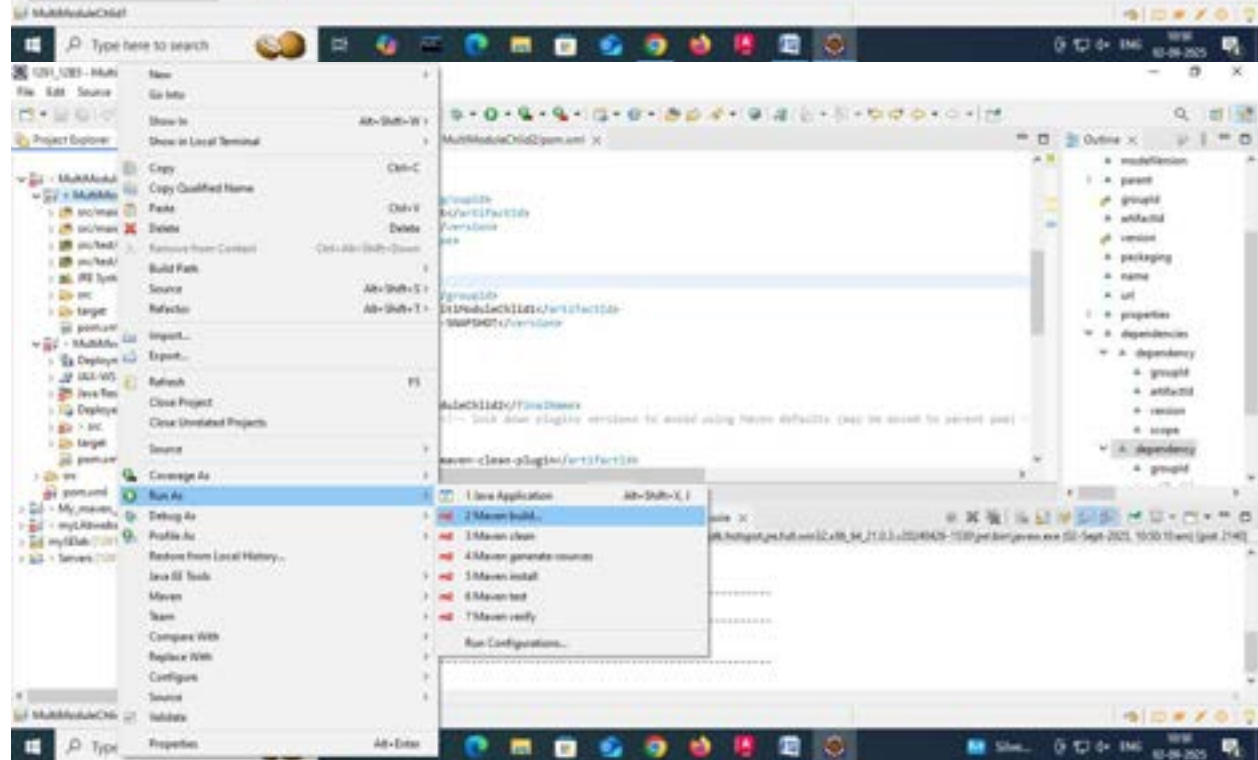
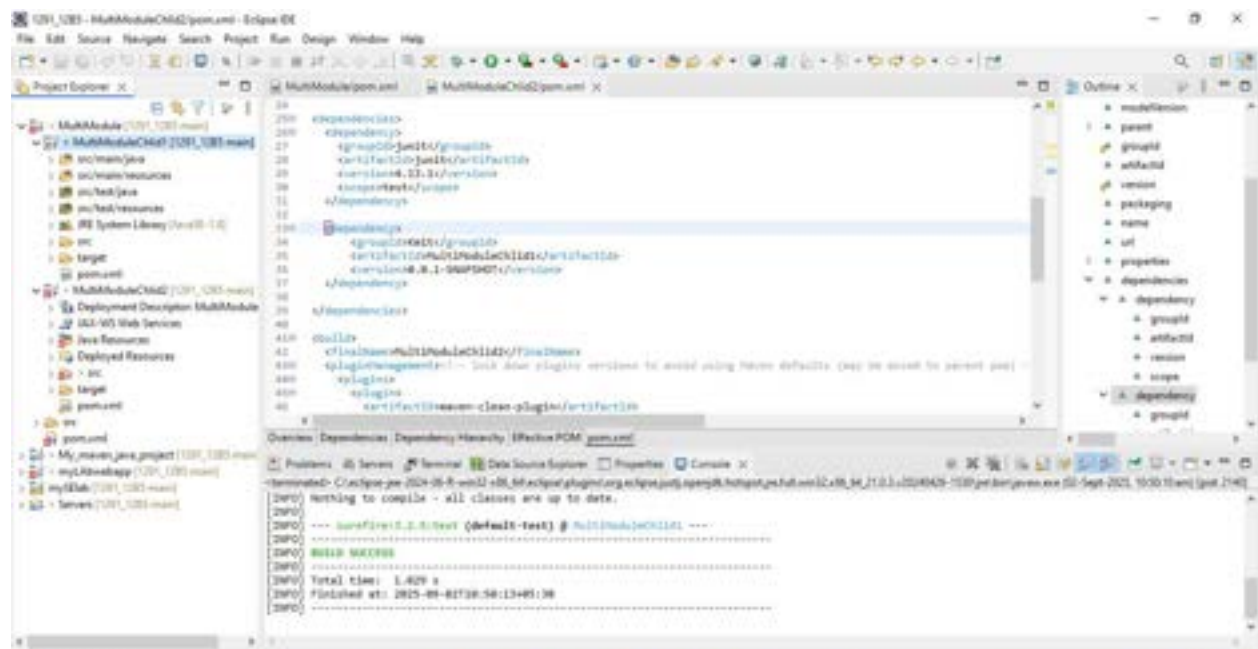
3. Now build the child2 by right clicking it and opting run as->maven build.
4. The build will be a *failure*. Because we haven't build the parent project and the child project. Child1 is dependent on parent. And Child2 is dependent on child2. SO we have to build ass the previous module first and then build the current child2.

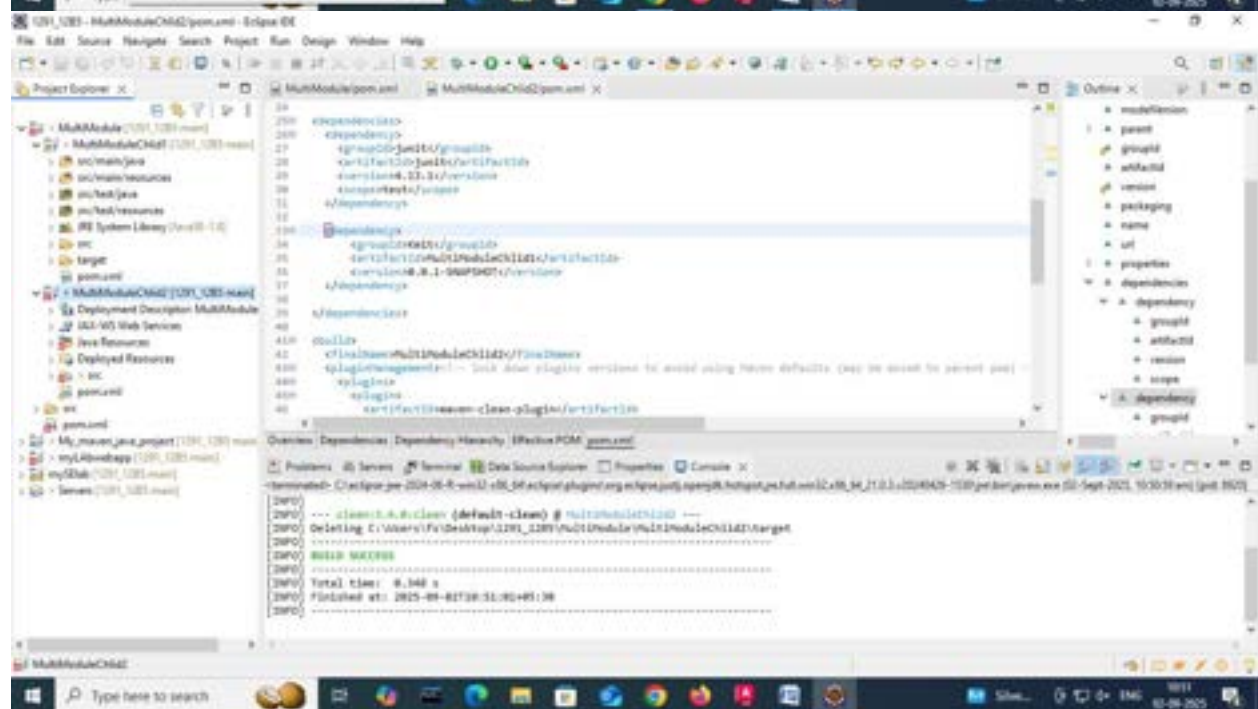
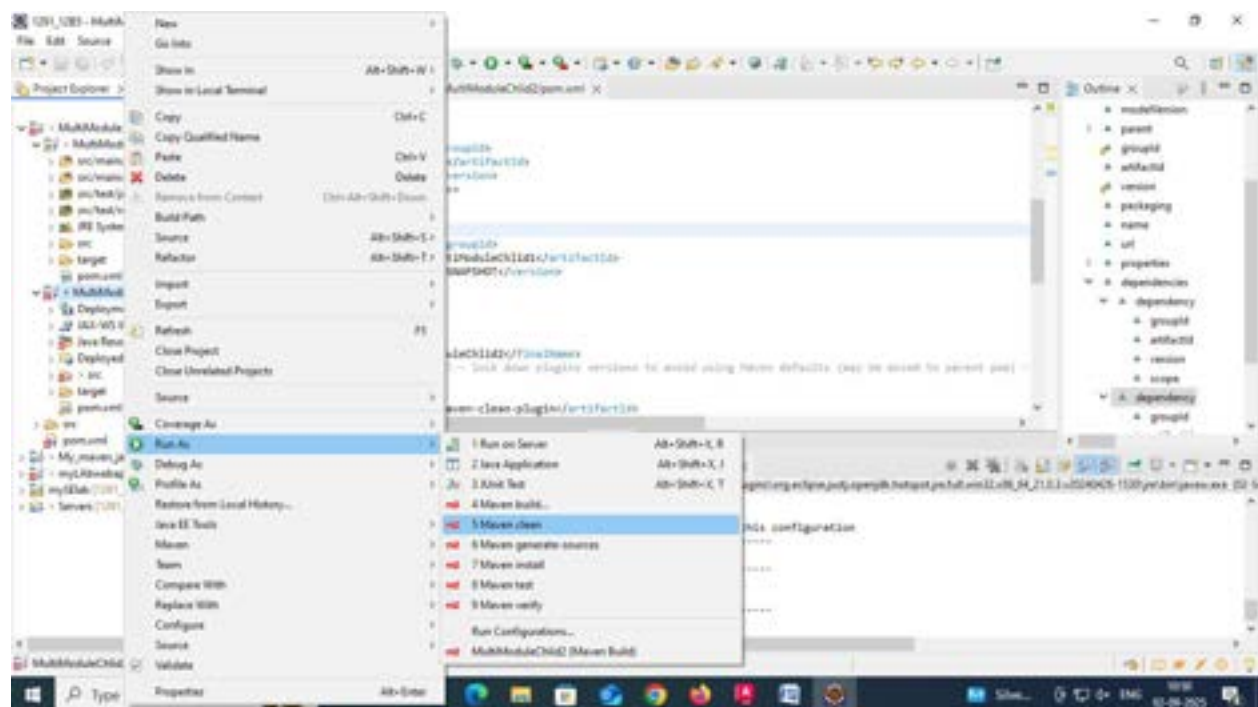


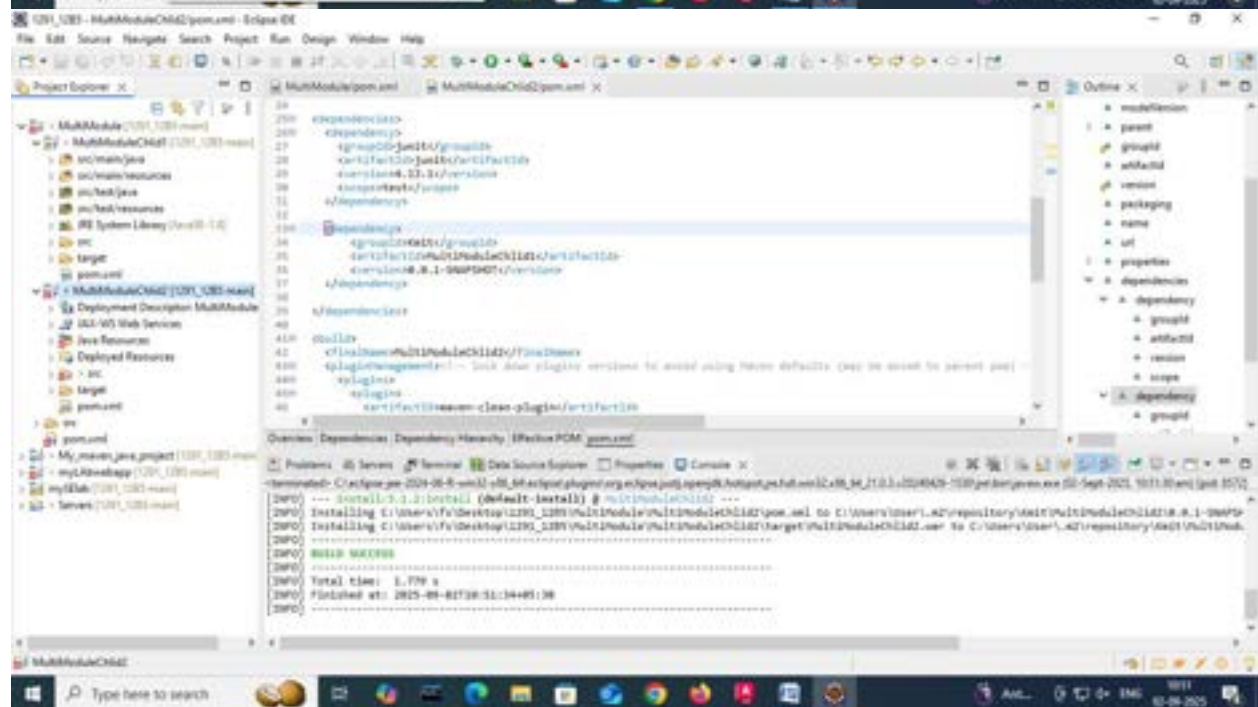
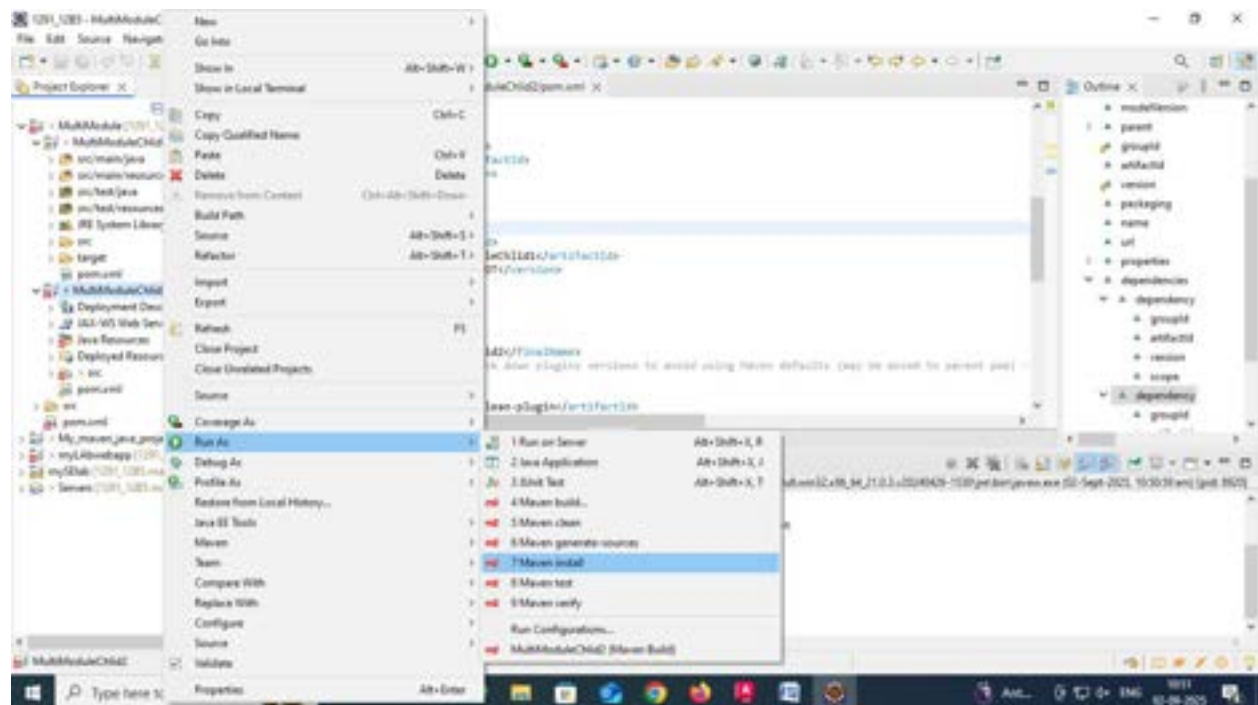
5. After building parent and child1, build child2 now the build will be a *Success*

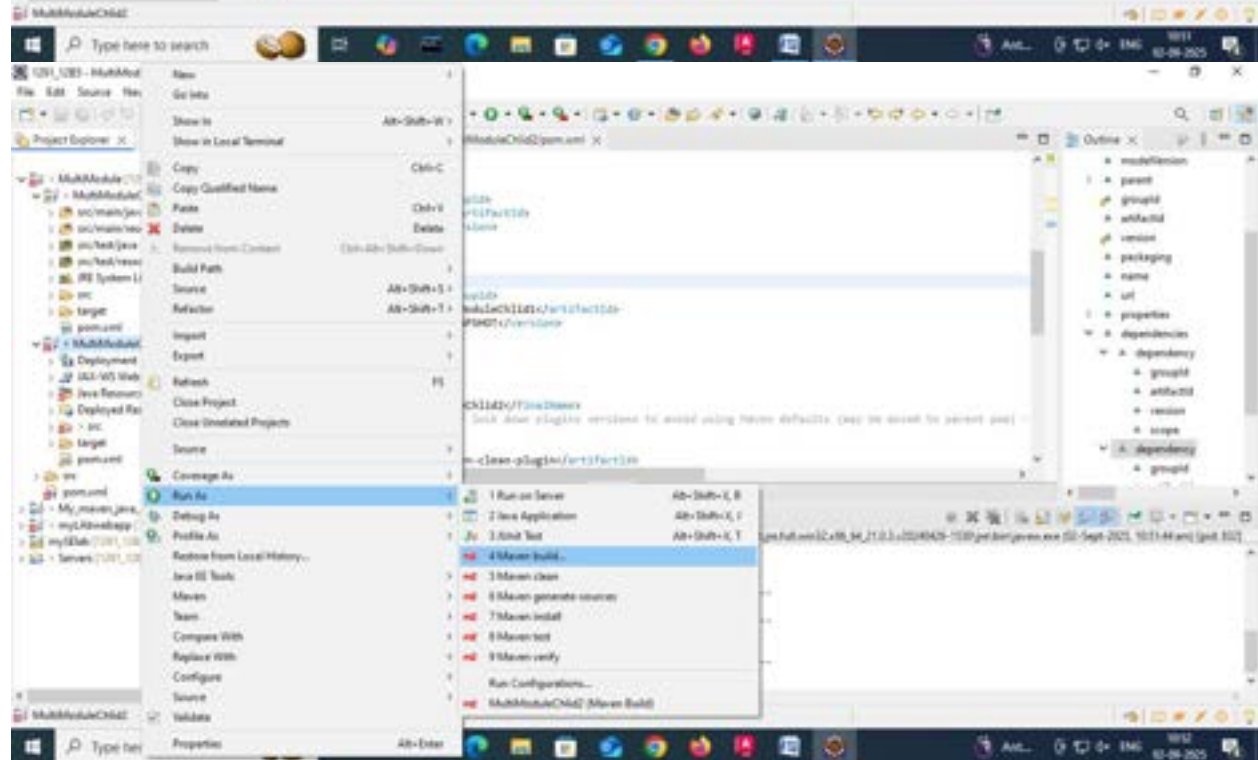
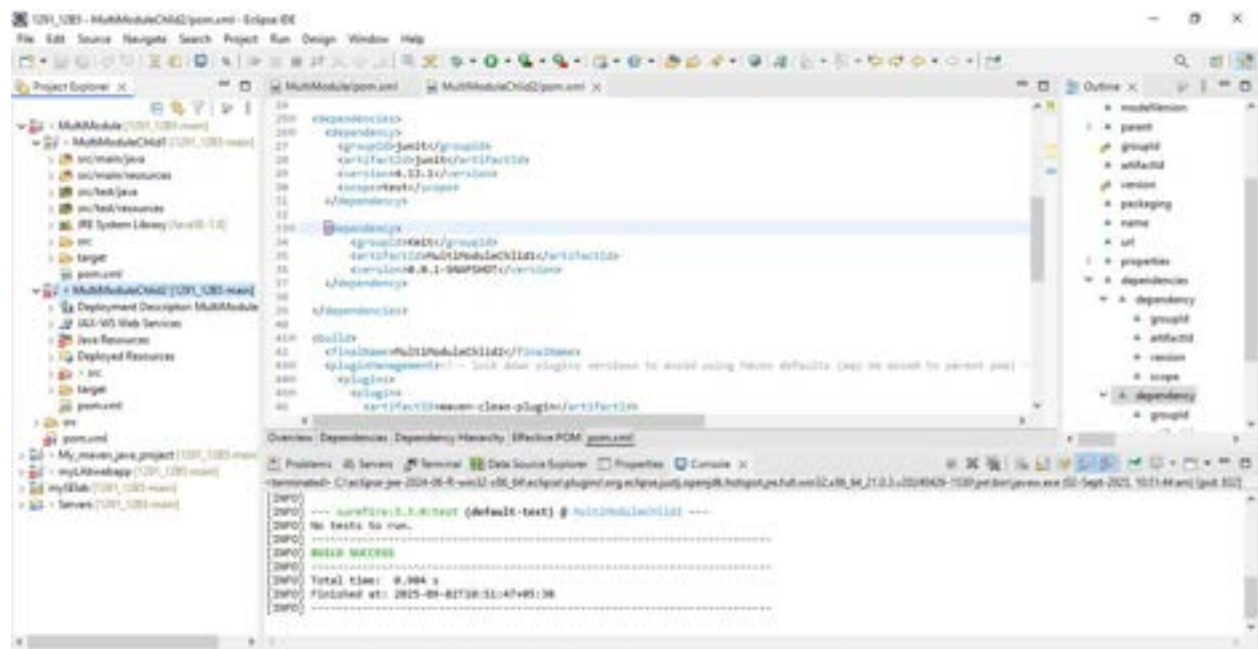


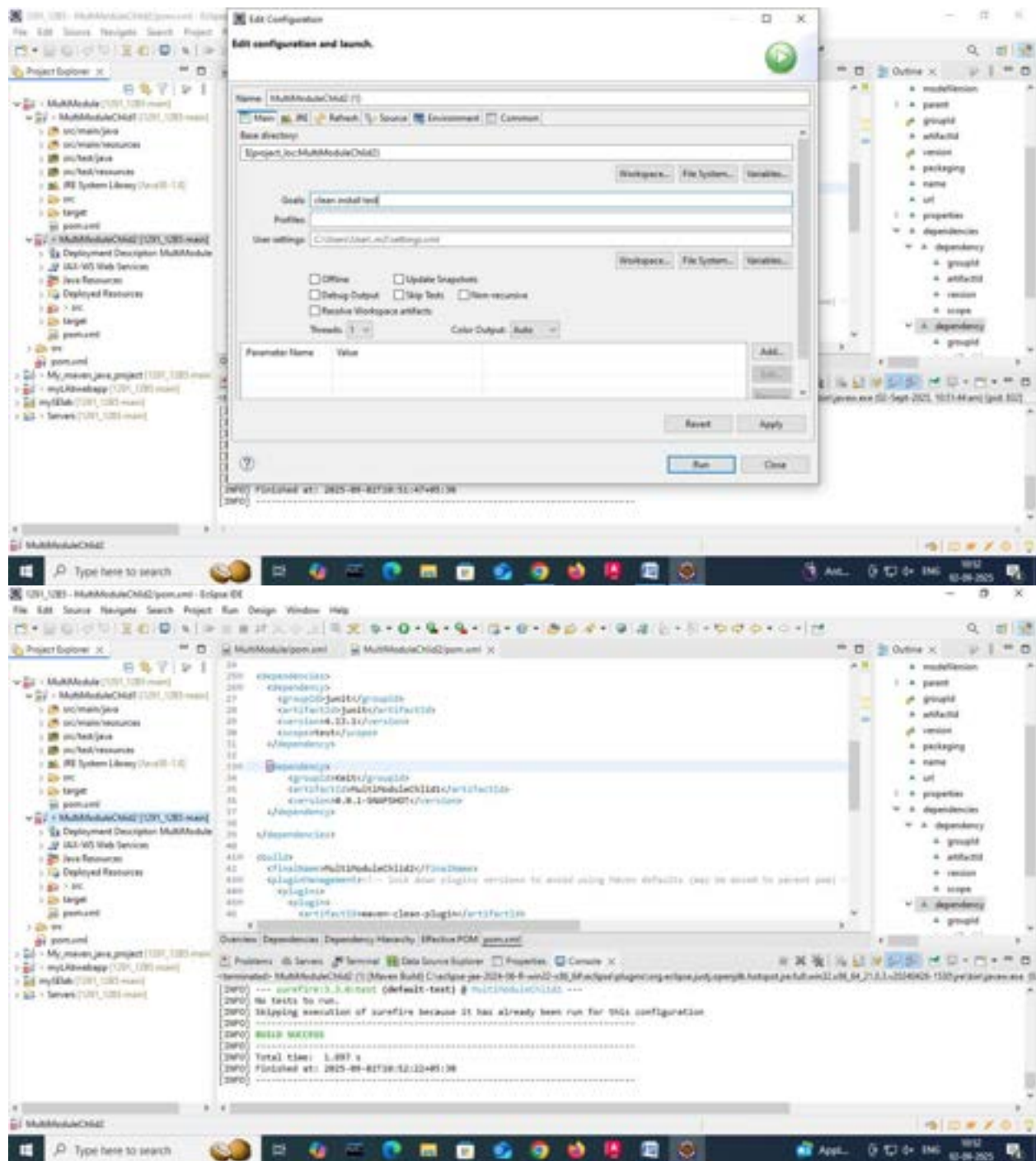












8. Executable JARs

Add to pom.xml:

```
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<configuration>
<archive>
<manifest>
<mainClass>com.example.Main</mainClass>
</manifest>
</archive>
</configuration>
</plugin>
</plugins>
</build>
```

Run:

```
mvn package
java -jar target/myapp.jar
```

9. Building a WAR File

Create a Maven web project with structure:

```
src/main/webapp/
└─ WEB-INF/web.xml
```

Add:

```
<packaging>war</packaging>
```

Command:

```
mvn package
```

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.

Week 8: Jenkins Automation

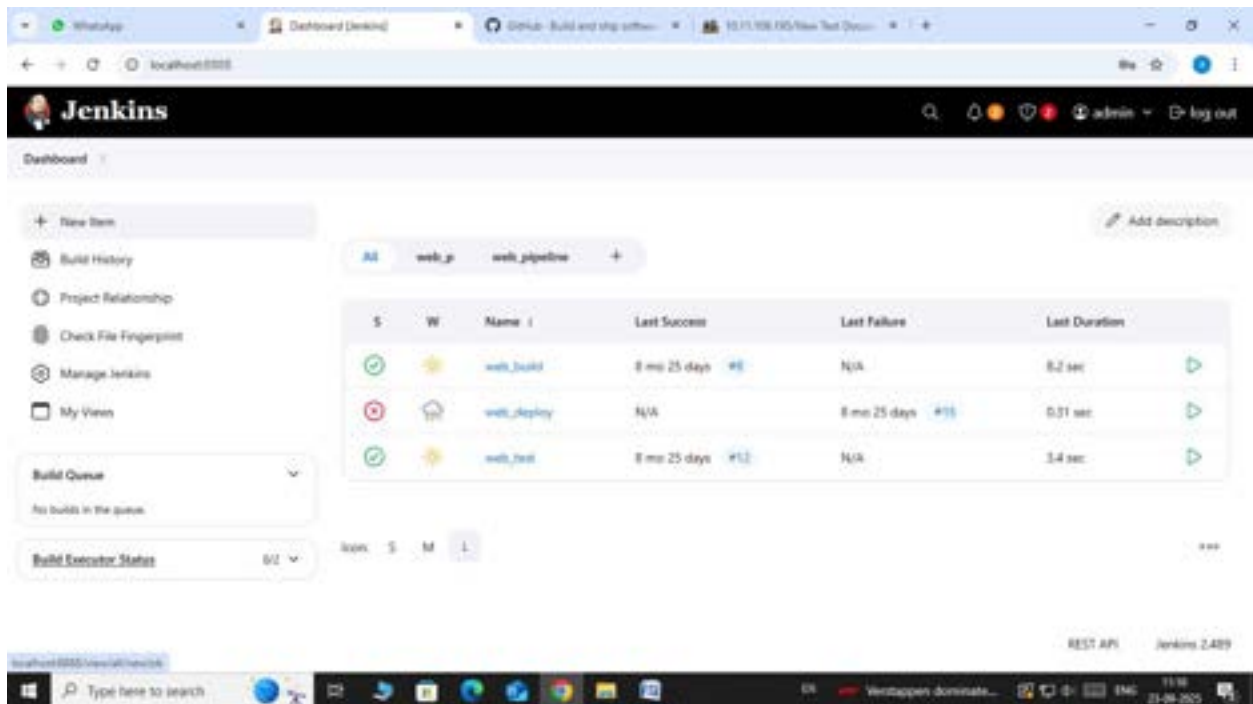
- I. Hands-on practice on manual creation of Jenkins pipeline using Maven projects from Github
- II. Create the job and build the pipeline for maven-java and maven-web project.
- III. Questions on Jenkins
- IV. Upload the Screenshots.

I. Steps for MavenJava Automation:

Maven Java Automation Steps:

Step 1: Open Jenkins (localhost:8080)

└─ Click on "New Item" (left side menu)



Step 2: Create Freestyle Project (e.g., MavenJava_Build)

└─ Enter project name (e.g., MavenJava_Build)

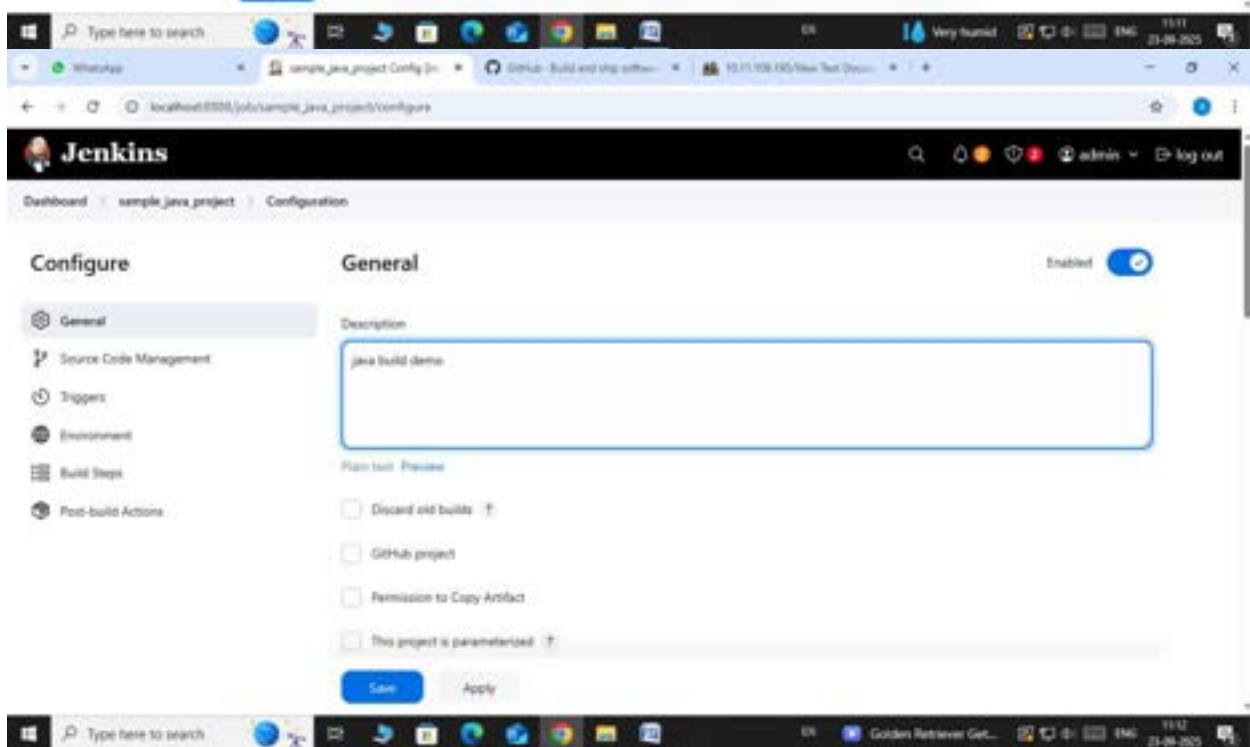
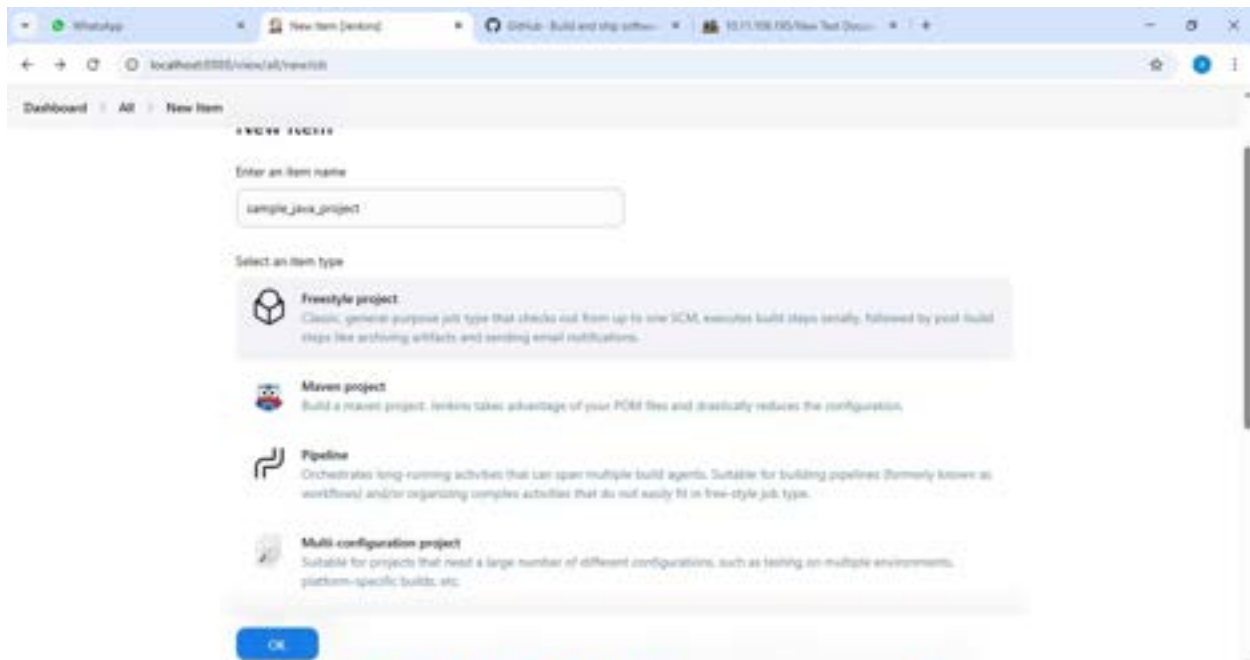
└─ Click "OK"

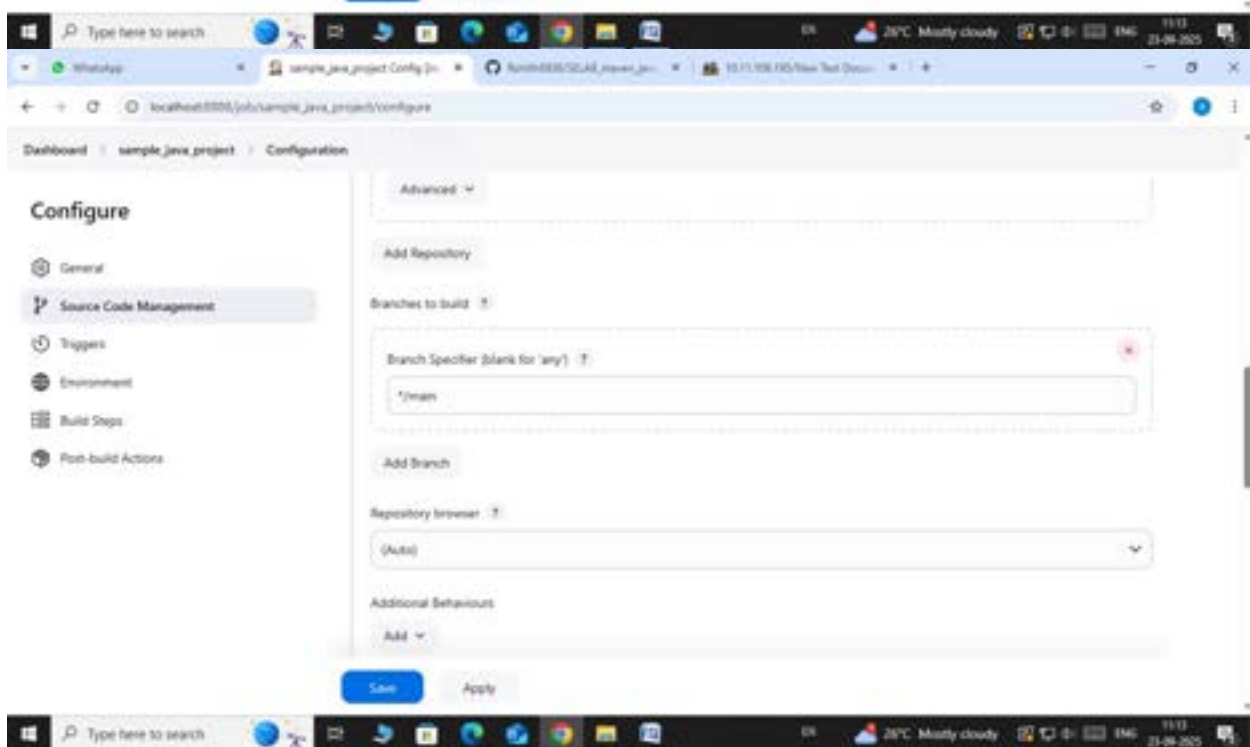
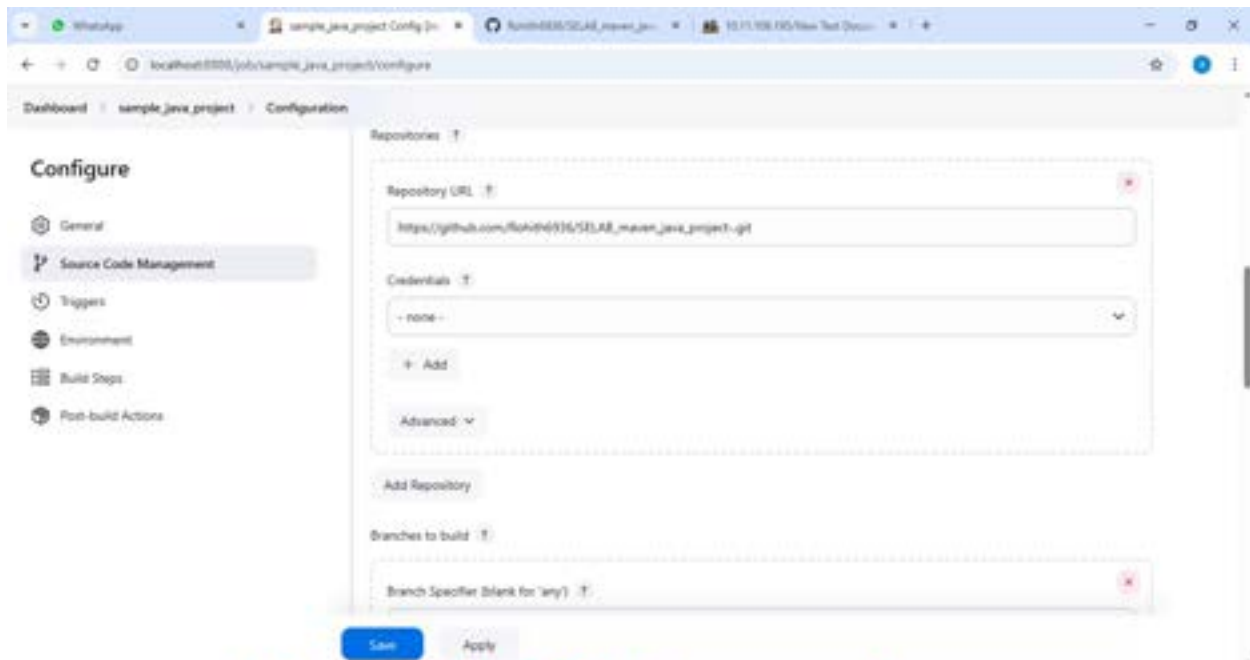
└─ **Configure the project:**

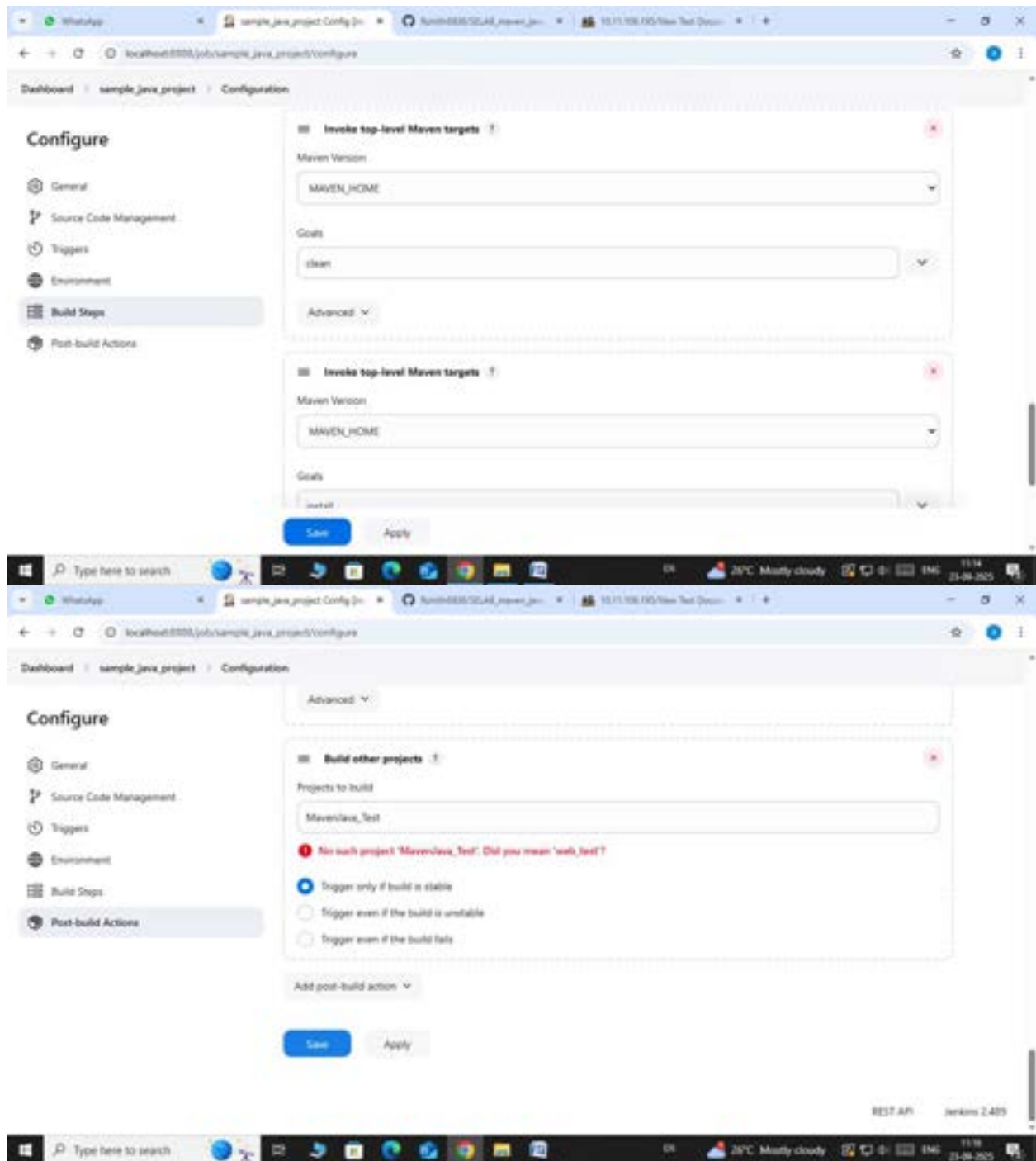
└─ **Description:** "Java Build demo"

└─ **Source Code Management:**

- └─ Git repository URL: [GitMavenJava repo URL]
- └─ **Branches to build:** */Main or */master
- └─ **Build Steps:**
 - └─ **Add Build Step** -> "Invoke top-level Maven targets"
 - └─ Maven version: MAVEN_HOME
 - └─ Goals: clean
 - └─ **Add Build Step** -> "Invoke top-level Maven targets"
 - └─ Maven version: MAVEN_HOME
 - └─ Goals: install
- └─ **Post-build Actions:**
 - └─ Add Post Build Action -> "Archive the artifacts"
 - └─ Files to archive: **/*
 - └─ Add Post Build Action -> "Build other projects"
 - └─ Projects to build: MavenJava_Test
 - └─ Trigger: Only if build is stable
- └─ Apply and Save







└─ Step 3: Create Freestyle Project (e.g., MavenJava_Test)

└─ Enter project name (e.g., MavenJava_Test)

└─ Click "OK"

└─ Configure the project:

- |— **Description:** "Test demo"

- |— **Build Environment:**

- └— Check: "Delete the workspace before build starts"

- |— **Add Build Step** -> "Copy artifacts from another project"

- └— Project name: MavenJava_Build

- └— Build: Stable build only // tick at this

- └— Artifacts to copy: **/*

- |— **Add Build Step** -> "Invoke top-level Maven targets"

- └— Maven version: MAVEN_HOME

- └— Goals: test

- └— Post-build Actions:

- |— **Add Post Build Action** -> "Archive the artifacts"

- └— Files to archive: **/*

- └— Apply and Save

- └— **Step 4:** Create Pipeline View for Maven Java project

- |— Click "+" beside "All" on the dashboard

- |— Enter name: MavenJava_Pipeline

- |— Select "**Build pipeline view**" // tick here

- |— create

- └— **Pipeline Flow:**

- |— **Layout:** Based on upstream/downstream relationship

- |— Initial job: MavenJava_Build

- └— Apply and Save OK

Dashboard > All > New Item

New Item

Enter an item name

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to nine SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project, Jenkins takes advantage of your POM files and drastically reduces the configuration.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

OK

Dashboard > MavenJava_Test > Configuration

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) [?](#)
- ☐ Build after other projects are built [?](#)
- ☐ Build periodically [?](#)
- ☐ GitHub hook trigger for GITScm polling [?](#)
- ☐ Poll SCM [?](#)

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

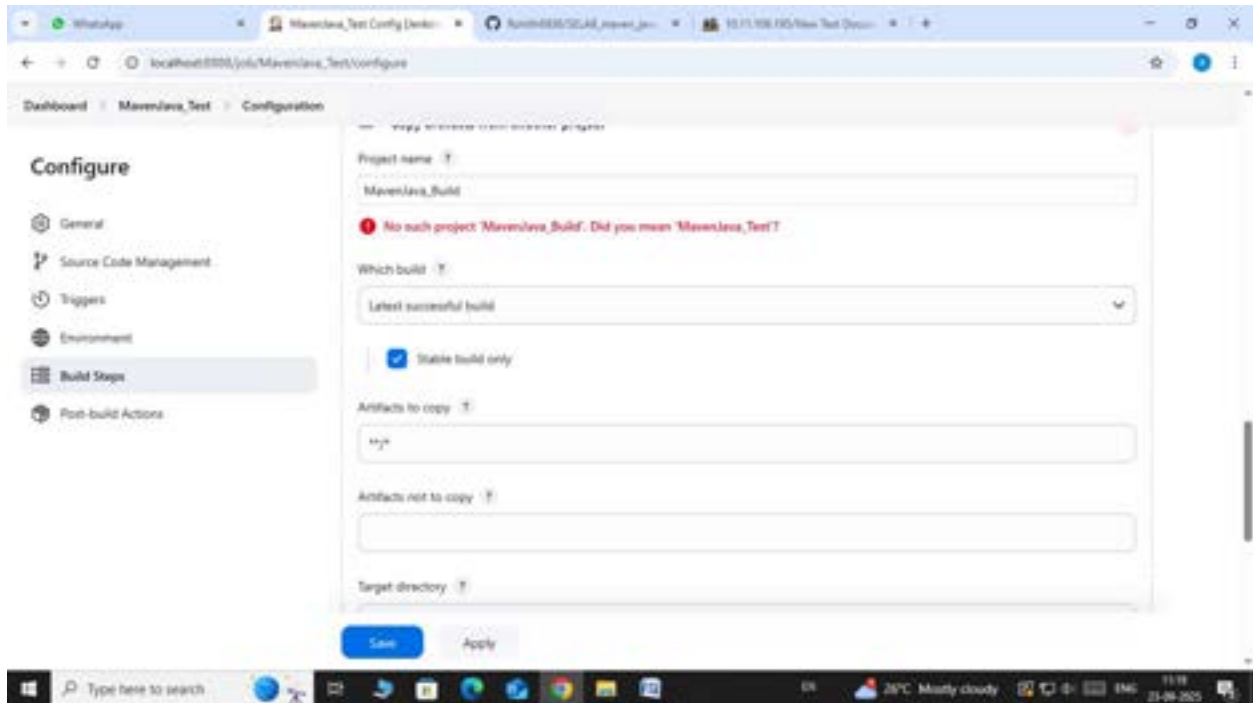
☒ Delete workspace before build starts

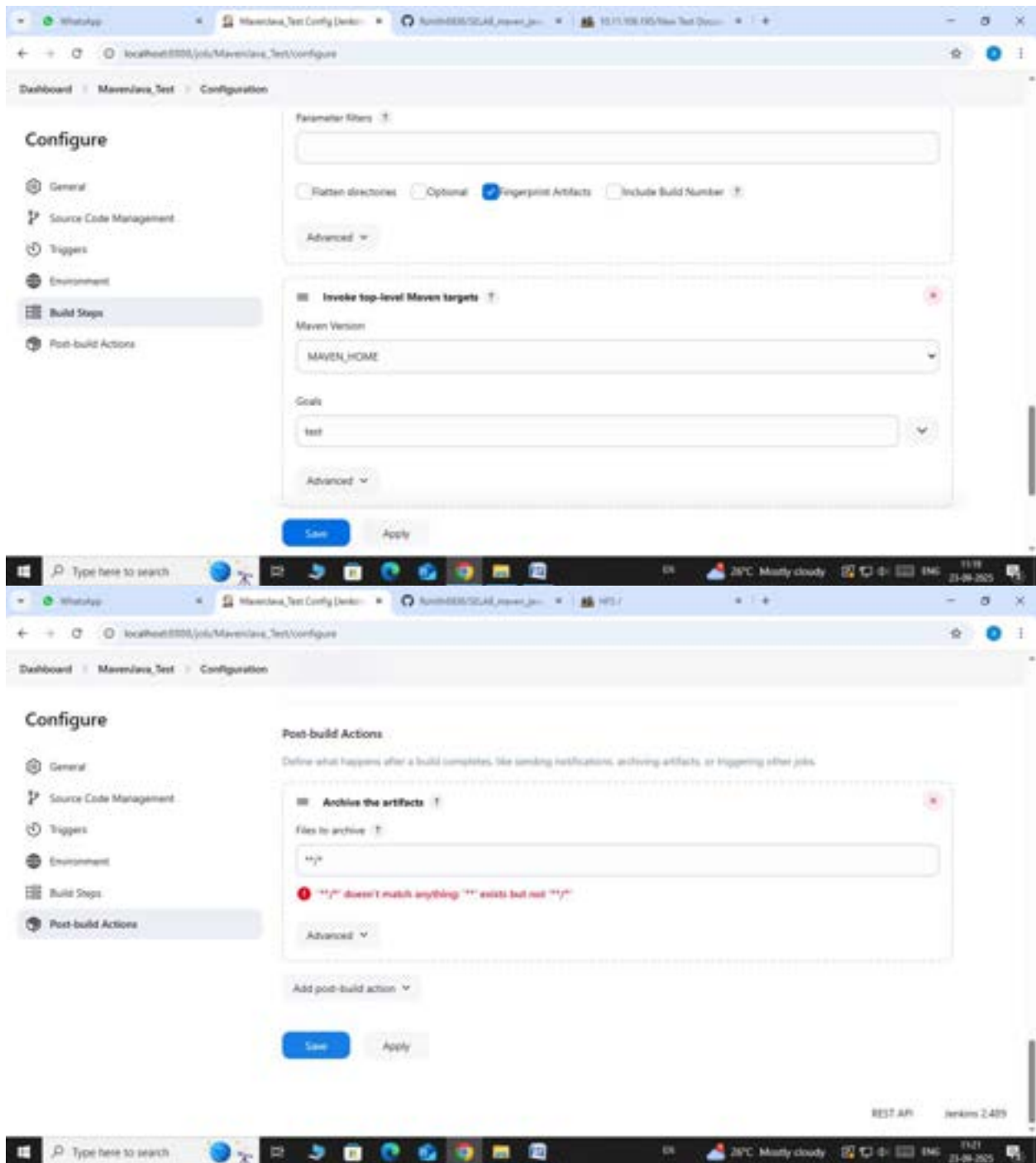
Advanced [?](#)

☐ Use secret text(s) or file(s) [?](#)

Save

Apply

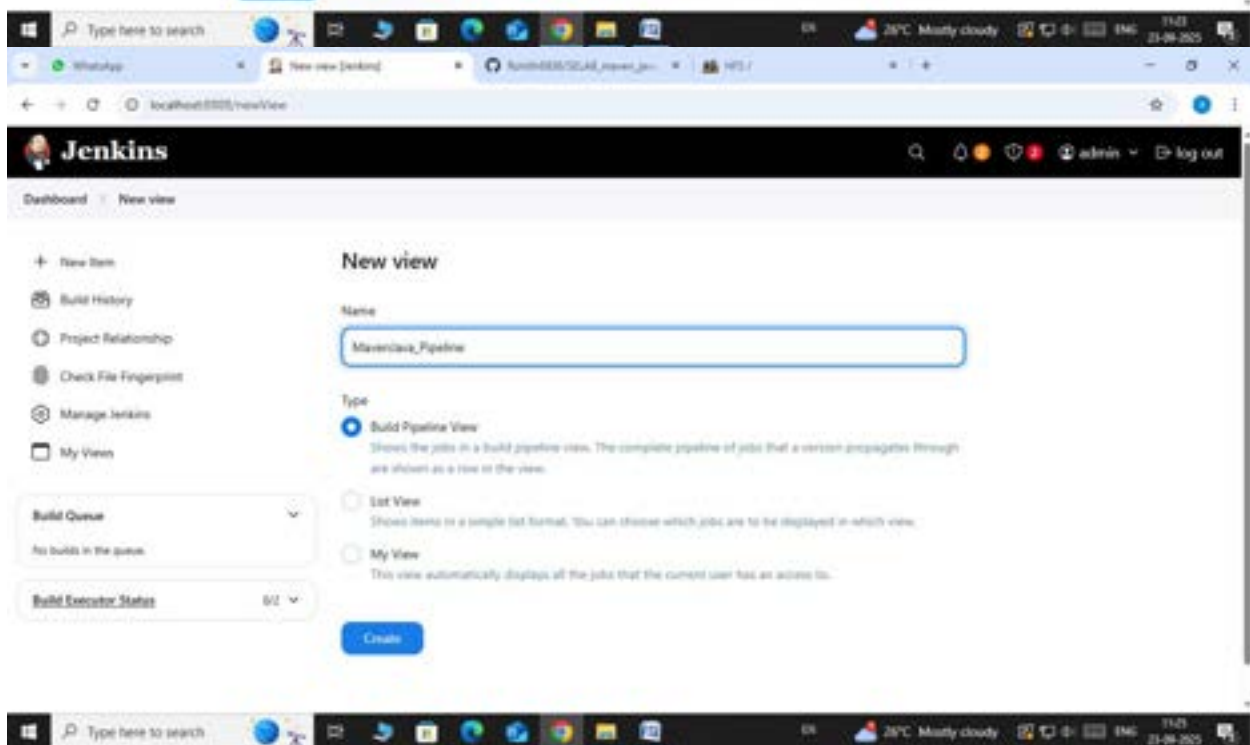
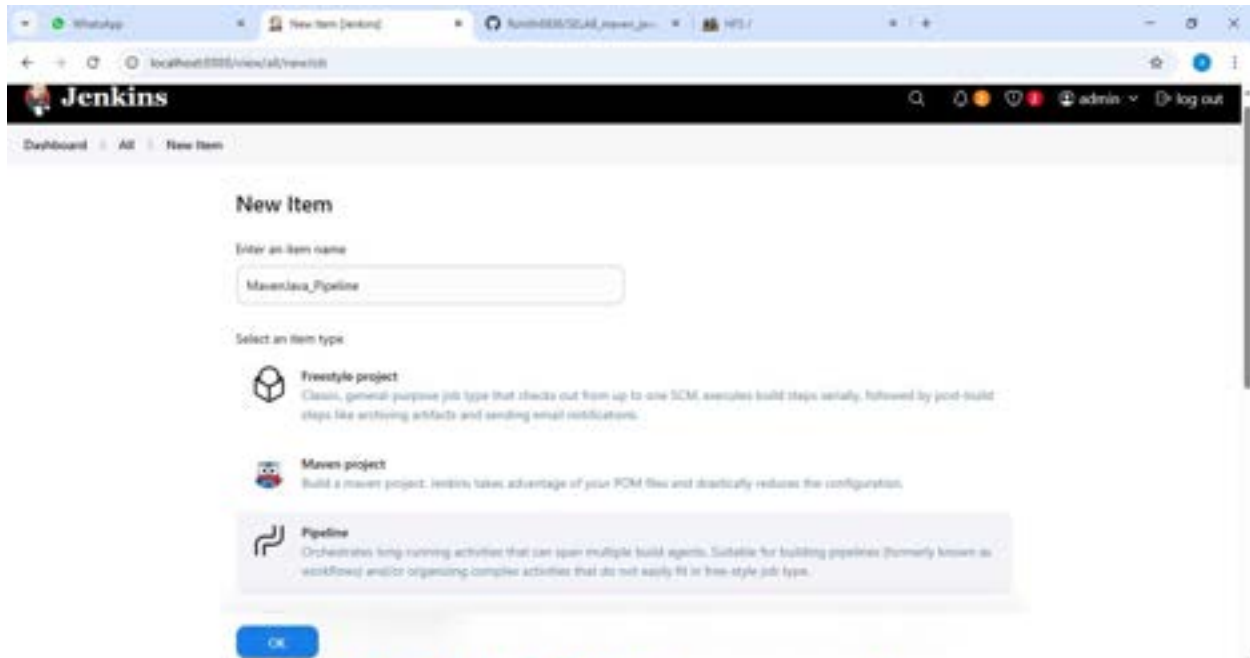


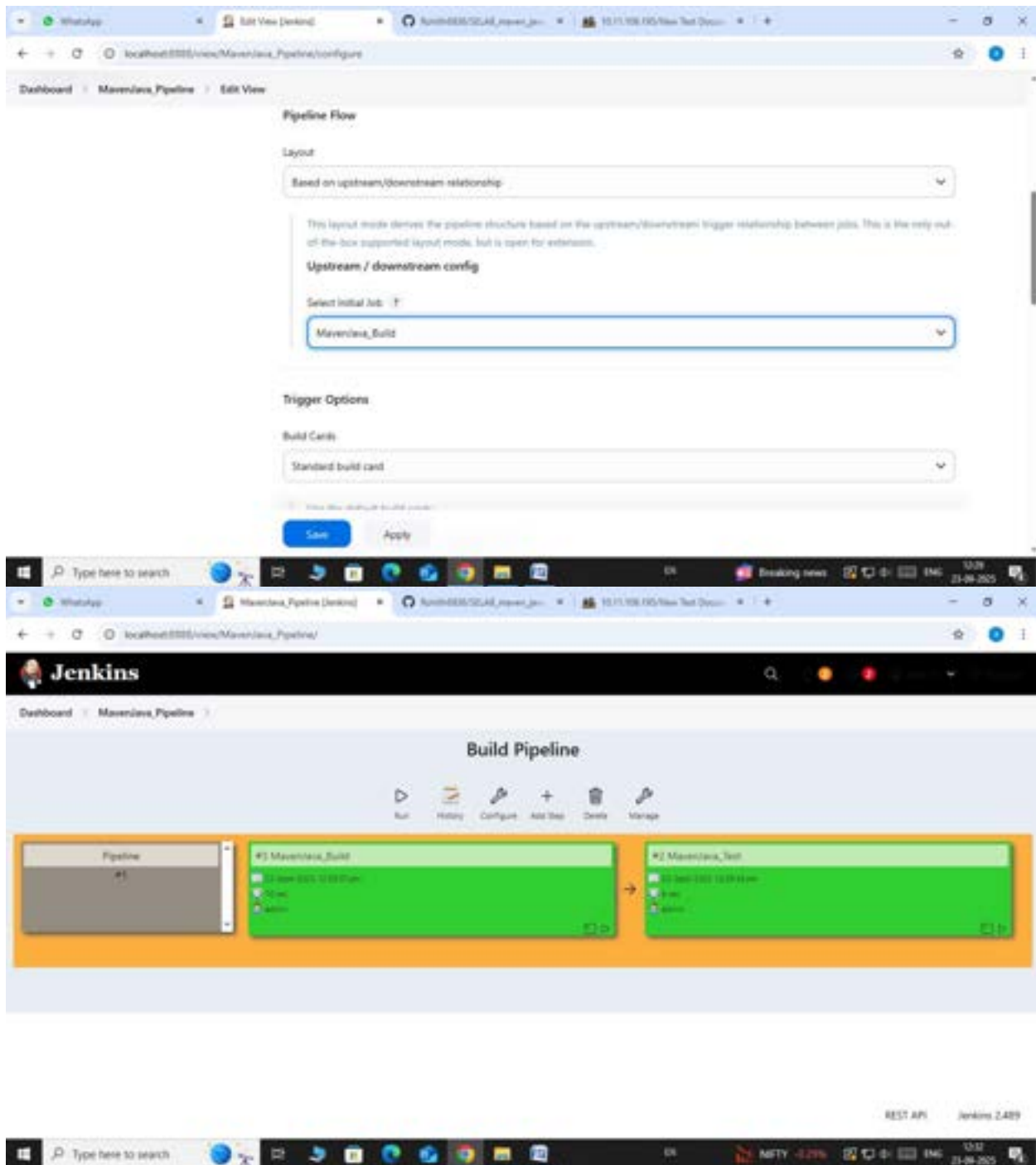


— Step 5: Run the Pipeline and Check Output

— Click on the trigger to run the pipeline

— click on the small black box to open the console to check if the build is success



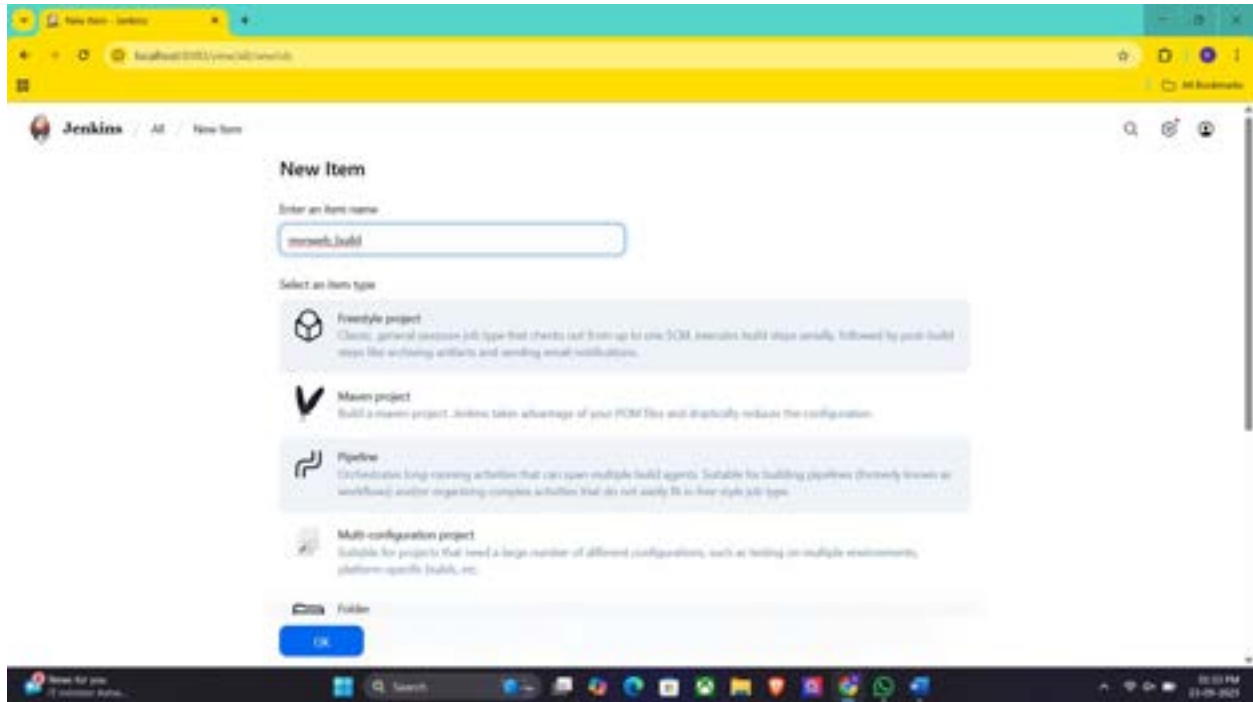


Note :

1. If build is success and the test project is also automatically triggered with name
“MavenJava_Test”
2. The pipeline is successful if it is in green color as shown ->check the console of the test project
3. The test project is successful and all the artifacts are archived successfully

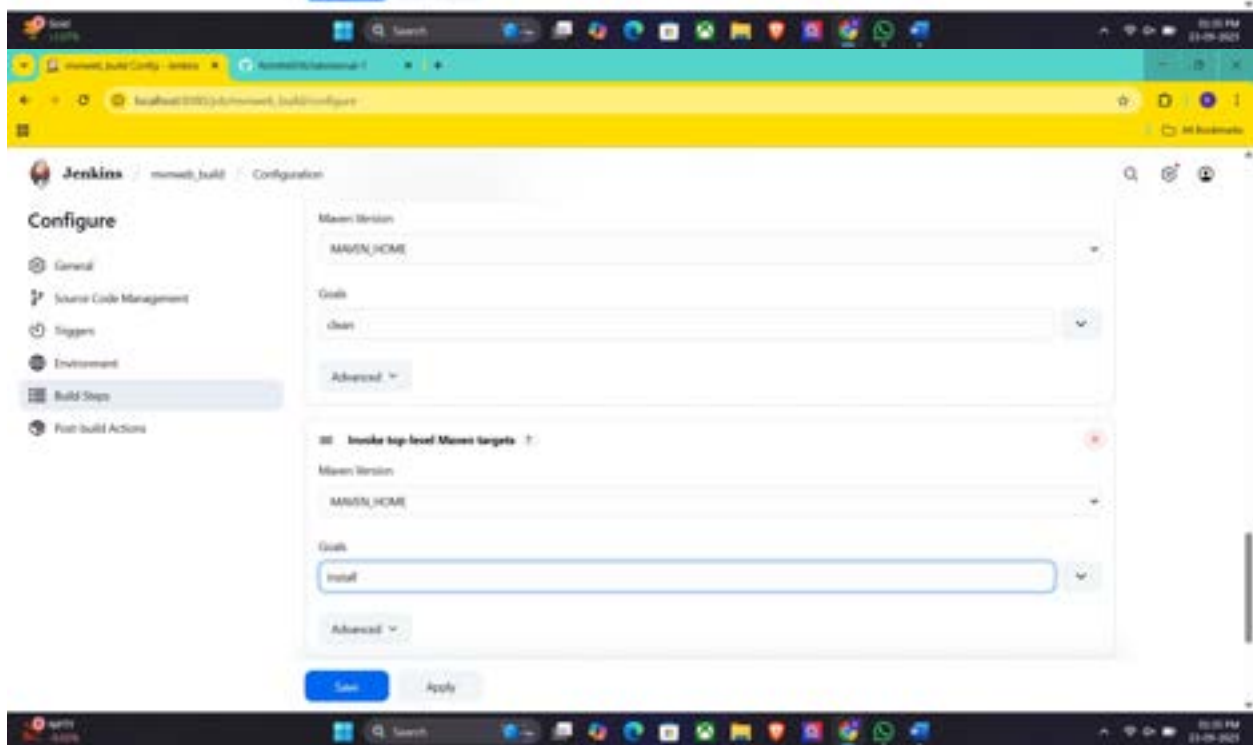
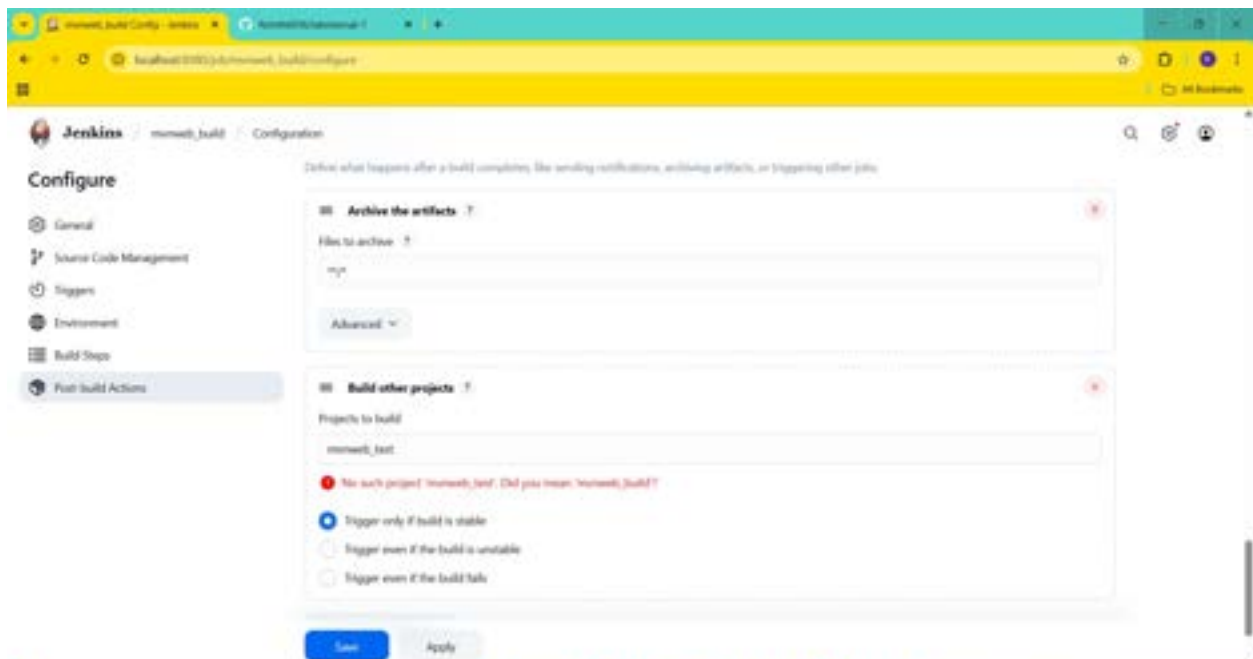
II. Maven Web Automation Steps:

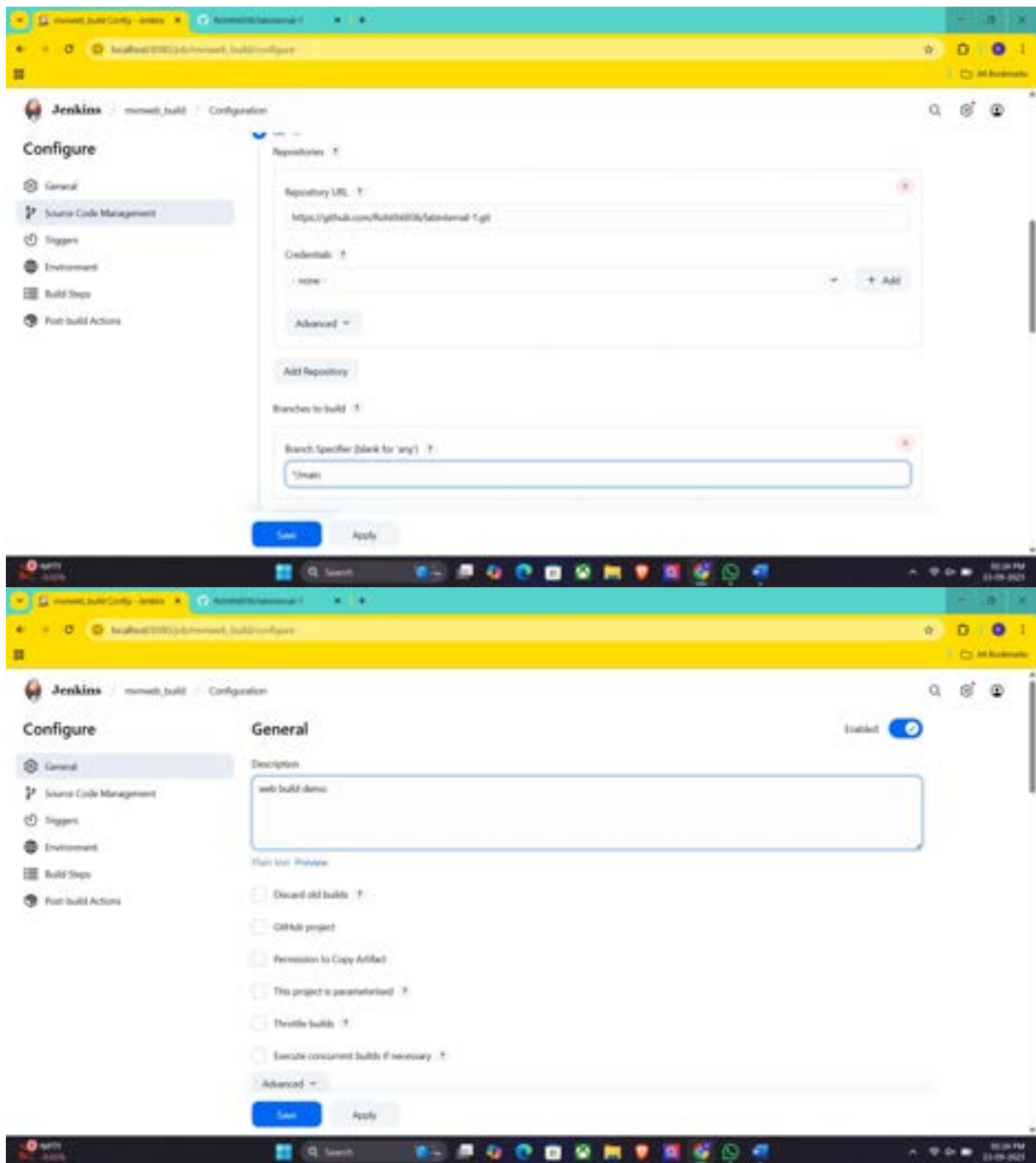
- └─ **Step 1:** Open Jenkins (localhost:8080)
 - └─ Click on "New Item" (left side menu)



- └─ **Step 2:** Create Freestyle Project (e.g., MavenWeb_Build)
 - └─ Enter project name (e.g., MavenWeb_Build)
 - └─ Click "OK"
 - └─ **Configure the project:**
 - └─ **Description:** "Web Build demo"
 - └─ **Source Code Management:**
 - └─ Git repository URL: [GitMavenWeb repo URL]
 - └─ *Branches to build:* */Main or master
 - └─ **Build Steps:**
 - └─ **Add Build Step** -> "Invoke top-level Maven targets"
 - └─ Maven version: MAVEN_HOME
 - └─ Goals: clean

- |— **Add Build Step** -> "Invoke top-level Maven targets"
 - └— Maven version: MAVEN_HOME
 - └— Goals: install
- |— **Post-build Actions:**
 - |— **Add Post Build Action** -> "Archive the artifacts"
 - └— Files to archive: **/*
 - |— **Add Post Build Action** -> "Build other projects"
 - └— Projects to build: MavenWeb_Test
 - └— Trigger: Only if build is stable
- |— Apply and Save





└─ **Step 3: Create Freestyle Project (e.g., MavenWeb_Test)**

└─ Enter project name (e.g., MavenWeb_Test)

└─ Click "OK"

└─ **Configure the project:**

└─ **Description: "Test demo"**

|— **Build Environment:**

└— Check: "Delete the workspace before build starts"

|— **Add Build Step -> "Copy artifacts from another project"**

└— Project name: MavenWeb_Build

└— Build: Stable build only

└— Artifacts to copy: **/*

|— **Add Build Step -> "Invoke top-level Maven targets"**

└— Maven version: MAVEN_HOME

└— Goals: test

|— **Post-build Actions:**

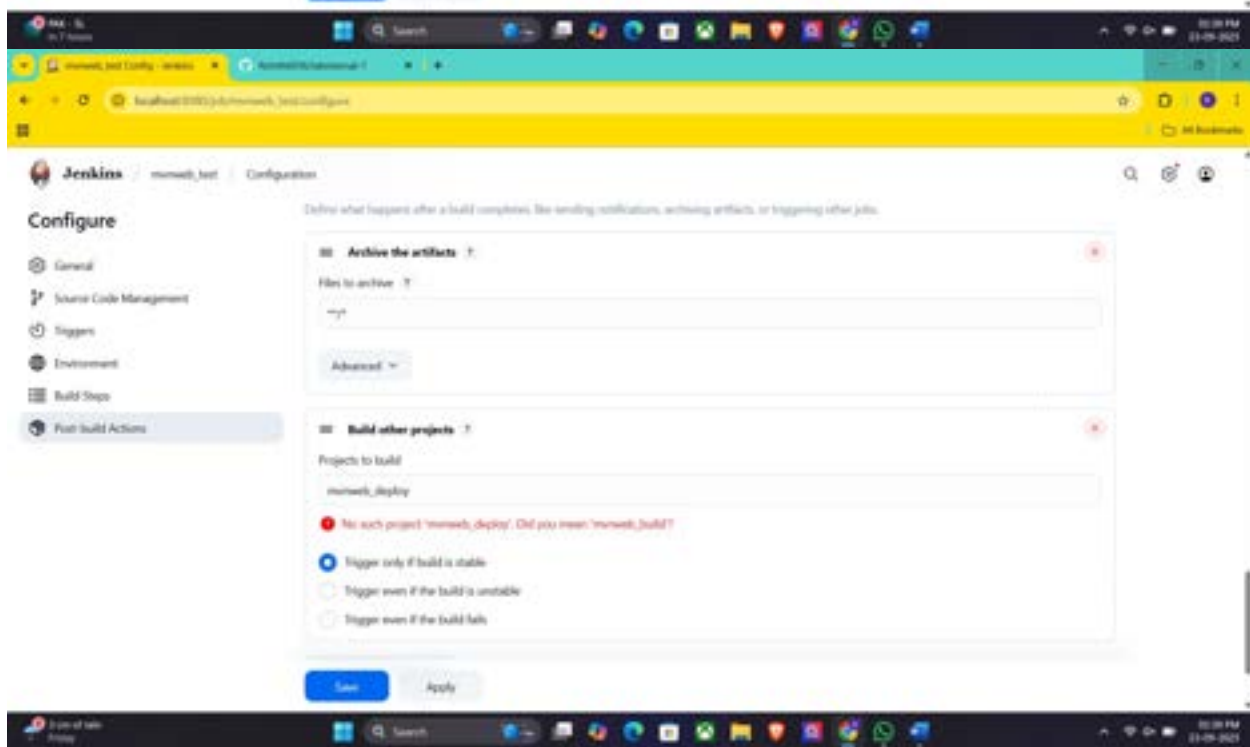
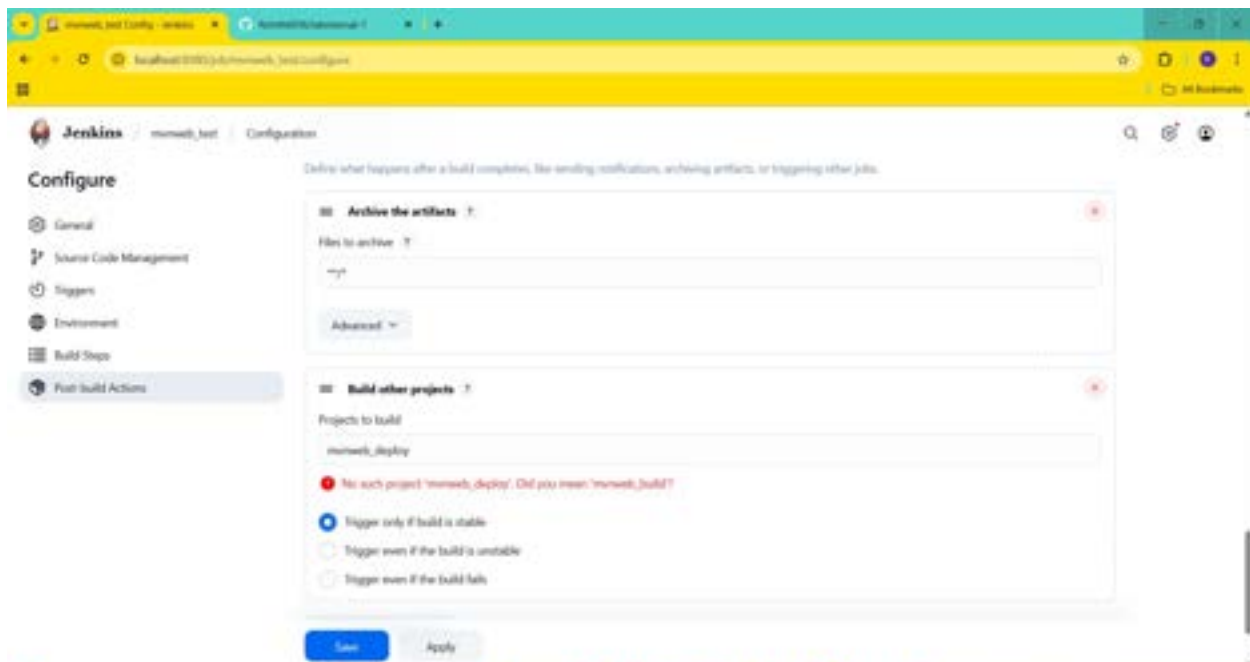
|— **Add Post Build Action -> "Archive the artifacts"**

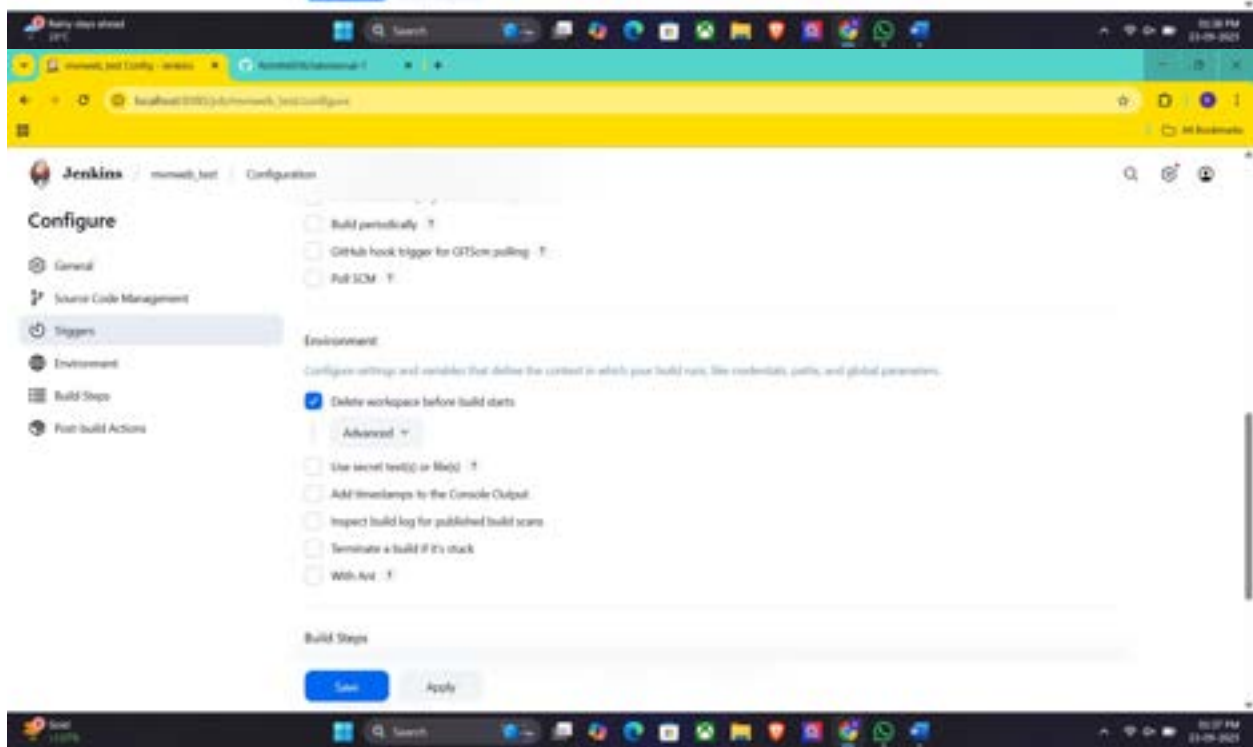
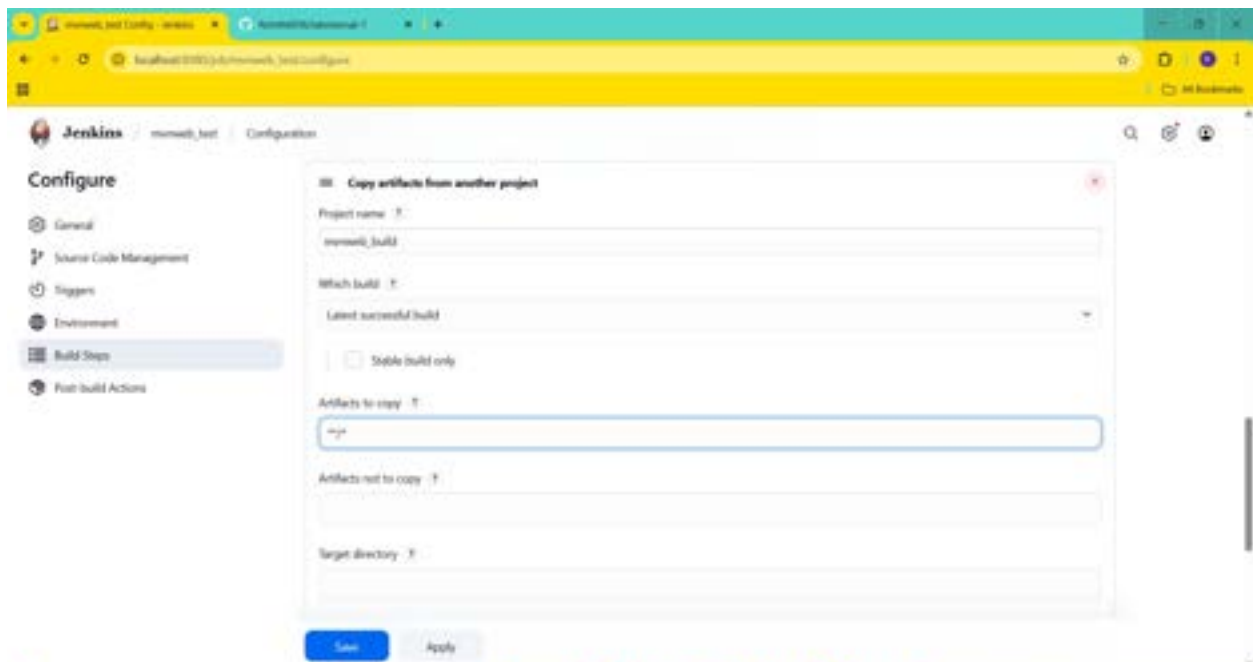
└— Files to archive: **/*

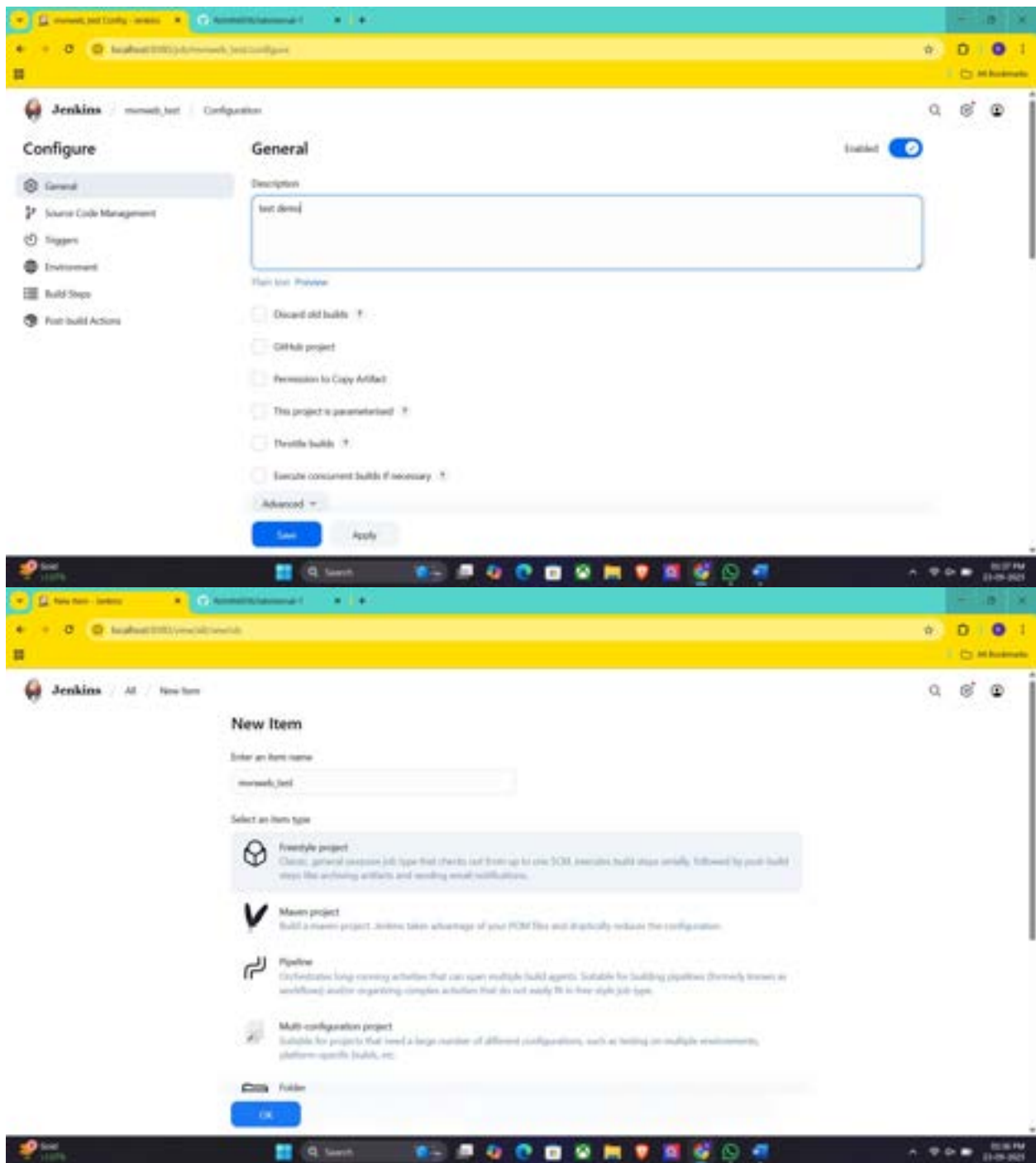
|— **Add Post Build Action -> "Build other projects"**

└— Projects to build: MavenWeb_Deploy

|— Apply and Save







└─ **Step 4: Create Freestyle Project (e.g., MavenWeb_Deploy)**

└─ Enter project name (e.g., MavenWeb_Deploy)

└─ Click "OK"

└─ **Configure the project:**

└─ **Description: "Web Code Deployment"**

|— **Build Environment:**

└— Check: "Delete the workspace before build starts"

|— **Add Build Step -> "Copy artifacts from another project"**

└— Project name: MavenWeb_Test

└— Build: Stable build only

└— Artifacts to copy: **/*

|— **Post-build Actions:**

|— **Add Post Build Action -> "Deploy WAR/EAR to a container"**

└— WAR/EAR File: **/*.war

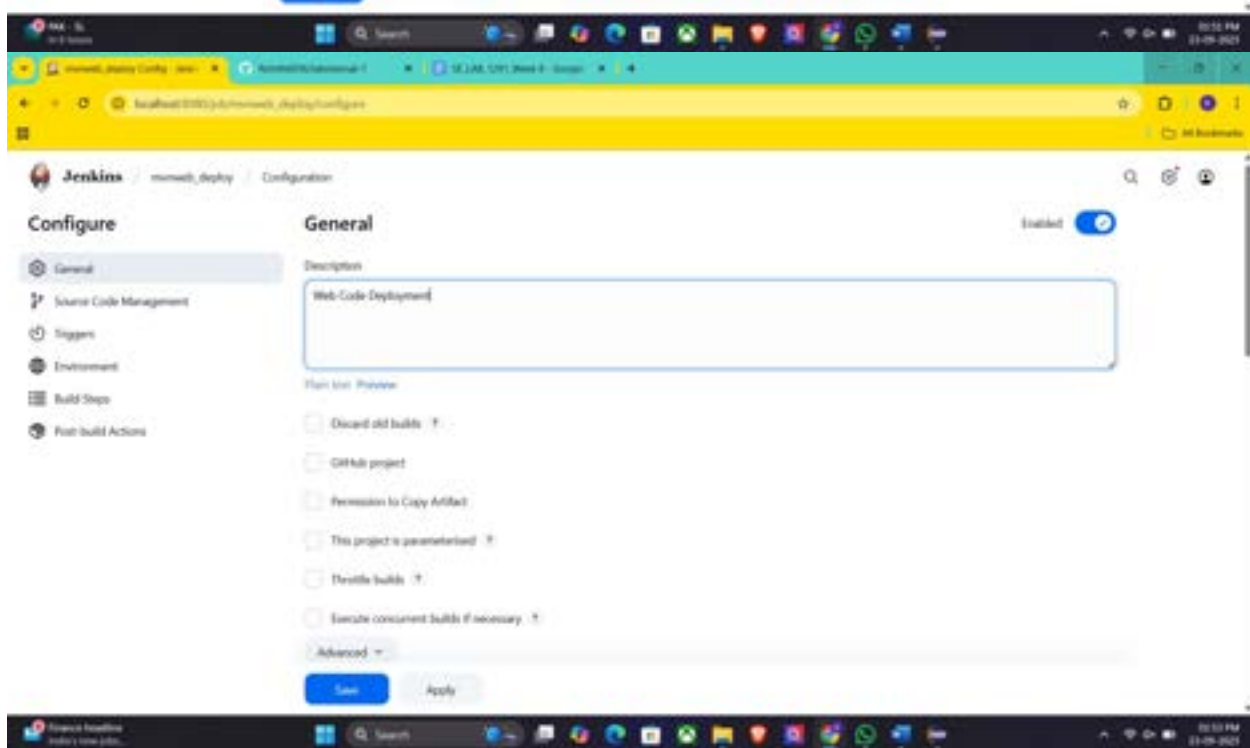
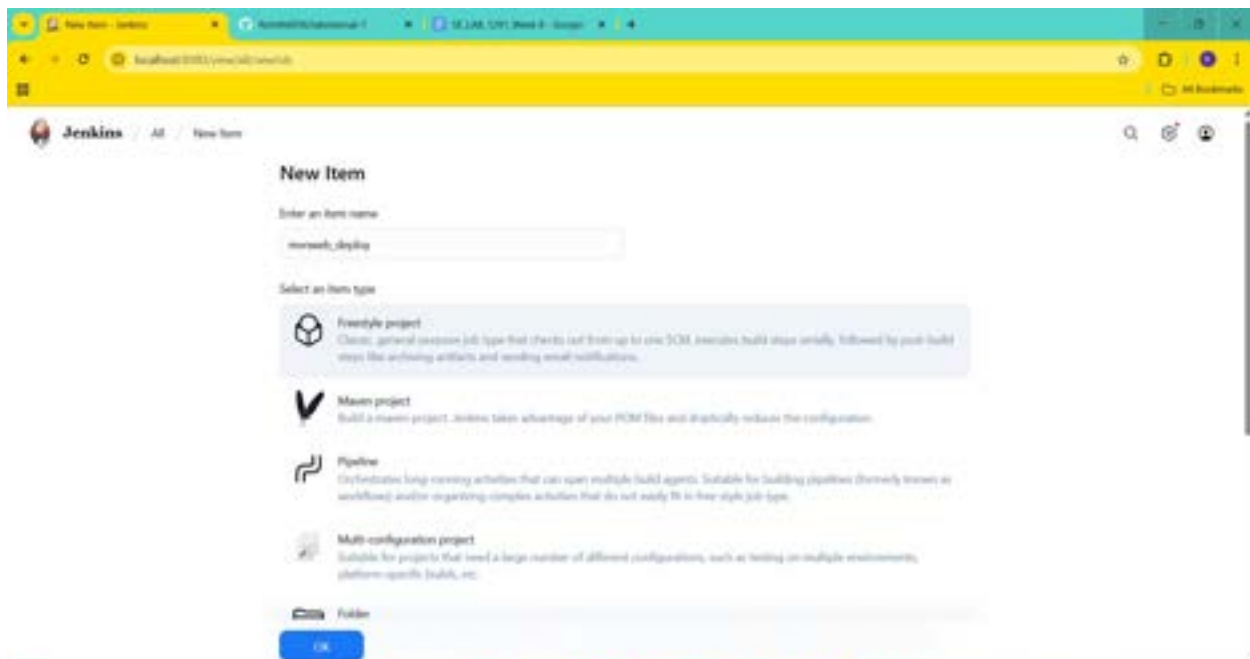
└— Context path: Webpath

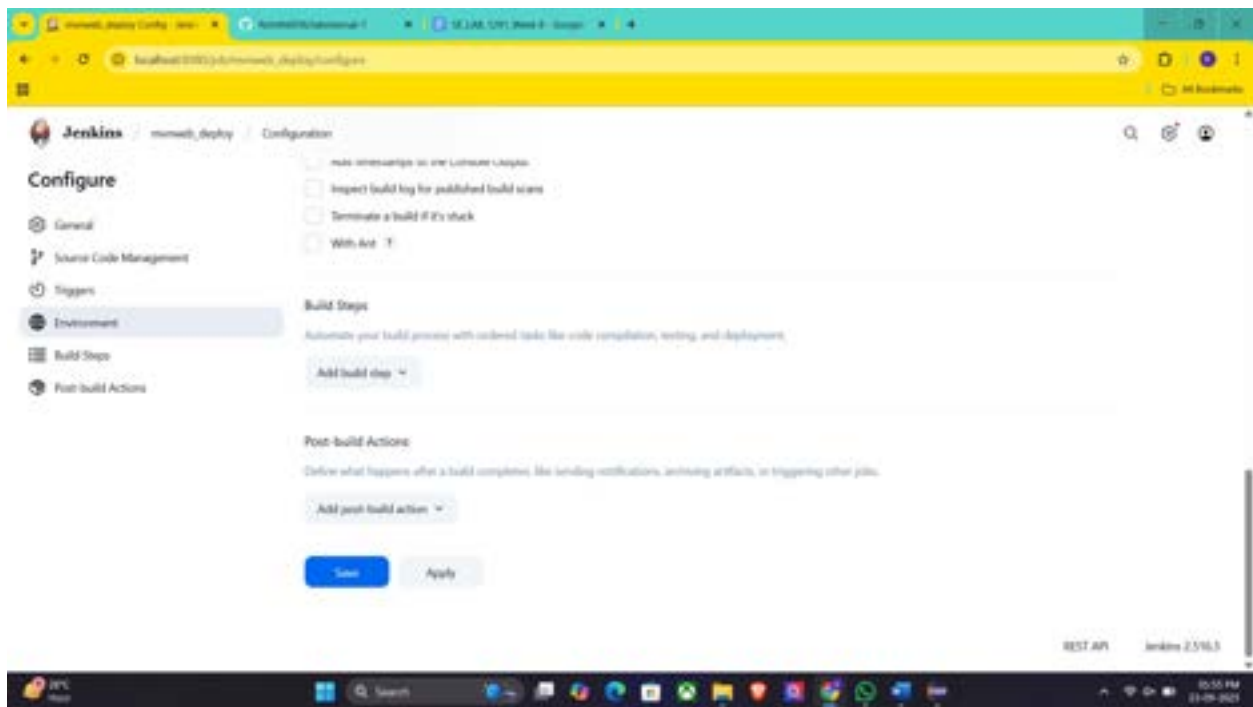
└— Add container -> Tomcat 9.x remote

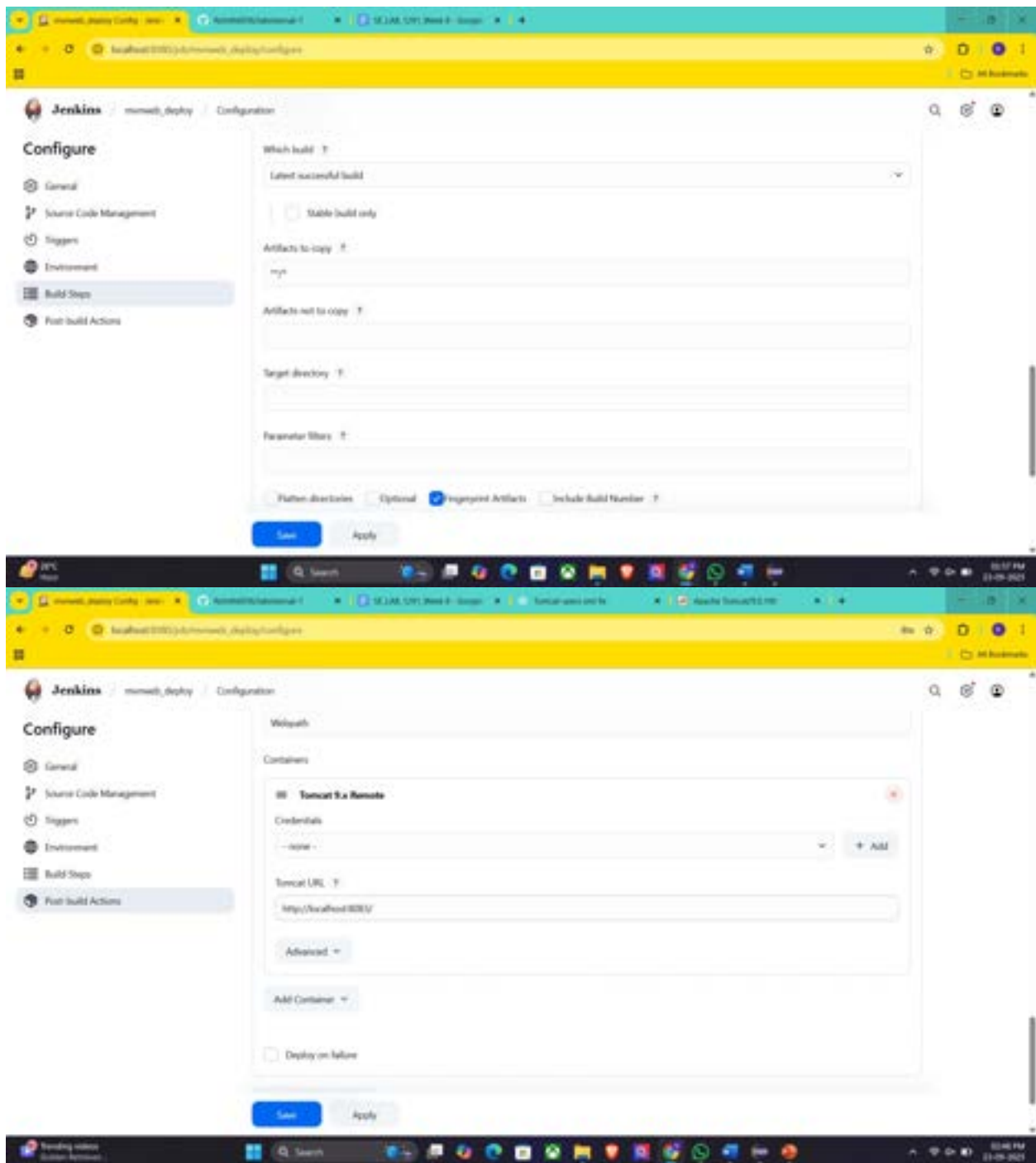
└— Credentials: Username: admin, Password: 1234

└— Tomcat URL: https://localhost:8085/

|— Apply and Save







└─ **Step 5: Create Pipeline View for MavenWeb**

└─ Click "+" beside "All" on the dashboard

└─ Enter name: MavenWeb_Pipeline

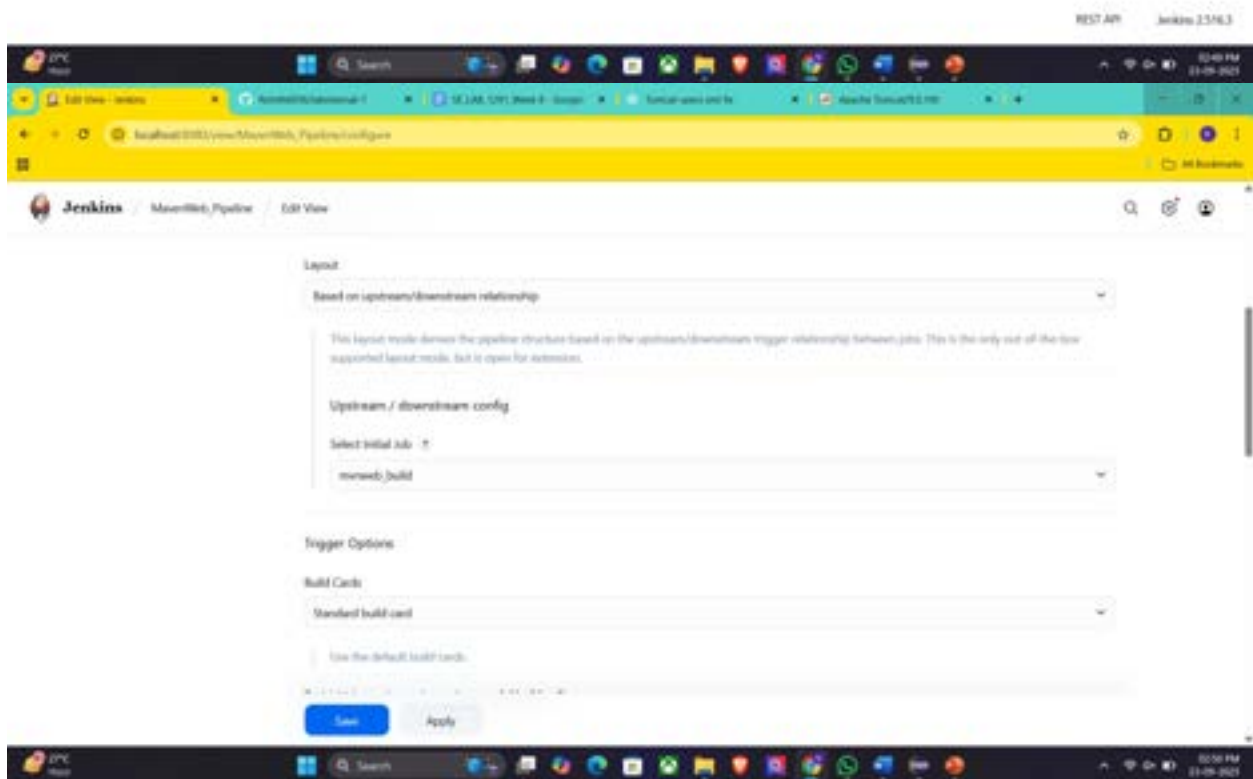
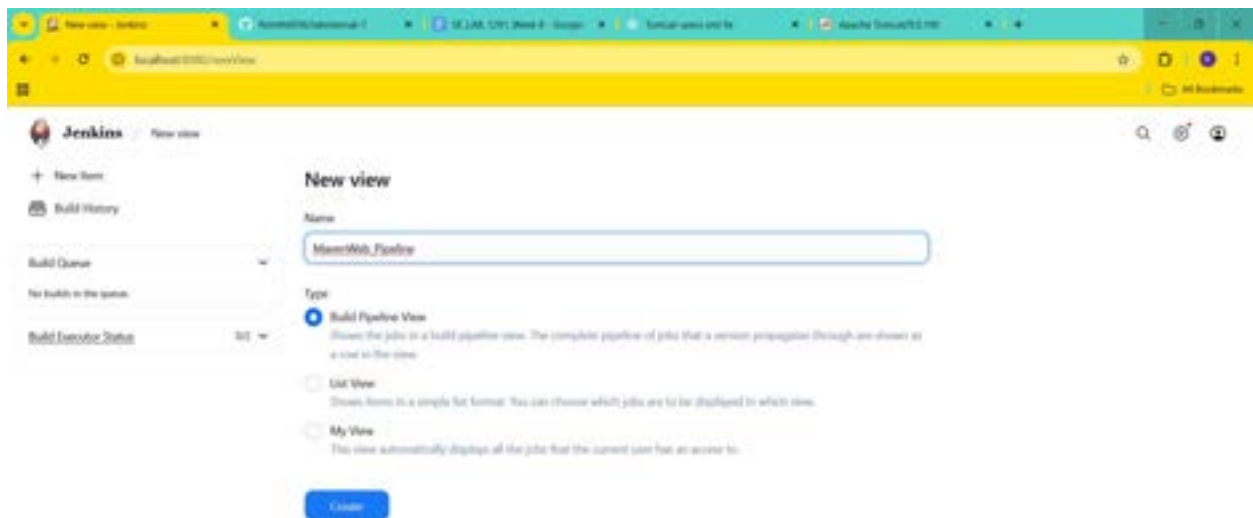
└─ **Select "Build pipeline view"**

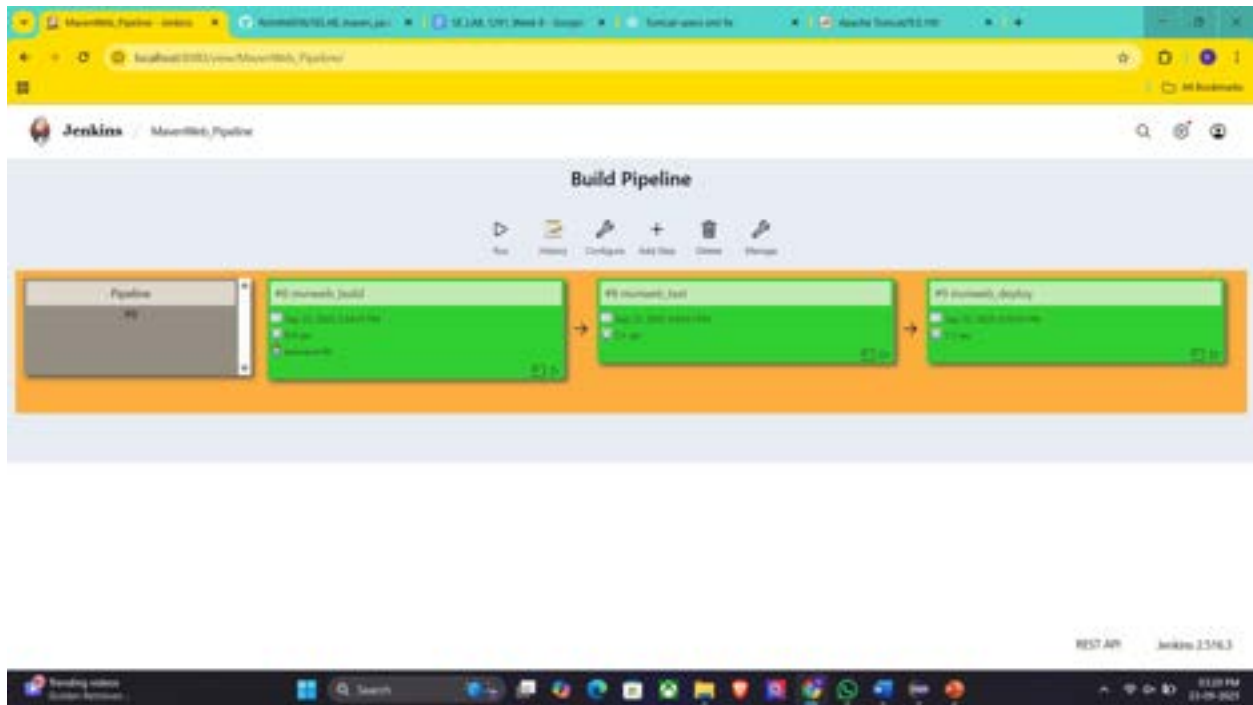
└─ **Pipeline Flow:**

└─ **Layout:** Based on upstream/downstream relationship

└─ Initial job: MavenWeb_Build

└─ Apply and Save





— Step 6: Run the Pipeline and Check Output

— Click on the trigger “**RUN**” to run the pipeline

Note:

1. After Click on Run -> click on the small black box to open the console to check if the build is success
2. Now we see all the build has success if it appears in green color

— Open Tomcat homepage in another tab

— Click on the "/webpath" option under the manager app

Note:

1. It ask for user credentials for login ,provide the credentials of tomcat.
2. It provide the page with out project name which is highlighted.
3. After clicking on our project we can see output.

Conclusion: In this week student learnt automating Maven projects through Jenkins.

Week 9: Pipeline Creation using script

1. Evaluation of Jenkins pipeline.
2. **WORKING ON BUILD TRIGGERS FOR LAST JENKINS PIPELINE**
3. Hands-on practice on creation of scripted Jenkins pipeline.
4. Take the screenshots for above task

Procedure

General :

Description :- provide the description of the project

Build triggers: here we can provide with build triggers of you choice

Advance project option:

Definition:

Choose pipeline script

Here code for pipeline -script

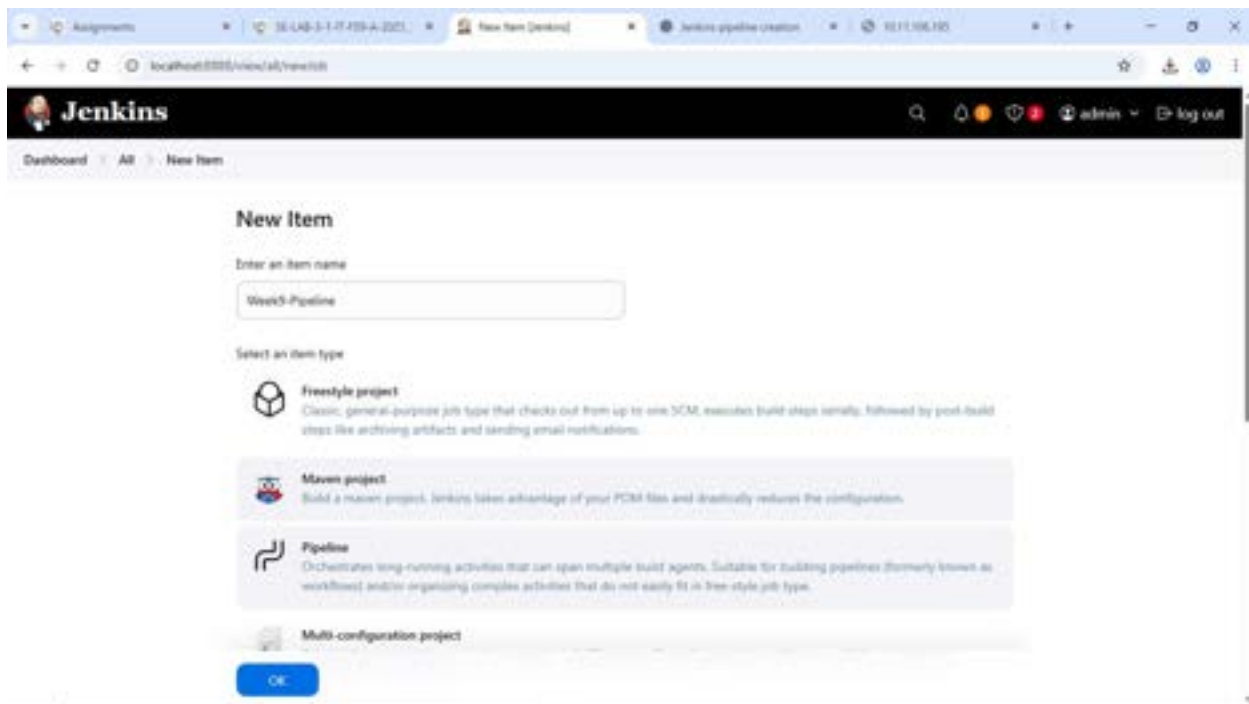
Copy the code there

```
pipeline {
  agent any
  tools{
    maven 'MAVEN-HOME'
  }
  stages {
    stage('git repo & clean') {
      steps {
        //bat "rmdir /s /q mavenjava"
        bat "git clone provide your github link"
        bat "mvn clean -f mavenjava"
      }
    }
    stage('install') {
      steps {
        bat "mvn install -f mavenjava"#project name#
      }
    }
    stage('test') {
      steps {
```

```

        bat "mvn test -f mavenjava"
    }
}
stage('package') {
    steps {
        bat "mvn package -f mavenjava"
    }
}
}
}
}

```



Assignments 16 LAB-5-1-IT-2024 Week9-Pipeline Config Dev Jenkins pipeline creation 10.11.2024

localhost:8080/job/Week9-Pipeline/configure

Dashboard > Week9-Pipeline > Configuration

Configure

- General
- Build Triggers**
- Advanced Project Options
- Pipeline

Build Triggers

☐ Build after other projects are built

☒ Build periodically

Schedule

H 18 ***

Would last have run at Monday, 6 October, 2025, 6:17:52 pm India Standard Time; would next run at Tuesday, 7 October, 2025, 6:17:52 pm India Standard Time.

☐ Build whenever a SNAPSHOT dependency is built

☒ GitHub hook trigger for GITScm polling

☒ Poll SCM

Schedule

H/5 ****

Save Apply

Assignments 16 LAB-5-1-IT-2024 Week9-Pipeline Config Dev Jenkins pipeline setup 10.11.2024

localhost:8080/job/Week9-Pipeline/configure

Dashboard > Week9-Pipeline > Configuration

Configure

- General
- Build Triggers
- Advanced Project Options
- Pipeline**

Pipeline

Definition

Pipeline script

Script

```
13+ }
14+ stage("Get Repo & Clone") {
15+     steps {
16+         git clone https://github.com/week9lab/week9lab.git
17+         git checkout -f develop
18+     }
19+ }
20+ stage("Install") {
21+     steps {
22+         mvn install -f pom.xml
23+     }
24+ }
25+ stage("Test") {
26+     steps {
27+         mvn test -f pom.xml
28+     }
29+ }
```

☒ Use Groovy Sandbox

Pipeline Syntax

Save Apply

Assignments 16 LAB-5-1-IT-2024-4-2025 Build log [W] Jenkins Jenkins pipeline setup

localhost:8080/jobs/Week9-Pipeline/5/pipeline-console/

Jenkins

Dashboard Week9-Pipeline #6 Pipeline Console

Build #6

Success 14 min ago in 55 sec

- Tool Install
- Welcome Message
- Clean Workspace & Clone Repo
- Install
- Test
- Package

```

24 [INFO] No tests to run.
25 [INFO]
26 [INFO] --- war:3.1.1:war (default-war) @ E-Ticketing ---
27 [INFO] Packaging webapp
28 [INFO] Assembling webapp [E-Ticketing] in [C:\ProgramData\Jenkins\jenkins\workspace\Week9-Pipeline\se-lab\target\E-
Ticketing-system]
29 [INFO] Processing war project
30 [INFO] Copying webapp resources [C:\ProgramData\Jenkins\jenkins\workspace\Week9-Pipeline\se-lab\src\main\webapp]
31 [INFO] Building war: C:\ProgramData\Jenkins\jenkins\workspace\Week9-Pipeline\se-lab\target\E-Ticketing-system.war
32 [INFO]
33 [INFO] BUILD SUCCESS
34 [INFO]
35 [INFO] Total time: 2.772 s
36 [INFO] Finished at: 2025-10-07T08:54:27+00:00
37 [INFO]

```

Jenkins 2.489

Assignments 16 LAB-5-1-IT-2024-4-2025 Week9-Pipeline [W] Jenkins Jenkins pipeline setup

localhost:8080/jobs/Week9-Pipeline/

Jenkins

Dashboard Week9-Pipeline

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

GitHub Hook Log

Polling Log

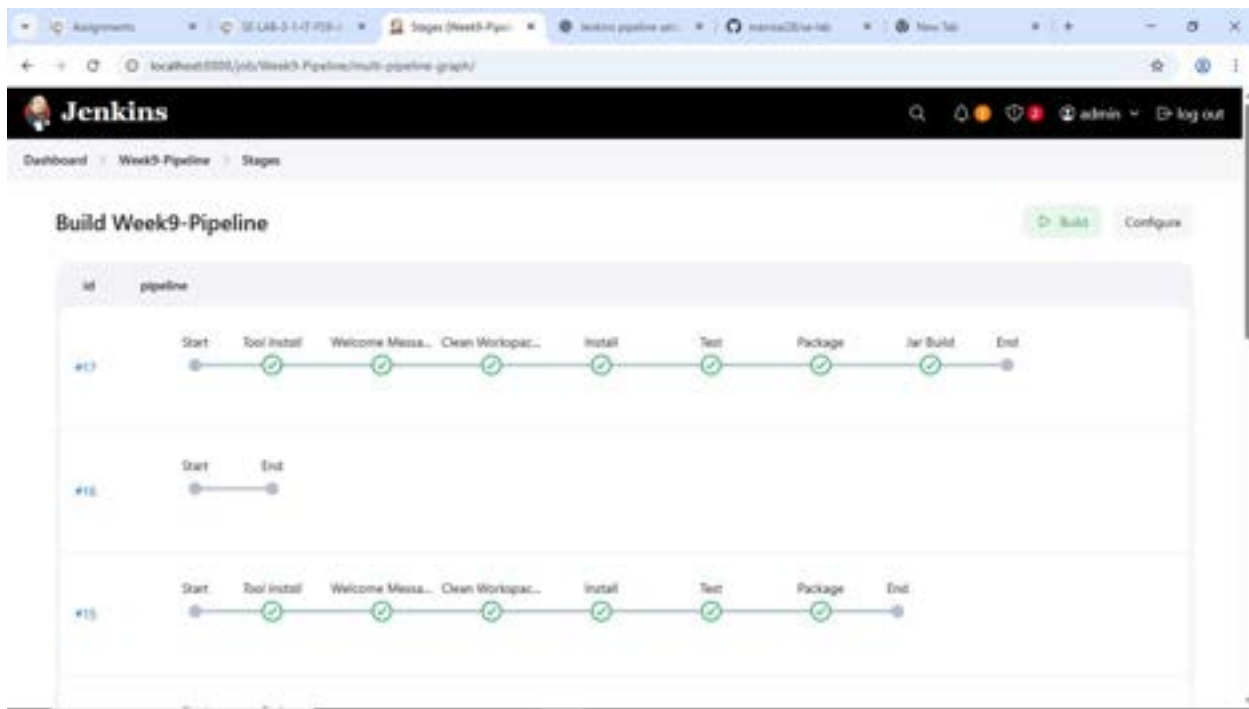
Builds

Week9-Pipeline

Week 9: Scripted Pipeline for Maven project with Git integration

Stage View

	Declarative: Tool Install	Welcome Message	Clean Workspace & Clone Repo	Install	Test	Package
Average stage times: (all run times ~29s)	105ms	320ms	10s	5s	3s	4s
Get #1 11:21	88ms	230ms	6s	4s	4s	7s
Get #1 11:30	138ms	301ms	7s	126ms	83ms	84ms
Get #1 11:21	255ms	345ms	8s	4s	4s	4s



Apply and save

LAB ACTION PLAN FOR WEEK 10

Working with minikube and Nagios

Hands-on practice of creating, running and scaling pods in minikube.

Running Nginx server on specified port number by explaining the Nginx monitoring tool

Running Nagios server and Understanding the Monitoring tool using Docker.

AWS-free Trier account Creation steps

Upload the screenshots for the tasks

Kubernetes:

Kubernetes is a tool that automates how we run and manage applications inside the container.

Dockers will only run containers, if in any case the container fails/stopped/killed, the docker will not help us, here is where Kubernetes plays an important role, Kubernetes cluster will be responsible in creating a new container and managing various containers.

POD: In Kubernetes, a Pod is the smallest deployable unit that you can create and manage.

Minikube:

Minikube creates a VM on your local machine and deploys a simple Kubernetes cluster with one node. It's a lightweight implementation. Minikube is a version of Kubernetes.

Nagios:

Nagios is an **open-source IT infrastructure monitoring tool**. It monitors

- Servers
- Network devices
- Applications and services

It **alerts administrators** when issues occur and notifies when they are resolved.

Step 1: Install Prerequisites

Before installing Minikube, ensure the following are installed:

Virtualization Support:

Verify virtualization is enabled:

Hypervisor:

Minikube supports multiple hypervisors (e.g., **Hyper-V**, **VirtualBox**, or **Docker** as a driver).

Install one of the following:

Hyper-V (pre-installed on Windows 10/11 Pro or Enterprise).

Docker Desktop (if you want to use Docker as the driver).

Step 2: Download Minikube

Open a PowerShell or Command Prompt with administrator privileges.

Download the latest Minikube executable using this command:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-installer.exe
```

Install Minikube by running the installer:

```
.\minikube-installer.exe
```

Step 3: Add Minikube to PATH

If Minikube is not automatically added to your PATH during installation:

Open **System Properties** → **Environment Variables**.

Add the directory where Minikube is installed (e.g., `C:\Program Files\Minikube`) to your PATH variable.

Step 4: Start Minikube

Open a terminal (PowerShell or CMD). Do the following commands

Start Minikube with a specified driver (e.g., Hyper-V, Docker, or VirtualBox). For example:

Hyper-V:

```
minikube start --driver=hyperv
```

Docker:

```
minikube start --driver=docker
```

Verify Minikube is running:

```
minikube status
```

Step 5: Interact with Minikube

kubectl is a command-line tool used in Kubernetes to interact with and manage Kubernetes clusters.

Once Minikube is running:

Use `kubectl` to interact with the cluster.

Install `kubectl` if not already installed:

```
minikube kubectl -- get pods -A
```

Or download it separately from the [official Kubernetes site](https://kubernetes.io/docs/tasks/tools/install-kubectl/).

Open the Minikube dashboard (optional):

```
minikube dashboard
```

Optional: Check Your Installation

Run the following to verify the installation:

Minikube version

```
kubectl version --client
```

Troubleshooting

If Minikube fails to start:

Ensure your hypervisor (Hyper-V/Docker/VirtualBox) is installed and running.

Check the Minikube logs:

```
minikube logs
```

Updating Minikube:

```
minikube update-check
```

```
minikube update
```

Minikube Automation Steps

Step 1: Start Minikube Cluster

Open your terminal and run the command:

```
minikube start
```

```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Asus>minikube start
* minikube v1.37.0 on Microsoft Windows 11 Home Single Language 10.0.26100.6584 Build 26100.6584
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.48 ...
* Restarting existing docker container for "minikube" ...
| Failing to connect to https://registry.k8s.io/ from inside the minikube container
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass

| C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.30.5, which may have incompatibilities with Kubernetes 1.34.0.
  - Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

C:\Users\Asus>minikube ip
192.168.49.2

C:\Users\Asus>
```

```
C:\Users\Asus>minikube kubectl -- get pods -A
> kubectl.exe.sha256: 64 B / 64 B [-----] 100.00% 7 p/s 0s
> kubectl.exe: 59.26 MiB / 59.26 MiB [-----] 100.00% 543.78 KiB p/s 1m52s
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-66bc5c9577-t6dcq	1/1	Running	6 (3m54s ago)	11h
kube-system	etcd-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-apiserver-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-controller-manager-minikube	1/1	Running	7 (3m54s ago)	11h
kube-system	kube-proxy-sl52j	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-scheduler-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	storage-provisioner	1/1	Running	11 (3m21s ago)	11h

```
C:\Users\Asus>minikube kubectl -- get pods -A
> kubectl.exe.sha256: 64 B / 64 B [-----] 100.00% 7 p/s 0s
> kubectl.exe: 59.26 MiB / 59.26 MiB [-----] 100.00% 543.78 KiB p/s 1m52s
```

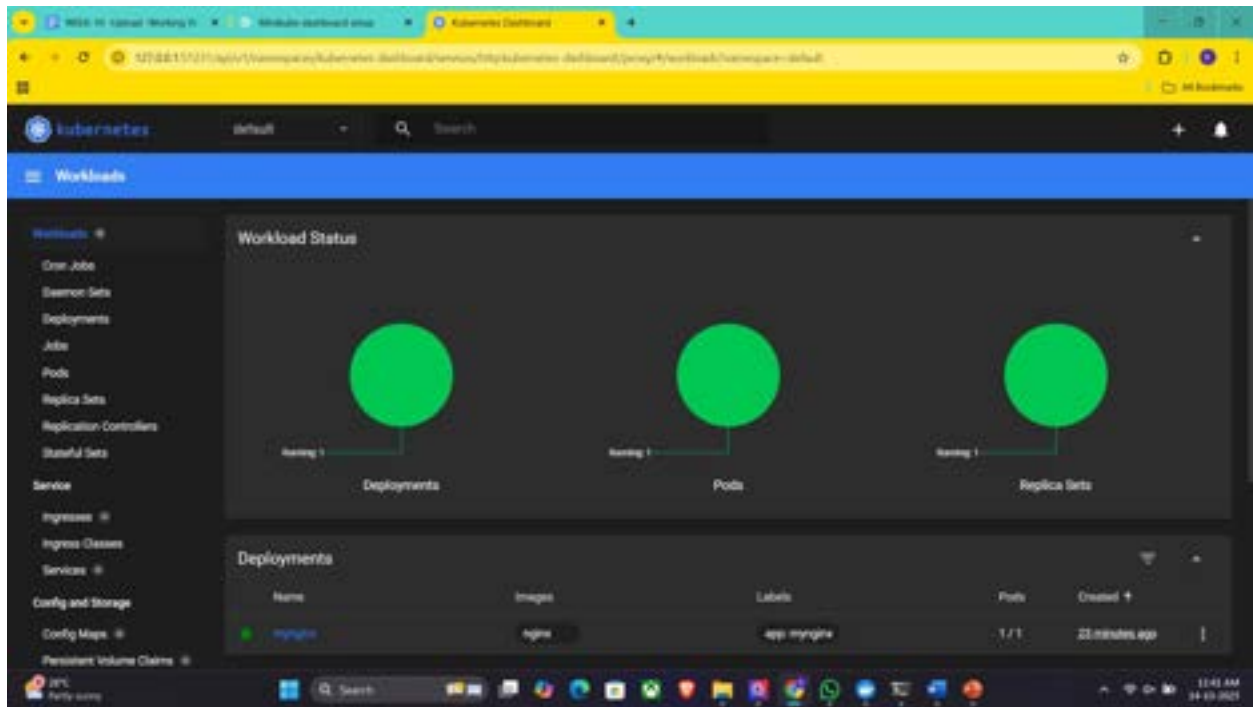
NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-66bc5c9577-t6dcq	1/1	Running	6 (3m54s ago)	11h
kube-system	etcd-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-apiserver-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-controller-manager-minikube	1/1	Running	7 (3m54s ago)	11h
kube-system	kube-proxy-sl52j	1/1	Running	6 (3m54s ago)	11h
kube-system	kube-scheduler-minikube	1/1	Running	6 (3m54s ago)	11h
kube-system	storage-provisioner	1/1	Running	11 (3m21s ago)	11h

```
C:\Users\Asus>kubectl create deployment mynginx --image=nginx
deployment.apps/mynginx created

C:\Users\Asus>minikube dashboard
* Enabling dashboard ...
  - Using image docker.io/kubernetes/metrics-scraper:v1.0.0
  - Using image docker.io/kubernetes/dashboard:v2.7.0
* Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

* Verifying dashboard health ...
* Launching proxy ...
* Verifying proxy health ...
* Opening http://127.0.0.1:51231/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
```



Step 2: Create and Manage Deployment

Create an application in Kubernetes:

Command:

```
kubectl create deployment mynginx --image=nginx
```

if already created then

```
kubectl set image deployment/mynginx nginx=nginx:latest
```

Verify the deployment using: Kubernetes responds by showing you a list that includes the names of your deployment groups

```
kubectl get deployments
```



```
C:\Users\Asus>kubectl get deployments
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
mynginx       1/1      1             1            32m

C:\Users\Asus>
```

Ensure mynginx appears in the output.

Check the following commands:

kubectl get pods

kubectl describe pods

```
C:\Users\Asus>kubectl get pods
NAME          READY    STATUS    RESTARTS    AGE
mynginx-645865c456-87dfg  1/1      Running   0           32m

C:\Users\Asus>kubectl describe pods
Name:          mynginx-645865c456-87dfg
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Tue, 14 Oct 2025 11:18:05 +0530
Labels:        app=mynginx
               pod-template-hash=645865c456
Annotations:   <none>
Status:        Running
IP:            10.244.0.9
IPs:           10.244.0.9
Controlled By: ReplicaSet/mynginx-645865c456
Containers:
  nginx:
    Container ID:  docker://7f27a5a82d8dc84282b8899d878fcb878a2bdc4ef4cc2c8815d58453bc821718
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:3b7732365933ca591ce9a8d386cb713ad96a1176b82f7979a8d4a9973486a8d8
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Tue, 14 Oct 2025 11:22:52 +0530
      Ready:       True
      Restart Count: 0
    Environment:   <none>
```

Expose Deployment as a Service:

Command:

```
kubectl expose deployment mynginx --type=NodePort --port=80 --
target-port=80
```

```
C:\Users\Asus>kubectl expose deployment mynginx --type=NodePort --port=80 --target-port=80
service/mynginx exposed

C:\Users\Asus>
```

Step 3: Scale the Deployment

Command: Scales the Nginx deployment to 4 replicas (pods).

```
kubectl scale deployment mynginx --replicas=4
```

```
kubectl get service mynginx
```

```
C:\Users\Asus>kubectl scale deployment mynginx --replicas=4
deployment.apps/mynginx scaled

C:\Users\Asus>kubectl get service mynginx
Error from server (NotFound): services "mynginx" not found

C:\Users\Asus>|
```

Step 4: Access the Nginx App

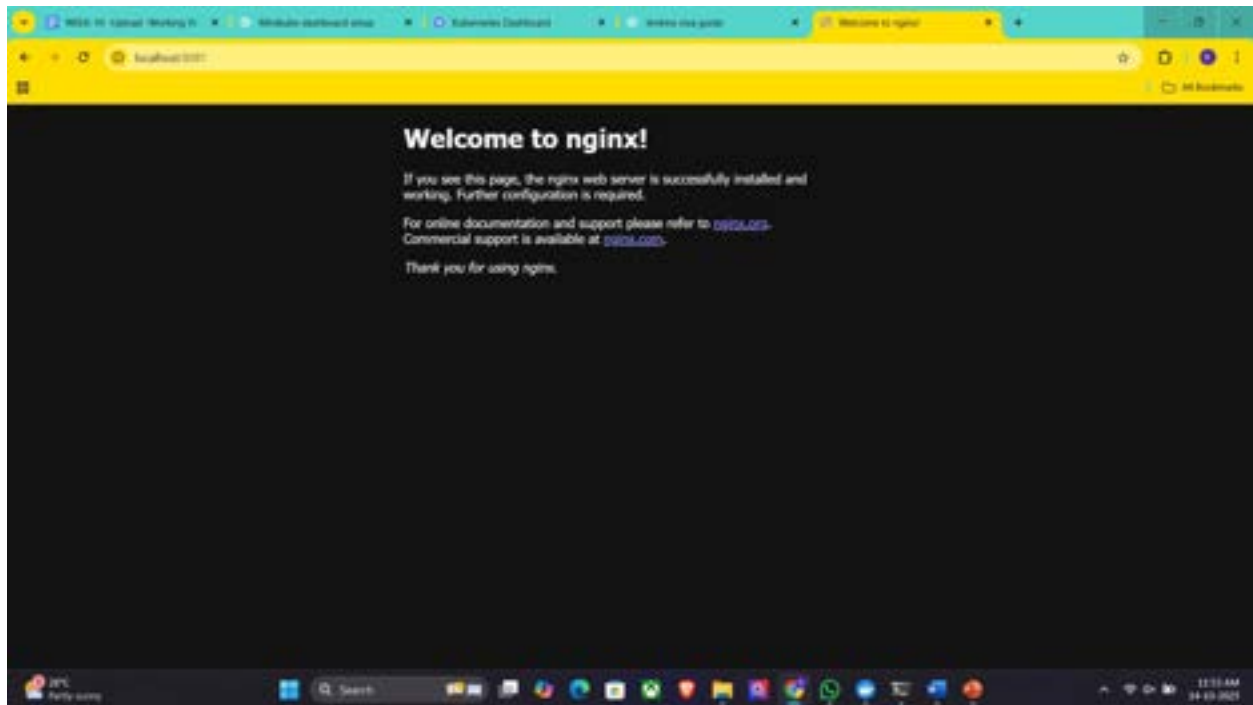
Using Port Forwarding:

Command:

```
kubectl port-forward svc/mynginx 8081:80
```

```
C:\Users\Asus>kubectl port-forward svc/mynginx 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
|
```

Access the app via <http://localhost:8081>.



If Error, use this option, **Using Minikube Tunnel:**

Start the tunnel:

```
minikube tunnel
```

Retrieve the service URL:

```
minikube service mynginx --url
```

```
C:\Users\Asus>minikube tunnel
* Tunnel successfully started

* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...

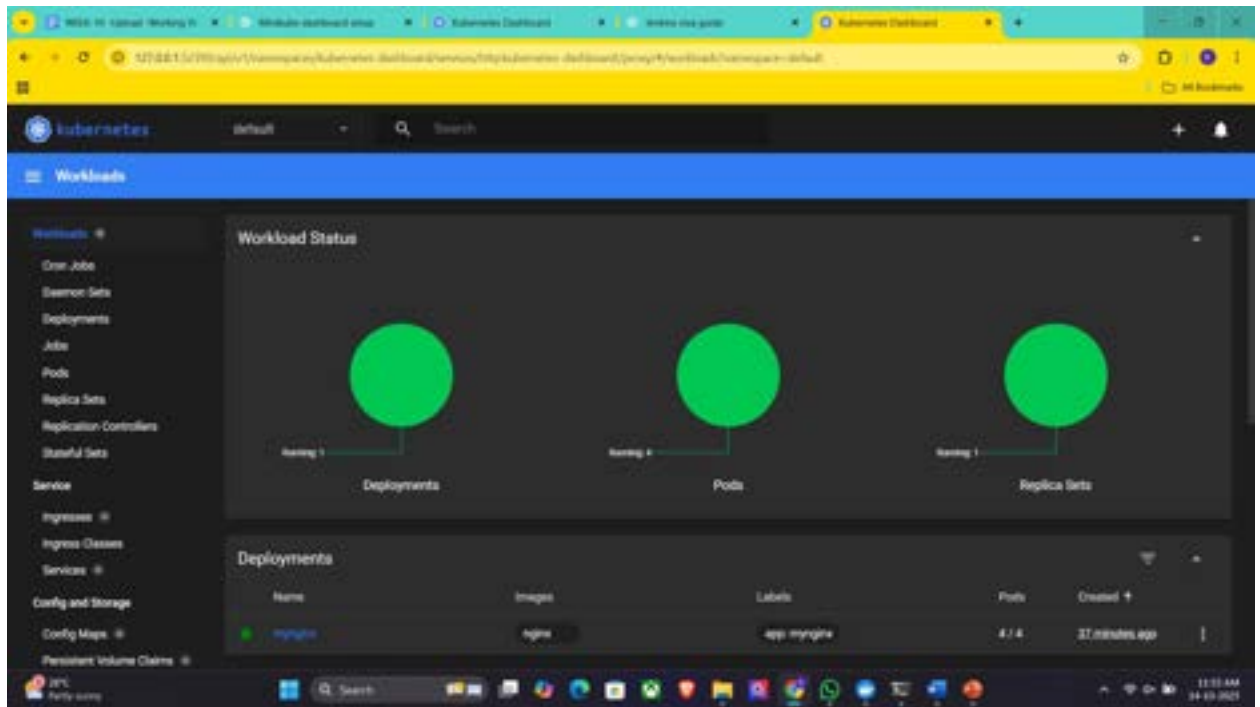
C:\Users\Asus>minikube service mynginx --url
http://127.0.0.1:57079
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Open the provided URL in your browser.

Open the kubernets dashboard

Open the minikube dashboard

Minikube dashboard



Step 5: Stop and Clean Up

Stop Nginx Deployment:

Commands:

```
kubectl delete deployment mynginx  
kubectl delete service mynginx
```

```
C:\Users\Asus>kubectl delete deployment mynginx  
deployment.apps "mynginx" deleted  
  
C:\Users\Asus>kubectl delete service mynginx  
service "mynginx" deleted  
  
C:\Users\Asus>
```

Stop Minikube (Optional):

Command:

```
minikube stop
```

Delete Minikube Cluster (Optional):

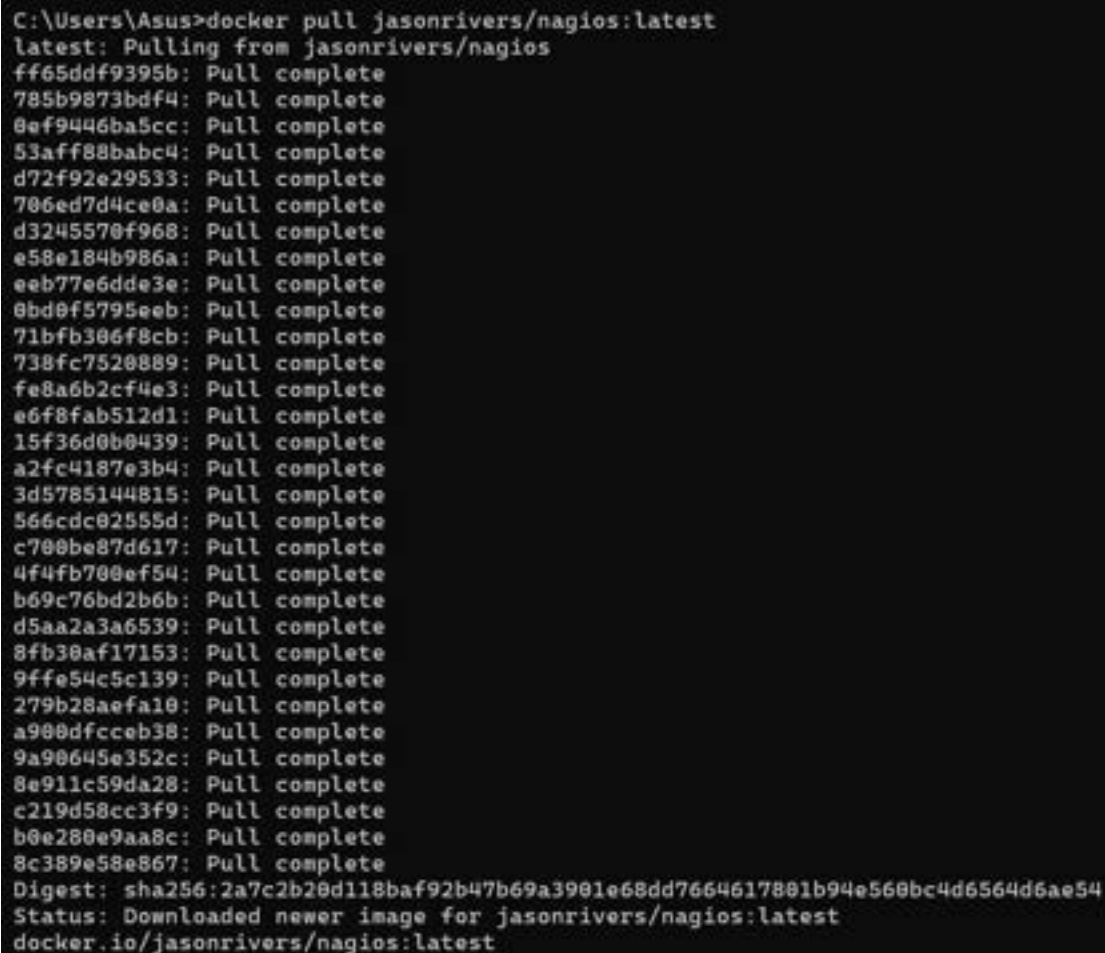
Command:minikube delete

Nagios Automation Steps

Step 1: Pull the Nagios Docker Image

Open a terminal and run:

```
docker pull jasonrivers/nagios:latest
```



```
C:\Users\Asus>docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
ff65ddf9395b: Pull complete
785b9873bdf4: Pull complete
8ef9446ba5cc: Pull complete
53aff88babcb: Pull complete
d72f92e29533: Pull complete
706ed7d4ce0a: Pull complete
d3245570f968: Pull complete
e58e184b986a: Pull complete
eeb77e6dde3e: Pull complete
0bd0f5795eeb: Pull complete
71bfb306f8cb: Pull complete
738fc7520889: Pull complete
fe8a6b2cf4e3: Pull complete
e6f8fab512d1: Pull complete
15f36d0b0439: Pull complete
a2fc4187e3b4: Pull complete
3d5785144815: Pull complete
566cdc02555d: Pull complete
c700be87d617: Pull complete
4f4fb700ef54: Pull complete
b69c76bd2b6b: Pull complete
d5aa2a3a6539: Pull complete
8fb30af17153: Pull complete
9ffe54c5c139: Pull complete
279b28aefa10: Pull complete
a900dfcceb38: Pull complete
9a90645e352c: Pull complete
8e911c59da28: Pull complete
c219d58cc3f9: Pull complete
b0e280e9aa8c: Pull complete
8c389e58e867: Pull complete
Digest: sha256:2a7c2b20d118baf92b47b69a3901e68dd7664617801b94e560bc4d6564d6ae54
Status: Downloaded newer image for jasonrivers/nagios:latest
docker.io/jasonrivers/nagios:latest
```

Step 2: Run Nagios

Command:

```
docker run --name nagiosdemo -p 8888:80 jasonrivers/nagios:latest
```

```
C:\Users\Asus>docker run --name nagiosdemo -p 8888:80 jasonrivers/nagios:latest
Adding password for user nagiosadmin
chown: warning: '.' should be ':' : 'nagios.nagios'
Started runsvdir, PID is 13
checking permissions for nagios & nagiosgraph
rsyslogd: [origin software="rsyslogd" swVersion="8.2312.0" x-pid="20" x-info="https://www.rsyslog.com"] start

Nagios Core 4.5.7
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2024-10-24
License: GPL

Website: https://www.nagios.org
Nagios 4.5.7 starting... (PID=25)
Local time is Tue Oct 14 06:48:32 UTC 2025
wproc: Successfully registered manager as @wproc with query handler
nagios: Nagios 4.5.7 starting... (PID=23)
nagios: Local time is Tue Oct 14 06:48:32 UTC 2025
nagios: LOG VERSION: 2.0
nagios: qh: Socket '/opt/nagios/var/rw/nagios.qh' successfully initialized
nagios: qh: core query handler registered
nagios: qh: echo service query handler registered
nagios: qh: help for the query handler registered
nagios: wproc: Successfully registered manager as @wproc with query handler
wproc: Registry request: name=Core Worker 42;pid=42
nagios: wproc: Registry request: name=Core Worker 42;pid=42
wproc: Registry request: name=Core Worker 43;pid=43
nagios: wproc: Registry request: name=Core Worker 43;pid=43
wproc: Registry request: name=Core Worker 44;pid=44
nagios: wproc: Registry request: name=Core Worker 44;pid=44
wproc: Registry request: name=Core Worker 45;pid=45
nagios: wproc: Registry request: name=Core Worker 45;pid=45
wproc: Registry request: name=Core Worker 47;pid=47
nagios: wproc: Registry request: name=Core Worker 47;pid=47
wproc: Registry request: name=Core Worker 46;pid=46
```

Step 3: Access Nagios Dashboard

Open your browser and navigate to:

<http://localhost:8888>

Login Credentials:

Username: nagiosadmin

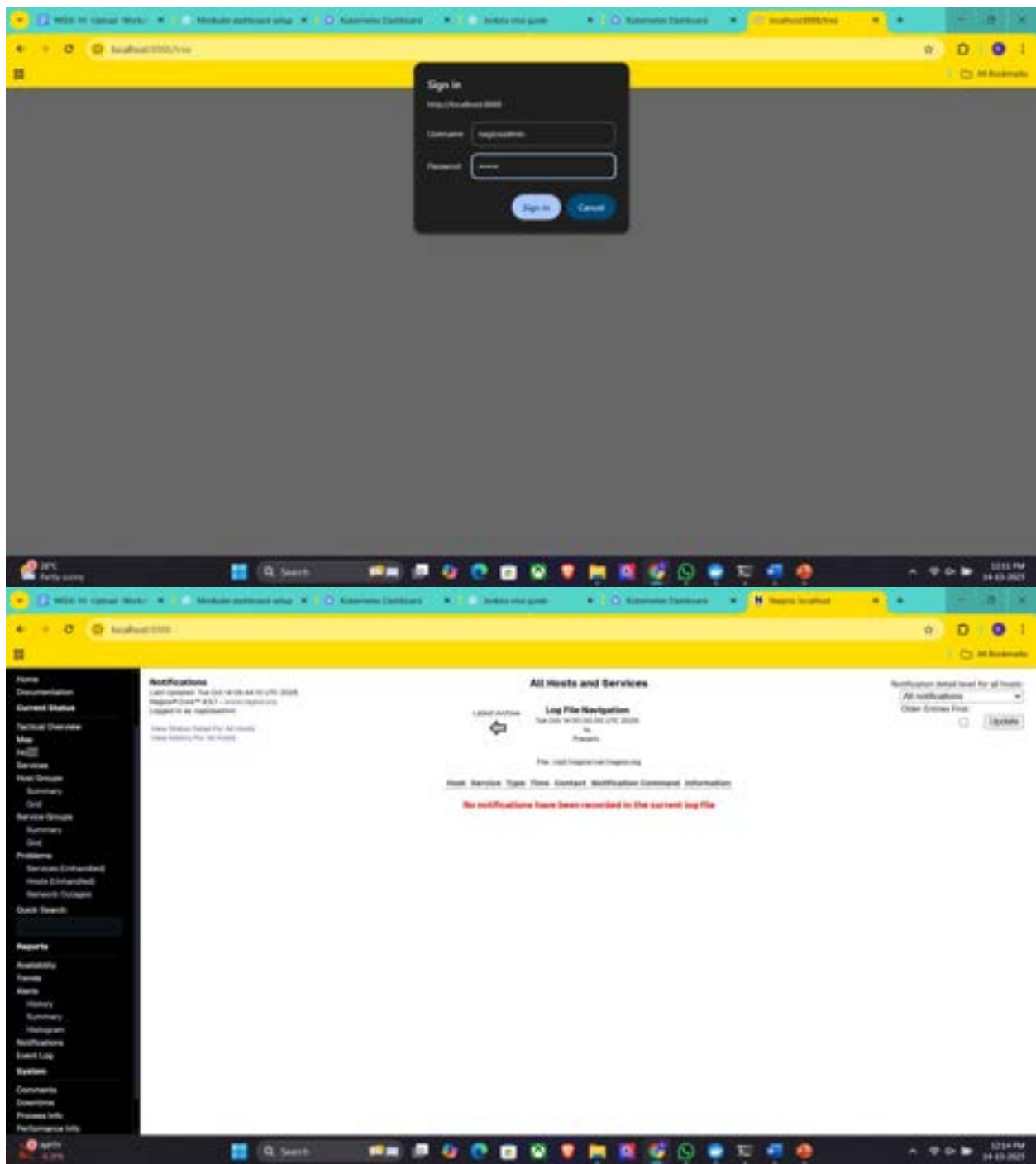
Password: nagios

Once logged in, explore:

Hosts: View systems being monitored.

Services: Check tasks being monitored (e.g., CPU usage).

Alerts: Access recent notifications.



WS26-10 Network Monitor

Module dashboard setup

Network Dashboard

Network info guide

Network Dashboard

Network Dashboard

localhost:3000

Home

Documentation

Current Status

Tactical Overview

Map

Hosts

Services

Host Groups

Summary

Grid

Service Groups

Summary

Grid

Problems

Services Expanded

Hosts Expanded

Network Outages

Quick Search

Reports

Availability

Events

Alerts

History

Summary

Histogram

Notifications

Event Log

System

Comments

Overview

Process Info

Performance Info

Current Network Status

Last updated: Tue Oct 14 09:43:29 UTC 2025

Updated every 60 seconds

Target IP: 192.168.1.1 - 192.168.1.255

Updated in 10 seconds

Host Status Totals

100 Hosts: Up, Down, Pending

100 Up, 0 Down, 0 Pending

100 Up, 0 Down, 0 Pending

Service Status Totals

100 Services: Up, Down, Pending

100 Up, 0 Down, 0 Pending

100 Up, 0 Down, 0 Pending

Service Status Details For All Hosts

Limit Results: 100

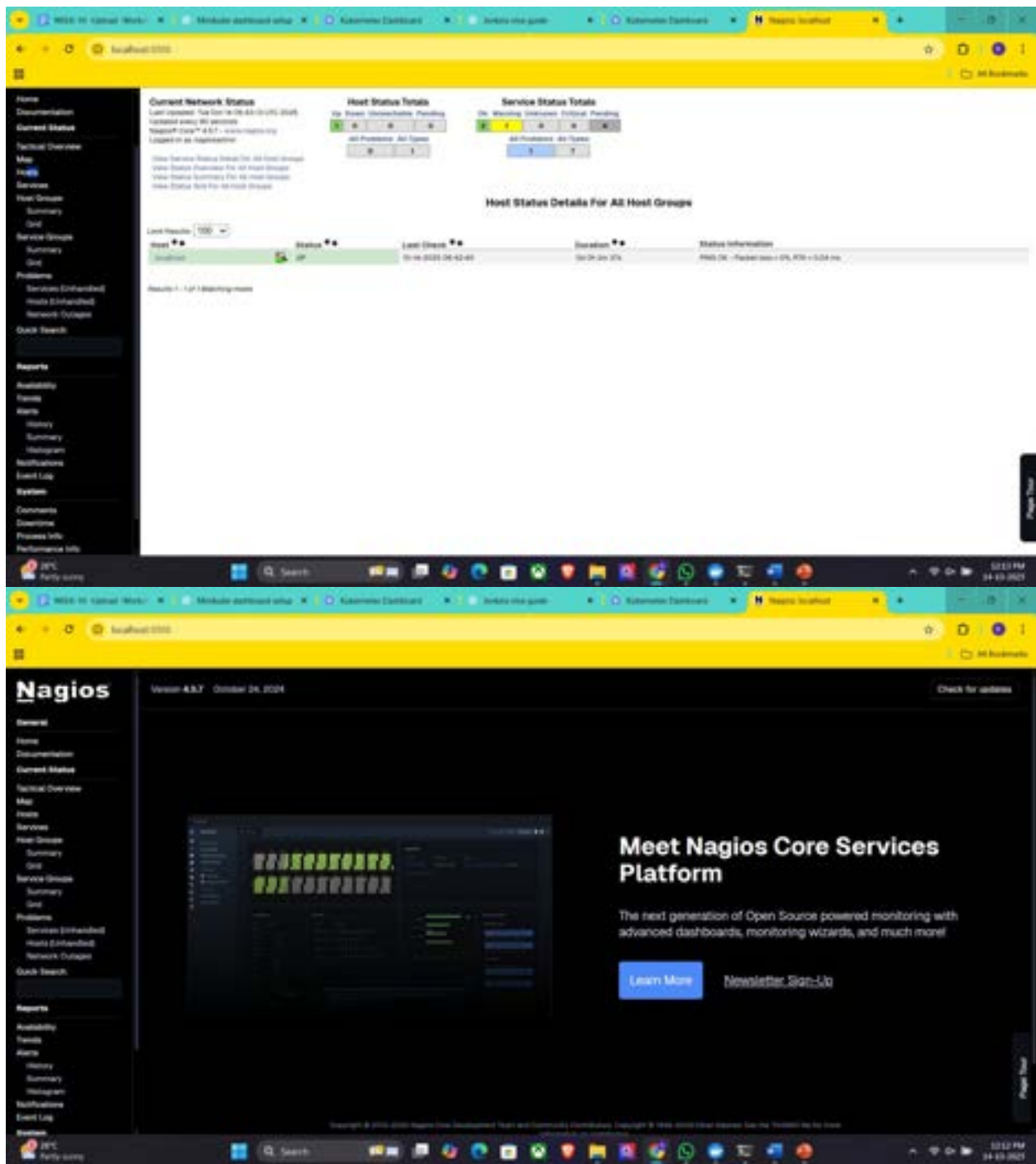
Host	Service	Status	Last Check	Duration	Attempts	Status Information
192.168.1.1	Current Load	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	OK - load average: 0.00, 0.00, 0.00
192.168.1.1	Current Load	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	OK - load average: 0.00, 0.00, 0.00
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time
192.168.1.1	HTTP	OK	10-14-2025 09:43:29	0s 0m 0s 0ms	1/1	HTTP 200 OK - 300 OK, 0.00s response time

Results: 1 - Full 7 monitoring devices

Page Top

10:13 PM

10-14-2025



Step 4: Monitoring Host Details

Navigate to the Host Information Page:

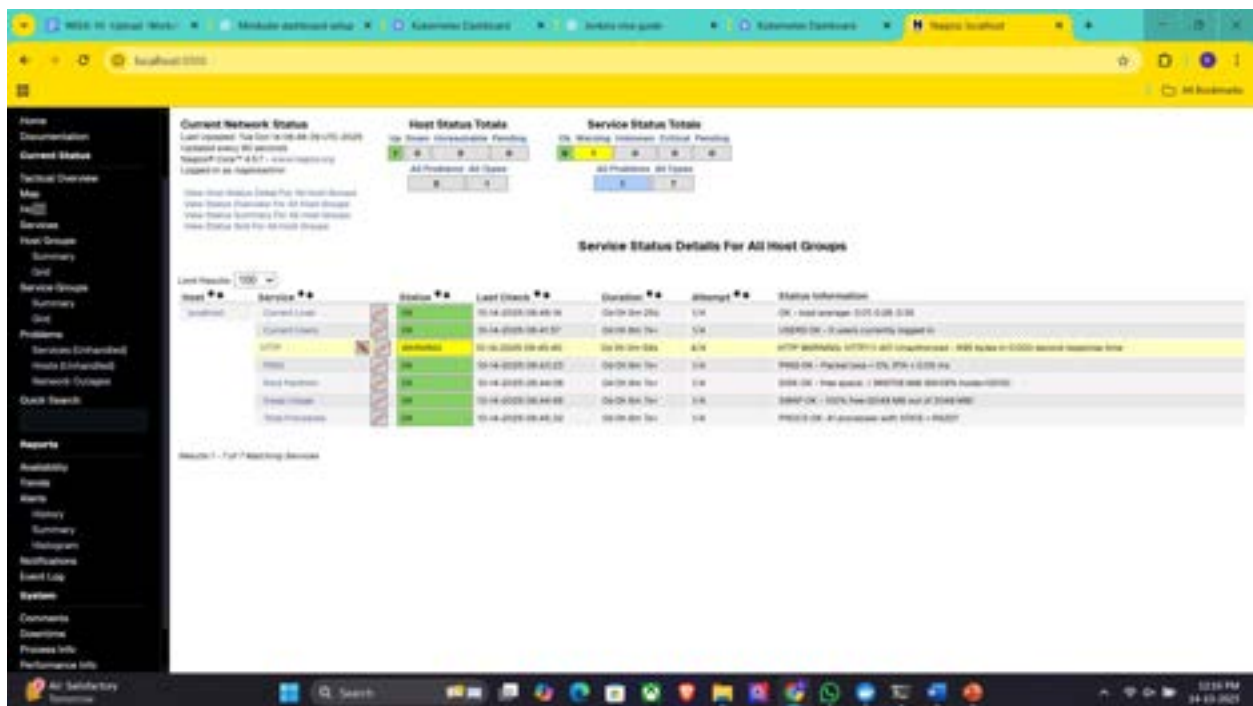
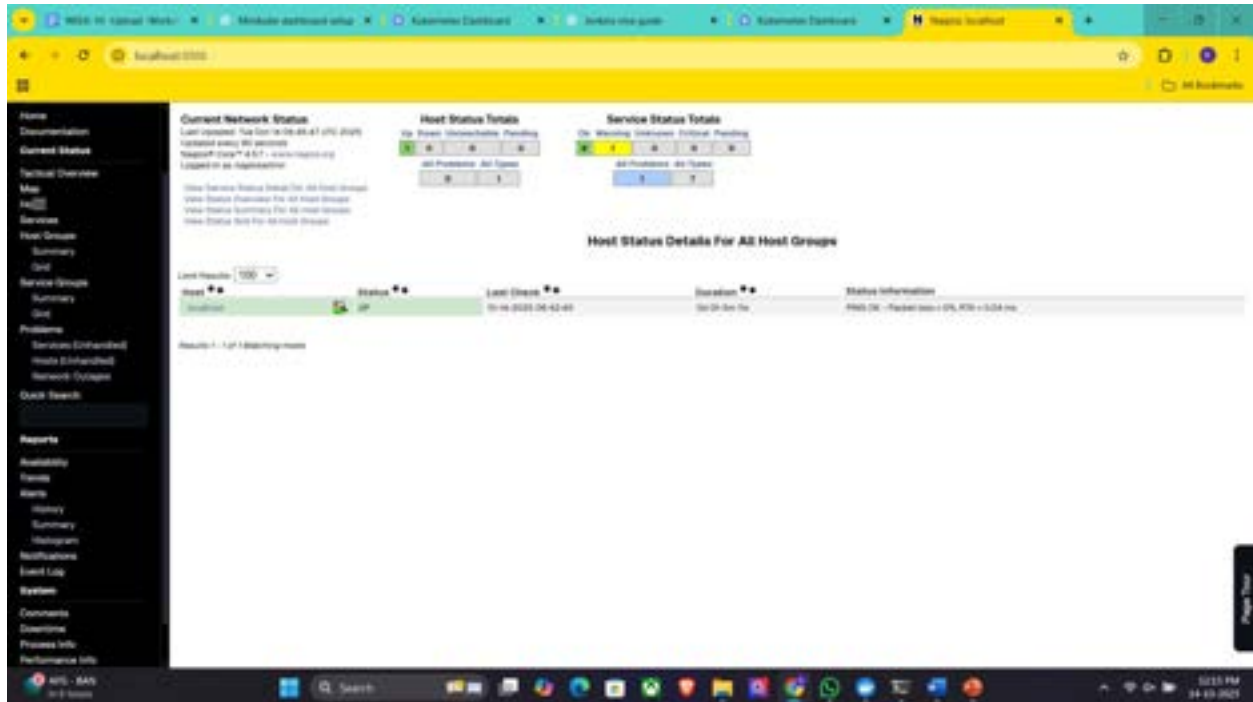
Select a host from the **Hosts** menu.

Key Details:

Host Status: Indicates if the system is UP or DOWN.

Metrics: View CPU usage, memory status, and network activity.

Actions: Reschedule checks, disable notifications, or schedule downtime.



Step 5: Stop and Remove Nagios

Stop the Container:

Command:

```
docker stop nagiosdemo
```

```
C:\Users\Asus>docker stop nagiosdemo
nagiosdemo
```

Delete the Container:

Command:

```
docker rm nagiosdemo
```

```
C:\Users\Asus>docker rm nagiosdemo
nagiosdemo
```

Remove the Image (Optional):

List images:

```
docker images
```

```
C:\Users\Asus>docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
gcr.io/google-containers/ubuntu   latest             8a0725624a07       5 months ago       247MB
redis               latest             8a0725624a07       5 months ago       247MB
ubuntu             latest             8a0725624a07       5 months ago       247MB
jasonrivers/nagios latest             8a0725624a07       5 months ago       247MB
docker/desktop-kubernetes          latest             8a0725624a07       5 months ago       247MB
registry.k8s.io/kube-apiserver     v1.30.5           8a0725624a07       5 months ago       247MB
registry.k8s.io/kube-scheduler     v1.30.5           8a0725624a07       5 months ago       247MB
registry.k8s.io/kube-controller-manager v1.30.5         8a0725624a07       5 months ago       247MB
registry.k8s.io/kube-proxy         v1.30.5           8a0725624a07       5 months ago       247MB
registry.k8s.io/coredns/coredns   v1.11.3           8a0725624a07       5 months ago       247MB
registry.k8s.io/etcd               3.6.15-0         8a0725624a07       5 months ago       247MB
registry.k8s.io/etcd               3.5.13-0         8a0725624a07       5 months ago       247MB
docker/desktop-vpnkit-controller   3.9              8a0725624a07       5 months ago       247MB
docker/desktop-storage-provisioner v2.0             8a0725624a07       5 months ago       247MB
```

Delete the Nagios image:

```
docker rmi jasonrivers/nagios:latest
```

Observe the docker containers in DockerHub, we can see the latest Nagios
Installed running on port:8888

```
C:\Users\Asus>docker rmi jasonrivers/nagios:latest
Untagged: jasonrivers/nagios:latest
Untagged: jasonrivers/nagios@sha256:2a7c2b20d118baf92b47b69a3901e68dd7664617801b94e560bc4d6564d6ae54
Deleted: sha256:2ab73fb5d162ad0be4ef495fb62e17772120b32721a6c3eb9091fcc800d2989b
Deleted: sha256:631ab139f760d68457a4186da4a60f5e5f11e65aa3b26864b5df237ef75b1311
Deleted: sha256:b10e2a7f4de8bcf15a69232335422c9a736c4a4a93ca288592a2fb53a2de6f45
Deleted: sha256:3a5a34bf7e5ab76f1a3054c5854a9bc7227fd5a0dec73d4a4673456f69cec753
Deleted: sha256:0603355dfe22b531eff49627391c7a8bb944163a735d9142cfcf02529e3c8fffd
Deleted: sha256:2cdfed24d1997cb2951dc561b1c4fbc1947069e22801aab9103cf0f0c1e8c532
Deleted: sha256:5d2ae66d21d91026745748ae98d112256472f76f72deaa904a368b9dd41ca987
Deleted: sha256:b9759ebfa6418db67d82fd861af913fd571bd06c6d3ccddfe760d637e49f15f2
Deleted: sha256:62b02d2bded263673e8e75637ddca61e85ad0fba2e2eaa48bcfda314197add5f4
Deleted: sha256:c8bf237c41fd124329567e29514d381ae822f6d202565e013df75bb47811c94d
Deleted: sha256:5e918b338fe317b5f88044bfc83c21c5bca1f8570092859d4a8c05a8e2e036ce
Deleted: sha256:24ce75f6abff9ace033a572c2adb3cccfla57d995baleaad9bf68b9a50d5bfbab
Deleted: sha256:bc291db0b02d187622e2bb6f1c85dc8b5baa3a37ce557e5357a628b3befbb0df3
Deleted: sha256:a0d43b5b4f90893c91713325e05539667219a1813fc95804acfa0efa1925b5120
Deleted: sha256:b985a1f962d4e9e6c23c7d823a68861a4e69c7dae1a4fd01f2eb425ba130d742
Deleted: sha256:eefaf4321bcf0520d56ae0d458508da8d6c62d6b7fd484ec08c99678254e84dd
Deleted: sha256:320894a8e0c97786c5249f30c572390355cd72953ee7dd70a3f6b11be9c2a047
Deleted: sha256:7c52f6b68cf3d137336b97a0fcd9beb2a8512b6c2512c8f535e2a81aa767cab1
Deleted: sha256:88100fb20cb3c5b2c708de42f7d16c2baf0992610f1fb4d435791d52ca008eb5
Deleted: sha256:067cbc3a33f52f4afbacc2ea3d35fbb2270ec8142a04d7e9837fc10f627a84d8
Deleted: sha256:4c463e1f44c076c2a117691f426399fc6236b7d8c92e375058e0045d12525e0f
Deleted: sha256:cd0357977469c3814df276354774e1d424ed389664ab653e4652940ac1d55a6d
Deleted: sha256:1277c94ea1944b9e52ab1fba34af10214509106536e43fe3fdb082bfd1c1aa02
Deleted: sha256:57ac56afc1593da088f8f3dbfbba11884f38aee15b2db2e92c98300f44574cf
Deleted: sha256:202680f0cbb41cf353db89a064dd727ecc16e12fecde216b99b22ec06f3a34f9
Deleted: sha256:5c005cf4164068516d0a2e7b15277509890438e5b934794ca055a0f22c37e35c
Deleted: sha256:0d9515684b30166985cc21960825aebb9a8cd7811c00175eb395f97806881f91
Deleted: sha256:c784deeb88028b6eb1574c623e477d1075cca6b6f68db14747c5d30a8c077
Deleted: sha256:757fa2edf084816630e3943408ca67e261ff5e0550da6ead903qb1d28b4d1cc5
Deleted: sha256:f77ac6741c76fba0772ea95bf85c6e2a5e33271aa9dd226efae3b3ebcf64117
Deleted: sha256:3f93222df00f5e6495196760d065ada12aa3c4768947558ac1ad167d3b6683e4
Deleted: sha256:a46a5fb872b554648d9d0262f302b2c1ded46eeb1ef4dc727ecc5274605937af
```

Steps for AWS-free trier account creation

open <https://aws.amazon.com>

click on create AWS account

Provide email id and name details for AWS account creation then a screen will appear as below

Enter verification code received in your mail

Provide your contact details and agree Terms and conditions, then click on create

Provide billing details, click on verify and continue

Amazon charges 2 rs and it will be credit back into your account once verification is over

Once payment success a screen appear as below, provide your working contact number after selecting country, and click on send SMS after entering the details

Provide verification code received in your mobile

Now select basic plan and click on complete sign up

Now a screen appear as follows then click on AWS management console

You will navigating to the aws page ..where you need to Select role as Academic/Researcher and interest as DevOps and click on submit

Sign into AWS console as Root user

A console screen appears aws will appear

Conclusion: In this week we learnt how to deploy pods in minikube and how to monitor our local system using Nagios tool and will know how to create the aws free trier account.

LAB ACTION PLAN FOR WEEK 11

Jenkins-CI/CD

1. CI-Continuous Integration using Webhooks .
2. Sending E-mail Notification on Build Failure or success
3. Upload the screenshots for the tasks

Lab

Setting Up Jenkins CI-----using GitHub Webhook with Jenkins

Step 1: Configure Webhook in GitHub

1. Go to your GitHub repository.
2. Navigate to Settings → **Webhooks**.
3. Click **"Add webhook"**.
4. In the Payload URL field:
 - Enter the Jenkins webhook URL in the format:
http://<jenkins-server-url>/github-webhook/

Note: If Jenkins is running on localhost, GitHub cannot access it directly.

Use [ngrok](#) to expose your local Jenkins to the internet:

- ngrok.exe http <Jenkins local host:8080>
 - Use the generated ngrok URL, e.g.:
 - http://abc123.ngrok.io/**github-webhook/**
5. Set Content type to:
application/json
 6. Under "Which events would you like to trigger this webhook?", select:
 - Just the push event
 7. Click "Add webhook" to save.

Step 2: Configure Jenkins to Accept GitHub Webhooks

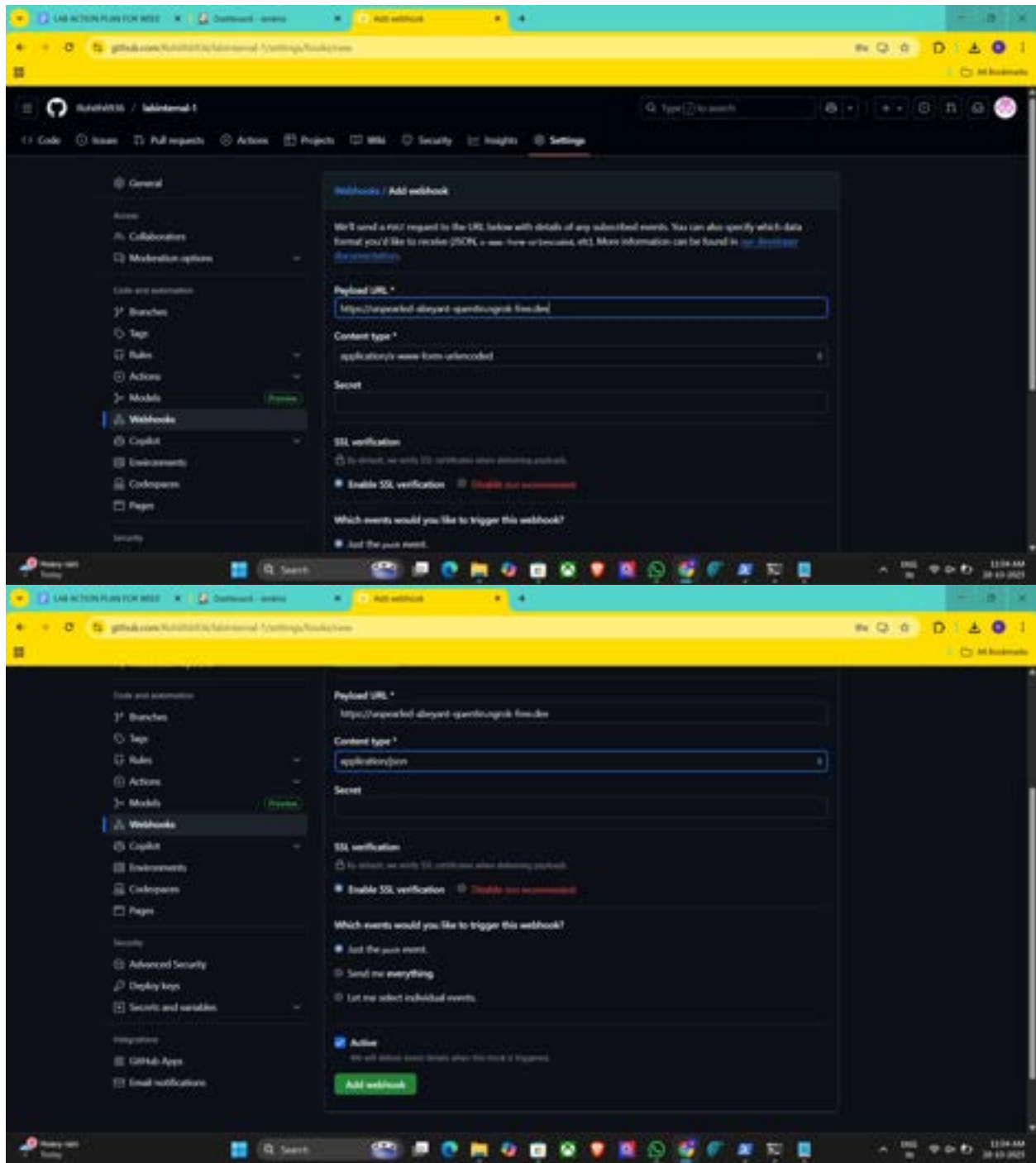
1. Open Jenkins Dashboard.
2. Select the job (freestyle or pipeline) you've already created.
3. Click Configure.
4. Scroll down to the Build Triggers section.
5. Check the box: ☒ GitHub hook trigger for GITScm polling
6. Click Save.

Step 3: Test the Setup

1. Make any code update in your local repo and push it to GitHub.
2. Once pushed, GitHub will trigger the webhook.
3. Jenkins will automatically detect the change and start the build pipeline.

outcome

- You've successfully connected GitHub and Jenkins using webhooks.
- Every time you push code to GitHub, Jenkins will automatically start building your project without manual intervention.



LAB ACTION PLAN FOR MSED | mmsweb_build | Jenkins | Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

[Build]

Additional Behaviours

Add

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts)
- ☐ Build after other projects are built
- ☐ Build periodically
- ☒ GitHub hook trigger for GITScm polling
- ☐ Poll SCM

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

Save Apply

LAB ACTION PLAN FOR MSED | mmsweb_build | Jenkins | Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

[Build]

Additional Behaviours

Add

Triggers

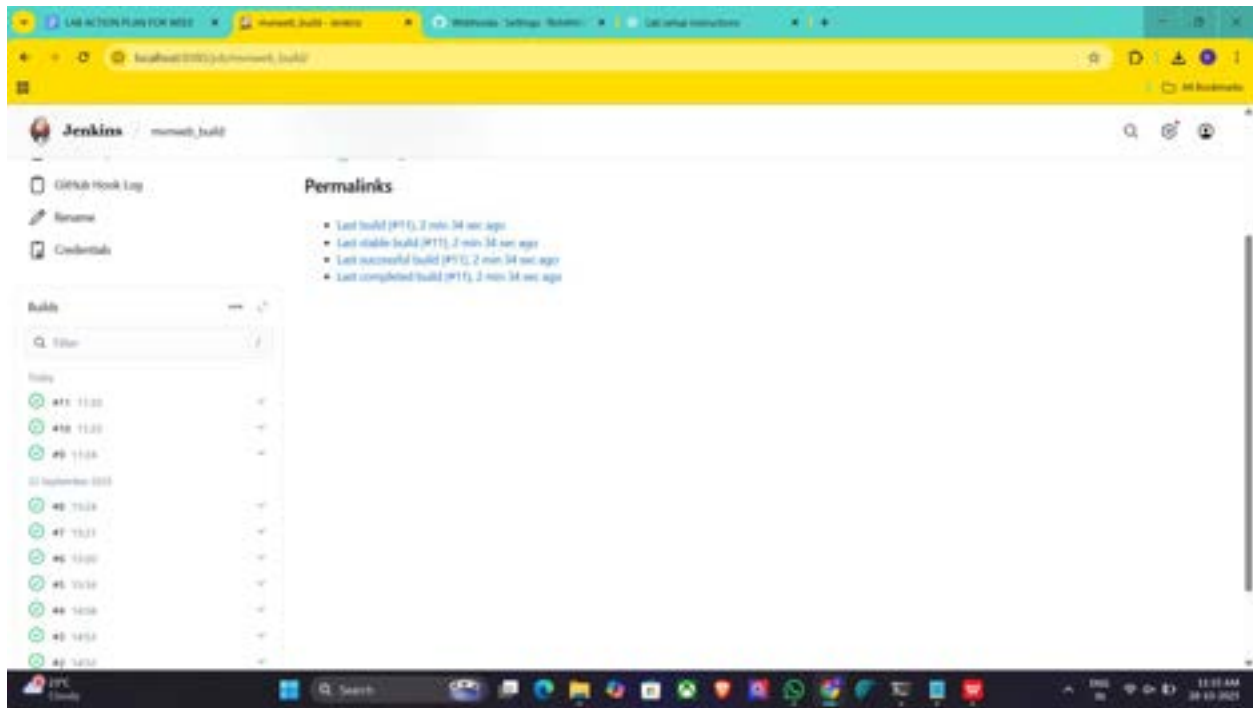
Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts)
- ☐ Build after other projects are built
- ☐ Build periodically
- ☒ GitHub hook trigger for GITScm polling
- ☐ Poll SCM

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

Save Apply



Set-uping the ngrok

How to Install and Use ngrok

Step 1. Download ngrok

<https://ngrok.com/download>

Download and extract it for your OS (Windows, macOS, or Linux).

Step 2. Connect Your ngrok Account (optional but useful)

After you sign up (free), ngrok gives you an auth token.

CREATE AUTHENTICATOR [<https://dashboard.ngrok.com/get-started/your-authtoken>]

Run this command (replace <your_token> with yours):

ngrok config add-authtoken <your_token>

This ensures stable sessions and more control.

Step 3. Start a Tunnel for Jenkins

Assuming Jenkins runs locally on port 8085:

ngrok http 8085

You'll see output like:

Session Status online

Forwarding <https://1234abcd.ngrok.io> -> <http://localhost:8080>

Copy the HTTPS URL (<https://1234abcd.ngrok.io>) — this is your public Jenkins URL for webhooks.

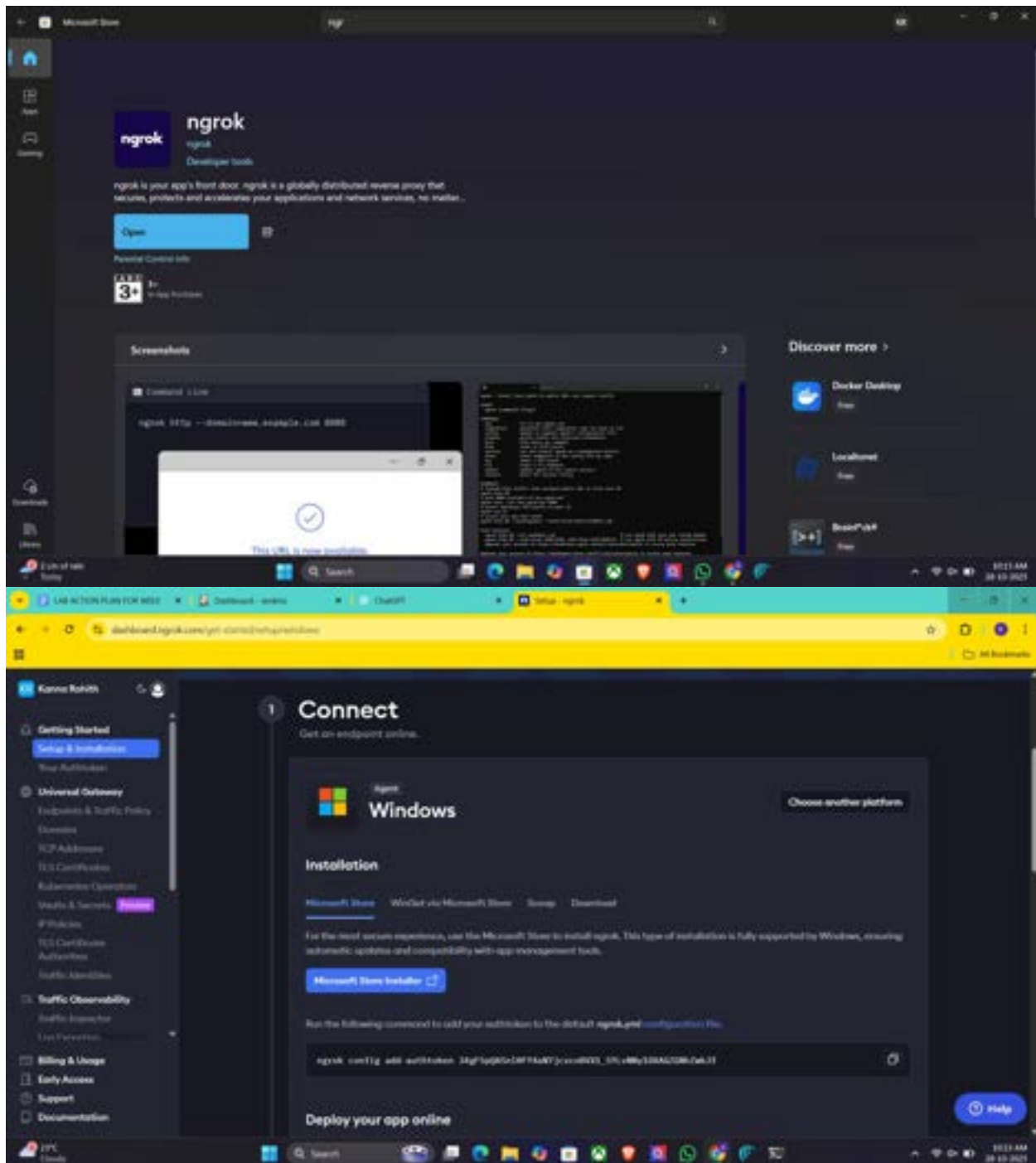
Step 4. Use it in GitHub Webhook

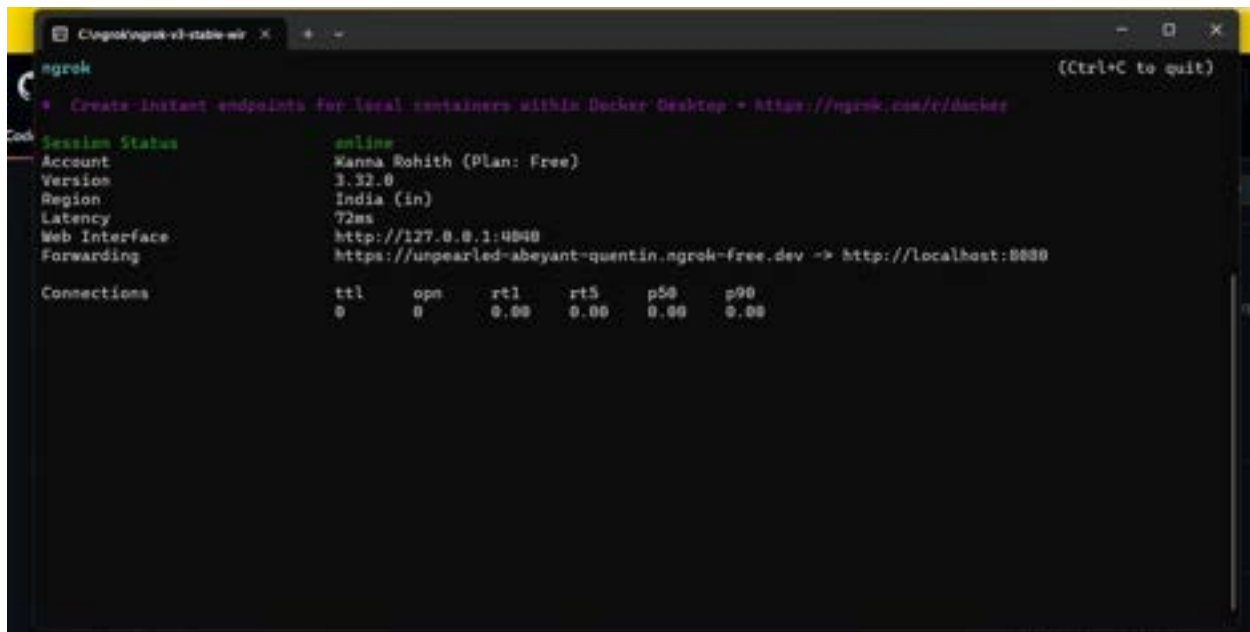
In your GitHub repo → Settings → Webhooks:

- Payload URL: *[paste the url generated by ngrok]*

<https://1234abcd.ngrok.io/github-webhook/> [please include this – remaining all default]

Now, whenever you push code, GitHub sends an event to that URL, which ngrok forwards to your local Jenkins.





```
ngrok
+ Create instant endpoints for local containers within Docker Desktop - https://ngrok.com/e/docker

Session Status      online
Account             Kanna Rohith (Plan: Free)
Version             3.32.0
Region              India (in)
Latency              72ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://unpearled-abeyant-quentin.ngrok-free.dev -> http://localhost:8080

Connections
  ttl    ops    rt1    rt5    p50    p90
    0     0    0.00    0.00    0.00    0.00
```

Setting Up Jenkins Email Notification Setup (Using Gmail with App Password)

Creation of app password

1. Gmail: Enable App Password (for 2-Step Verification)

i. Go to: <https://myaccount.google.com>

ii. Enable 2-Step Verification

- Navigate to:
 - Security → 2-Step Verification
 - Turn it **ON**
 - Complete the OTP verification process (via phone/email)

iii. Generate App Password for Jenkins

- Go to:
 - Security → App passwords
- Select:
 - **App:** Other (Custom name)
 - **Name:** Jenkins-Demo
- Click **Generate**
- Copy the **16-digit app password**
 - Save it in a secure location (e.g., Notepad)

2. Jenkins Plugin Installation

i. Open Jenkins Dashboard

ii. Navigate to:

- Manage Jenkins → Manage Plugins

iii. Install Plugin:

- Search for and install:
 - Email Extension Plugin
-

3. Configure Jenkins Global Email Settings

i. Go to:

- Manage Jenkins → Configure System

A. E-mail Notification Section

Field	Value
SMTP Server	smtp.gmail.com
Use SMTP Auth	✔ Enabled
User Name	Your Gmail ID (e.g., archanareddykmit@gmail.com)
Password	Paste the 16-digit App Password
Use SSL	✔ Enabled
SMTP Port	465

Reply-To Address Your Gmail ID (same as above)

➤ Test Configuration

- Click: Test configuration by sending test e-mail
- Provide a valid email address to receive a test mail
- ✔ Should receive email from Jenkins

B. Extended E-mail Notification Section

Field	Value
SMTP Server	smtp.gmail.com
SMTP Port	465
Use SSL	✔ Enabled
Credentials	Add Gmail ID and App Password as Jenkins credentials
Default Content Type	text/html or leave default
Default Recipients	Leave empty or provide default emails
Triggers	Select as per needs (e.g., Failure)

4. Configure Email Notifications for a Jenkins Job

i. Go to:

- Jenkins → Select a Job → Configure

ii. In the Post-build Actions section:

- Click: Add post-build action → **Editable Email Notification**

A. Fill in the fields:

Field	Value
Project Recipient List	Add recipient email addresses (comma-separated)
Content Type	Default (text/plain) or text/html
Triggers	Select events (e.g., Failure, Success, etc.)
Attachments	(Optional) Add logs, reports, etc.

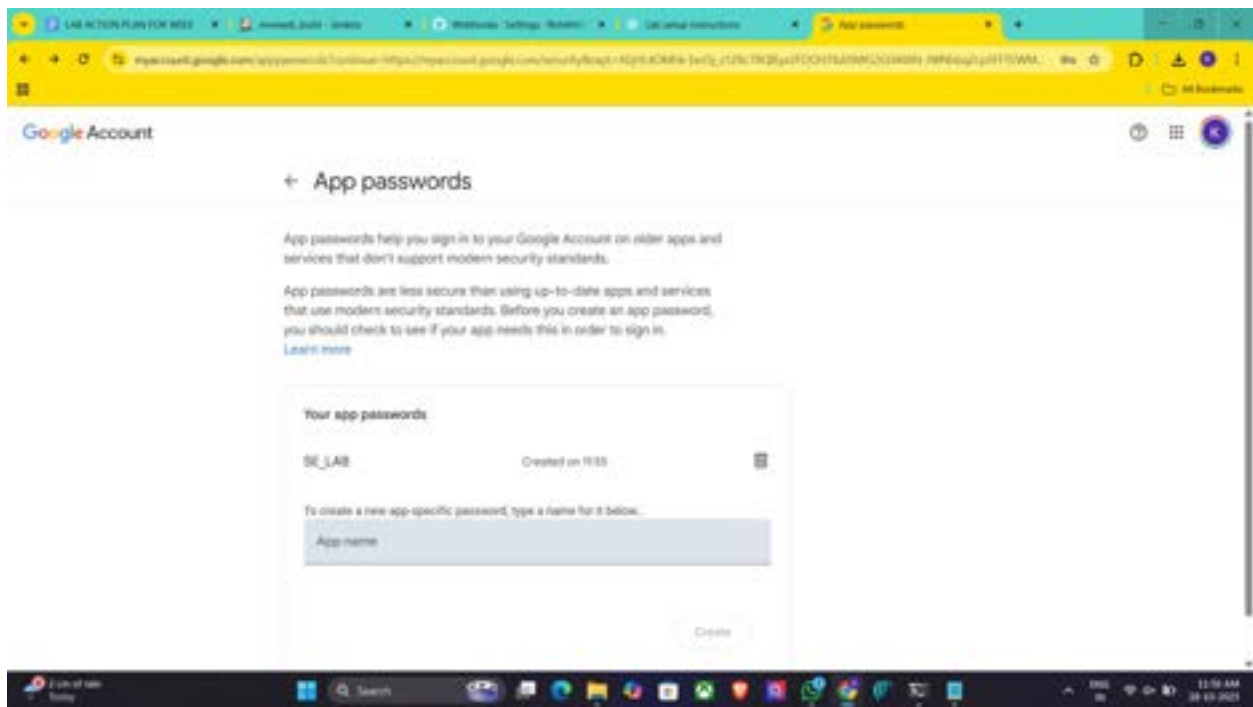
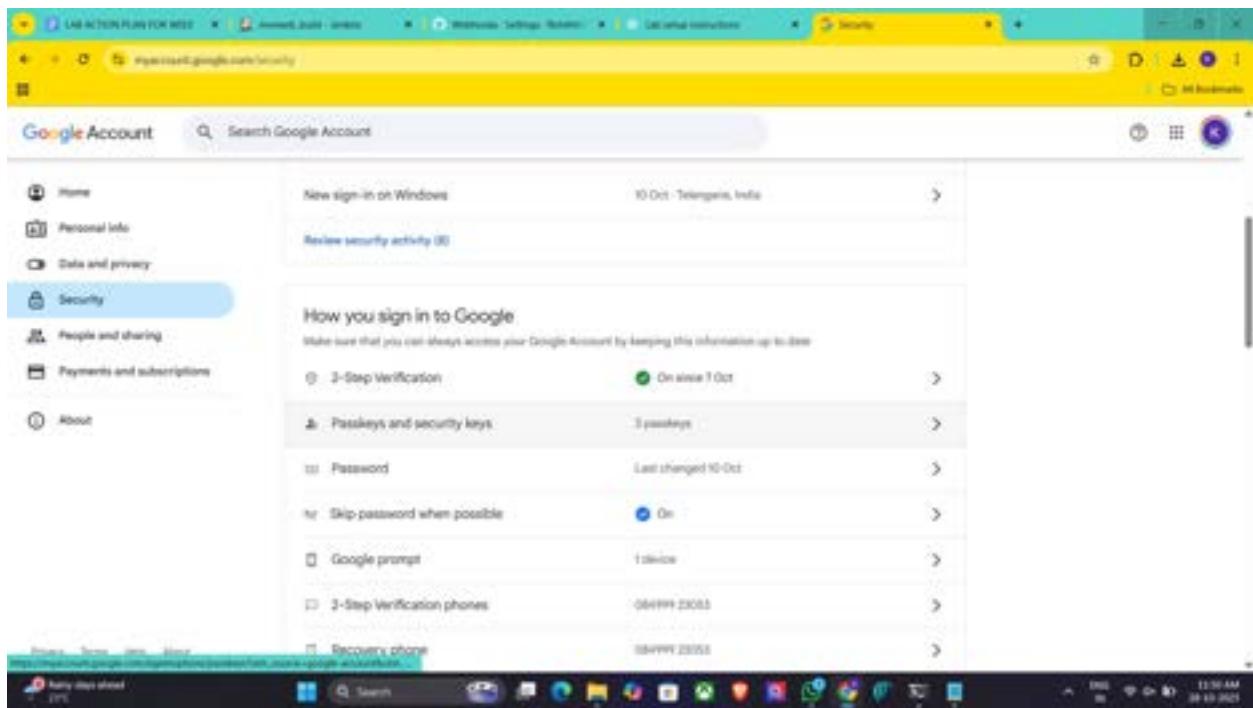
iii. Click Save

Now your Jenkins job is set up to send email notifications based on the build status!

- ---

Takeaway :

Students learned how to integrate Jenkins with GitHub using webhooks to automate build triggers and configure email notifications to monitor build success or failure effectively.



LAB ACTION PLAN FOR MISC | Installed plugins: Plugins | Webhooks: Settings: Tokens | Lab virtual instructions | Add elements

localhost:8080/manage2/pluginsManager/installed

Jenkins Manage Jenkins / Plugins

Plugins

Updates Available plugins Installed plugins Advanced settings

Search: email

Name	Health	Enabled
Email Extension 1.07.1 v1.07.1 (26, 1044) This plugin is a replacement for Jenkins's email publisher. It allows to configure every aspect of email notifications: when an email is sent, who should receive it and what the email says. Report an issue with this plugin	OK	☒
Mail Plugin 1.22 v1.22 (16, 51, 3) This plugin allows you to configure email notifications for build results. Report an issue with this plugin	OK	☐

REST API Jenkins 2.376.3

LAB ACTION PLAN FOR MISC | Available plugins: Plugins | Webhooks: Settings: Tokens | Lab virtual instructions | Add elements

localhost:8080/manage2/pluginsManager/available

Jenkins Manage Jenkins / Plugins

Plugins

Updates Available plugins Installed plugins Advanced settings

Search: Search available plugins

Install	Name	Released	Health
☐	Pipeline Graph Analysis 2.0 v2.0 (11, 3) Library plugin (for use by other plugins) Provides a REST API to access pipeline and pipeline run data.	1 mo 16 days ago	OK
☐	OIDC Authentication 1.12 Security Adds OpenID Connect (OIDC) support to Jenkins.	7 mo 27 days ago	OK
☐	LocalMail API 1.0.2 v1 Library plugin (for use by other plugins) This plugin provides the LocalMail API for other plugins.	8 mo 7 days ago	OK
☐	Command Agent Launcher 1.03 v1.03 (14, 10) Agent Management Allows agents to be launched using a specified command.	6 mo 20 days ago	OK
☐	Oracle Java SE Development Kit Installer 0.0.1 v0.0.1 (1, 1) Allows the Oracle Java SE Development Kit (JDK) to be installed via download from Oracle's website.	9 mo 9 days ago	OK
☐	SSH server 1.27 v1.27, 102 v1.27 Adds SSH server functionality to Jenkins, exposing CI2 commands through it.	2 mo 16 days ago	OK

Jenkins Manage Jenkins

Building on the built-in node can be a security issue. You should set up distributed builds. See [Ext.DistributedBuilds]

Warnings have been published for the following currently installed components

Build Pipeline Plugin 2.3.2
(Known XSS vulnerability (see fix available))
No fixes for these issues are available. It is recommended that you review the security advisory and apply mitigations if possible, or uninstall this plugin.

Go to plugin manager Configure which of these warnings are shown

System Configuration

- System: Configure global settings and paths.
- Tools: Configure tools, their locations and automatic updates.
- Plugins: Add, remove, disable or enable plugins that can extend the functionality of Jenkins.
- Nodes: Add, remove, control and monitor the various nodes that Jenkins runs jobs on.
- Clouds: Add, remove, and configure cloud instances to provision agents on-demand.
- Appearance: Configure the look and feel of Jenkins.

Security

Global Properties

Global Tool Configuration

Global Security

Global Labels

Global Tags

Global Credentials

Global Variables

Global Environment

Global Parameters

Global Scripts

Global Templates

Global Views

Global Dashboards

Jenkins Manage Jenkins

System

E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail address

Address

Use SMTP authentication

Use SMTP

Use TLS

SMTP host

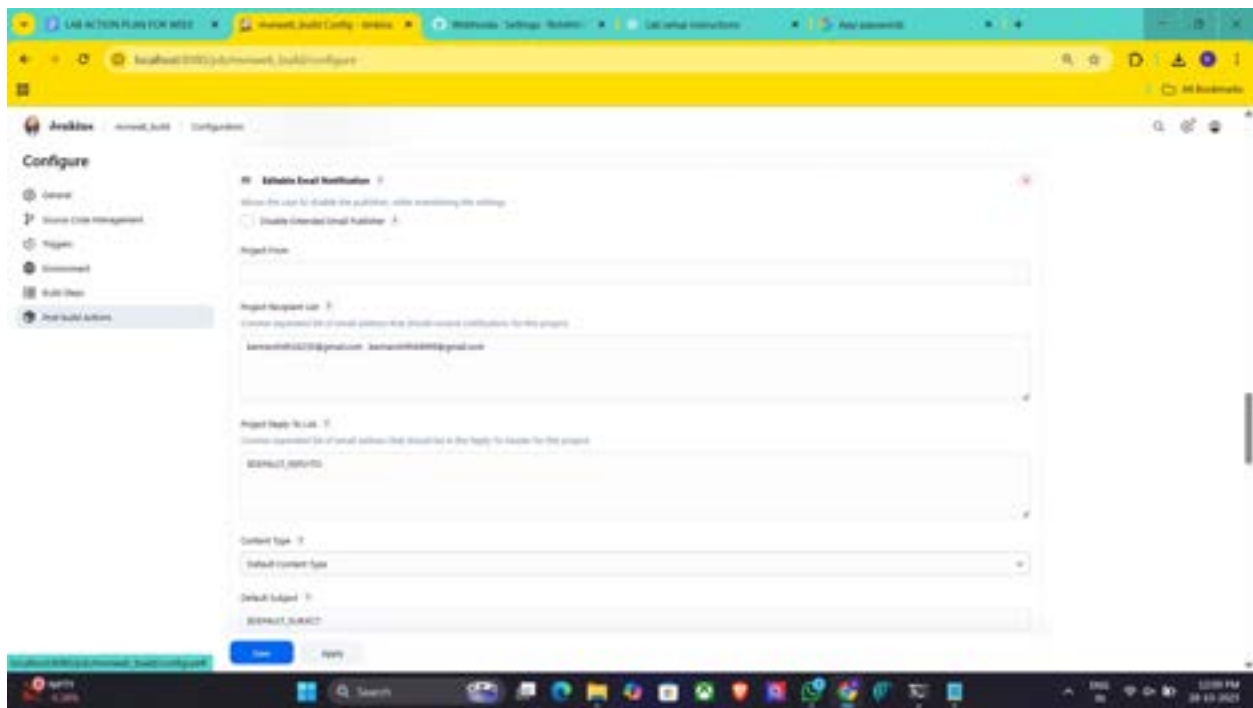
Host

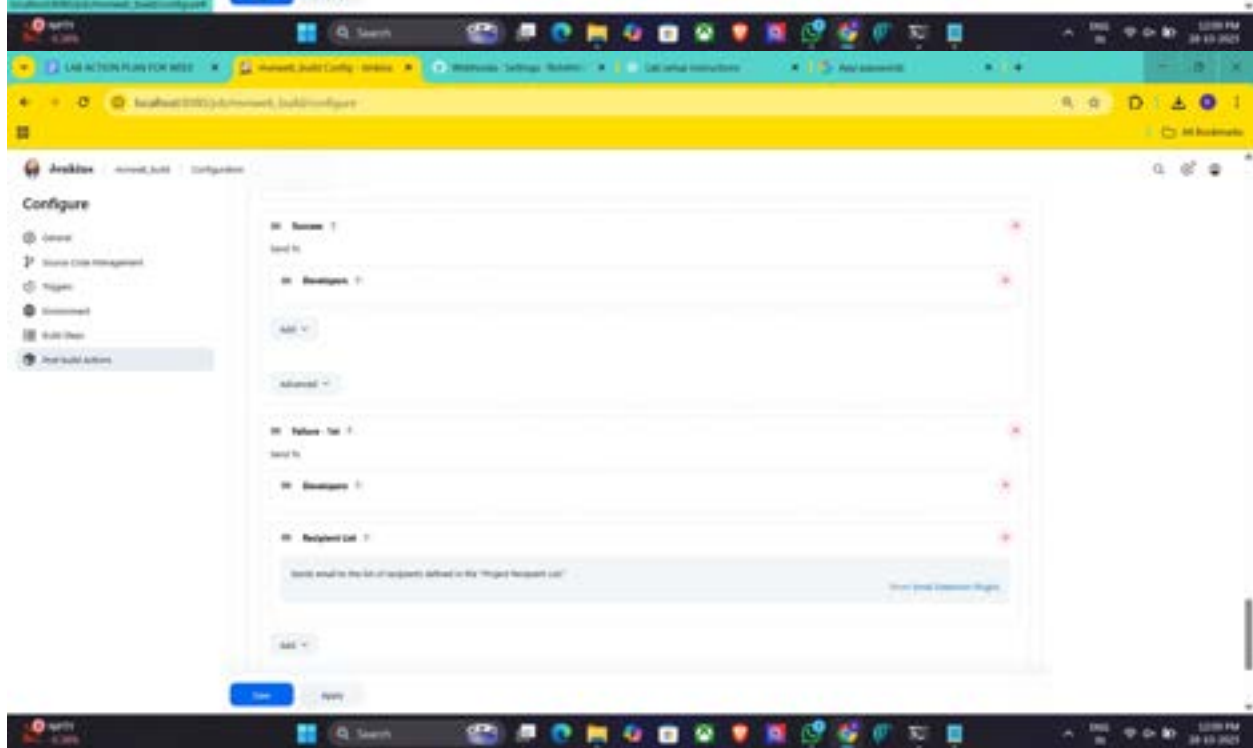
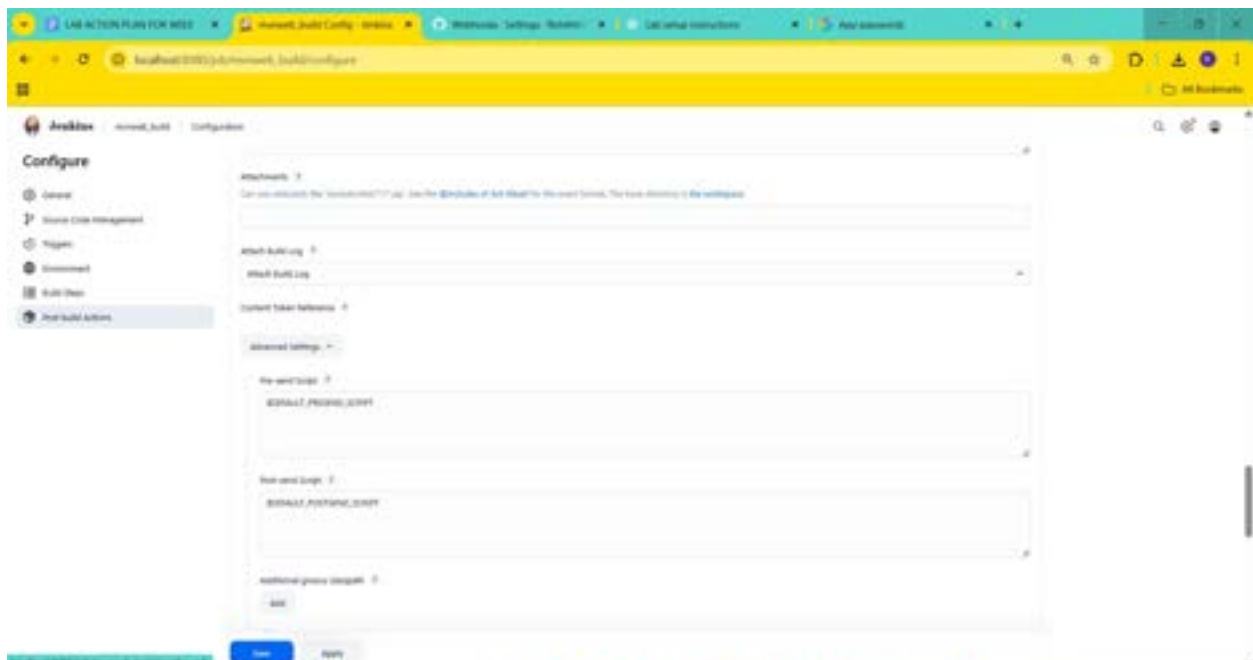
SMTP host address

smtp.gmail.com

Save Apply

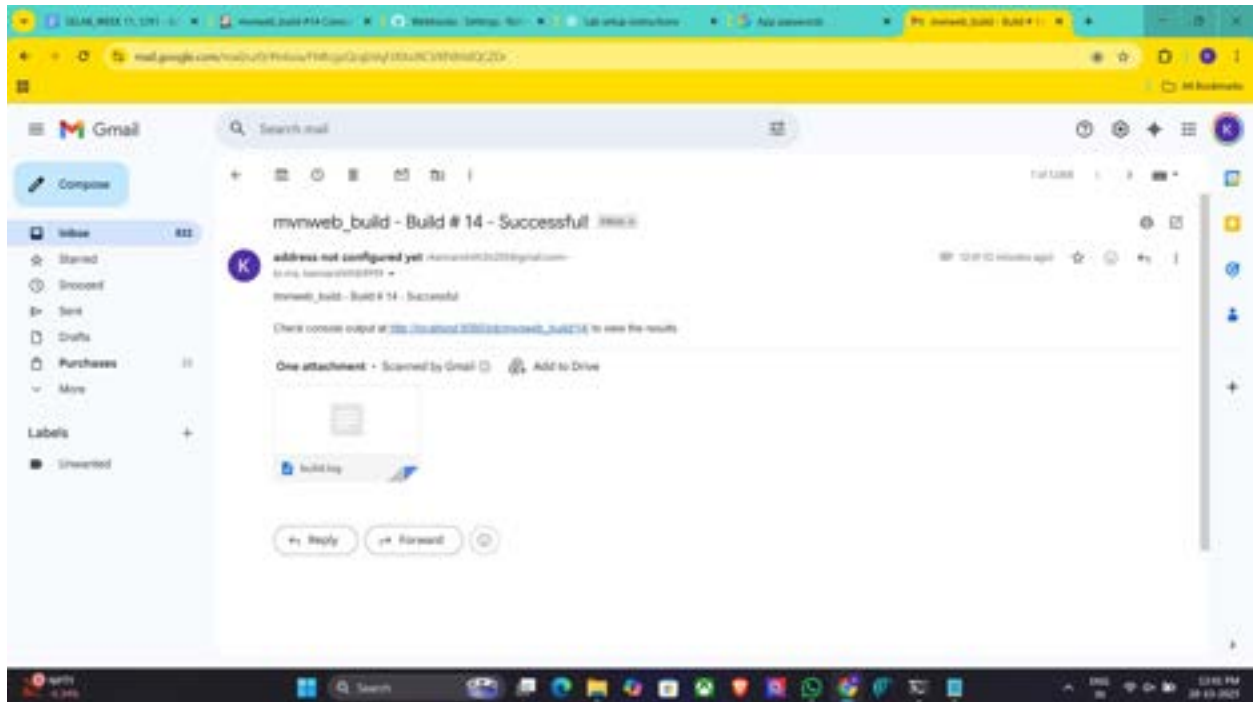
The screenshot shows the AWS Amplify console interface. The top navigation bar includes the AWS logo and the text 'Configure'. The left sidebar contains a 'Configure' section with a 'General' tab selected. The main content area is titled 'General' and features a 'Description' field with the placeholder text 'add build details'. Below this, there are several checkboxes for build configuration: 'Skip all builds', 'Skip project', 'Rebuild on code commit', 'This project is untested', 'Use this build', 'Exclude untested builds of necessary', and 'Advanced'. The 'Source Code Management' section is also visible, showing a dropdown menu for 'Provider' with 'None' selected. The bottom of the console shows a 'Save' button and an 'Apply' button.






```
[INFO] Processing our project
[INFO] Copying webapp resources [/ProgramData/Jenkins/Jenkins/workspace/mymweb_build/./src/main/webapp]
[INFO] Building war: /ProgramData/Jenkins/Jenkins/workspace/mymweb_build/target/mymweb.war
[INFO]
[INFO] --- install:3.1.2:install (default-install) @ mymweb ---
[INFO] Installing /ProgramData/Jenkins/Jenkins/workspace/mymweb_build/target/mymweb.war to /C:/Users/robert/Local Settings/Application Data/Jenkins/Jenkins/workspace/mymweb_build/.m2/repository/org/jenkins-ci/plugins/mymweb/1.0-SNAPSHOT/mymweb.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.588 s
[INFO] Finished at: 2020-06-27 12:45:06 GMT
[INFO] -----
Archiving artifacts
[INFO] Task triggered for: Always
Sending email for trigger: Always
Sending email for: robert@robert@gmail.com robert@robert@gmail.com
Triggering a new build of mymweb
Finished: SUCCESS
```

8:27 AM Jenkins 2.191.3



Week 12: Creation of virtual machine for Ubuntu OS and Deploying the web application

1. Creation of virtual machine
 2. Deploying the web application
 3. Accessing it publicly
-

Deploying an application into cloud

Steps for Deploying application into the cloud

- I. Create application and Push into github
- II. Create the virtual machine and connect to it.
- III. Clone the application from github, Write the Dockerfile
- IV. Create the image
- V. Run the image and access it public ip of virtual machine

I. Create Maven-web-java project in eclipse & push into github

II. Create the virtual machine (EC2--instance) in aws and connect to

Using Amazon EC2 eliminates your need to invest in hardware up front, so you can develop and deploy applications faster.

Ex : **Launch ubuntu instnace**

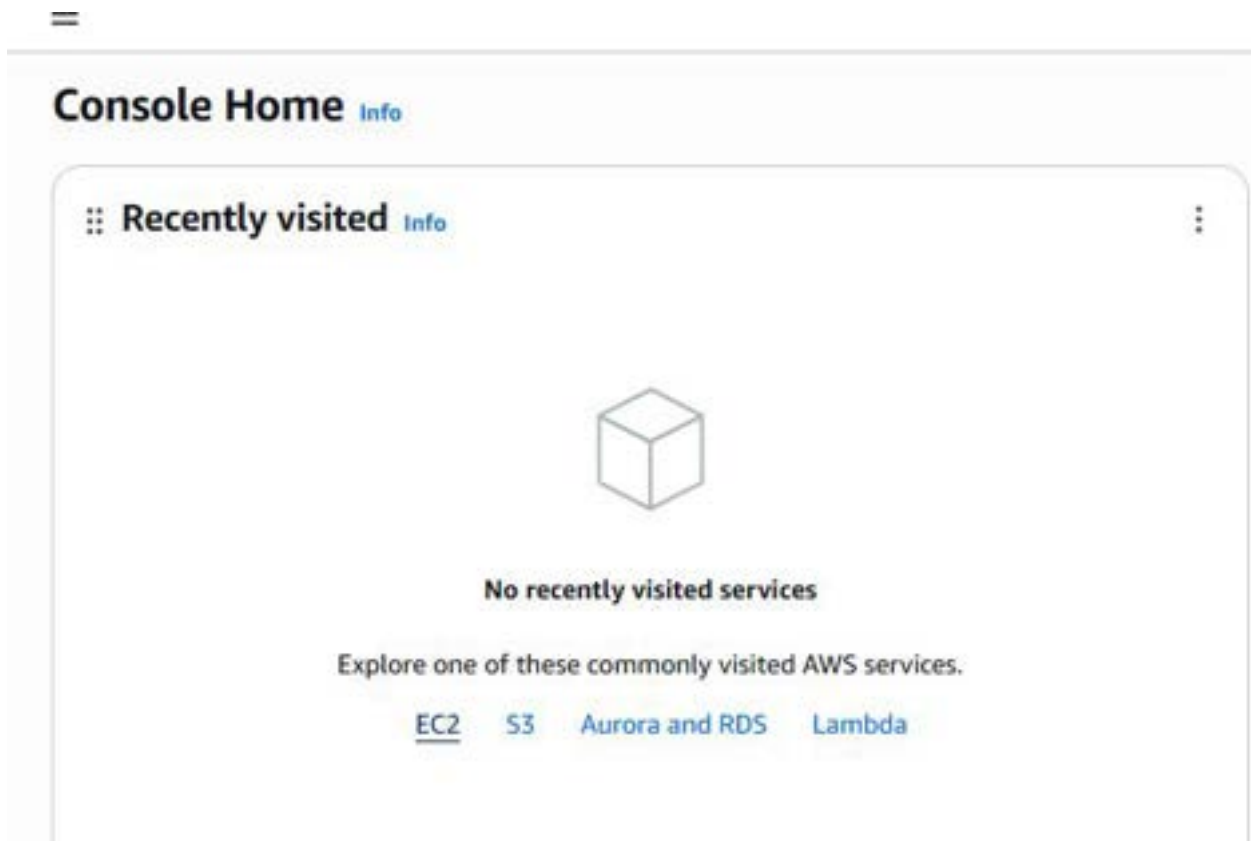
Step 1: Login to AWS /canvas account



Step 2: Services -- EC2

Step 3: Choose region which is near ?

Services -- EC2 --- Launch Instance



Stage 1 --Name (Giving name to the machine) ubuntu

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

Stage 2 -- Select AMI (Note: Select free tier eligible) ubuntu server

Stage 3 -- Architecture as 64-bit

[EC2](#) > [Instances](#) > Launch an instance

Linux

aws

Mac

ubuntu

Microsoft

Red Hat

SUSE

al

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0ecb62995f68bb549 (64-bit (x86)) / ami-01b9f1e7dc427266e (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

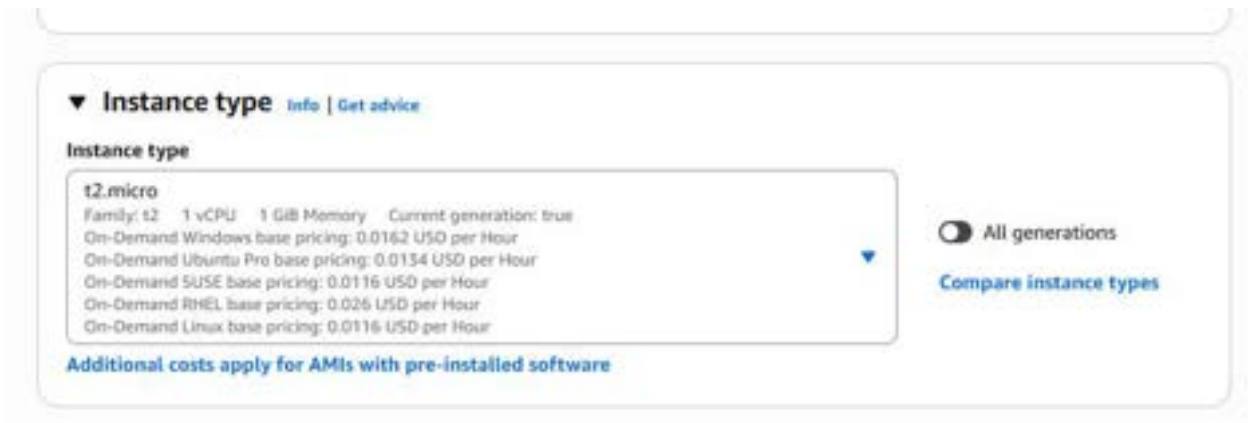
Description

Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

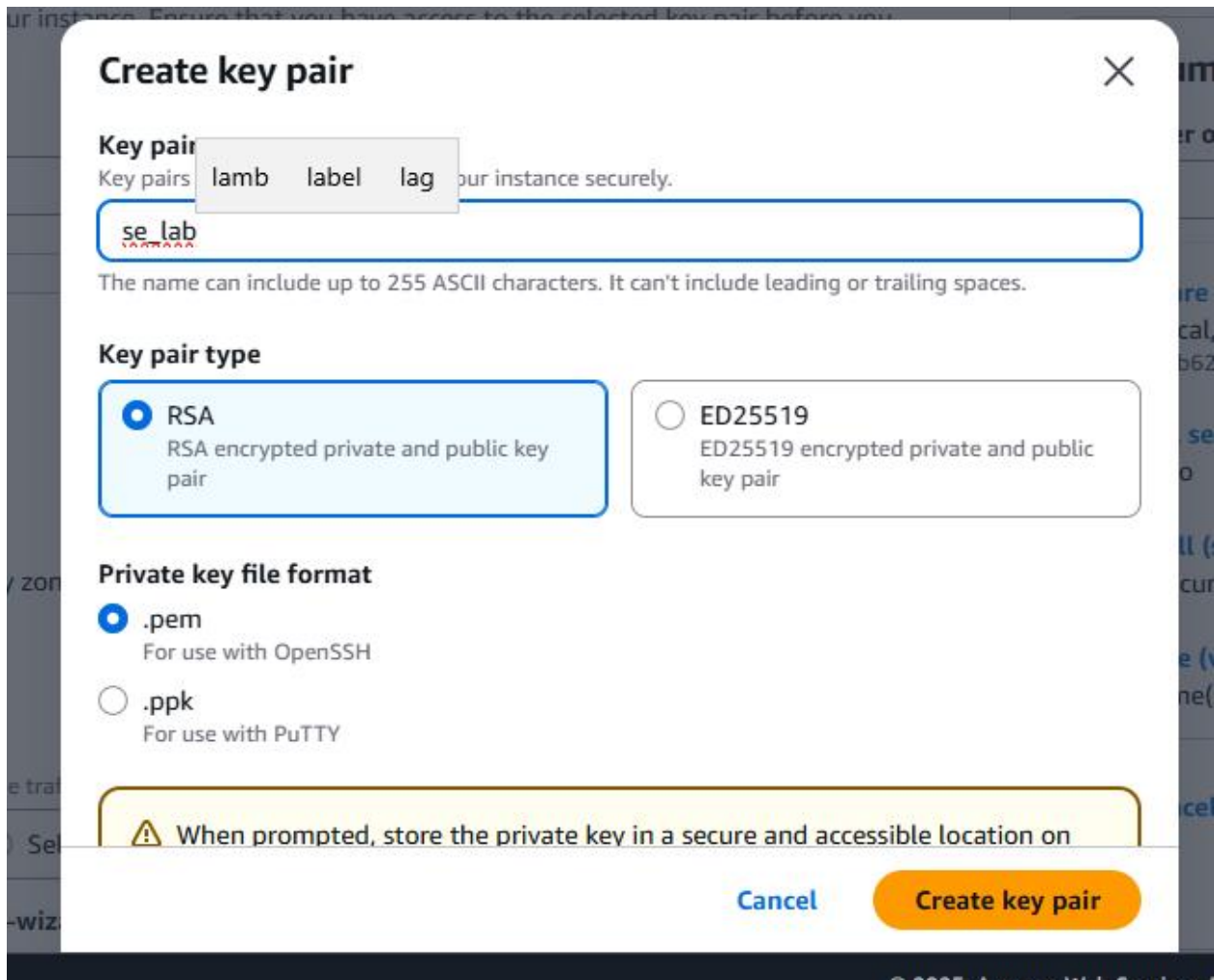
Architecture	AMI ID	Publish Date	Username	
64-bit (x...	ami-0ecb62995f68bb549	2025-10-22	ubuntu	Verified provider

Stage 4 -- **Instance type** ---- t2.micro(default 1 CPU,1 GB RAM)



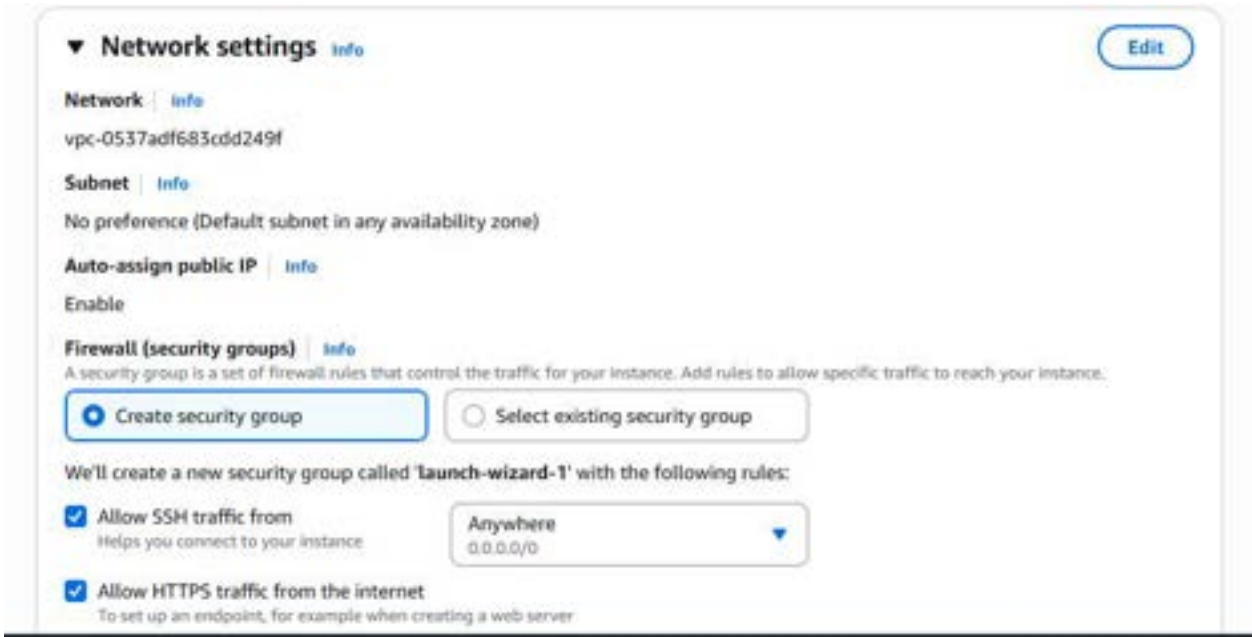
Stage 5 -- Create a new keypair---a keypair will be downloaded with extension .pem

Store key in folder AWS



Stage 6 -- Network Setting ----Create Security group -- (It deals with ports)

(Note for understanding We have 0 to 65535 ports. Every port is dedicated to special purpose)



The screenshot shows the 'Network settings' panel in the AWS console. It includes sections for Network (vpc-0537adf683cdd249f), Subnet (No preference), Auto-assign public IP (Enabled), and Firewall (security groups). Under Firewall, there are two radio buttons: 'Create security group' (selected) and 'Select existing security group'. Below this, it states 'We'll create a new security group called 'launch-wizard-1' with the following rules:'. There are two checked rules: 'Allow SSH traffic from' (source: Anywhere, 0.0.0.0/0) and 'Allow HTTPS traffic from the internet' (description: To set up an endpoint, for example when creating a web server).

Do this step : HERE select http and https

Stage 7 -- Storage - 8GB (Observation - we have root - it is same as C Drive)

Stage 8 --- click on launch instance

Stage 9: Number of instances ---1

+++++

Observation - One machines created

▼ Summary

Number of instances | Info

1

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...read more
ami-0ecb62995f68bb549

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

Success

Successfully initiated launch of instance i-0f5cdd76343330f

Do this step:---once it is created **select that instance and click on connect**

EC2

Instances

Dashboard

EC2 Global View

Events

Instances

Instance Types

Instances (1/1)

Connect

Instance state

Actions

Launch instances

Find instance by attribute or tag (case-sensitive)

All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
ubuntu	i-0f5cdd76343330f	Running	t2.micro	Initializing	View alarms	us-east-1c

Here copy the ssh – i command from SSH client connect tab

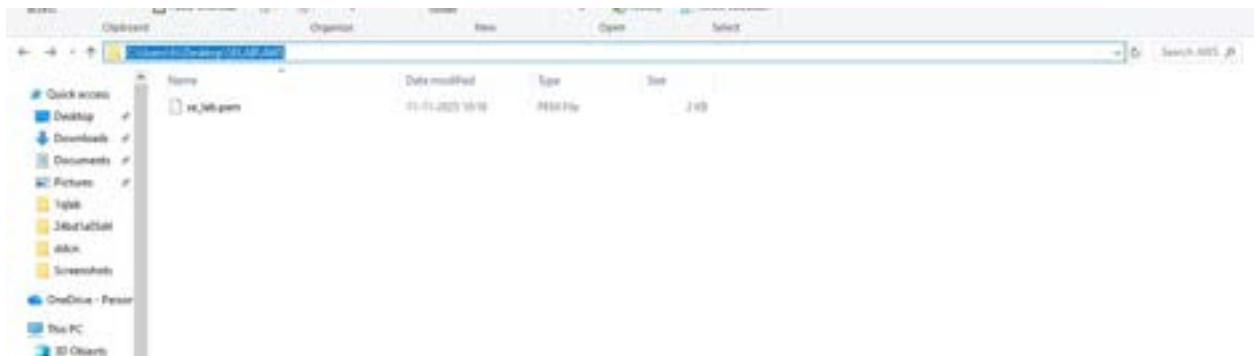
We can use powershell /gitbash /webconsole , to connect to ubuntu machine.

NOTE:- cd path of AWS folder // change path

To connect to above terminals we need to go into the path of the keypair.and

paste the

ssh -i command from the aws console



III. Clone the application from github, Write the Dockerfile

once connected to instance

Step 1:- **Run the following commands to install s/w**

1. Update all softwares in Ubuntu by command

sudo apt update

```
ubuntu@ip-172-31-16-33:~$ sudo apt install udo
ubuntu@ip-172-31-16-33:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1305 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [215 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9336 B]
```

2. Install docker by command

sudo apt-get install docker.io

```
ubuntu@ip-172-31-16-33:~$ sudo apt-get install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd das-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debconf-tzdata docker-buildx docker-compose-v2 docker-doc rinse sfs-fuse | sfsutils
The following NEW packages will be installed:
  bridge-utils containerd das-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 10 not upgraded.
Need to get 76.0 MB of archives.
After this operation, 288 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.1.3-0ubuntu1-24.04.2 [8815 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.28-0ubuntu1-24.04.1 [38.4 MB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 das-root-data all 2024071801-ubuntu0.24.04.1 [5918 B]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 dnsmasq-base amd64 2.90-2ubuntu0.1 [376 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 28.2.2-0ubuntu1-24.04.1 [28.3 MB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 ubuntu-fan all 0.12.16+24.04.1 [34.2 kB]
Fetched 76.0 MB in 1s (81.9 MB/s)
Reconfiguring packages ...
Selecting previously unselected package pigz.
(Reading database ... 71735 files and directories currently installed.)
```

3. Install git by command

sudo apt install git

```
ubuntu@ip-172-31-16-33:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
ubuntu@ip-172-31-16-33:~$
```

4. Install nano(text editor) by command

Sudo apt install nano

```
ubuntu@ip-172-31-16-33:~$ sudo apt install nano
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nano is already the newest version (7.2-2ubuntu0.1).
nano set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
ubuntu@ip-172-31-16-33:~$
```

step 2:- git clone <paste the github link of maven-web-java project>

```
ubuntu@ip-172-31-16-33:~$ git clone https://github.com/Rohith4936/SELAS_maven_java_project-.git
Cloning into 'SELAS_maven_java_project'...
remote: Enumerating objects: 53, done.
remote: Counting objects: 100% (53/53), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 53 (delta 1), reused 53 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (53/53), 9.62 MiB | 1.92 MiB/s, done.
Resolving deltas: 100% (1/1), done.
ubuntu@ip-172-31-16-33:~$
```

step 3:- navigate to the maven-web-java project

VI. Create the image

Step 4:- nano Dockerfile

```
GNU nano 8.2 Dockerfile
# Use an official Java runtime as a parent image
FROM tomcat:9-jdk11

# Copy the built WAR file to the Tomcat webapps directory
COPY *.war /usr/local/tomcat/webapps/

# Expose the port Tomcat is running on
# EXPOSE 8080

# Start Tomcat server
# CMD ["catalina.sh", "run"]
```

V. Run the image and access it with public ip of virtual machine

Step 1:- build your image

`docker build -t <imagename> .(dot)`

```
ubuntu@ip-172-31-16-33:~/labinternal-1$ sudo docker build -t webapp .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.229MB
Step 1/3 : FROM tomcat:9.0
9.0: Pulling from library/tomcat
4b3ffd8ccb52: Already exists
b48f960b380d: Pulling fs layer
58424d8c3e86: Pulling fs layer
4f4fb700ef54: Pulling fs layer
37b617836889: Pulling fs layer
891b6ad931b7: Pulling fs layer
ac0beccecf50: Pulling fs layer
37b617836889: Waiting
891b6ad931b7: Waiting
ac0beccecf50: Waiting
4f4fb700ef54: Verifying Checksum
4f4fb700ef54: Download complete
37b617836889: Verifying Checksum
37b617836889: Download complete
891b6ad931b7: Verifying Checksum
891b6ad931b7: Download complete
b48f960b380d: Verifying Checksum
b48f960b380d: Download complete
58424d8c3e86: Verifying Checksum
58424d8c3e86: Download complete
```

Step 2:- check for images

```
ubuntu@ip-172-31-16-33:~/labinternal-1$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
webapp	latest	e1b44d1dddf92	8 seconds ago	414MB
tomcat	9-jdk11	f79dfb7f6fee	11 hours ago	422MB
tomcat	9.0	2e4887a16e43	11 hours ago	413MB

```
ubuntu@ip-172-31-16-33:~/labinternal-1$
```

Step 3:- run image

```
docker run -d --name app-demo -p 6060:8080 <image name>
```

```

root@kali:~# docker run -d -p 6063:8080 webapp
1525261c67c767e4d3b4d788062e141f45c082552674795a13240b
root@kali:~# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED          STATUS          PORTS
1525261c67c767e4d3b4d788062e141f45c082552674795a13240b  webapp             "catalina.sh run"        2 minutes ago   Up 2 minutes   0.0.0.0:6063->8080/tcp, [::]:6063->8080/tcp
1725261746b34      webapp             "catalina.sh run"        10 minutes ago  Up 10 minutes  8080/tcp, 0.0.0.0:6063->8082/tcp, [::]:6063->8082/tcp

```

Step:-4 Accessing the app by public ip of virtual machine

Note:-if your are not able to connect change the inbound rules..

